
Oracle Database 11g: Develop PL/SQL Program Units

Electronic Presentation

D49986GC12
Edition 1.2
April 2008

ORACLE®

Author

Lauran K. Serhal

**Technical Contributors
and Reviewers**

Don Bates	Bryn Llewellyn
Claire Bennett	Lakshmi Narapareddi
Zarko Cesljas	Essi Parast
Purjanti Chang	Alan Paulson
Ashita Dhir	Manish Pawar
Peter Driver	Srinivas Putrevu
Gerlinde Frenzen	Bryan Roberts
Steve Friedberg	Grant Spencer
Nancy Greenberg	Tulika Srivastava
Thomas Hoogerwerf	Glenn Stokol
Akira Kinutani	Jenny Tsai-Smith
Chaitanya Koratamaddi	Lex Van Der Werff
Timothy Leblanc	Ted Witiuk

Graphic Designer

Asha Thampy

Editors

Nita Pavitran
Aju Kumar

Publishers

Sheryl Domingue
Syed Ali

Copyright © 2009, Oracle. All rights reserved.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

1

Introduction

Lesson Objectives

After completing this lesson, you should be able to do the following:

- Discuss the goals of the course
- Identify the available environments that can be used in this course
- Describe the database schema and tables that are used in the course
- List the available documentation and resources

Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course and the available PL/SQL development in this course
- Overview of Oracle SQL Developer
- Oracle 11g documentation and additional resources

Course Objectives

After completing this course, you should be able to do the following:

- Create, execute, and maintain:
 - Procedures and functions with OUT parameters
 - Package constructs
 - Database triggers
- Manage PL/SQL subprograms and triggers
- Use a subset of Oracle-supplied packages to:
 - Generate screen and file output

Course Agenda

Day 1:

- Introduction
- Creating Procedures
- Creating Functions
- Creating Packages
- Working with Packages

Day 2:

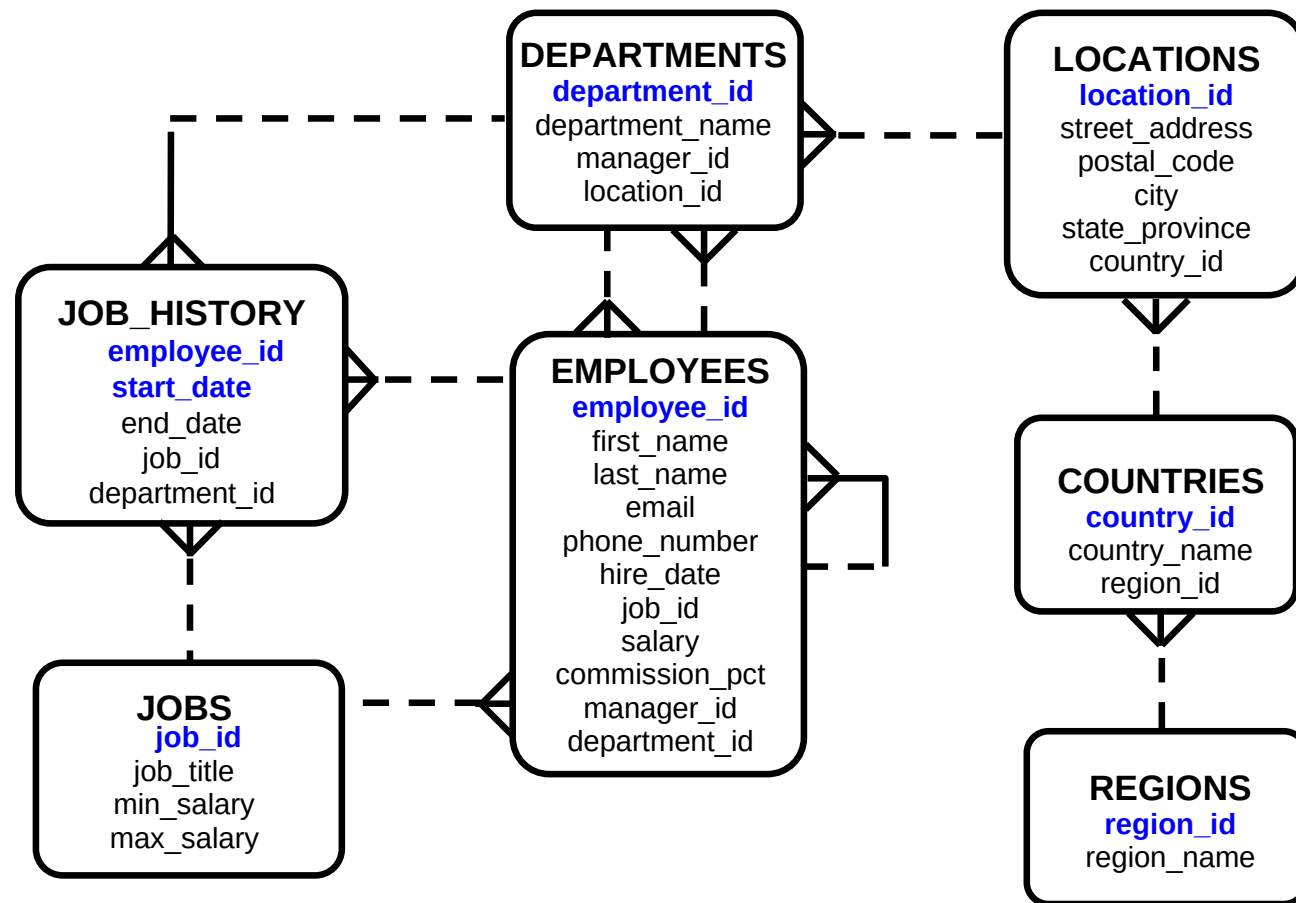
- Using Oracle-Supplied Packages in Application Development
- Using Dynamic SQL
- Design Considerations for PL/SQL Code
- Creating Triggers

Course Agenda

Day 3:

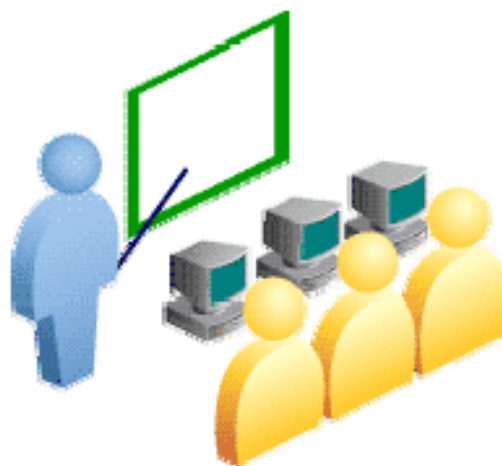
- Creating Compound, DDL, and Event Database Triggers
- Using the PL/SQL Compiler
- Managing PL/SQL Code
- Managing Dependencies

The Human Resources (HR) Schema That Is Used in This Course



Class Account Information

- Cloned HR account IDs are set up for you.
- Your account IDs are ora61 – ora80.
- The password matches your account ID.
- Each machine is assigned one account.
- The instructor has a separate ID.



Appendixes Used in This Course

- Appendix A: Practices and Solutions
- Appendix APS: Additional Practice: Solutions
- Appendix B: Table Descriptions
- Appendix C: Using SQL Developer
- Appendix D: Review of PL/SQL
- Appendix E: Using SQL*Plus
- Appendix F: Studies for Implementing Triggers
- Appendix G: Using the DBMS_SCHEDULER and HTP Packages
- Appendix H: Review of JDeveloper

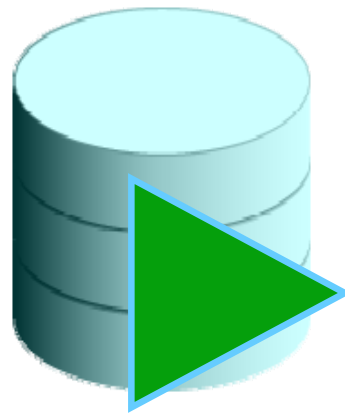
PL/SQL Development Environments

This course setup provides the following tools for developing PL/SQL code:

- Oracle SQL Developer (used in this course)
- Oracle SQL*Plus
- Oracle JDeveloper IDE

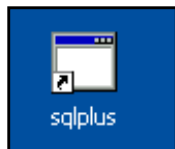
What Is Oracle SQL Developer?

- Oracle SQL Developer is a free graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema using standard Oracle database authentication.
- You use SQL Developer in this course.



SQL Developer

Coding PL/SQL in SQL*Plus



```
C:\ sqlplus

SQL*Plus: Release 11.1.0.4.0 - Beta on Mon Apr 30 08:01:20 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.

SQL> connect ora61/ora61@orcl
Connected.
SQL> set serveroutput on
SQL> create or replace procedure hello is
  2 begin
  3   dbms_output.put_line('Hello Class!');
  4 end;
  5 /

Procedure created.

SQL> execute hello
Hello Class!

PL/SQL procedure successfully completed.

SQL> _
```

Coding PL/SQL in Oracle JDeveloper



Run

The screenshot shows the Oracle JDeveloper IDE interface. The **System - Navigator** on the left displays a tree view of the project structure. The **HR** package is selected, and the **HELLO** procedure is highlighted with a red box. A red arrow points from this box to the **Messages** window at the bottom, which shows the execution output: "Connecting to the database hr.", "Executing PL/SQL: CALL HR.'JDEV_TMP_PROC_1'()", "Hello World", and "Process exited.".

Edit

The **Procedure.HR.HELLO.pls** editor window shows the following PL/SQL code:

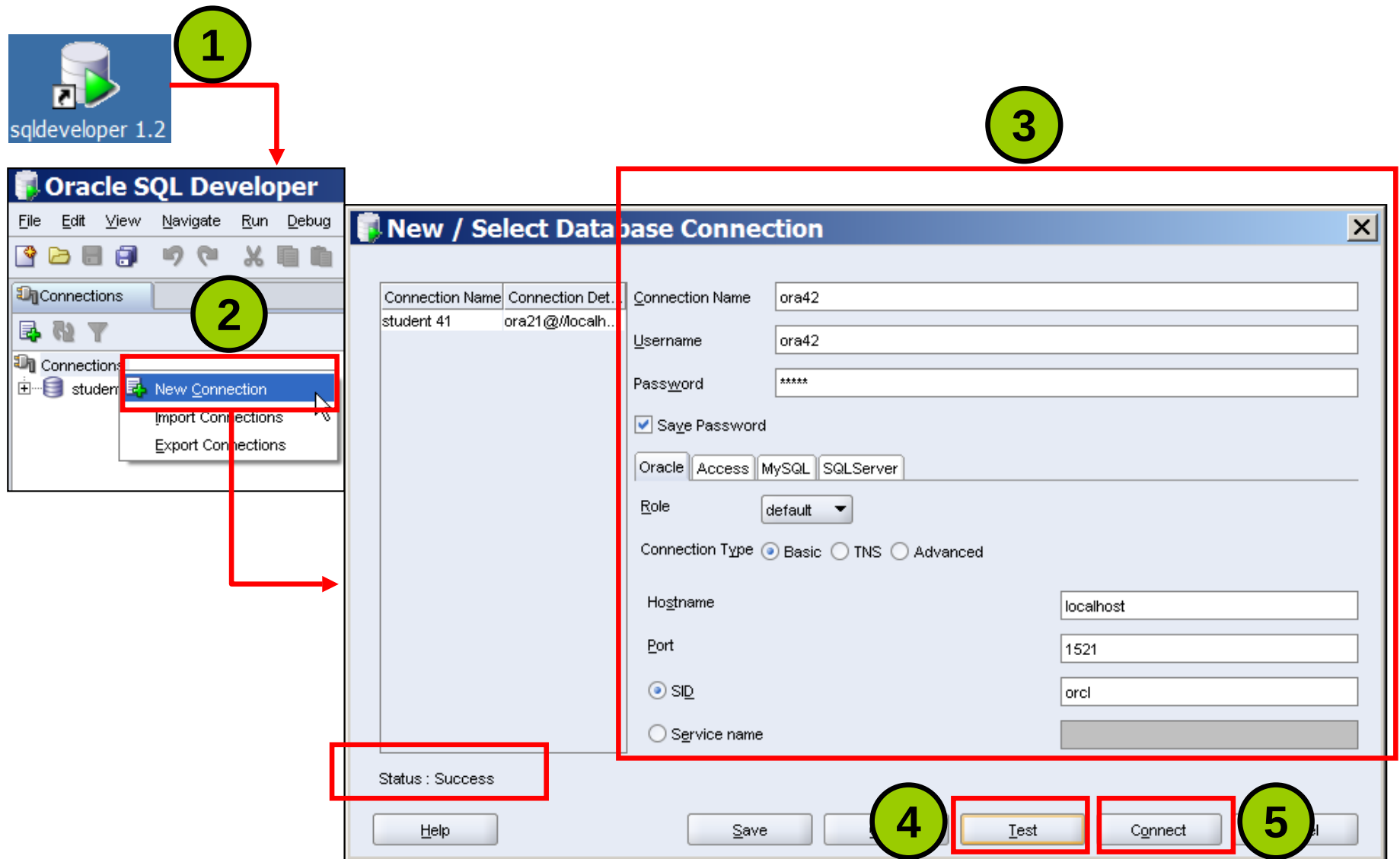
```
1 PROCEDURE HELLO AS
2 BEGIN
3   dbms_output.put_line('Hello World');
4 END ;
5
6
```

A red arrow points from the **HELLO** procedure in the **System - Navigator** to the **BEGIN** statement in the code editor.

Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course and the available PL/SQL development in this course
- Overview of Oracle SQL Developer
- Oracle 11g documentation and additional resources

Starting SQL Developer and Creating a Database Connection

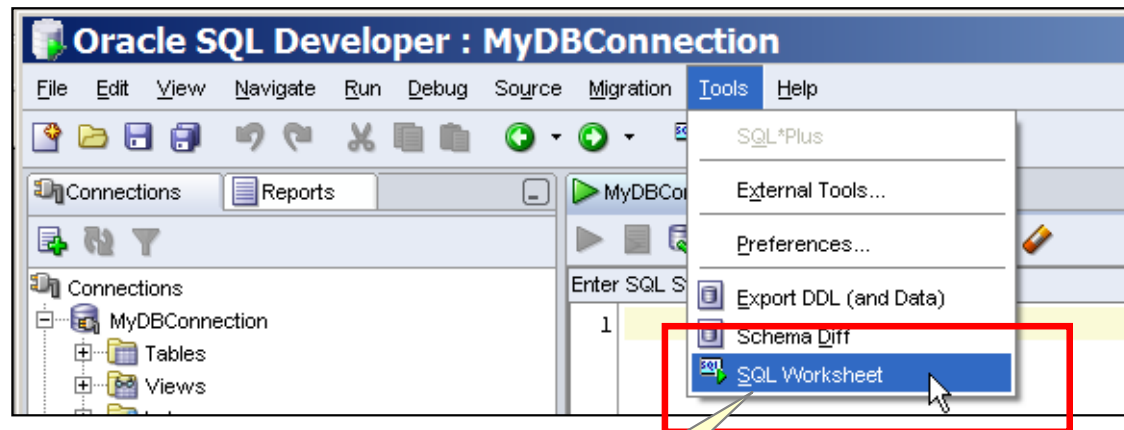


Creating Schema Objects

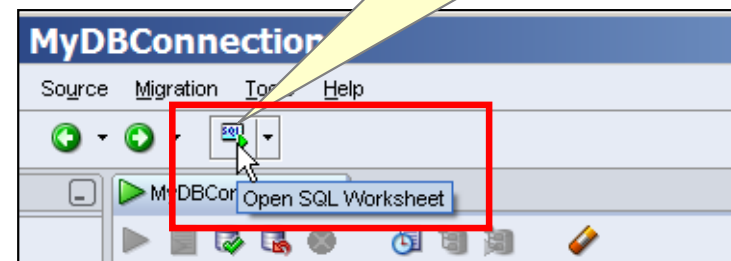
- You can create any schema object in SQL Developer using one of the following methods:
 - Executing a SQL statement in the SQL Worksheet
 - Using the context menu
- Edit the objects using an edit dialog box or one of the many context-sensitive menus.
- View the DDL for adjustments such as creating a new object or editing an existing schema object.

Using the SQL Worksheet

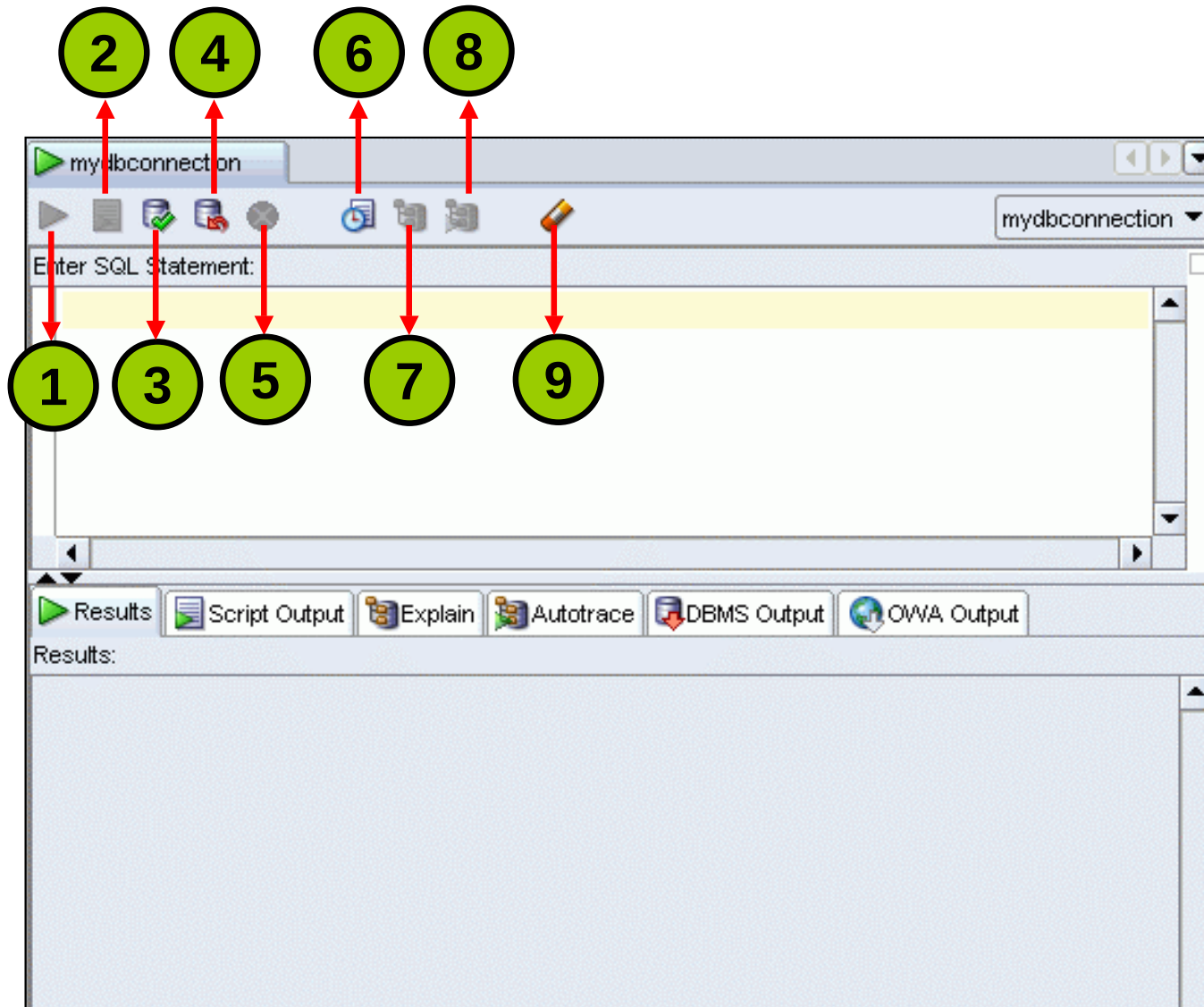
- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Click the **Open SQL Worksheet** icon.



Using the SQL Worksheet



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

The screenshot shows the Oracle SQL Developer interface. At the top, a tab labeled 'MyDBConnection' is active. Below it, a toolbar contains various icons, and a status bar shows '2.03304029 seconds'. The main area is divided into two panes. The top pane, titled 'Enter SQL Statement:', contains two SQL queries. The first query is:
1 SELECT last_name, salary
2 FROM employees
3 WHERE salary > 10000;
4
The second query is:
5 SELECT last_name "Name", salary*12 "Annual Salary"
6 FROM employees;
The bottom pane has a tabbed interface with 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. The 'Script Output' tab is selected and highlighted with a red box. It displays the results of the first query as a table with two columns: 'last_name' and 'salary'. The results are:
Ozer 11500
Abel 11000
Below this, it says '15 rows selected'. The second query's results are also visible, showing a table with columns 'Name' and 'Annual Salary'. The results are:
OConnell 31200
Grant 31200
Whalen 52800

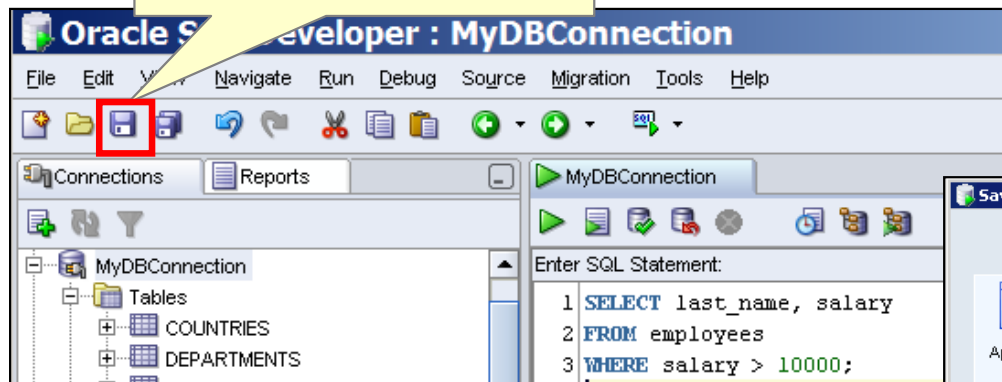
Use the **Enter SQL Statement** box to enter single or multiple SQL statements.

View the results on the **Script Output** tabbed page.

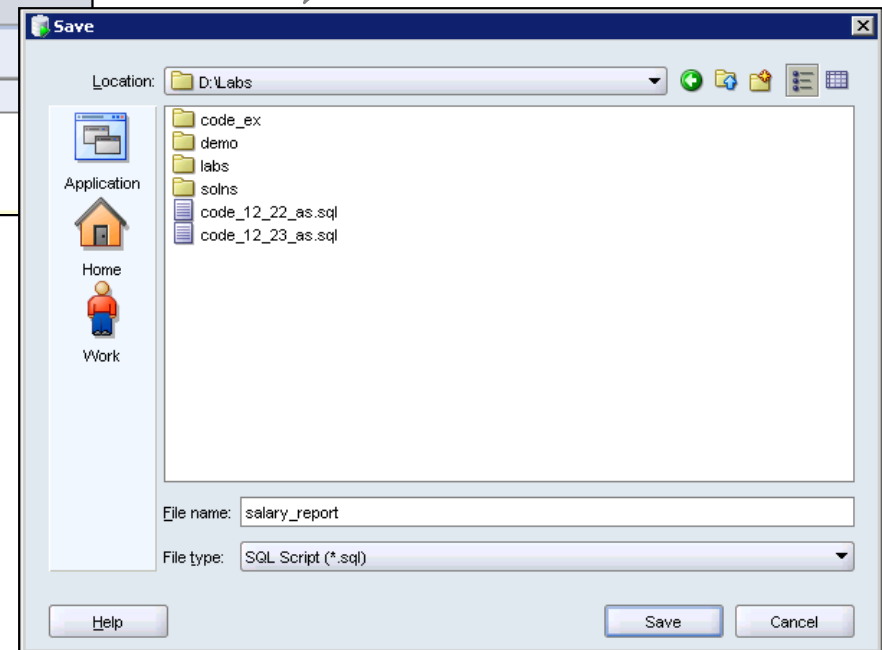
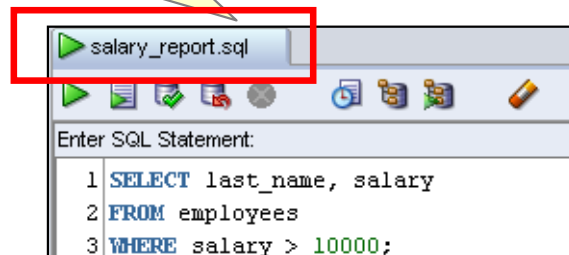
Saving SQL Scripts

Click the **Save** icon to save your SQL statement to a file.

Enter a file name and identify a location to save the file, and then click **Save**.



The contents of the saved file are visible and editable in your SQL Worksheet window.



Executing Saved Script Files: Method 1

Right-click in the SQL Worksheet area, and then select **Open File** from the shortcut menu.

Select (or navigate to) the script file that you want to open.

Click **Open**.

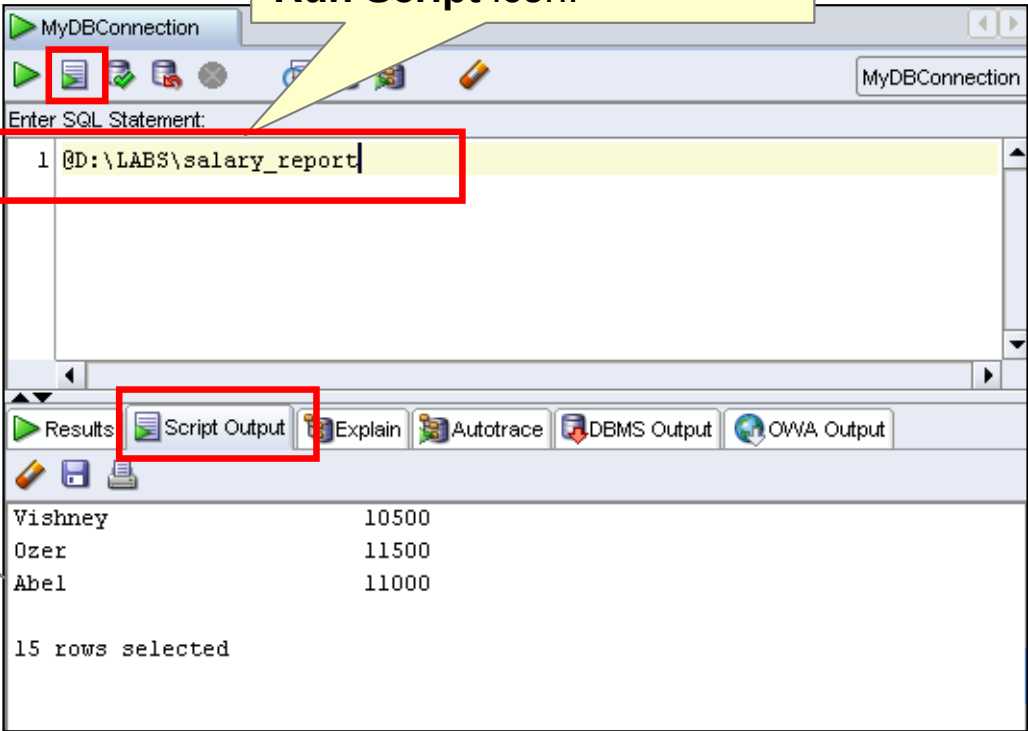
To run the code, click the **Run Script (F5)** icon.

```
CREATE PROCEDURE getemp IS -- header
emp_id employees.employee_id%type;
lname employees.last_name%type;
BEGIN
emp_id := 100;
SELECT last_name INTO lname
FROM EMPLOYEES
WHERE employee_id = emp_id;
DBMS_OUTPUT.PUT_LINE('Last name: '||lname);
END;
```

Executing Saved SQL Scripts: Method 2

Use the @ command followed by the location and name of the file you want to execute, and then click the **Run Script** icon.

The output from the script is displayed on the Script Output tabbed page.



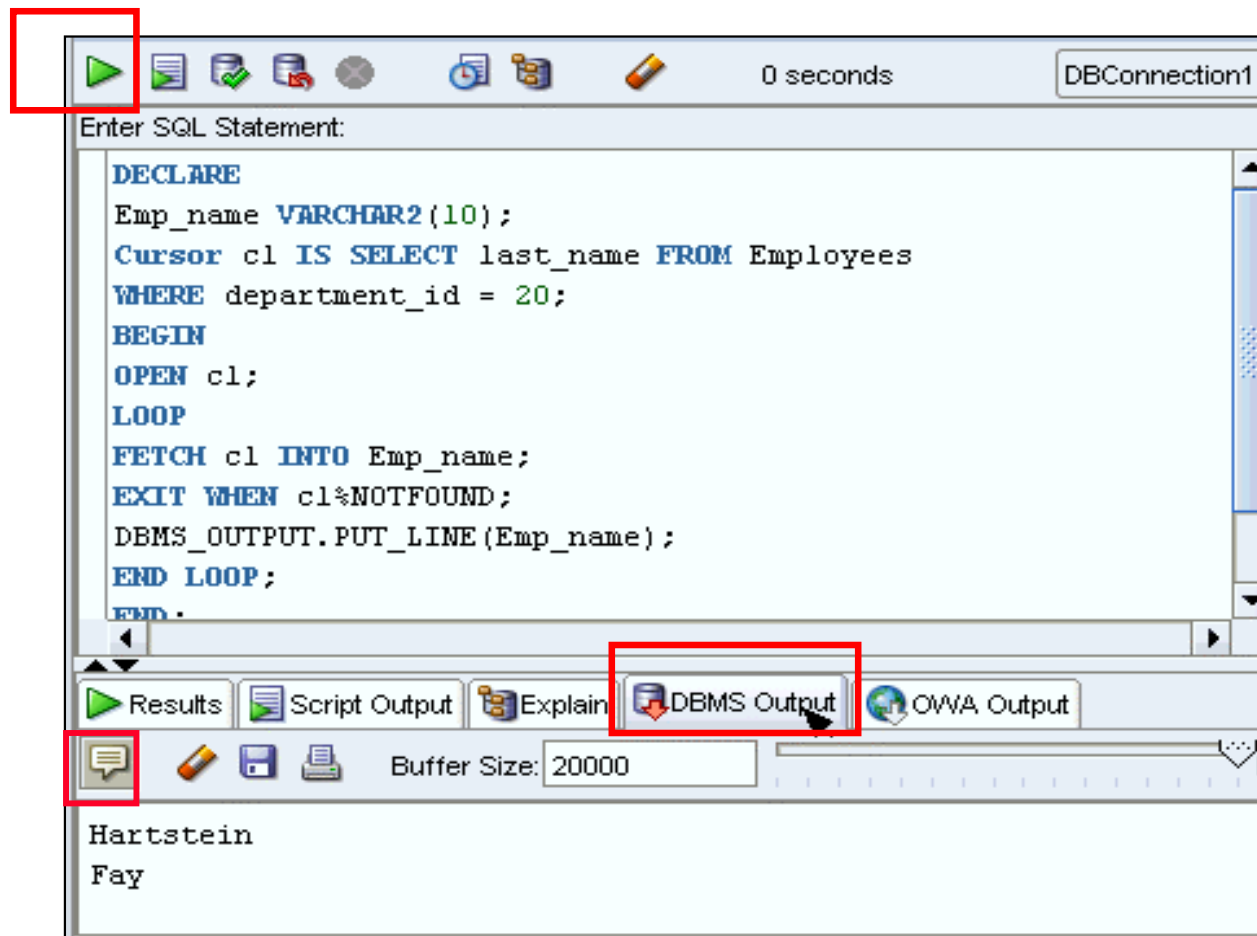
The screenshot shows the Oracle SQL Developer interface. At the top, a toolbar contains several icons; the 'Run Script' icon (a document with a play button) is highlighted with a red box. Below the toolbar, the 'Enter SQL Statement:' text box contains the text '1 @D:\LABS\salary_report', which is also highlighted with a red box. At the bottom, a tabbed interface shows several tabs: 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. The 'Script Output' tab is selected and highlighted with a red box. Below the tabs, the output of the script is displayed as a table with two columns: names and salaries. The names listed are Vishney, Ozer, and Abel, with salaries of 10500, 11500, and 11000 respectively. Below the table, it states '15 rows selected'.

Vishney	10500
Ozer	11500
Abel	11000

15 rows selected

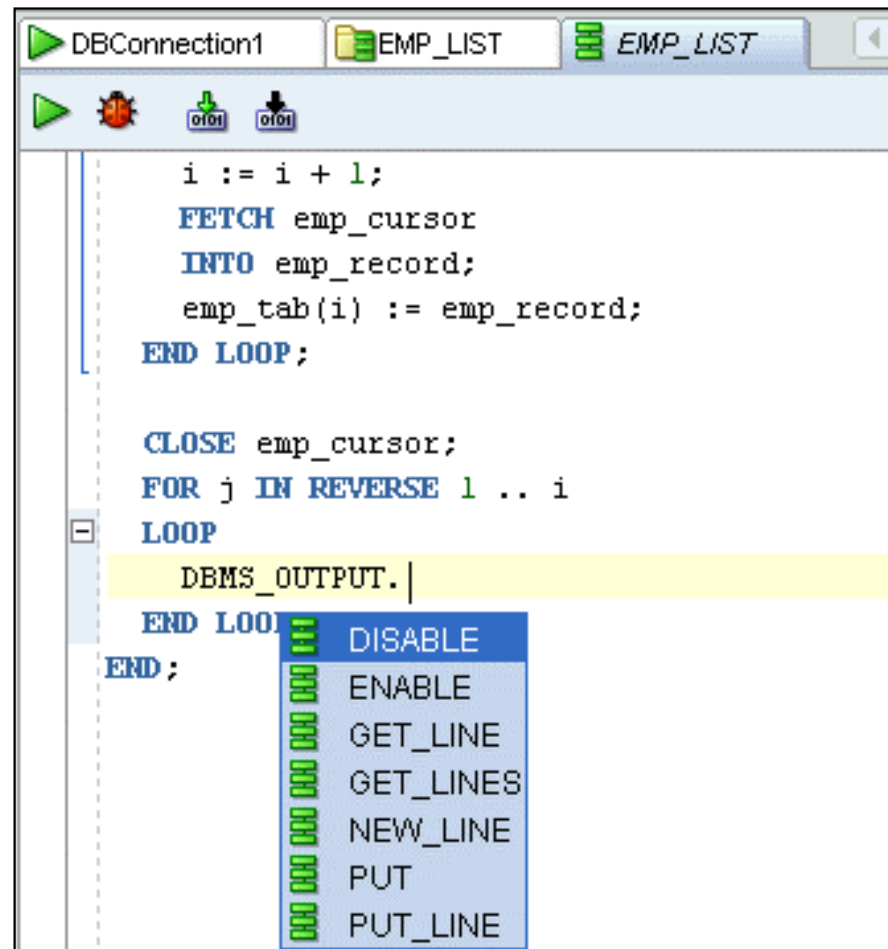
Creating an Anonymous Block

Create an anonymous block and display the output of the DBMS_OUTPUT package.



Editing the PL/SQL Code

Use the full-featured editor for PL/SQL program units.



Lesson Agenda

- Course objectives and course agenda
- The schema and appendices used in this course and the available PL/SQL development in this course
- Overview of Oracle SQL Developer
- Oracle 11g documentation and additional resources

Oracle 11g SQL and PL/SQL Documentation

- *Oracle Database New Features Guide 11g Release 1 (11.1)*
- *Oracle Database Advanced Application Developer's Guide 11g Release 1 (11.1)*
- *Oracle Database PL/SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Concepts 11g Release 1 (11.1)*
- *Oracle Database PL/SQL Packages and Types Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Developer User's Guide Release 1.1.2*

Additional Resources

For additional information about the new Oracle 11g SQL and PL/SQL new features, refer to the following:

- Oracle Database 11g: New Features eStudies
- Oracle by Example (OBE) series: Oracle Database 11g:
 - http://www.oracle.com/technology/obe/11gr1_db/admin/11gr1db.html
- What's New in PL/SQL in Oracle Database 11g on the Oracle Technology Network (OTN):
 - http://www.oracle.com/technology/tech/pl_sql/

Summary

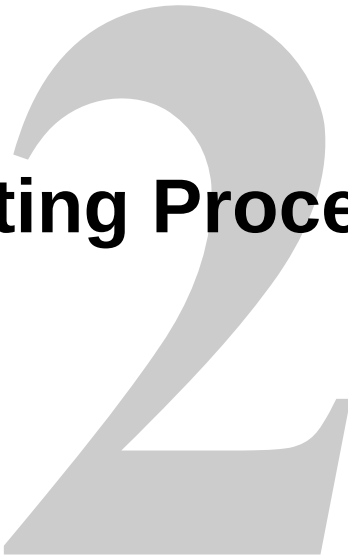
In this lesson, you should have learned how to:

- Discuss the goals of the course
- Identify the available environments that can be used in this course
- Describe the database schema and tables that are used in the course
- List the available documentation and resources

Practice 1 Overview: Getting Started

This practice covers the following topics:

- Reviewing the available SQL Developer resources
- Starting SQL Developer and creating a new database connection and browsing the HR tables
- Setting some SQL Developer preferences
- Executing SQL statements and an anonymous PL/SQL block using SQL Worksheet
- Accessing and bookmarking the Oracle Database 11g documentation and other useful Web sites



Creating Procedures

Objectives

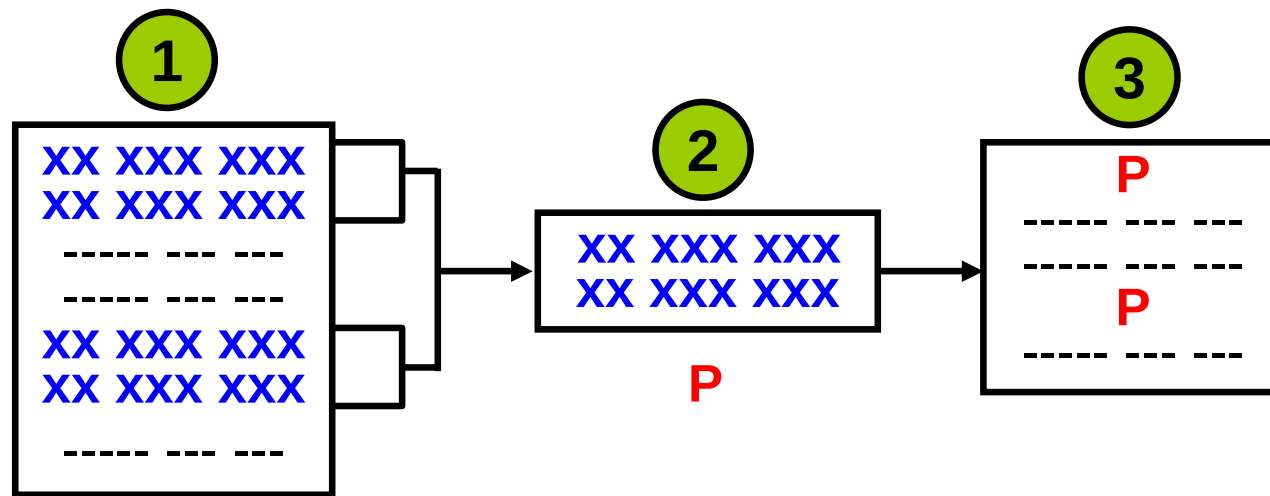
After completing this lesson, you should be able to do the following:

- Identify the benefits of modularized and layered subprogram design
- Create and call procedures
- Use formal and actual parameters
- Use positional, named, or mixed notation for passing parameters
- Identify the available parameter-passing modes
- Handle exceptions in procedures
- Remove a procedure
- Display the procedures' information

Lesson Agenda

- Using a modularized and layered subprogram design and identifying the benefits of subprograms
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, and displaying the procedures' information

Creating a Modularized Subprogram Design



Modularize code into subprograms.

1. Locate code sequences repeated more than once.
2. Create subprogram P containing the repeated code
3. Modify original code to invoke the new subprogram.

Creating a Layered Subprogram Design

Create subprogram layers for your application.

- Data access subprogram layer with SQL logic
- Business logic subprogram layer, which may or may not use the data access layer

Modularizing Development with PL/SQL Blocks

- PL/SQL is a block-structured language. The PL/SQL code block helps modularize code by using:
 - Anonymous blocks
 - Procedures and functions
 - Packages
 - Database triggers
- The benefits of using modular program constructs are:
 - Easy maintenance
 - Improved data security and integrity
 - Improved performance
 - Improved code clarity

Anonymous Blocks: Overview

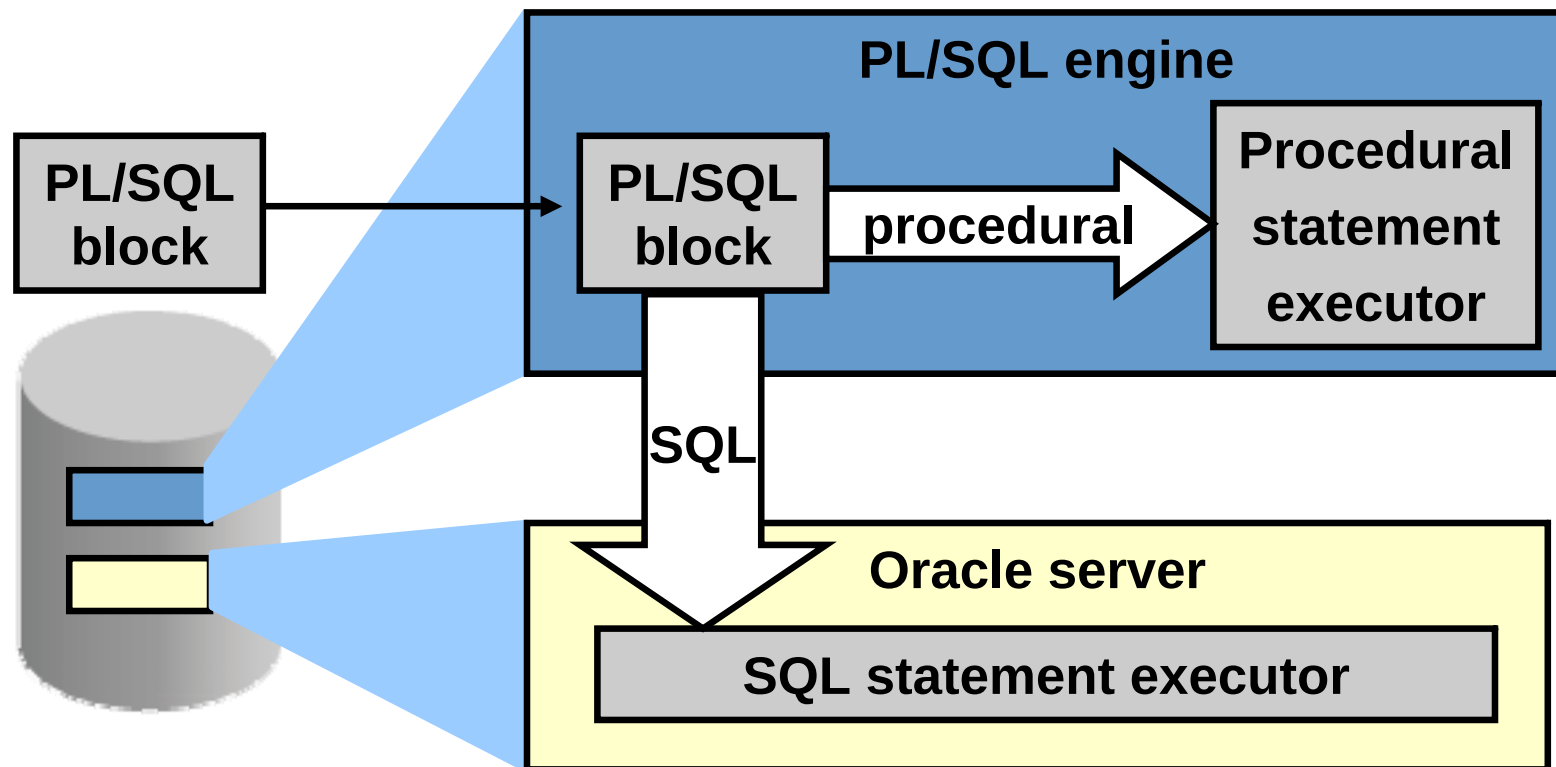
Anonymous blocks:

- Form the basic PL/SQL block structure
- Initiate PL/SQL processing tasks from applications
- Can be nested within the executable section of any PL/SQL block

```
[DECLARE      -- Declaration Section (Optional)
  variable declarations; ... ]
BEGIN          -- Executable Section (Mandatory)
  SQL or PL/SQL statements;
[EXCEPTION    -- Exception Section (Optional)
  WHEN exception THEN statements; ]
END;          -- End of Block (Mandatory)
```

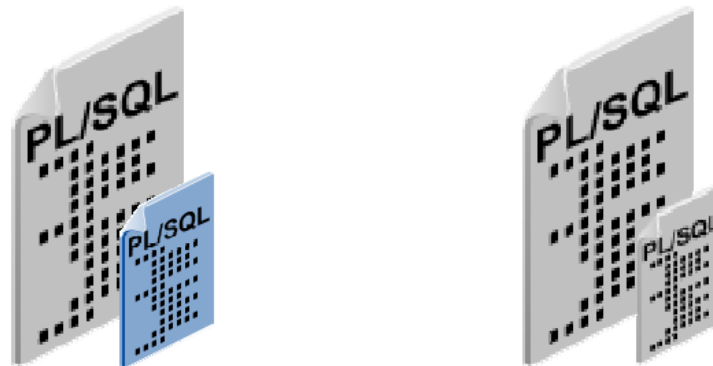
PL/SQL Execution Environment

The PL/SQL run-time architecture:

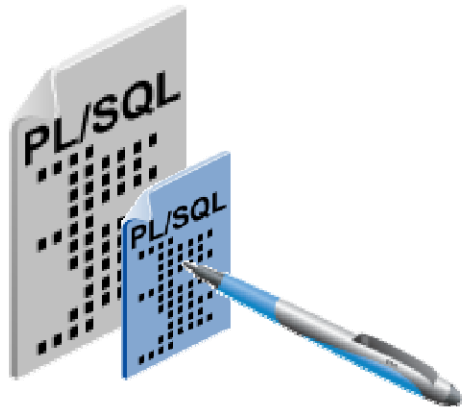


What Are PL/SQL Subprograms?

- A PL/SQL subprogram is a named PL/SQL block that can be called with a set of parameters.
- You can declare and define a subprogram within either a PL/SQL block or another subprogram.
- A subprogram consists of a specification and a body.
- A subprogram can be a procedure or a function.
- Typically, you use a procedure to perform an action and a function to compute and return a value.



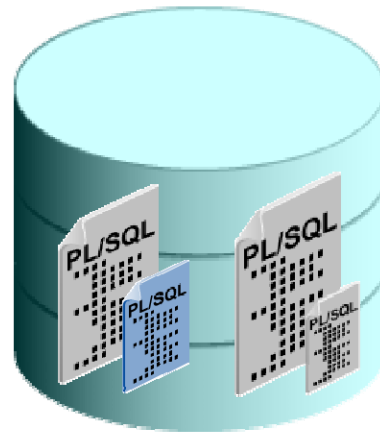
The Benefits of Using PL/SQL Subprograms



Easy maintenance



Improved code clarity



**Subprograms:
Stored procedures
and functions**



**Improved data
security and integrity**



Improved performance

Differences Between Anonymous Blocks and Subprograms

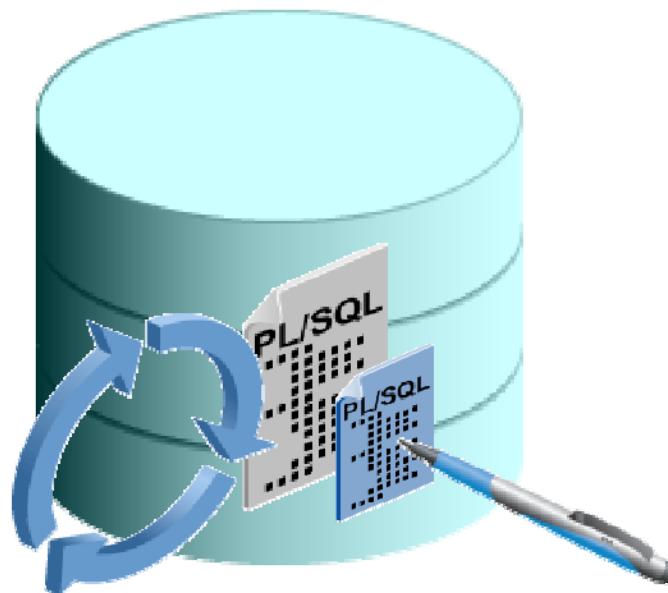
Anonymous Blocks	Subprograms
Unnamed PL/SQL blocks	Named PL/SQL blocks
Compiled every time	Compiled only once
Not stored in the database	Stored in the database
Cannot be invoked by other applications	Named and, therefore, can be invoked by other applications
Do not return values	Subprograms called functions must return values.
Cannot take parameters	Can take parameters

Lesson Agenda

- Using a modularized and layered subprogram design and identifying the benefits of subprograms
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, and displaying the procedures' information

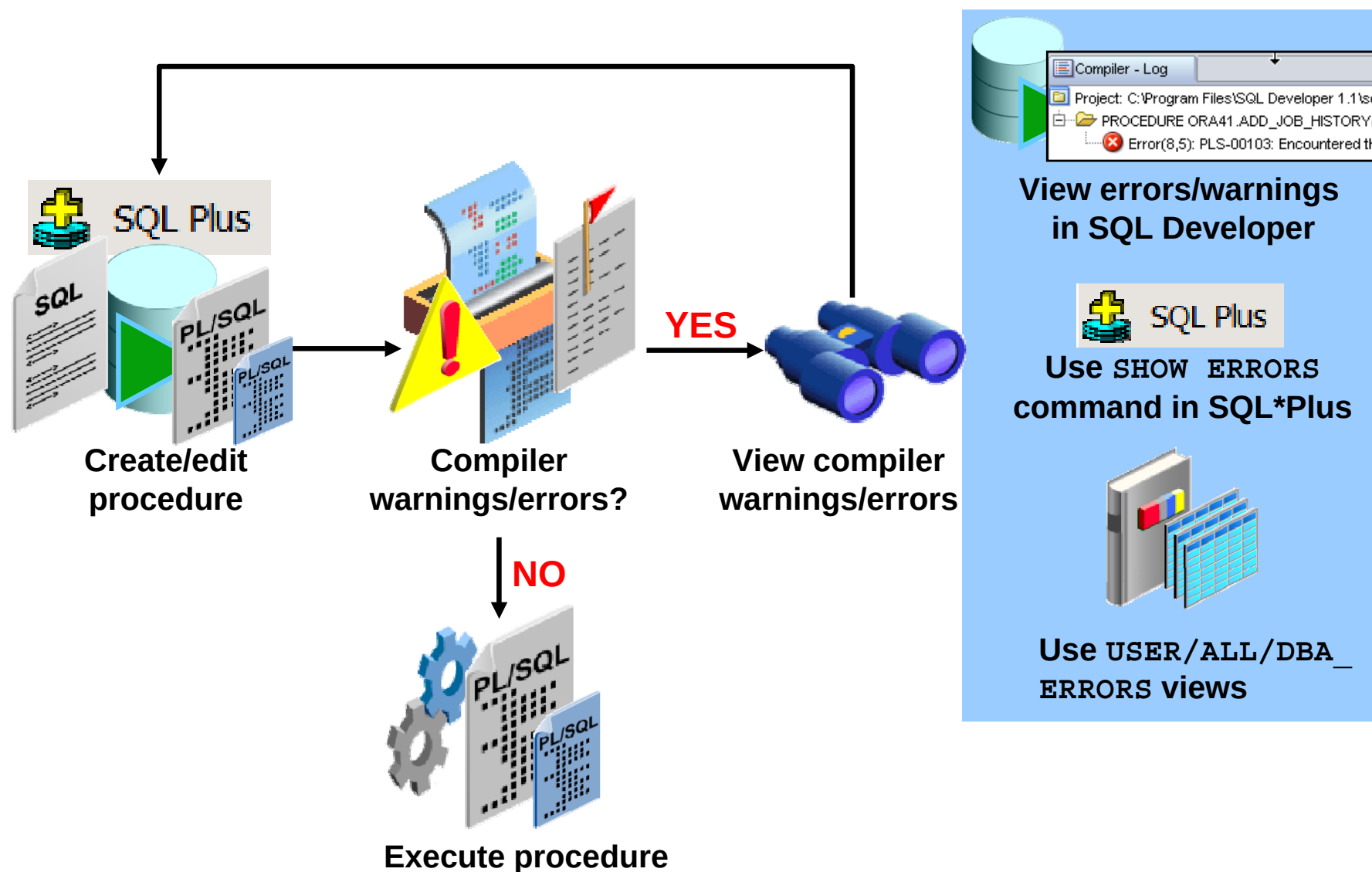
What Are Procedures?

- Are a type of subprogram that perform an action
- Can be stored in the database as a schema object
- Promote reusability and maintainability



Procedures

Creating Procedures: Overview



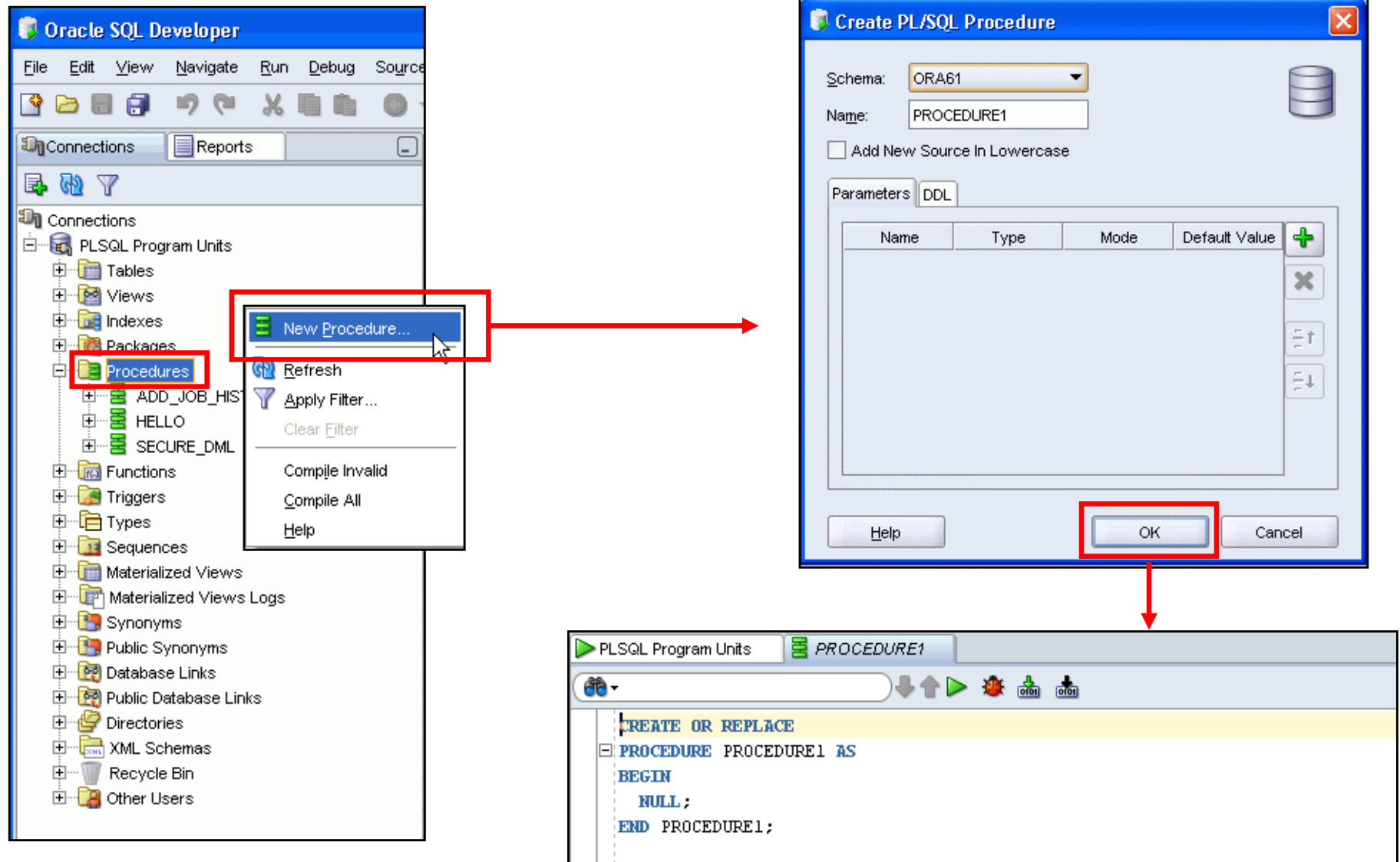
Creating Procedures with the SQL CREATE OR REPLACE Statement

- Use the CREATE clause to create a stand-alone procedure that is stored in the Oracle database.
- Use the OR REPLACE option to overwrite an existing procedure.

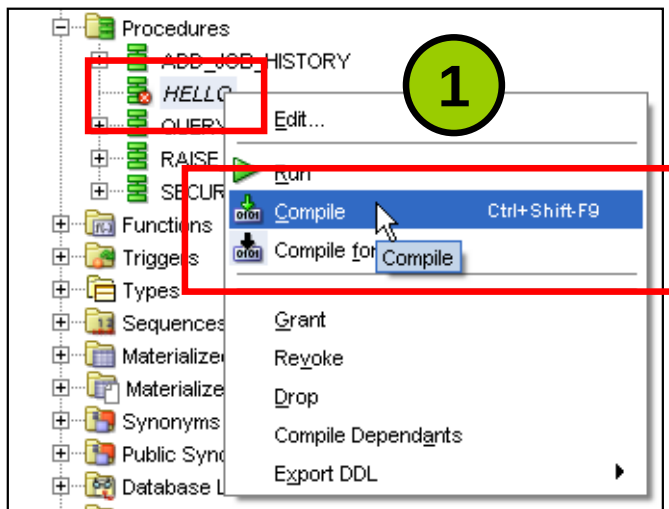
```
CREATE [OR REPLACE] PROCEDURE procedure_name
  [(parameter1 [mode] datatype1,
    parameter2 [mode] datatype2, ...)]
IS | AS
  [local_variable_declarations; ...]
BEGIN
  -- actions;
END [procedure_name];
```

PL/SQL block

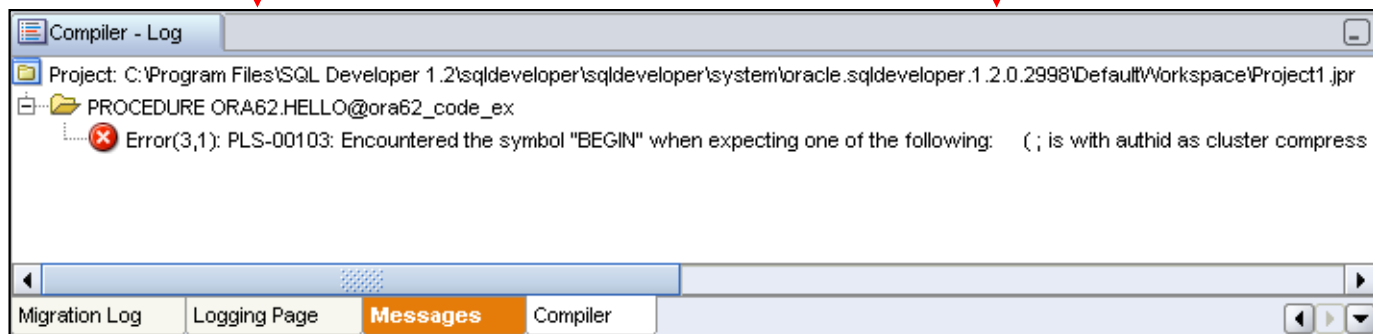
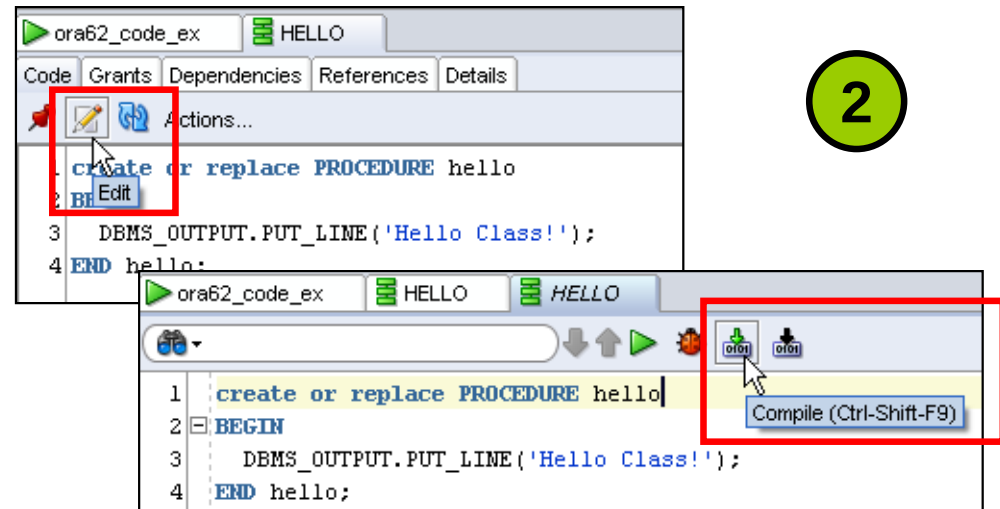
Creating Procedures Using SQL Developer



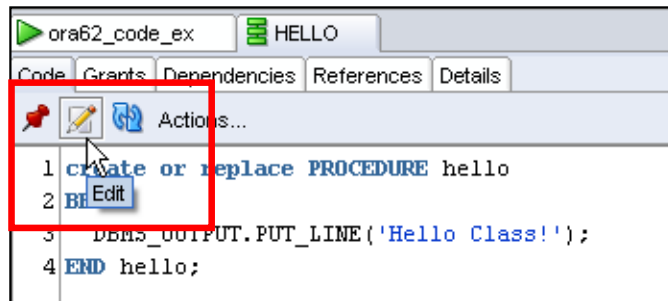
Compiling Procedures and Displaying Compilation Errors in SQL Developer



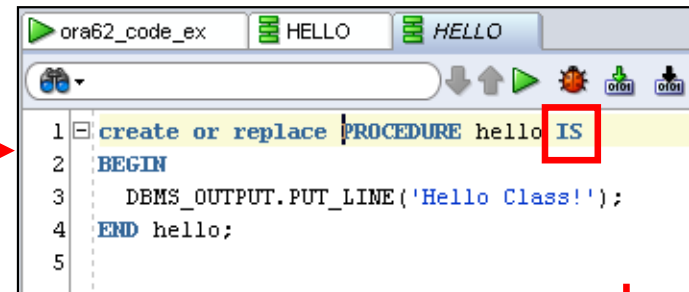
OR



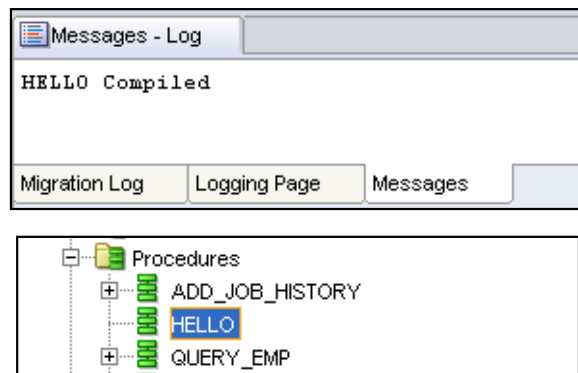
Correcting Compilation Errors in SQL Developer



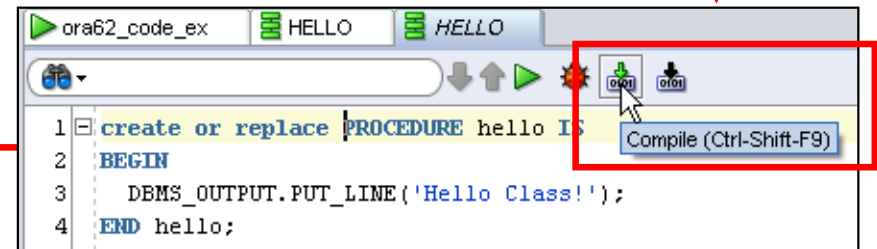
Edit procedure



Correct error



Recompilation successful



Recompile procedure

Naming Conventions of PL/SQL Structures Used in This Course

PL/SQL Structure	Convention	Example
Variable	<i>v_variable_name</i>	v_rate
Constant	<i>c_constant_name</i>	c_rate
Subprogram parameter	<i>p_parameter_name</i>	p_id
Bind (host) variable	<i>b_bind_name</i>	b_salary
Cursor	<i>cur_cursor_name</i>	cur_emp
Record	<i>rec_record_name</i>	rec_emp
Type	<i>type_name_type</i>	ename_table_type
Exception	<i>e_exception_name</i>	e_products_invalid
File handle	<i>f_file_handle_name</i>	f_file

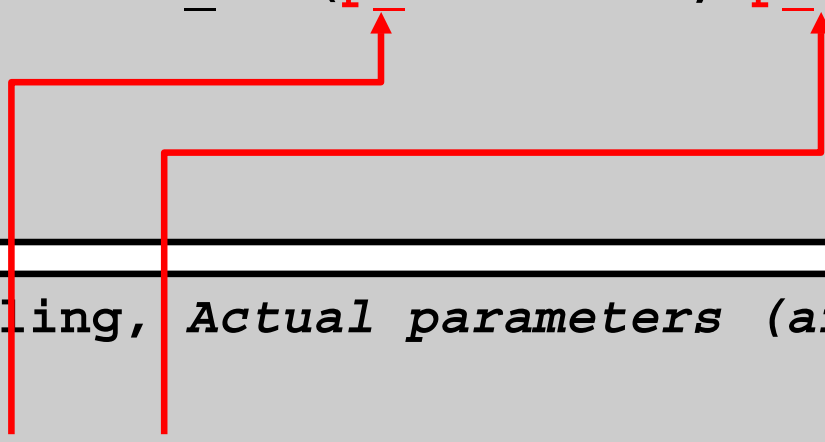
What Are Parameters and Parameter Modes?

- Are declared after the subprogram name in the PL/SQL header
- Pass or communicate data between the caller and the subprogram
- Are used like local variables but are dependent on their parameter-passing mode:
 - An `IN` parameter mode (the default) provides values for a subprogram to process
 - An `OUT` parameter mode returns a value to the caller
 - An `IN OUT` parameter mode supplies an input value, which may be returned (output) as a modified value

Formal and Actual Parameters

- Formal parameters: Local variables declared in the parameter list of a subprogram specification
- Actual parameters (or arguments): Literal values, variables, and expressions used in the parameter list of the calling subprogram

```
-- Procedure definition, Formal parameters  
CREATE PROCEDURE raise_sal(p_id NUMBER, p_sal NUMBER) IS  
BEGIN  
    . . .  
END raise_sal;
```

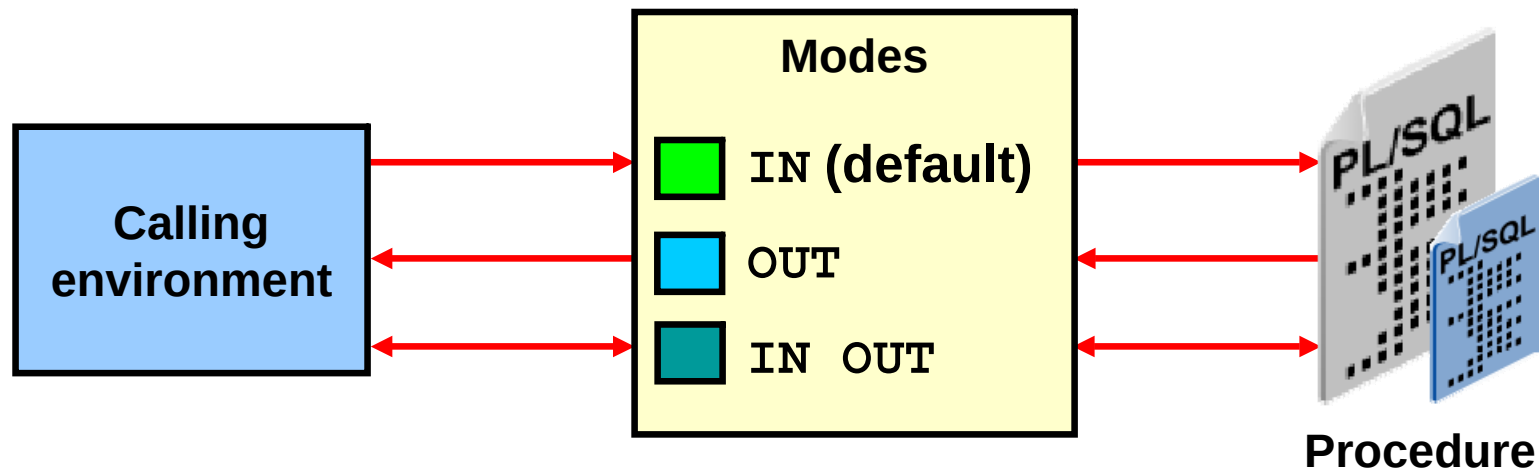


```
-- Procedure calling, Actual parameters (arguments)  
v_emp_id := 100;  
raise_sal(v_emp_id, 2000)
```

Procedural Parameter Modes

- Parameter modes are specified in the formal parameter declaration, after the parameter name and before its data type.
- The `IN` mode is the default if no mode is specified.

```
CREATE PROCEDURE proc_name(param_name [mode] datatype)  
...
```



Comparing the Parameter Modes

IN	OUT	IN OUT
Default mode	Must be specified	Must be specified
Value is passed into subprogram	Returned to calling environment	Passed into subprogram; returned to calling environment
Formal parameter acts as a constant	Uninitialized variable	Initialized variable
Actual parameter can be a literal, expression, constant, or initialized variable	Must be a variable	Must be a variable
Can be assigned a default value	Cannot be assigned a default value	Cannot be assigned a default value

Using the IN Parameter Mode: Example

```
CREATE OR REPLACE PROCEDURE raise_salary
(p_id      IN employees.employee_id%TYPE,
 p_percent IN NUMBER)
IS
BEGIN
    UPDATE employees
    SET     salary = salary * (1 + p_percent/100)
    WHERE  employee_id = p_id;
END raise_salary;
/
```



```
EXECUTE raise_salary(176, 10)
```

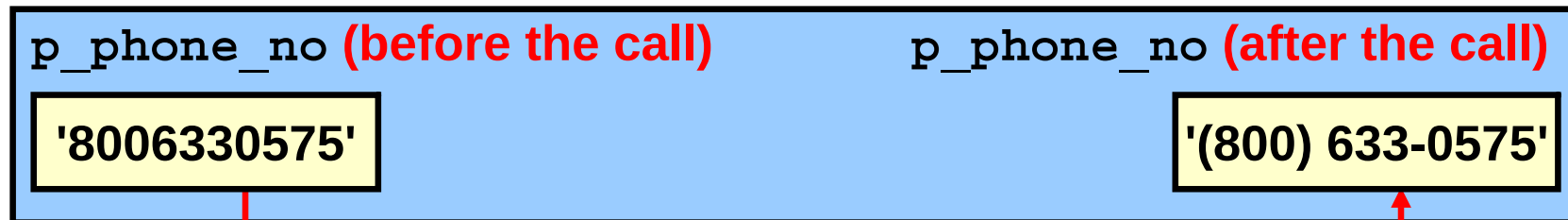
Using the OUT Parameter Mode: Example

```
CREATE OR REPLACE PROCEDURE query_emp
  (p_id      IN  employees.employee_id%TYPE,
   p_name    OUT employees.last_name%TYPE,
   p_salary  OUT employees.salary%TYPE) IS
BEGIN
  SELECT  last_name, salary INTO p_name, p_salary
  FROM    employees
  WHERE   employee_id = p_id;
END query_emp;
/
```

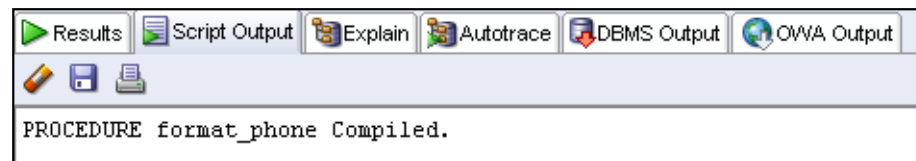
```
DECLARE
  v_emp_name employees.last_name%TYPE;
  v_emp_sal  employees.salary%TYPE;
BEGIN
  query_emp(171, v_emp_name, v_emp_sal);
  DBMS_OUTPUT.PUT_LINE(v_emp_name || ' earns ' ||
    to_char(v_emp_sal, '$999,999.00'));
END; /
```


Using the IN OUT Parameter Mode: Example

Calling environment



```
CREATE OR REPLACE PROCEDURE format_phone
  (p_phone_no IN OUT VARCHAR2) IS
BEGIN
  p_phone_no := '(' || SUBSTR(p_phone_no,1,3) ||
                ')' || SUBSTR(p_phone_no,4,3) ||
                '-' || SUBSTR(p_phone_no,7);
END format_phone;
/
```



Viewing the OUT Parameters:

Using the DBMS_OUTPUT.PUT_LINE Subroutine

Use PL/SQL variables that are printed with calls to the DBMS_OUTPUT.PUT_LINE procedure.

```
SET SERVEROUTPUT ON

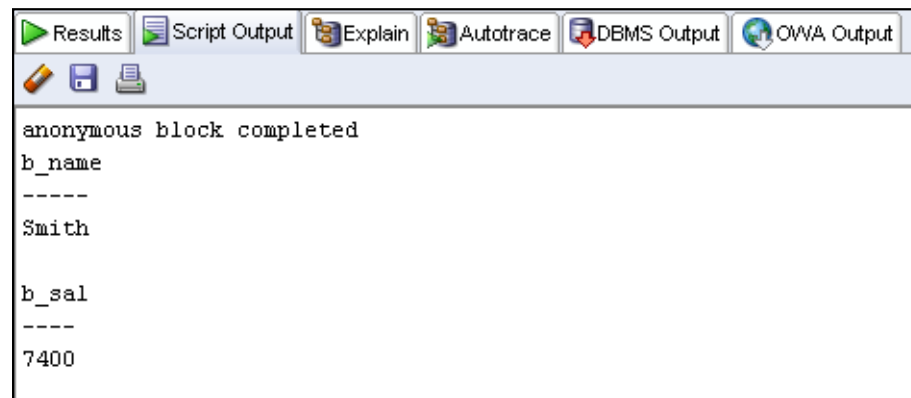
DECLARE
  v_emp_name employees.last_name%TYPE;
  v_emp_sal   employees.salary%TYPE;
BEGIN
  query_emp(171, v_emp_name, v_emp_sal);
  DBMS_OUTPUT.PUT_LINE('Name: ' || v_emp_name);
  DBMS_OUTPUT.PUT_LINE('Salary: ' || v_emp_sal);
END;
```

```
anonymous block completed
Name: Smith
Salary: 7400
```

Viewing OUT Parameters: Using SQL*Plus Host Variables

1. Use SQL*Plus host variables.
2. Execute QUERY_EMP using host variables.
3. Print the host variables.

```
VARIABLE b_name    VARCHAR2(25)
VARIABLE b_sal      NUMBER
EXECUTE query_emp(171, :b_name, :b_sal)
PRINT b_name b_sal
```



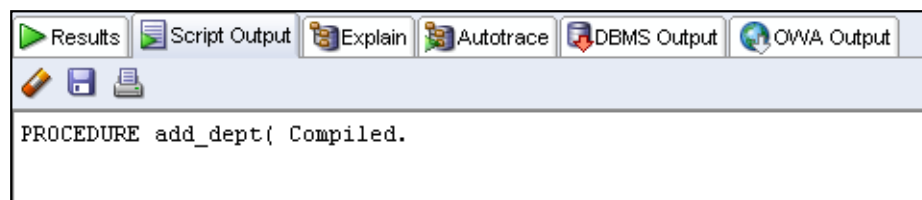
Available Notations for Passing Actual Parameters

When calling a subprogram, you can write the actual parameters using the following notations:

- Positional:
 - Lists the actual parameters in the same order as the formal parameters
- Named:
 - Lists the actual parameters in arbitrary order and uses the association operator (`=>`) to associate a named formal parameter with its actual parameter
- Mixed:
 - Lists some of the actual parameters as positional and some as named

Passing Actual Parameters: Creating the add_dept Procedure

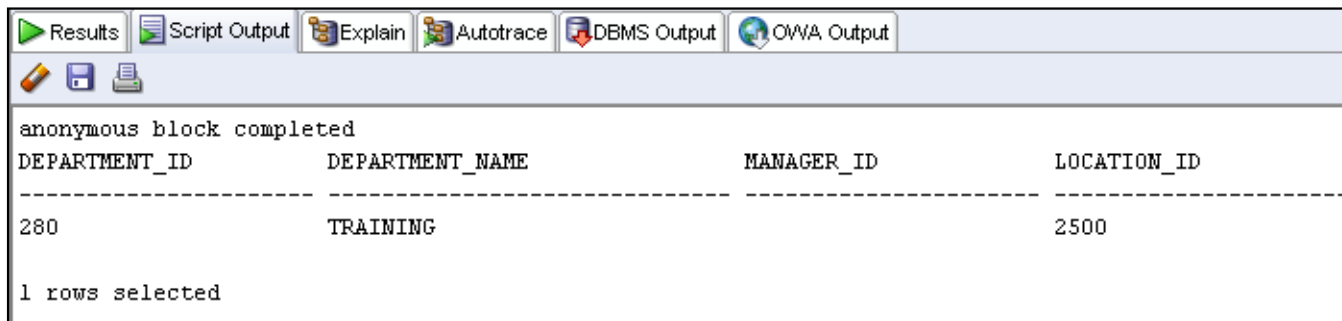
```
CREATE OR REPLACE PROCEDURE add_dept(  
  p_name IN departments.department_name%TYPE,  
  p_loc  IN departments.location_id%TYPE) IS  
BEGIN  
  INSERT INTO departments(department_id,  
                           department_name, location_id)  
  VALUES (departments_seq.NEXTVAL, p_name , p_loc );  
END add_dept;  
/
```



Passing Actual Parameters: Examples

-- Passing parameters using the *positional* notation.

```
EXECUTE add_dept ('TRAINING', 2500)
```



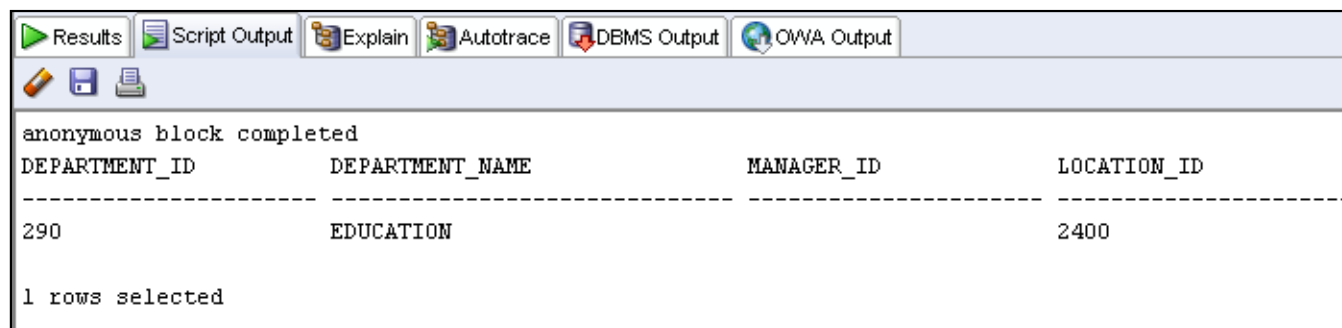
The screenshot shows the SQL Developer interface with the 'Results' tab selected. It displays the output of an anonymous block that executed the add_dept procedure with positional parameters. The output is a table with four columns: DEPARTMENT_ID, DEPARTMENT_NAME, MANAGER_ID, and LOCATION_ID. A single row is shown with values 280, TRAINING, and 2500. The MANAGER_ID column is empty. Below the table, it states '1 rows selected'.

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
280	TRAINING		2500

1 rows selected

-- Passing parameters using the *named* notation.

```
EXECUTE add_dept (p_loc=>2400, p_name=>'EDUCATION')
```



The screenshot shows the SQL Developer interface with the 'Results' tab selected. It displays the output of an anonymous block that executed the add_dept procedure with named parameters. The output is a table with four columns: DEPARTMENT_ID, DEPARTMENT_NAME, MANAGER_ID, and LOCATION_ID. A single row is shown with values 290, EDUCATION, and 2400. The MANAGER_ID column is empty. Below the table, it states '1 rows selected'.

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
290	EDUCATION		2400

1 rows selected

Using the DEFAULT Option for the Parameters

- Defines default values for parameters.
- Provides flexibility by combining the positional and named parameter-passing syntax.

```
CREATE OR REPLACE PROCEDURE add_dept(  
  p_name departments.department_name%TYPE := 'Unknown',  
  p_loc  departments.location_id%TYPE DEFAULT 1700)  
IS  
BEGIN  
  INSERT INTO departments (department_id,  
    department_name, location_id)  
  VALUES (departments_seq.NEXTVAL, p_name, p_loc);  
END add_dept;
```

```
EXECUTE add_dept  
EXECUTE add_dept ('ADVERTISING', p_loc => 1200)  
EXECUTE add_dept (p_loc => 1200)
```

Calling Procedures

You can call procedures using anonymous blocks, another procedure, or packages.

```
CREATE OR REPLACE PROCEDURE process_employees
IS
    CURSOR cur_emp_cursor IS
        SELECT employee_id
        FROM   employees;
BEGIN
    FOR emp_rec IN cur_emp_cursor
    LOOP
        raise_salary(emp_rec.employee_id, 10);
    END LOOP;
    COMMIT;
END process_employees;
/
```

```
PROCEDURE process_employees Compiled.
```


Calling Procedures Using SQL Developer

1 Select the **RAISE_SALARY** procedure in the Connections tree.

2 Click the **Run** button in the context menu.

3 In the **Run PL/SQL** dialog, replace the placeholder values for **ID** and **PERCENT** with actual values.

4 Click the **OK** button to execute the procedure.

Run PL/SQL

Target: **RAISE_SALARY**

Parameters:

Parameter	Data Type	Mode
ID	NUMBER	IN
PERCENT	NUMBER	IN

PL/SQL Block

```
DECLARE
  ID NUMBER;
  PERCENT NUMBER;
BEGIN
  ID := NULL;
  PERCENT := NULL;

  RAISE SALARY(
    ID => ID,
    PERCENT => PERCENT
  );
END;
```

PL/SQL Block

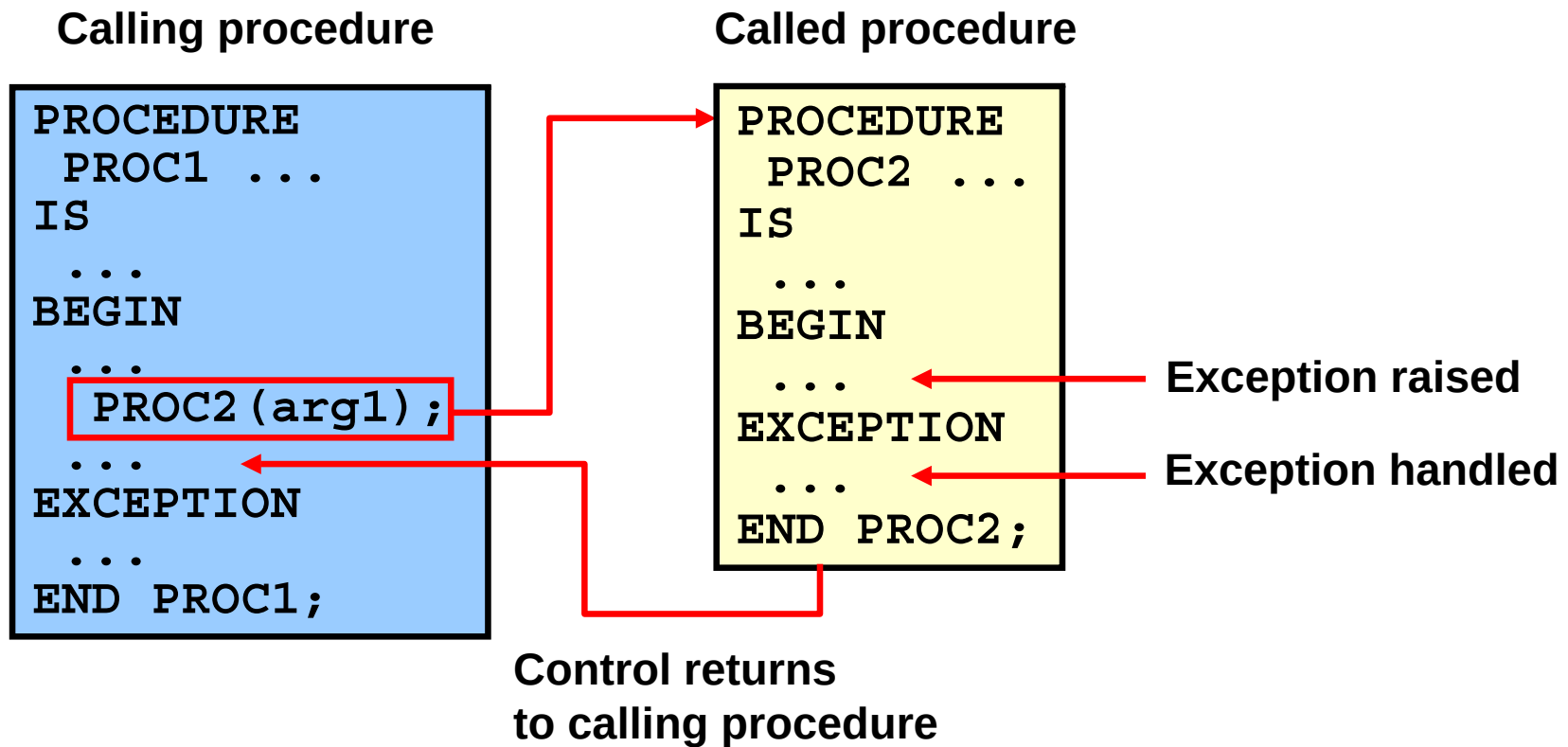
```
DECLARE
  ID NUMBER;
  PERCENT NUMBER;
BEGIN
  ID := NULL;
  PERCENT := NULL;

  RAISE SALARY(
    ID => 176,
    PERCENT => 10
  );
END;
```

Lesson Agenda

- Using a modularized and layered subprogram design and identifying the benefits of subprograms
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, and displaying the procedures' information

Handled Exceptions



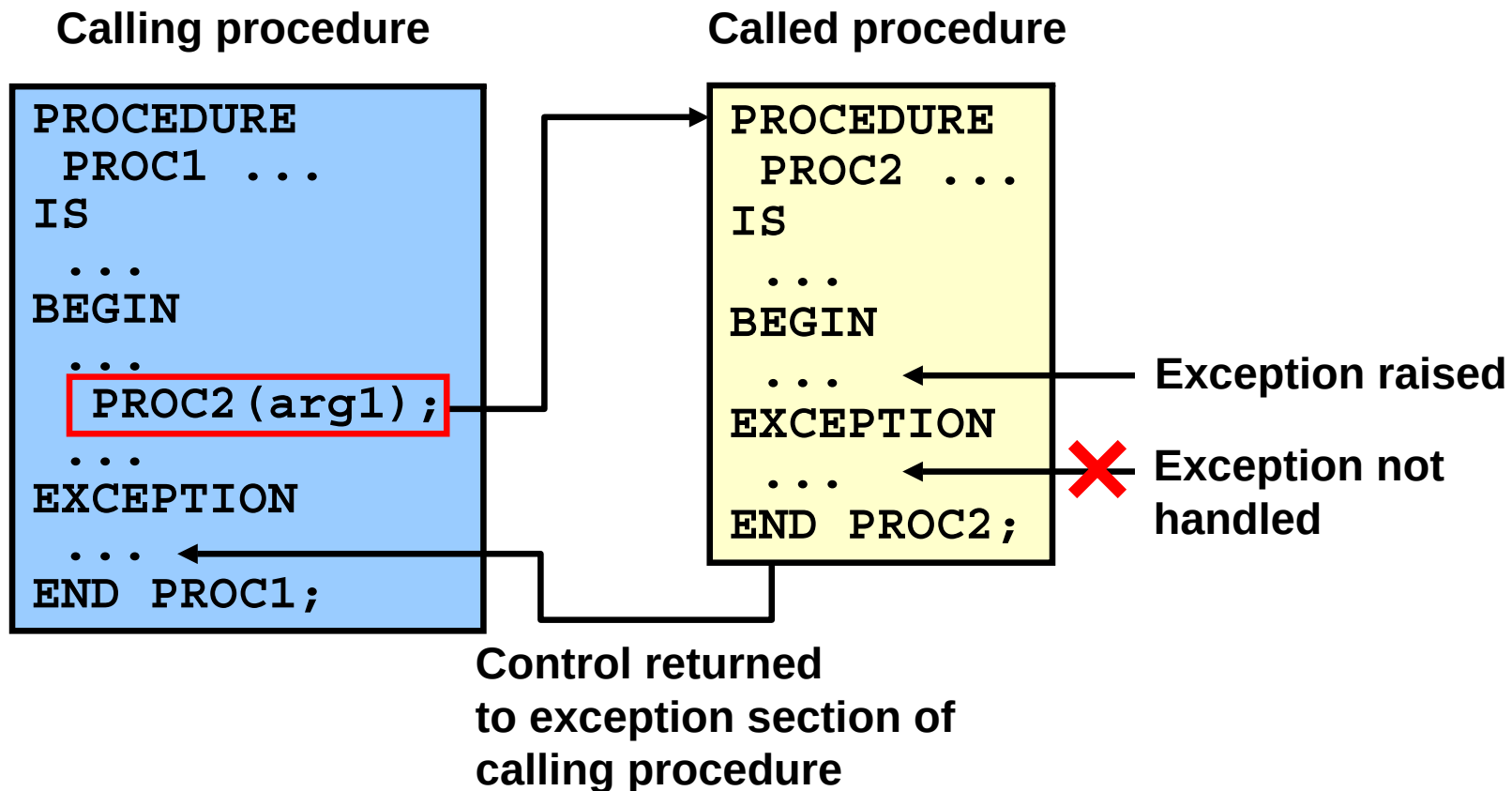
Handled Exceptions: Example

```
CREATE PROCEDURE add_department(  
    p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS  
BEGIN  
    INSERT INTO DEPARTMENTS (department_id,  
        department_name, manager_id, location_id)  
    VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);  
    DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Err: adding dept: ' || p_name);  
END;
```

```
CREATE PROCEDURE create_departments IS  
BEGIN  
    add_department('Media', 100, 1800);  
    add_department('Editing', 99, 1800);  
    add_department('Advertising', 101, 1800);  
END;
```



Exceptions Not Handled



Exceptions Not Handled: Example

```
SET SERVEROUTPUT ON
CREATE PROCEDURE add_department_noex(
    p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS
BEGIN
    INSERT INTO DEPARTMENTS (department_id,
        department_name, manager_id, location_id)
    VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);
    DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);
END;
```

```
CREATE PROCEDURE create_departments_noex IS
BEGIN
    add_department_noex('Media', 100, 1800);
    add_department_noex('Editing', 99, 1800);
    add_department_noex('Advertising', 101, 1800);
END;
```

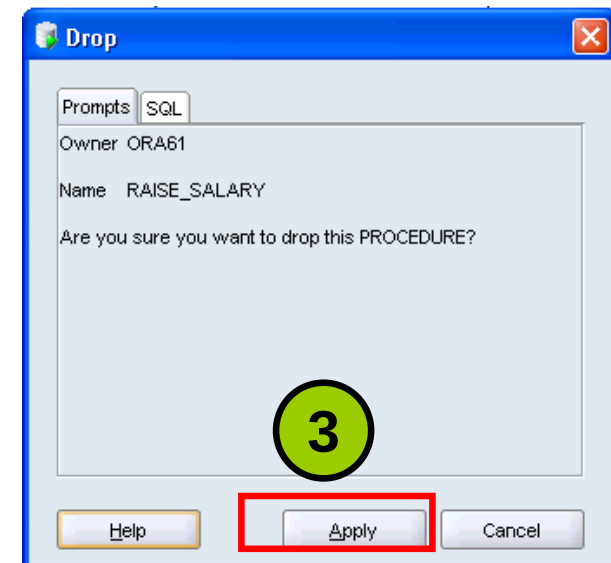
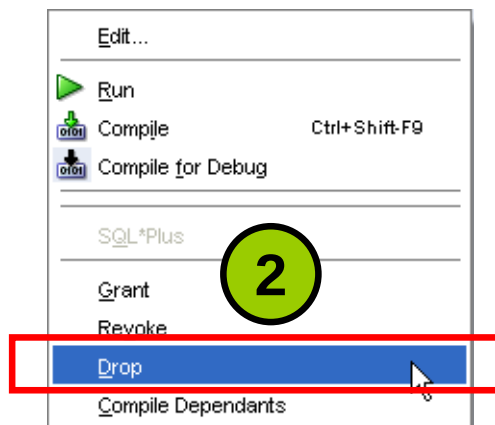
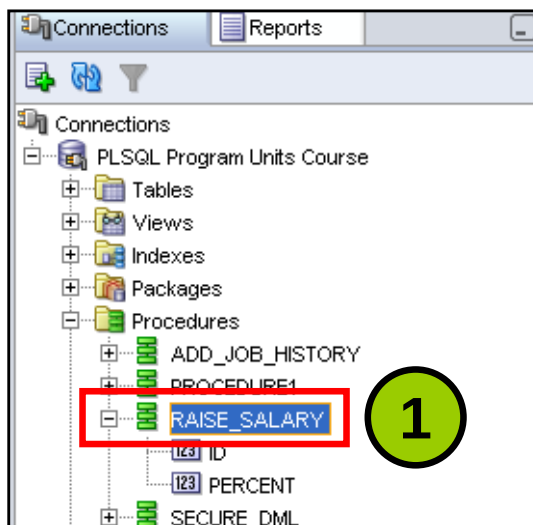


Removing Procedures: Using the DROP SQL Statement or SQL Developer

- Using the DROP statement:

```
DROP PROCEDURE raise_salary;
```

- Using SQL Developer:



Viewing Procedure Information Using the Data Dictionary Views

```
DESCRIBE user_source
```

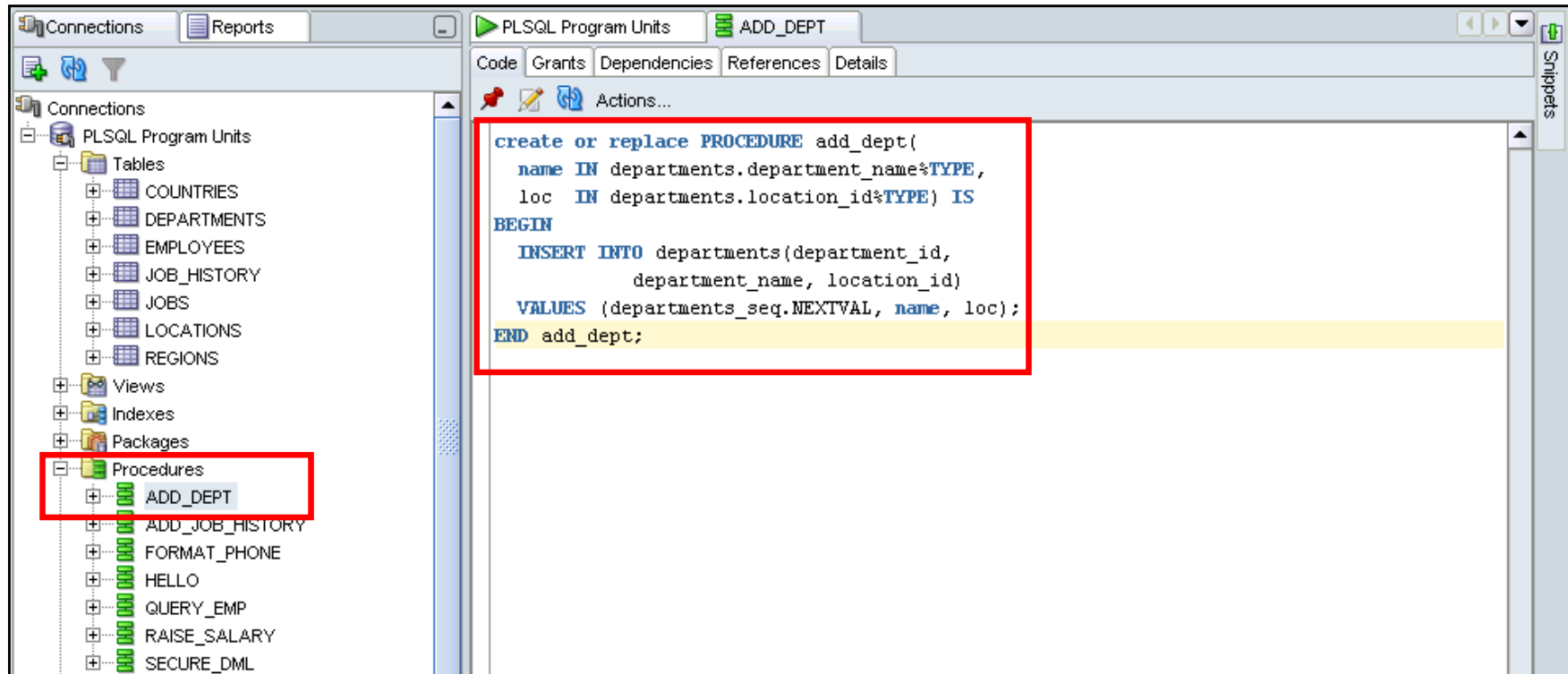
```
DESCRIBE user_source
Name                               Null    Type
-----
NAME                               VARCHAR2(30)
TYPE                               VARCHAR2(12)
LINE                               NUMBER
TEXT                               VARCHAR2(4000)

4 rows selected
```

```
SELECT text
FROM   user_source
WHERE  name = 'ADD_DEPT' AND type = 'PROCEDURE'
ORDER BY line;
```

	TEXT
1	PROCEDURE add_dept(
2	p_name IN departments.department_name%TYPE,
3	p_loc IN departments.location_id%TYPE) IS
4	
5	BEGIN
6	INSERT INTO departments(department_id, department_name, location_id)
7	VALUES (departments_seq.NEXTVAL, p_name, p_loc);
8	END add_dept;

Viewing Procedure Information Using SQL Developer



Quiz

Formal parameters are literal values, variables, and expressions used in the parameter list of the calling subprogram

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Identify the benefits of modularized and layered subprogram design
- Create and call procedures
- Use formal and actual parameters
- Use positional, named, or mixed notation for passing parameters
- Identify the available parameter-passing modes
- Handle exceptions in procedures
- Remove a procedure
- Display the procedures' information

Practice 2 Overview: Creating, Compiling, and Calling Procedures

This practice covers the following topics:

- Creating stored procedures to:
 - Insert new rows into a table using the supplied parameter values
 - Update data in a table for rows that match the supplied parameter values
 - Delete rows from a table that match the supplied parameter values
 - Query a table and retrieve data based on supplied parameter values
- Handling exceptions in procedures
- Compiling and invoking procedures



Creating Functions

Objectives

After completing this lesson, you should be able to do the following:

- Differentiate between a procedure and a function
- Describe the uses of functions
- Create stored functions
- Invoke a function
- Remove a function

Overview of Stored Functions

A function:

- Is a named PL/SQL block that returns a value
- Can be stored in the database as a schema object for repeated execution
- Is called as part of an expression or is used to provide a parameter value

Creating Functions

The PL/SQL block must have at least one RETURN statement.

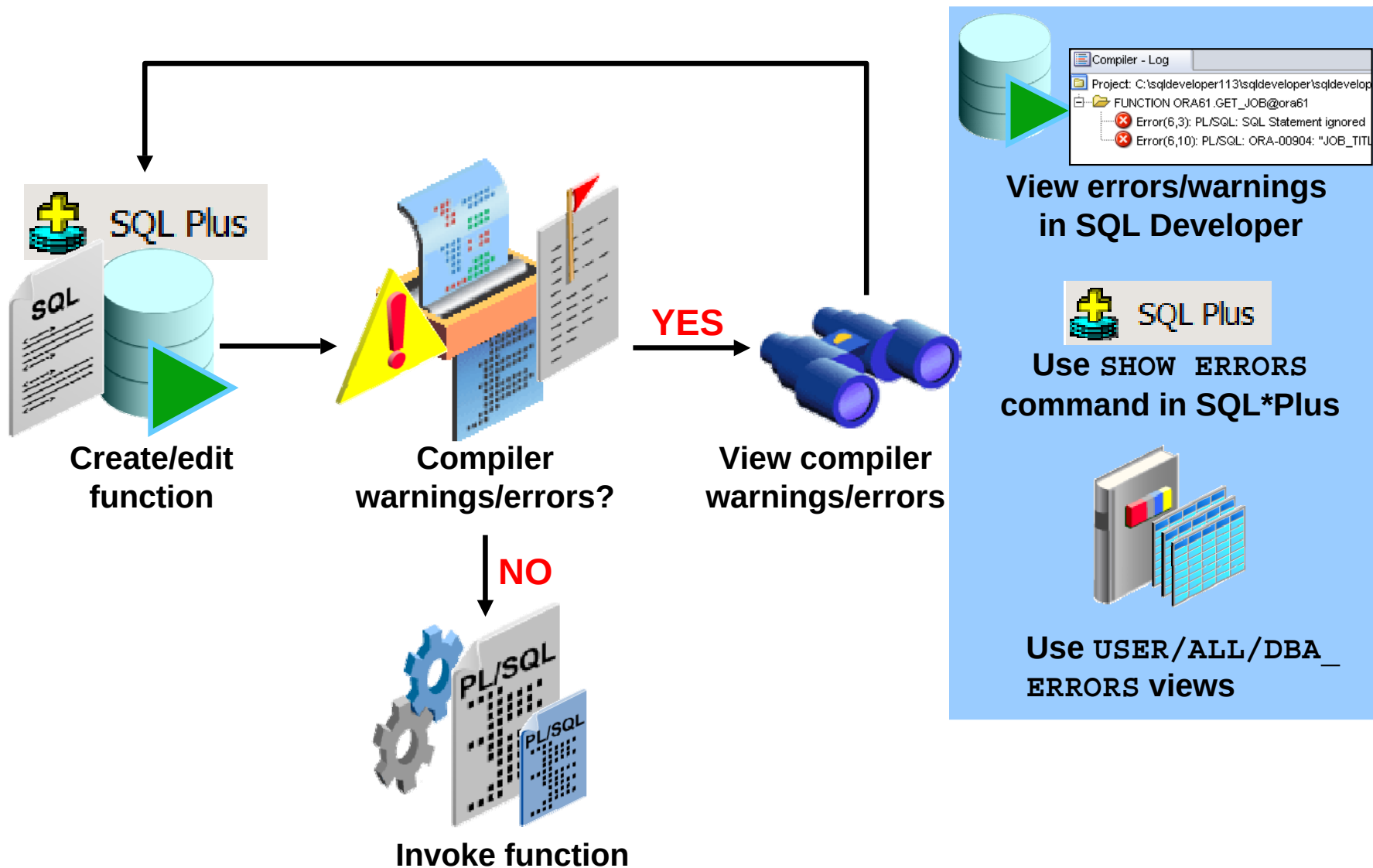
```
CREATE [OR REPLACE] FUNCTION function_name
  [(parameter1 [mode1] datatype1, . . .)]
RETURN datatype IS|AS
  [local_variable_declarations;
   . . .]
BEGIN
  -- actions;
  RETURN expression;
END [function_name];
```

PL/SQL Block

The Difference Between Procedures and Functions

Procedures	Functions
Execute as a PL/SQL statement	Invoke as part of an expression
Do not contain RETURN clause in the header	Must contain a RETURN clause in the header
Can pass values (if any) using output parameters	Must return a single value
Can contain a RETURN statement without a value	Must contain at least one RETURN statement

Creating and Running Functions: Overview



Creating and Invoking a Stored Function Using the CREATE FUNCTION Statement: Example

```
CREATE OR REPLACE FUNCTION get_sal
(p_id employees.employee_id%TYPE) RETURN NUMBER IS
  v_sal employees.salary%TYPE := 0;
BEGIN
  SELECT salary
  INTO    v_sal
  FROM    employees
  WHERE   employee_id = p_id;
  RETURN v_sal;
END get_sal; /
```

```
FUNCTION get_sal Compiled.
```

```
-- Invoke the function as an expression or as
-- a parameter value.
```

```
EXECUTE dbms_output.put_line(get_sal(100))
```

```
anonymous block completed
24000
```

Using Different Methods for Executing Functions

```
-- As a PL/SQL expression, get the results using host variables  
  
VARIABLE b_salary NUMBER  
EXECUTE :b_salary := get_sal(100)
```

```
anonymous block completed  
b_salary  
-----  
24000
```

```
-- As a PL/SQL expression, get the results using a local  
-- variable
```

```
DECLARE  
    sal employees.salary%type;  
BEGIN  
    sal := get_sal(100);  
    DBMS_OUTPUT.PUT_LINE('The salary is: ' || sal);  
END;/
```

```
anonymous block completed  
The salary is: 24000
```

Using Different Methods for Executing Functions

```
-- Use as a parameter to another subprogram  
  
EXECUTE dbms_output.put_line(get_sal(100))
```

```
anonymous block completed  
24000
```

```
-- Use in a SQL statement (subject to restrictions)  
  
SELECT job_id, get_sal(employee_id) FROM employees;
```

JOB_ID	GET_SAL(EMPLOYEE_ID)
SH_CLERK	2600
SH_CLERK	2600
AD_ASST	4400
MK_MAN	13000

...

SH_CLERK	3100
SH_CLERK	3000

107 rows selected

Creating and Compiling Functions Using SQL Developer

The diagram illustrates the process of creating and compiling a PL/SQL function in SQL Developer, consisting of four numbered steps:

- Step 1:** In the **Connections** pane, the **Functions** folder under the **student 41** connection is selected.
- Step 2:** The **New Function...** option is chosen from the context menu.
- Step 3:** The **Create PL/SQL Function** dialog is shown. The **Schema** is **ORA41** and the **Name** is **get_sal**. The **Parameters** tab is active, showing a return value of **VARCHAR2**. The **OK** button is highlighted.
- Step 4:** The **SQL Editor** shows the SQL script for the function. The **Run** button (a green play icon) is highlighted.

The SQL script in the editor is as follows:

```
CREATE OR REPLACE FUNCTION get_sal(id employees.employee_id%TYPE) RETURN NUMBER
IS
  sal employees.salary%TYPE := 0;
BEGIN
  SELECT salary
  INTO sal
  FROM employees
  WHERE employee_id = id;
  RETURN sal;
END get_sal;
```

Executing Functions Using SQL Developer

1

2

Run PL/SQL

Target: GET_SAL

Parameter	Data Type	Mode
<Return Value>	NUMBER	OUT
ID	NUMBER	IN

PL/SQL Block

```
DECLARE
  ID NUMBER;
  v_Return NUMBER;
BEGIN
  ID := NULL;
  v_Return := GET_SAL(
    ID => ID
  );
  DBMS_OUTPUT.PUT_LINE('v_Return = ' || v_Return);
END;
```

Replace ID with the actual value

3

Run PL/SQL

Target: GET_SAL

Parameter	Data Type	Mode
<Return Value>	NUMBER	OUT
ID	NUMBER	IN

PL/SQL Block

```
DECLARE
  ID NUMBER;
  v_Return NUMBER;
BEGIN
  ID := NULL;
  v_Return := GET_SAL(
    ID => 100
  );
  DBMS_OUTPUT.PUT_LINE('v_Return = ' || v_Return);
END;
```

OK

Running - Log

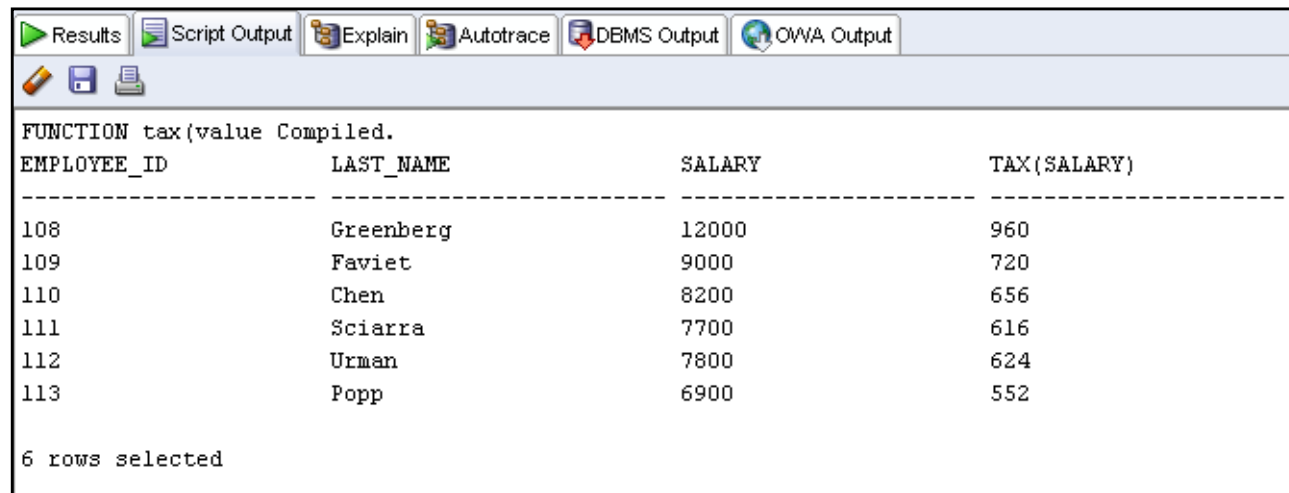
```
Connecting to the database student 41.
v_Return = 24000
Process exited.
Disconnecting from the database student 41.
```

Advantages of User-Defined Functions in SQL Statements

- Can extend SQL where activities are too complex, too awkward, or unavailable with SQL
- Can increase efficiency when used in the `WHERE` clause to filter data, as opposed to filtering the data in the application
- Can manipulate data values

Using a Function in a SQL Expression: Example

```
CREATE OR REPLACE FUNCTION tax(p_value IN NUMBER)
  RETURN NUMBER IS
BEGIN
  RETURN (p_value * 0.08);
END tax;
/
SELECT employee_id, last_name, salary, tax(salary)
FROM   employees
WHERE  department_id = 100;
```



EMPLOYEE_ID	LAST_NAME	SALARY	TAX(SALARY)
108	Greenberg	12000	960
109	Faviet	9000	720
110	Chen	8200	656
111	Sciarra	7700	616
112	Urman	7800	624
113	Popp	6900	552

6 rows selected

Calling User-Defined Functions in SQL Statements

User-defined functions act like built-in single-row functions and can be used in:

- The `SELECT` list or clause of a query
- Conditional expressions of the `WHERE` and `HAVING` clauses
- The `CONNECT BY`, `START WITH`, `ORDER BY`, and `GROUP BY` clauses of a query
- The `VALUES` clause of the `INSERT` statement
- The `SET` clause of the `UPDATE` statement

Restrictions When Calling Functions from SQL Expressions

- User-defined functions that are callable from SQL expressions must:
 - Be stored in the database
 - Accept only `IN` parameters with valid SQL data types, not PL/SQL-specific types
 - Return valid SQL data types, not PL/SQL-specific types
- When calling functions in SQL statements:
 - You must own the function or have the `EXECUTE` privilege

Controlling Side Effects When Calling Functions from SQL Expressions

Functions called from:

- A `SELECT` statement cannot contain DML statements
- An `UPDATE` or `DELETE` statement on a table `T` cannot query or contain DML on the same table `T`
- SQL statements cannot end transactions (that is, cannot execute `COMMIT` or `ROLLBACK` operations)

Note: Calls to subprograms that break these restrictions are also not allowed in the function.

Restrictions on Calling Functions from SQL: Example

```
CREATE OR REPLACE FUNCTION dml_call_sql(p_sal NUMBER)
  RETURN NUMBER IS
BEGIN
  INSERT INTO employees(employee_id, last_name,
                        email, hire_date, job_id, salary)
  VALUES(1, 'Frost', 'jfrost@company.com',
          SYSDATE, 'SA_MAN', p_sal);
  RETURN (p_sal + 100);
END;
```

```
UPDATE employees
  SET salary = dml_call_sql(2000)
WHERE employee_id = 170;
```

```
FUNCTION dml_call_sql(p_sal Compiled.

Error starting at line 1 in command:
UPDATE employees
  SET salary = dml_call_sql(2000)
WHERE employee_id = 170
Error report:
SQL Error: ORA-04091: table ORA62.EMPLOYEES is mutating, trigger/function may not see it
ORA-06512: at "ORA62.DML_CALL_SQL", line 4
04091. 00000 - "table %s.%s is mutating, trigger/function may not see it"
*Cause:      A trigger (or a user defined plsql function that is referenced in
              this statement) attempted to look at (or modify) a table that was
              in the middle of being modified by the statement which fired it.
*Action:     Rewrite the trigger (or function) so it does not read that table.
```

Named and Mixed Notation from SQL

- PL/SQL allows arguments in a subroutine call to be specified using positional, named, or mixed notation
- Prior to Oracle Database 11g, only the positional notation is supported in calls from SQL
- Starting in Oracle Database 11g, named and mixed notation can be used for specifying arguments in calls to PL/SQL subroutines from SQL statements
- For long parameter lists, with most having default values, you can omit values from the optional parameters
- You can avoid duplicating the default value of the optional parameter at each call site

Named and Mixed Notation from SQL: Example

```
CREATE OR REPLACE FUNCTION f(  
    p_parameter_1 IN NUMBER DEFAULT 1,  
    p_parameter_5 IN NUMBER DEFAULT 5)  
RETURN NUMBER  
IS  
    v_var number;  
BEGIN  
    v_var := p_parameter_1 + (p_parameter_5 * 2);  
    RETURN v_var;  
END f;  
/
```

```
FUNCTION f( Compiled.
```

```
SELECT f(p_parameter_5 => 10) FROM DUAL;
```

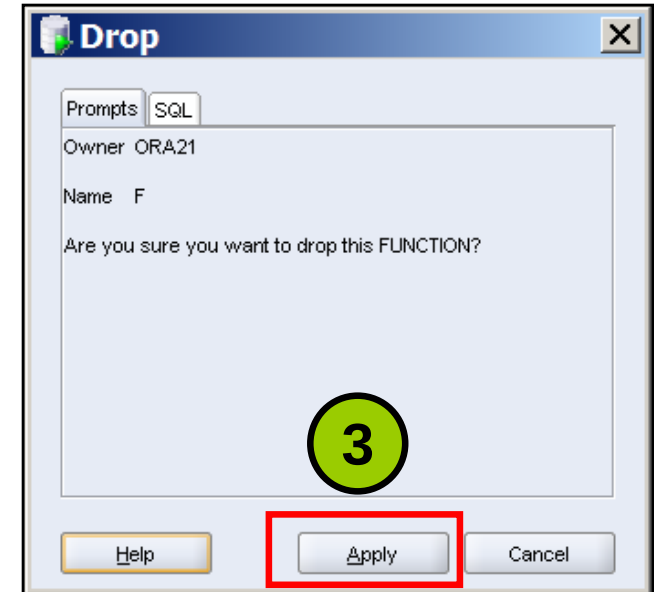
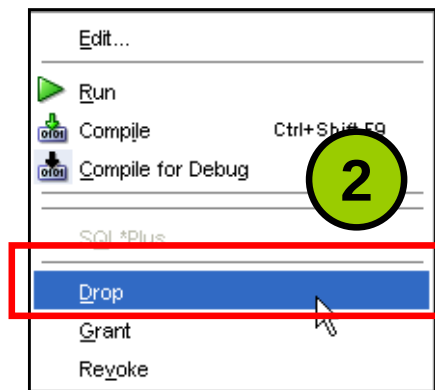
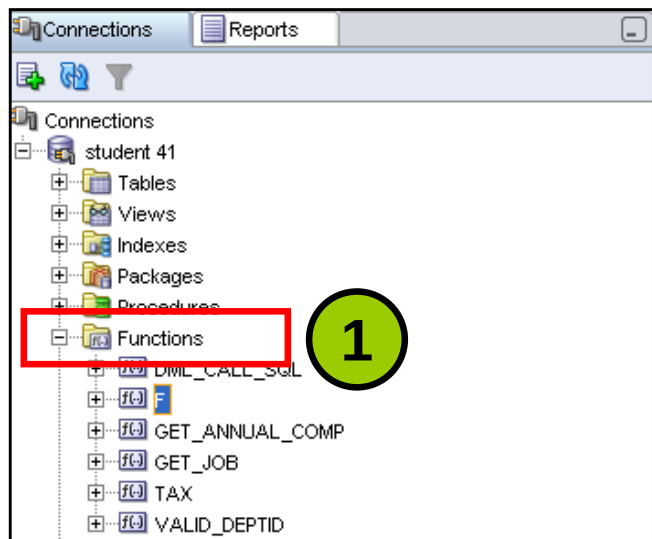
```
F(P_PARAMETER_5=>10)  
-----  
21  
  
1 rows selected
```

Removing Functions: Using the DROP SQL Statement or SQL Developer

- Using the DROP statement:

```
DROP FUNCTION f;
```

- Using SQL Developer:



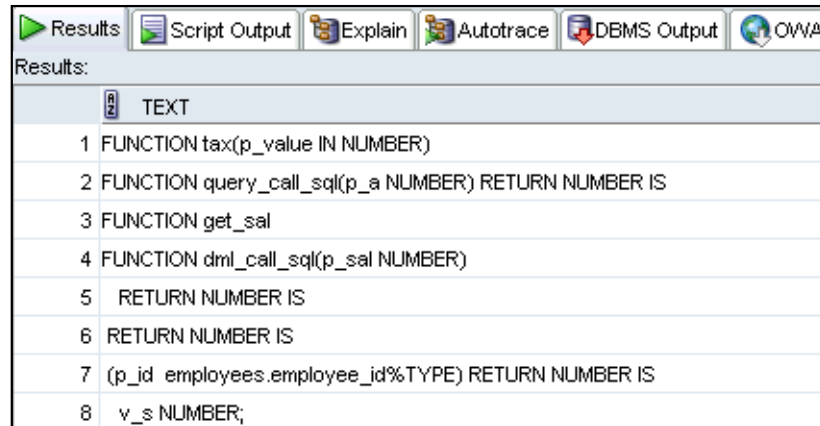
Viewing Functions Using Data Dictionary Views

```
DESCRIBE USER_SOURCE
```

```
DESCRIBE user_source
Name                Null    Type
-----
NAME                VARCHAR2(30)
TYPE                VARCHAR2(12)
LINE                NUMBER
TEXT                VARCHAR2(4000)

4 rows selected
```

```
SELECT  text
FROM    user_source
WHERE   type = 'FUNCTION'
ORDER  BY line;
```



	TEXT
1	FUNCTION tax(p_value IN NUMBER)
2	FUNCTION query_call_sql(p_a NUMBER) RETURN NUMBER IS
3	FUNCTION get_sal
4	FUNCTION dml_call_sql(p_sal NUMBER)
5	RETURN NUMBER IS
6	RETURN NUMBER IS
7	(p_id employees.employee_id%TYPE) RETURN NUMBER IS
8	v_s NUMBER;

...

Quiz

A PL/SQL function:

1. Can be invoked as part of an expression
2. Must contain a RETURN clause in the header
3. Must return a single value
4. Must contain at least one RETURN statement
5. Does not contain a RETURN clause in the header

Summary

In this lesson, you should have learned how to:

- Differentiate between a procedure and a function
- Describe the uses of functions
- Create stored functions
- Invoke a function
- Remove a function

Practice 3: Overview

This practice covers the following topics:

- Creating stored functions:
 - To query a database table and return specific values
 - To be used in a SQL statement
 - To insert a new row, with specified parameter values, into a database table
 - Using default parameter values
- Invoking a stored function from a SQL statement
- Invoking a stored function from a stored procedure

4

Creating Packages

Objectives

After completing this lesson, you should be able to do the following:

- Describe packages and list their components
- Create a package to group together related variables, cursors, constants, exceptions, procedures, and functions
- Designate a package construct as either public or private
- Invoke a package construct
- Describe the use of a bodiless package

Lesson Agenda

- Identifying the benefits and the components of packages
- Working with packages:
 - Creating the package specification and body
 - Invoking the package subprograms
 - Removing a package
 - Displaying the package information

What Are PL/SQL Packages?

- A package is a schema object that groups logically related PL/SQL types, variables, and subprograms.
- Packages usually have two parts:
 - A specification (spec)
 - A body
- The specification is the interface to the package. It declares the types, variables, constants, exceptions, cursors, and subprograms that can be referenced from outside the package.
- The body defines the queries for the cursors and the code for the subprograms.
- Enable the Oracle server to read multiple objects into memory at once.

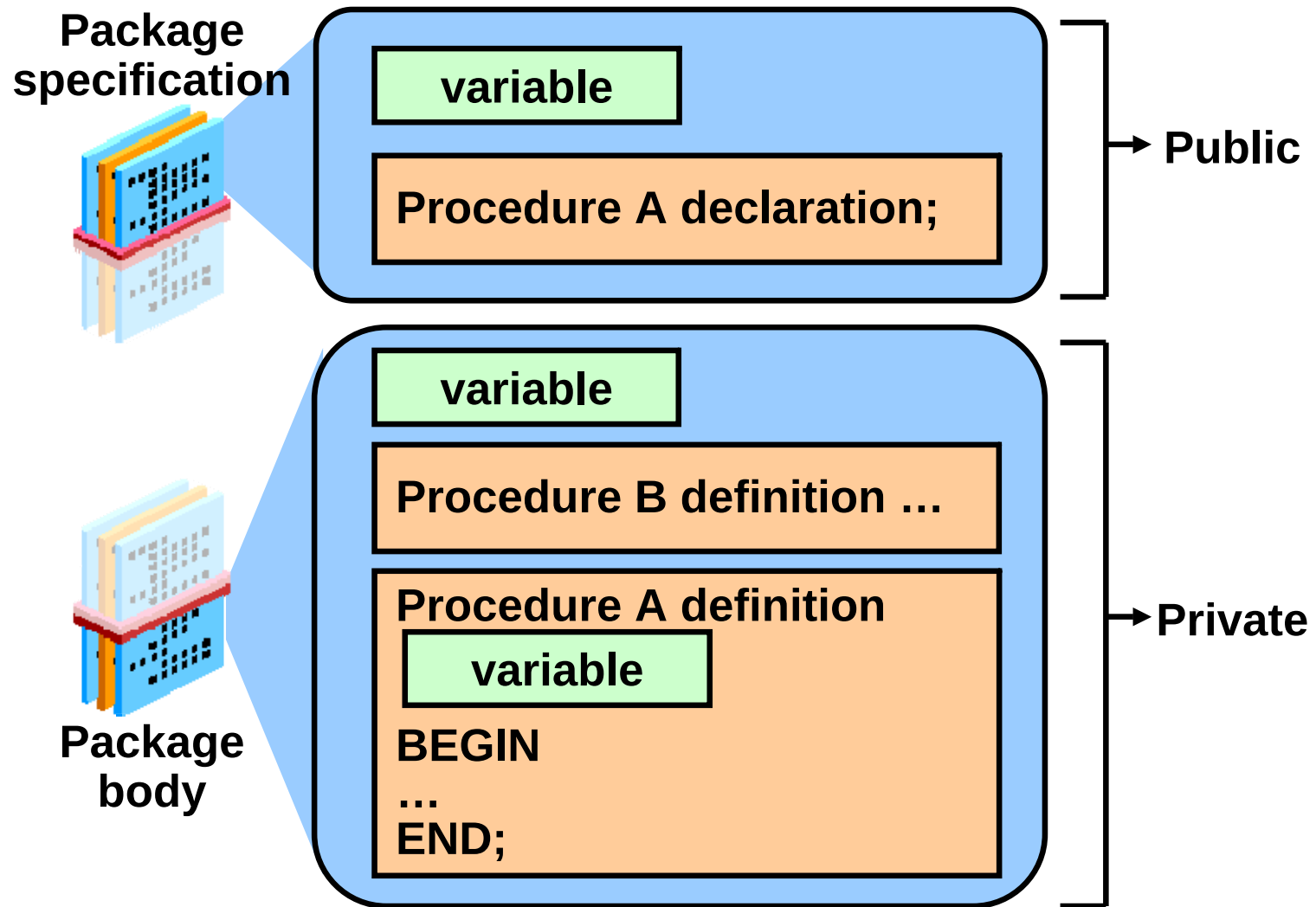
Advantages of Using Packages

- Modularity: Encapsulating related constructs
- Easier maintenance: Keeping logically related functionality together
- Easier application design: Coding and compiling the specification and body separately
- Hiding information:
 - Only the declarations in the package specification are visible and accessible to applications
 - Private constructs in the package body are hidden and inaccessible
 - All coding is hidden in the package body

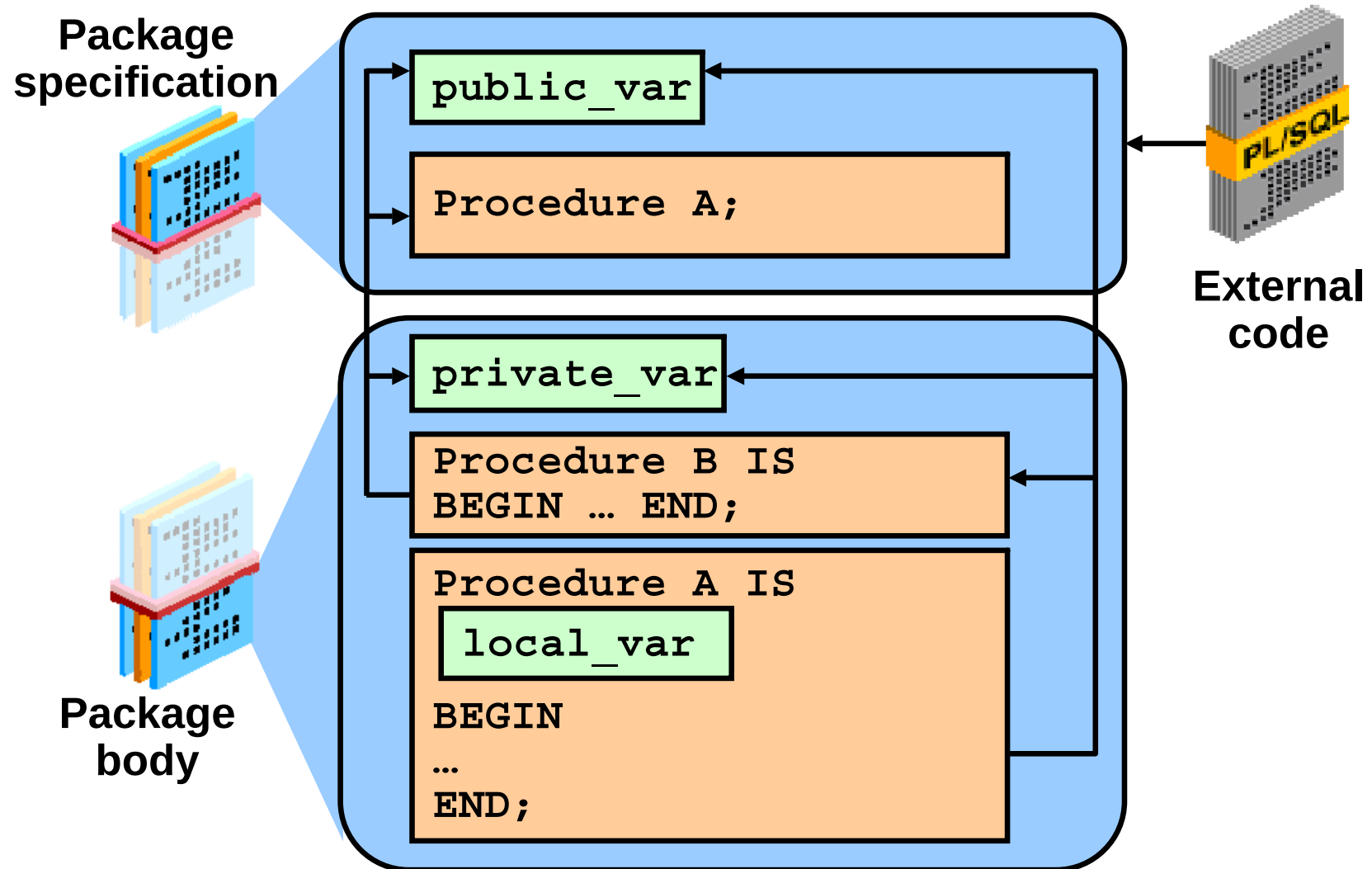
Advantages of Using Packages

- Added functionality: Persistency of public variables and cursors
- Better performance:
 - The entire package is loaded into memory when the package is first referenced.
 - There is only one copy in memory for all users.
 - The dependency hierarchy is simplified.
- Overloading: Multiple subprograms of the same name

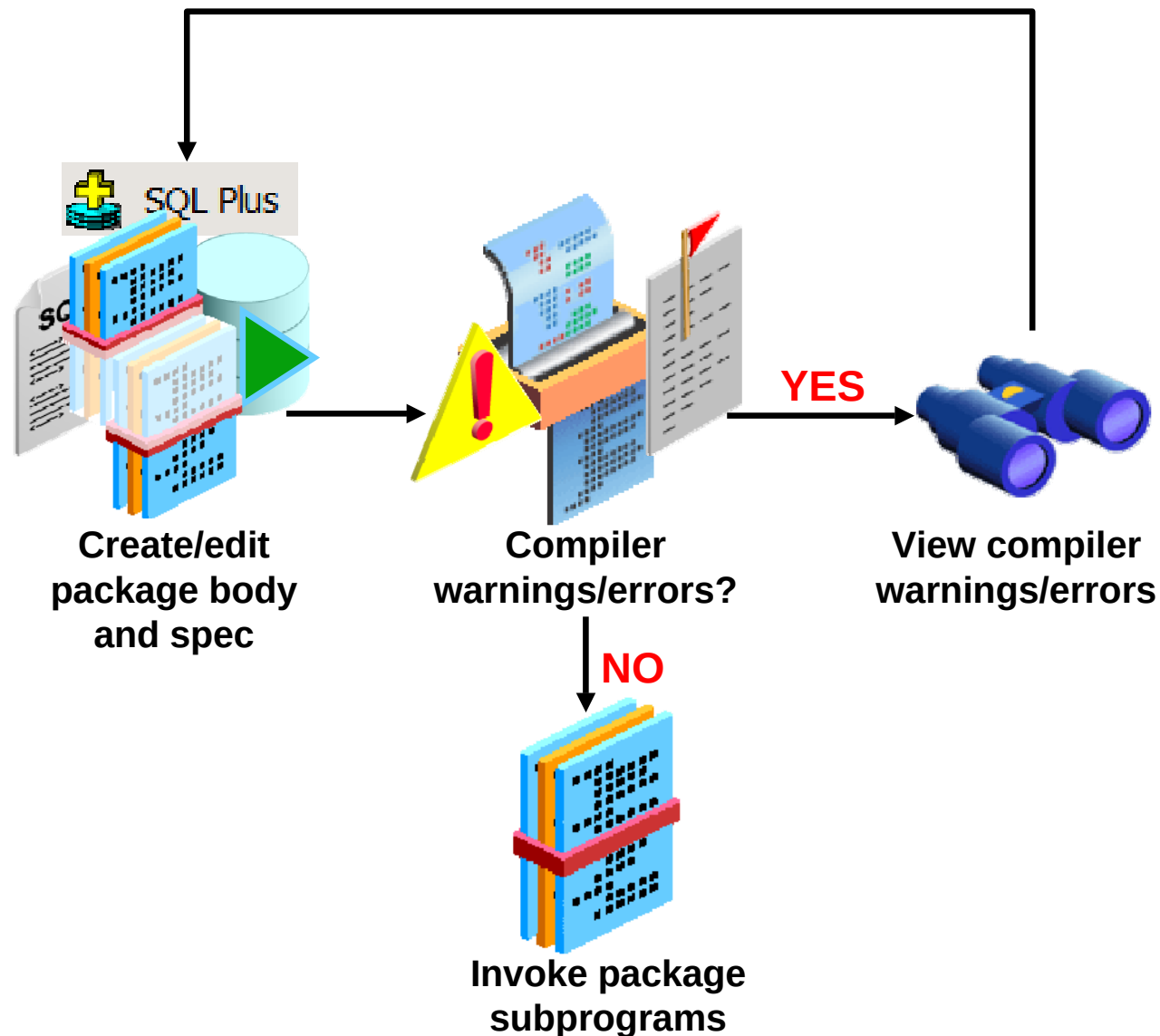
Components of a PL/SQL Package



The Visibility of a Package's Components



Developing PL/SQL Packages: Overview



Project: C:\Program Files\SQL Developer 1.1\sql
PROCEDURE ORA41.ADD_JOB_HISTORY@
Error(8,5): PLS-00103: Encountered the

View errors/warnings in SQL Developer

Use SHOW ERRORS command in SQL*Plus

Use USER/ALL/DBA_ERRORS views

Lesson Agenda

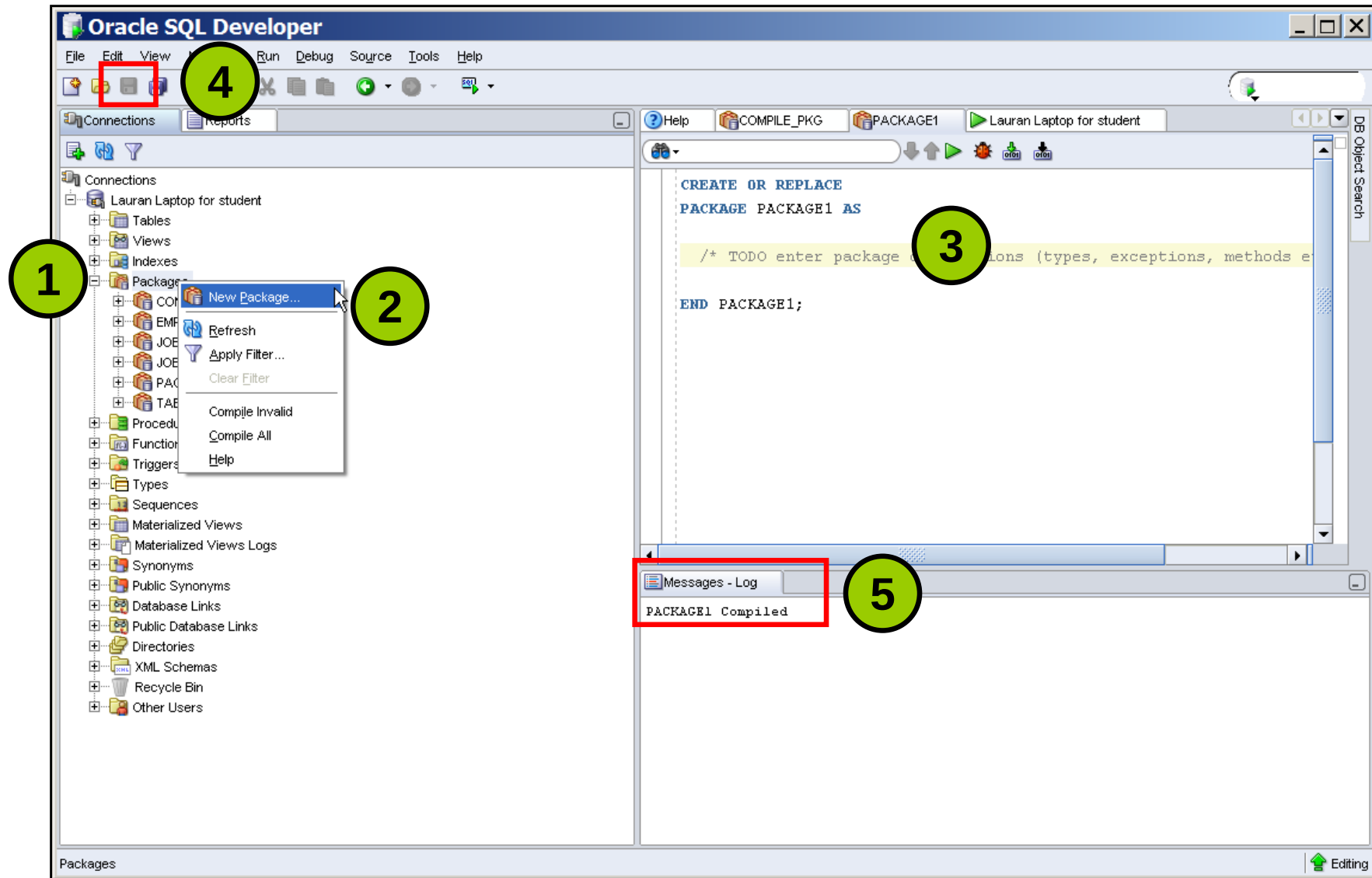
- Identifying the benefits and the components of packages
- Working with packages:
 - Creating the package specification and body
 - Invoking the package subprograms
 - Removing a package
 - Displaying the package information

Creating the Package Specification: Using the CREATE PACKAGE Statement

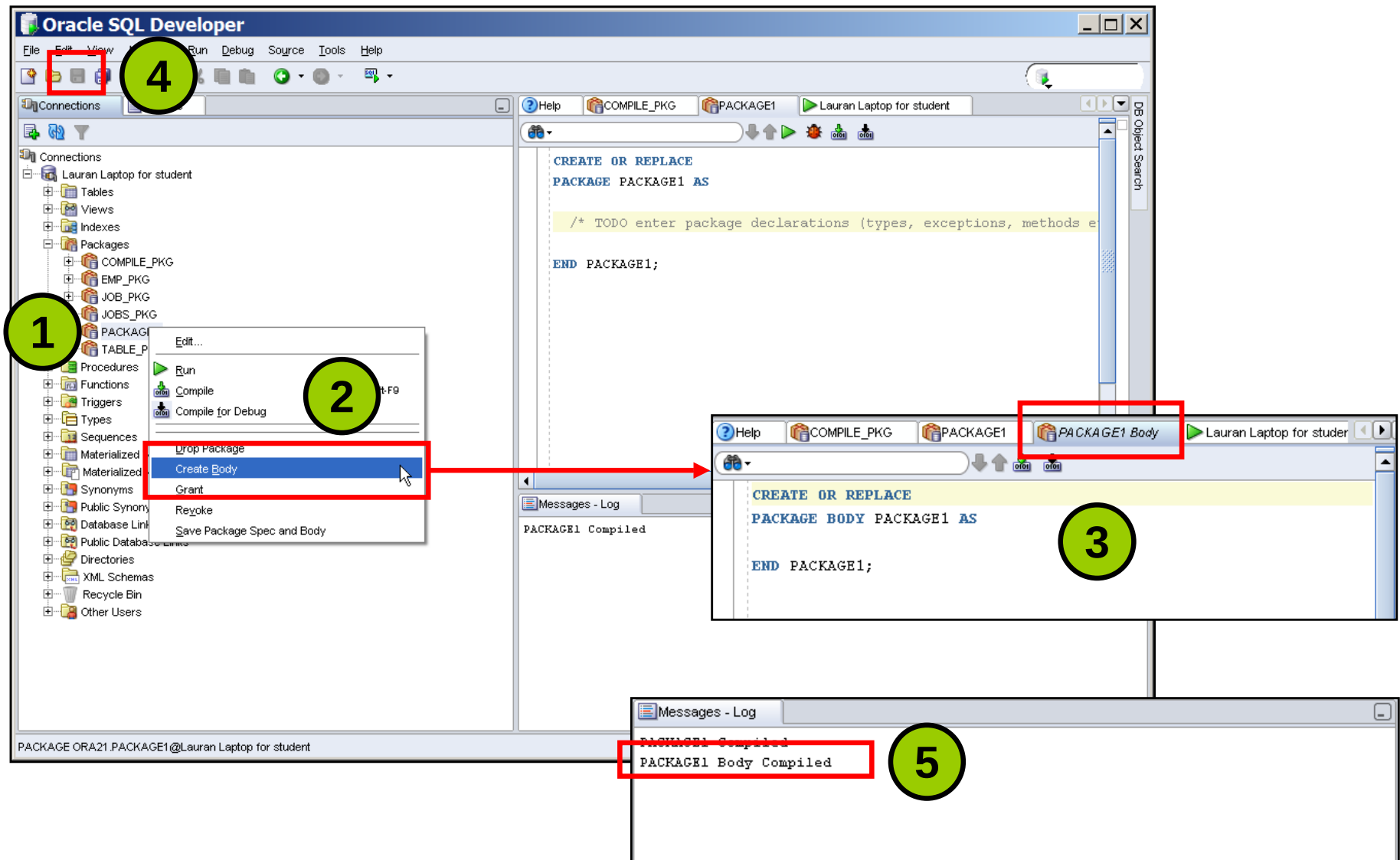
```
CREATE [OR REPLACE] PACKAGE package_name IS | AS  
    public type and variable declarations  
    subprogram specifications  
END [package_name];
```

- The OR REPLACE option drops and re-creates the package specification.
- Variables declared in the package specification are initialized to NULL by default.
- All the constructs declared in a package specification are visible to users who are granted privileges on the package.

Creating the Package Specification: Using SQL Developer



Creating the Package Body: Using SQL Developer



Example of a Package Specification: comm_pkg

```
-- The package spec with a public variable and a
-- public procedure that are accessible from
-- outside the package.

CREATE OR REPLACE PACKAGE comm_pkg IS
    v_std_comm NUMBER := 0.10;  --initialized to 0.10
    PROCEDURE reset_comm(p_new_comm NUMBER);
END comm_pkg;
/
```

- V_STD_COMM is a *public* global variable initialized to 0.10.
- RESET_COMM is a *public* procedure used to reset the standard commission based on some business rules. It is implemented in the package body.

Creating the Package Body

```
CREATE [OR REPLACE] PACKAGE BODY package_name IS | AS  
    private type and variable declarations  
    subprogram bodies  
    [BEGIN initialization statements]  
END [package_name];
```

- The OR REPLACE option drops and re-creates the package body.
- Identifiers defined in the package body are *private* and not visible outside the package body.
- All *private* constructs must be declared before they are referenced.
- Public constructs are visible to the package body.

Example of a Package Body: comm_pkg

```
CREATE OR REPLACE PACKAGE BODY comm_pkg IS
  FUNCTION validate(p_comm NUMBER) RETURN BOOLEAN IS
    v_max_comm      employees.commission_pct%type;
  BEGIN
    SELECT MAX(commission_pct) INTO v_max_comm
    FROM   employees;
    RETURN (p_comm BETWEEN 0.0 AND v_max_comm);
  END validate;

  PROCEDURE reset_comm (p_new_comm NUMBER) IS BEGIN
    IF validate(p_new_comm) THEN
      v_std_comm := p_new_comm; -- reset public var
    ELSE  RAISE_APPLICATION_ERROR(
          -20210, 'Bad Commission');
    END IF;
  END reset_comm;
END comm_pkg;
```

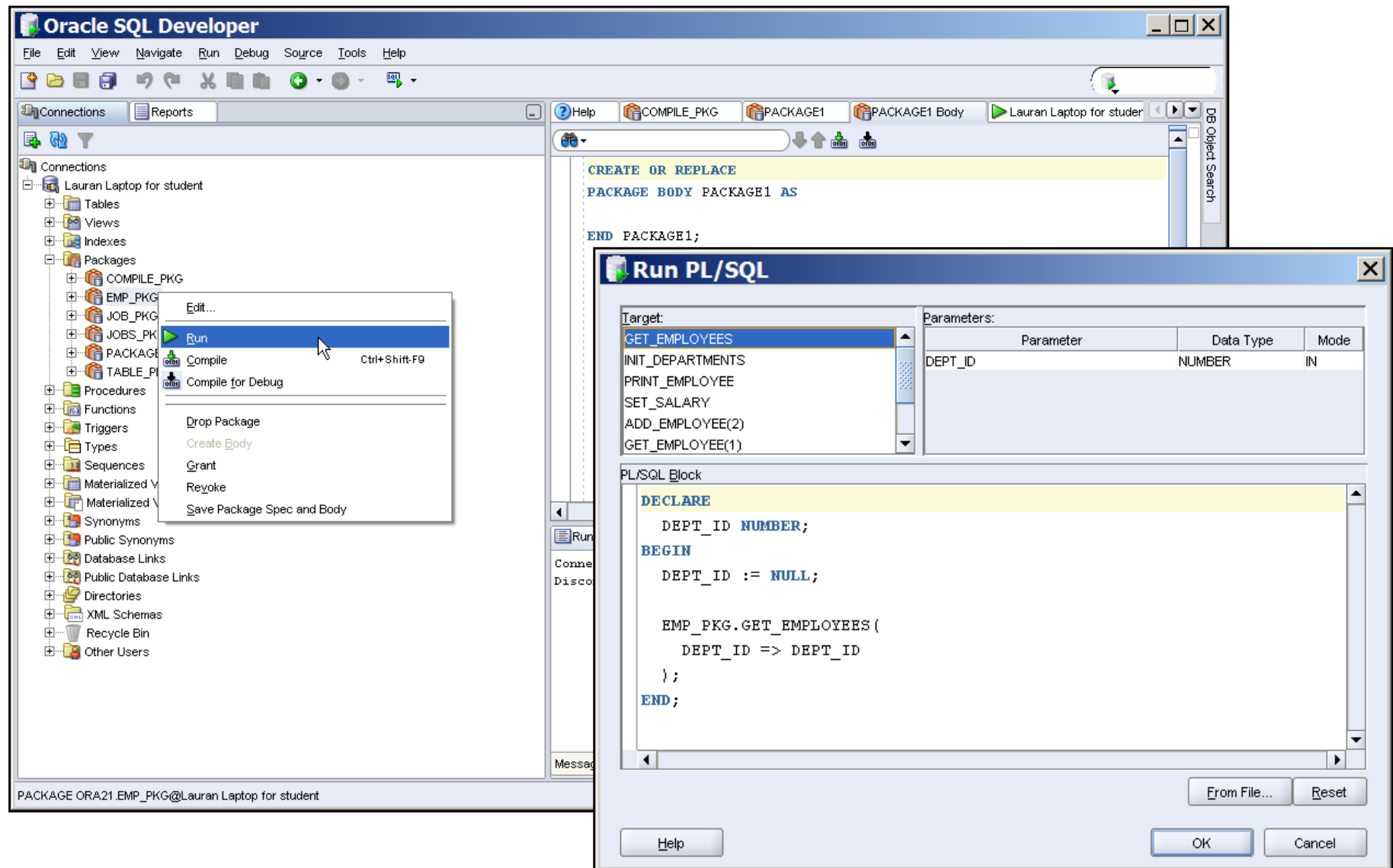
Invoking the Package Subprograms: Examples

```
-- Invoke a function within the same packages:  
CREATE OR REPLACE PACKAGE BODY comm_pkg IS ...  
  PROCEDURE reset_comm(p_new_comm NUMBER) IS  
  BEGIN  
    IF validate(p_new_comm) THEN  
      v_std_comm := p_new_comm;  
    ELSE ...  
    END IF;  
  END reset_comm;  
END comm_pkg;
```

```
-- Invoke a package procedure from SQL*Plus:  
EXECUTE comm_pkg.reset_comm(0.15)
```

```
-- Invoke a package procedure in a different schema:  
EXECUTE scott.comm_pkg.reset_comm(0.15)
```

Invoking the Package Subprograms: Using SQL Developer



Creating and Using Bodiless Packages

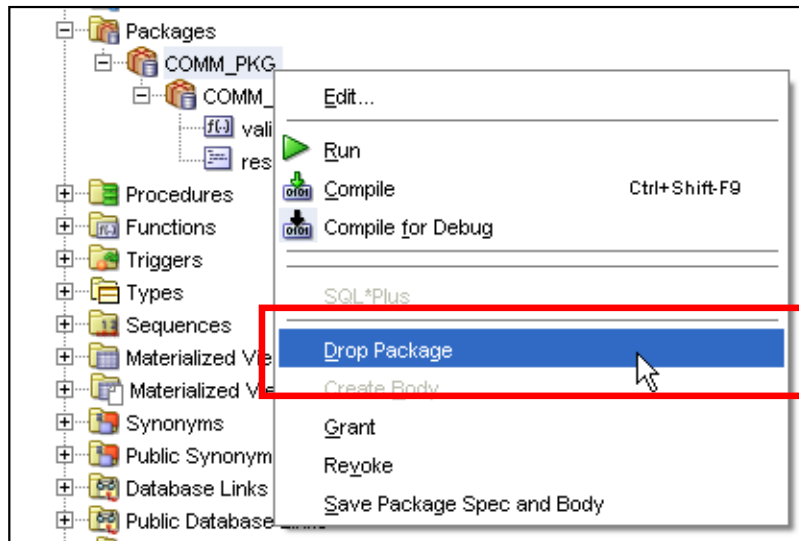
```
CREATE OR REPLACE PACKAGE global_consts IS
  c_mile_2_kilo      CONSTANT  NUMBER  :=  1.6093;
  c_kilo_2_mile      CONSTANT  NUMBER  :=  0.6214;
  c_yard_2_meter     CONSTANT  NUMBER  :=  0.9144;
  c_meter_2_yard     CONSTANT  NUMBER  :=  1.0936;
END global_consts;
```

```
BEGIN  DBMS_OUTPUT.PUT_LINE('20 miles = ' ||
    20 * global_consts.c_mile_2_kilo || ' km');
END;
```

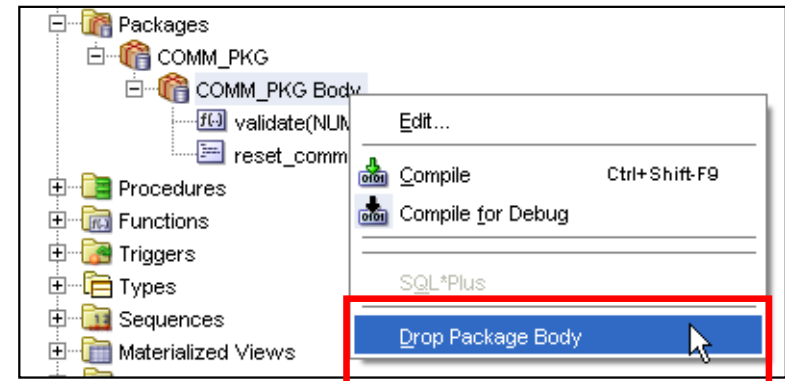
```
CREATE FUNCTION mtr2yrd(p_m NUMBER) RETURN NUMBER IS
BEGIN
  RETURN (p_m * global_consts.c_meter_2_yard);
END mtr2yrd;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(mtr2yrd(1))
```

Removing Packages: Using SQL Developer or the SQL DROP Statement

Drop package specification and body



Drop package body only



```
-- Remove the package specification and body  
DROP PACKAGE package_name;
```

```
-- Remove the package body only  
DROP PACKAGE BODY package_name;
```


Viewing Packages Using the Data Dictionary

```
-- View the package specification.  
SELECT text  
FROM    user_source  
WHERE   name = 'COMM_PKG' AND type = 'PACKAGE';
```

	TEXT
1	PACKAGE comm_pkg IS
2	std_comm NUMBER := 0.10; --initialized to 0.10
3	PROCEDURE reset_comm(new_comm NUMBER);
4	END comm_pkg;

```
-- View the package body.  
SELECT text  
FROM    user_source  
WHERE   name = 'COMM_PKG' AND type = 'PACKAGE BODY';
```

	TEXT
1	PACKAGE BODY comm_pkg IS
2	FUNCTION validate(comm NUMBER) RETURN BOOLEAN IS
3	max_comm employees.commission_pct%type;
4	BEGIN
5	SELECT MAX(commission_pct) INTO max_comm
6	FROM employees;
7	RETURN (comm BETWEEN 0.0 AND max_comm);

Guidelines for Writing Packages

- Develop packages for general use.
- Define the package specification before the body.
- The package specification should contain only those constructs that you want to be public.
- Place items in the declaration part of the package body when you must maintain them throughout a session or across transactions.
- The fine-grain dependency management reduces the need to recompile referencing subprograms when a package specification changes.
- The package specification should contain as few constructs as possible.

Quiz

The package specification is the interface to your applications. It declares the public types, variables, constants, exceptions, cursors, and subprograms available for use. The package specification may also include PRAGMAs, which are directives to the compiler.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Describe packages and list their components
- Create a package to group related variables, cursors, constants, exceptions, procedures, and functions
- Designate a package construct as either public or private
- Invoke a package construct
- Describe the use of a bodiless package

Practice 4 Overview: Creating and Using Packages

This practice covers the following topics:

- Creating packages
- Invoking package program units

5

Working with Packages

Objectives

After completing this lesson, you should be able to do the following:

- Overload package procedures and functions
- Use forward declarations
- Create an initialization block in a package body
- Manage persistent package data states for the life of a session
- Use PL/SQL tables and records in packages

Lesson Agenda

- Overloading package subprograms, using forward declarations, and creating an initialization block in a package body
- Managing persistent package data states for the life of a session and using PL/SQL tables and records in packages

Overloading Subprograms in PL/SQL

- Enables you to create two or more subprograms with the same name
- Requires that the subprogram's formal parameters differ in number, order, or data type family
- Enables you to build flexible ways for invoking subprograms with different data
- Provides a way to extend functionality without loss of existing code; that is, adding new parameters to existing subprograms
- Provides a way to overload local subprograms, package subprograms, and type methods, but not stand-alone subprograms

Overloading Procedures Example: Creating the Package Specification

```
CREATE OR REPLACE PACKAGE dept_pkg IS
  PROCEDURE add_department
    (p_deptno departments.department_id%TYPE,
     p_name departments.department_name%TYPE := 'unknown',
     p_loc departments.location_id%TYPE := 1700);

  PROCEDURE add_department
    (p_name departments.department_name%TYPE := 'unknown',
     p_loc departments.location_id%TYPE := 1700);
END dept_pkg;
/
```

Overloading Procedures Example: Creating the Package Body

```
CREATE OR REPLACE PACKAGE BODY dept_pkg IS
PROCEDURE add_department -- First procedure's declaration
    (p_deptno departments.department_id%TYPE,
     p_name    departments.department_name%TYPE := 'unknown',
     p_loc     departments.location_id%TYPE := 1700) IS
BEGIN
    INSERT INTO departments (department_id,
                             department_name, location_id)
    VALUES (p_deptno, p_name, p_loc);
END add_department;

PROCEDURE add_department -- Second procedure's declaration
    (p_name    departments.department_name%TYPE := 'unknown',
     p_loc     departments.location_id%TYPE := 1700) IS
BEGIN
    INSERT INTO departments (department_id,
                             department_name, location_id)
    VALUES (departments_seq.NEXTVAL, p_name, p_loc);
END add_department;
END dept_pkg; /
```

Overloading and the STANDARD Package

- A package named STANDARD defines the PL/SQL environment and built-in functions.
- Most built-in functions are overloaded. An example is the TO_CHAR function:

```
FUNCTION TO_CHAR (p1 DATE) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p2 NUMBER) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p1 DATE, P2 VARCHAR2) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p1 NUMBER, P2 VARCHAR2) RETURN  
    VARCHAR2;  
.  
.  
.
```

- A PL/SQL subprogram with the same name as a built-in subprogram overrides the standard declaration in the local context, unless qualified by the package name.

Illegal Procedure Reference

- Block-structured languages such as PL/SQL must declare identifiers before referencing them.
- Example of a referencing problem:

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
  PROCEDURE award_bonus(. . .) IS
  BEGIN
    calc_rating(. . .);      --illegal reference
  END;

  PROCEDURE calc_rating(. . .) IS
  BEGIN
    ...
  END;
END forward_pkg;
/
```

Using Forward Declarations to Solve Illegal Procedure Reference

In the package body, a forward declaration is a private subprogram specification terminated by a semicolon.

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
PROCEDURE calc_rating (...); -- forward declaration

-- Subprograms defined in alphabetical order

PROCEDURE award_bonus(...) IS
BEGIN
calc_rating (...);           -- reference resolved!
. . .
END;

PROCEDURE calc_rating (...) IS -- implementation
BEGIN
. . .
END;
END forward_pkg;
```



Initializing Packages

The block at the end of the package body executes once and is used to initialize public and private package variables.

```
CREATE OR REPLACE PACKAGE taxes IS
    v_tax    NUMBER;
    ... -- declare all public procedures/functions
END taxes;
/
CREATE OR REPLACE PACKAGE BODY taxes IS
    ... -- declare all private variables
    ... -- define public/private procedures/functions
BEGIN
    SELECT    rate_value INTO v_tax
    FROM      tax_rates
    WHERE     rate_name = 'TAX';
END taxes;
/
```

Using Package Functions in SQL

- You use package functions in SQL statements.
- To execute a SQL statement that calls a member function, the Oracle database must know the function's purity level.
- Purity level is the extent to which the function is free of side effects, which refers to accessing database tables, package variables, and so on, for reading or writing.
- It is important to control side effects because they can:
 - Prevent the proper parallelization of a query
 - Produce order-dependent and, therefore, indeterminate results
 - Require impermissible actions such as the maintenance of package state across user sessions

Controlling Side Effects of PL/SQL Subprograms

To be callable from SQL statements, a stored function must obey the following purity rules to control side effects:

- When called from a `SELECT` or a parallelized DML statement, the function cannot modify any database tables.
- When called from a DML statement, the function cannot query or modify any database tables modified by that statement.
- When called from a `SELECT` or DML statement, the function cannot execute SQL transaction control, session control, or system control statements.

Package Function in SQL: Example

```
CREATE OR REPLACE PACKAGE taxes_pkg IS
    FUNCTION tax (p_value IN NUMBER) RETURN NUMBER;
END taxes_pkg;
/
CREATE OR REPLACE PACKAGE BODY taxes_pkg IS
    FUNCTION tax (p_value IN NUMBER) RETURN NUMBER IS
        v_rate NUMBER := 0.08;
    BEGIN
        RETURN (p_value * v_rate);
    END tax;
END taxes_pkg;
/
```

```
SELECT taxes_pkg.tax(salary), salary, last_name
FROM   employees;
```

Lesson Agenda

- Overloading package subprograms, using forward declarations, and creating an initialization block in a package body
- Managing persistent package data states for the life of a session and using PL/SQL tables and records in packages

Persistent State of Packages

The collection of package variables and the values define the package state. The package state is:

- Initialized when the package is first loaded
- Persistent (by default) for the life of the session:
 - Stored in the User Global Area (UGA)
 - Unique to each session
 - Subject to change when package subprograms are called or public variables are modified
- Not persistent for the session but persistent for the life of a subprogram call when using `PRAGMA SERIALLY_REUSABLE` in the package specification

Persistent State of Package Variables: Example

<i>State for Scott</i>				<i>State for Jones</i>	
Time	Events	v_std_commission [variable]	MAX (comm_pc) [column]	v_std_commission [variable]	MAX (comm_pc) [Column]
9:00	Scott> EXECUTE comm_pkg.reset_comm(0.25)	0.10 0.25	0.4	-	0.4
9:30	Jones> INSERT INTO employees(last_name, commission_pct) VALUES('Madonna', 0.8);	0.25	0.4		0.8
9:35	Jones> EXECUTE comm_pkg.reset_comm (0.5)	0.25	0.4	0.1 0.5	0.8
10:00	Scott> EXECUTE comm_pkg.reset_comm(0.6) <i>Err -20210 'Bad Commission'</i>	0.25	0.4	0.5	0.8
11:00	Jones> ROLLBACK;	0.25	0.4	0.5	0.4
11:01	EXIT ...	0.25	0.4	-	0.4
12:00	EXEC comm_pkg.reset_comm(0.2)	0.25	0.4	0.2	0.4

Persistent State of a Package Cursor: Example

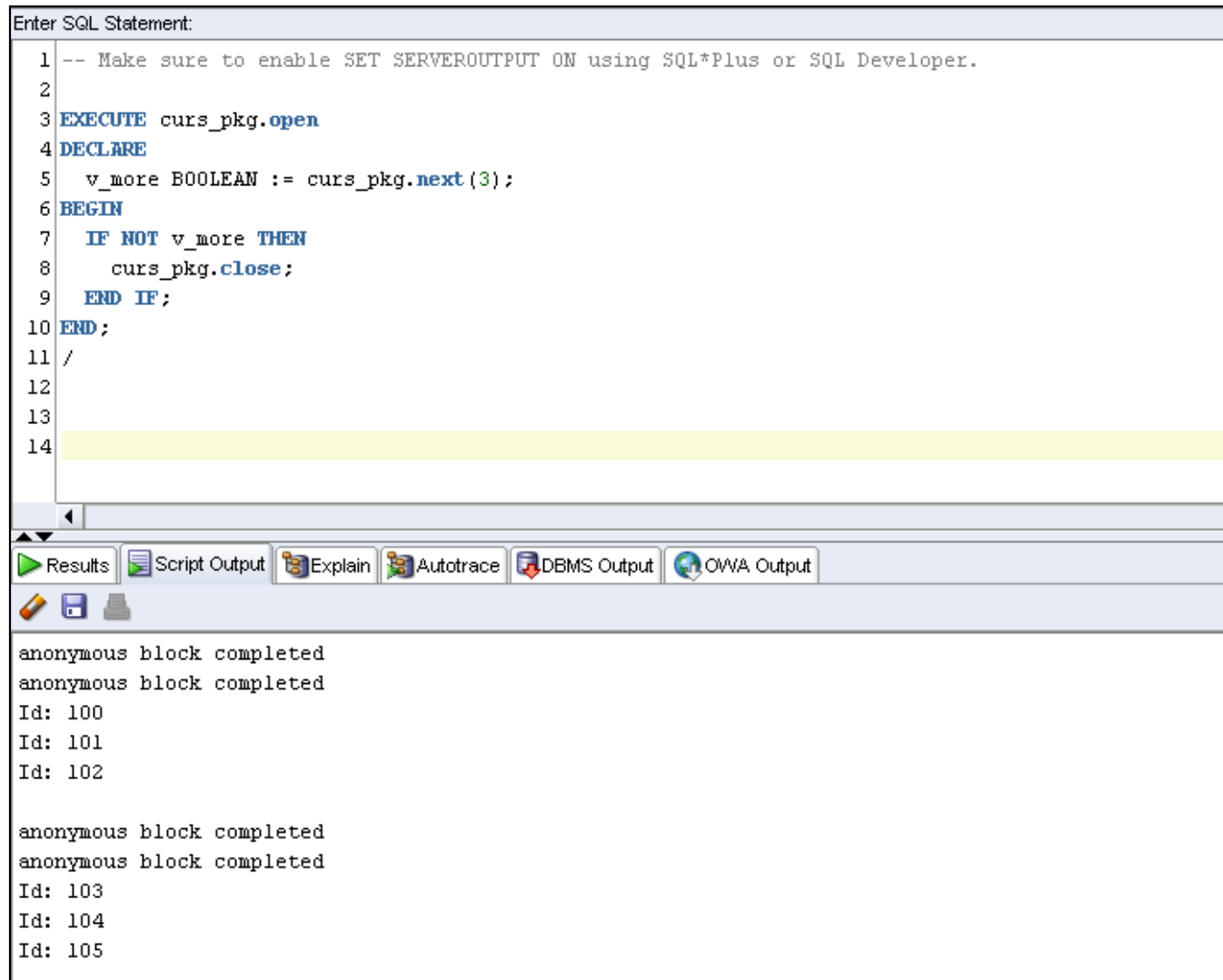
```
CREATE OR REPLACE PACKAGE curs_pkg IS -- Package spec
    PROCEDURE open;
    FUNCTION next(p_n NUMBER := 1) RETURN BOOLEAN;
    PROCEDURE close;
END curs_pkg;

CREATE OR REPLACE PACKAGE BODY curs_pkg IS
-- Package body
    CURSOR cur_c IS
        SELECT employee_id FROM employees;
    PROCEDURE open IS
    BEGIN
        IF NOT cur_c%ISOPEN THEN
            OPEN cur_c;
        END IF;
    END open;
    . . . -- code continued on next slide
```

Persistent State of a Package Cursor: Example

```
. . .  
FUNCTION next(p_n NUMBER := 1) RETURN BOOLEAN IS  
    v_emp_id employees.employee_id%TYPE;  
BEGIN  
    FOR count IN 1 .. p_n LOOP  
        FETCH cur_c INTO v_emp_id;  
        EXIT WHEN cur_c%NOTFOUND;  
        DBMS_OUTPUT.PUT_LINE('Id: ' || (v_emp_id));  
    END LOOP;  
    RETURN cur_c%FOUND;  
END next;  
PROCEDURE close IS  
    BEGIN  
        IF cur_c%ISOPEN THEN  
            CLOSE cur_c;  
        END IF;  
    END close;  
END curs_pkg;
```

Executing the CURS_PKG Package



```
1 -- Make sure to enable SET SERVEROUTPUT ON using SQL*Plus or SQL Developer.
2
3 EXECUTE curs_pkg.open
4 DECLARE
5     v_more BOOLEAN := curs_pkg.next(3);
6 BEGIN
7     IF NOT v_more THEN
8         curs_pkg.close;
9     END IF;
10 END;
11 /
12
13
14
```

Results | Script Output | Explain | Autotrace | DBMS Output | OWA Output

anonymous block completed
anonymous block completed
Id: 100
Id: 101
Id: 102

anonymous block completed
anonymous block completed
Id: 103
Id: 104
Id: 105

Using PL/SQL Tables of Records in Packages

```
CREATE OR REPLACE PACKAGE emp_pkg IS
  TYPE emp_table_type IS TABLE OF employees%ROWTYPE
    INDEX BY BINARY_INTEGER;
  PROCEDURE get_employees(p_emps OUT emp_table_type);
END emp_pkg;
```

```
CREATE OR REPLACE PACKAGE BODY emp_pkg IS
  PROCEDURE get_employees(p_emps OUT emp_table_type) IS
    v_i BINARY_INTEGER := 0;
  BEGIN
    FOR emp_record IN (SELECT * FROM employees)
    LOOP
      emps(v_i) := emp_record;
      v_i := v_i + 1;
    END LOOP;
  END get_employees;
END emp_pkg;
```

Quiz

Overloading subprograms in PL/SQL:

1. Enables you to create two or more subprograms with the same name
2. Requires that the subprogram's formal parameters differ in number, order, or data type family
3. Enables you to build flexible ways for invoking subprograms with different data
4. Provides a way to extend functionality without loss of existing code; that is, adding new parameters to existing subprograms

Summary

In this lesson, you should have learned how to:

- Create and call overloaded subprograms
- Use forward declarations for subprograms
- Write package initialization blocks
- Maintain persistent package state

Practice 5: Overview

This practice covers the following topics:

- Using overloaded subprograms
- Creating a package initialization block
- Using a forward declaration



Using Oracle-Supplied Packages in Application Development

Objectives

After completing this lesson, you should be able to do the following:

- Describe how the DBMS_OUTPUT package works
- Use UTL_FILE to direct output to operating system files
- Describe the main features of UTL_MAIL

Lesson Agenda

- Identifying the benefits of using the Oracle-supplied packages and listing some of those packages
- Using the following Oracle-supplied packages:
 - DBMS_OUTPUT
 - UTL_FILE
 - UTL_MAIL

Using Oracle-Supplied Packages

- The Oracle-supplied packages:
 - Are provided with the Oracle server
 - Extend the functionality of the database
 - Enable access to certain SQL features that are normally restricted for PL/SQL
- For example, the `DBMS_OUTPUT` package was originally designed to debug PL/SQL programs.

Examples of Some Oracle-Supplied Packages

Here is an abbreviated list of some Oracle-supplied packages:

- DBMS_OUTPUT
- UTL_FILE
- UTL_MAIL
- DBMS_ALERT
- DBMS_LOCK
- DBMS_SESSION
- HTP
- DBMS_SCHEDULER

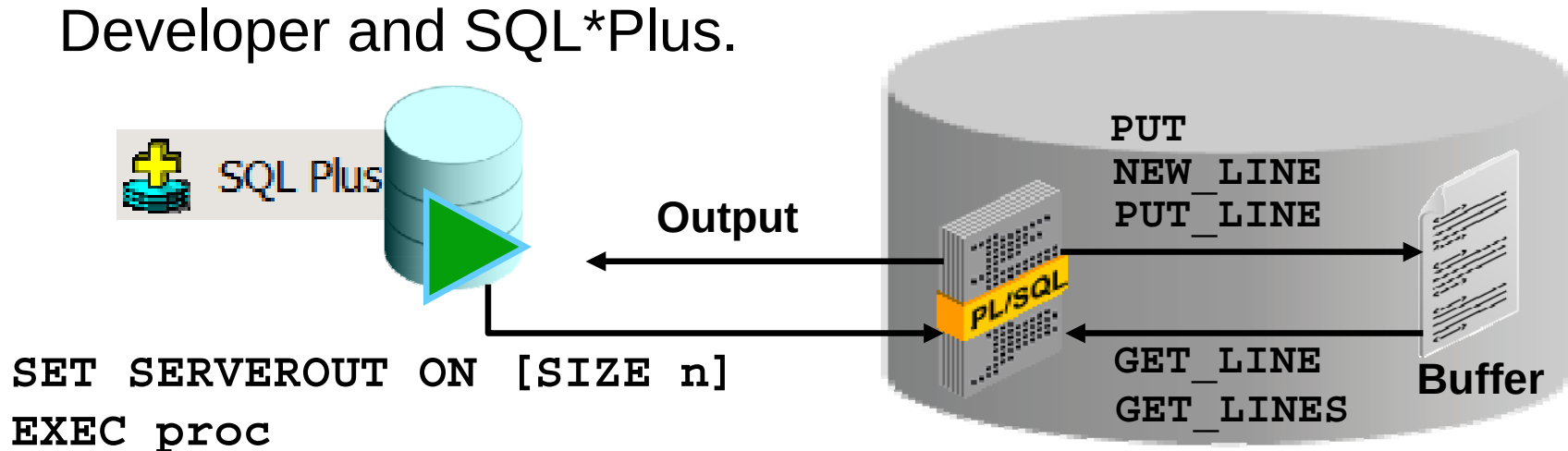
Lesson Agenda

- Identifying the benefits of using the Oracle-supplied packages and listing some of those packages
- Using the following Oracle-supplied packages:
 - DBMS_OUTPUT
 - UTL_FILE
 - UTL_MAIL

How the DBMS_OUTPUT Package Works

The DBMS_OUTPUT package enables you to send messages from stored subprograms and triggers.

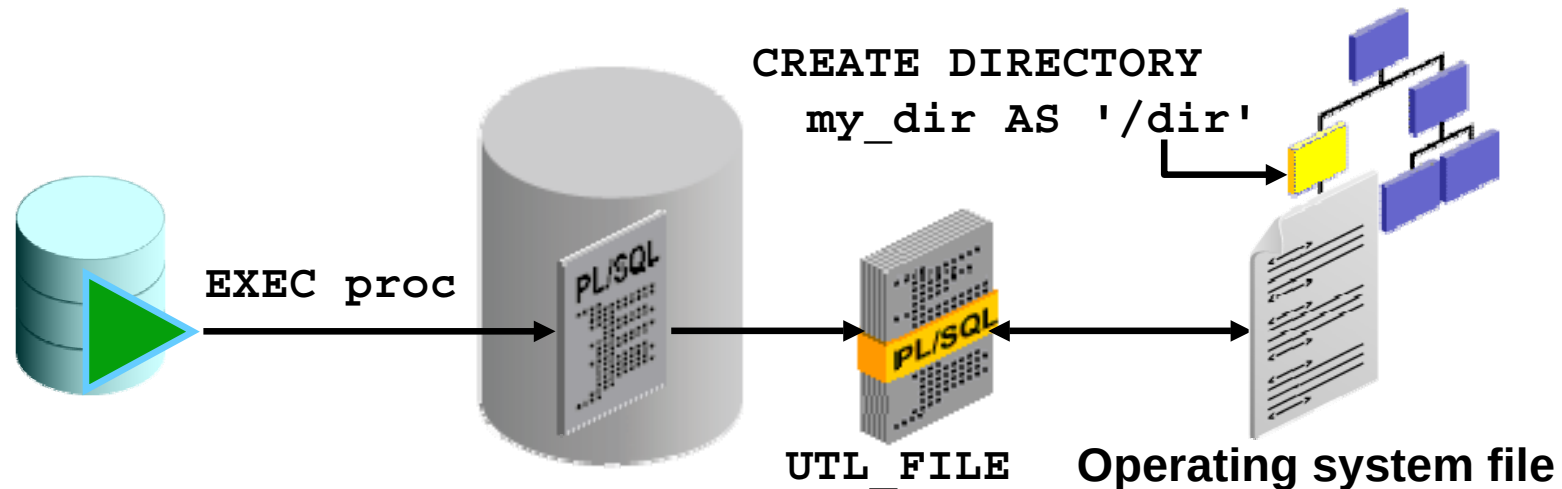
- PUT and PUT_LINE place text in the buffer.
- GET_LINE and GET_LINES read the buffer.
- Messages are not sent until the sending subprogram or trigger completes.
- Use SET SERVEROUTPUT ON to display messages in SQL Developer and SQL*Plus.



Using the UTL_FILE Package to Interact with Operating System Files

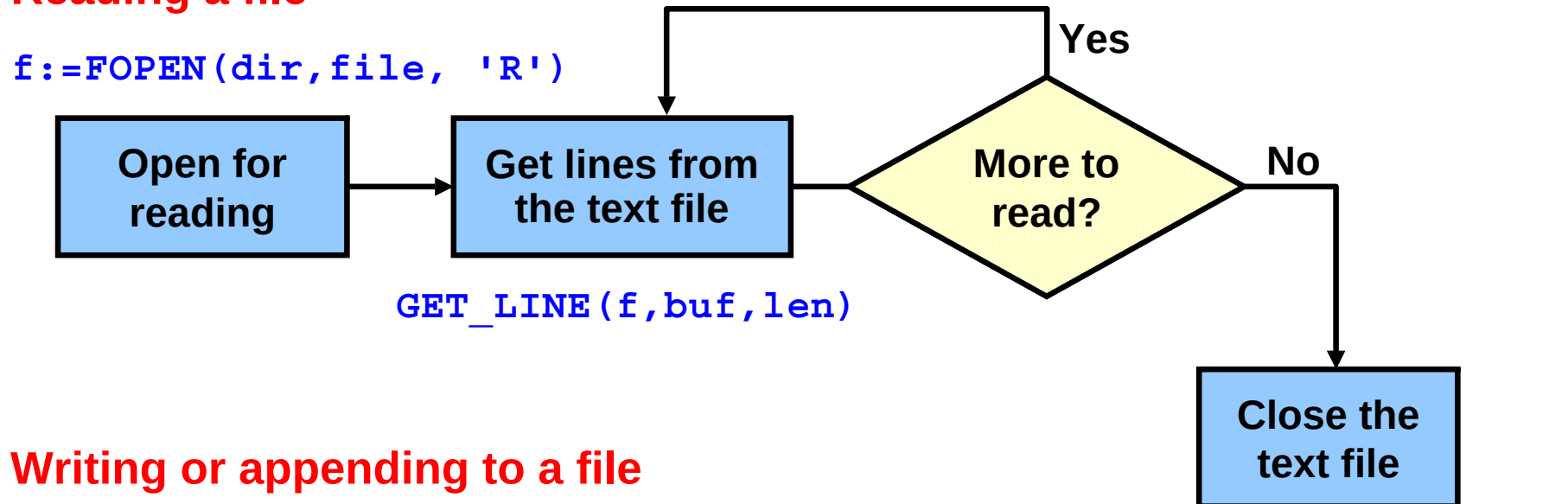
The UTL_FILE package extends PL/SQL programs to read and write operating system text files:

- Provides a restricted version of operating system stream file I/O for text files
- Can access files in operating system directories defined by a CREATE DIRECTORY statement

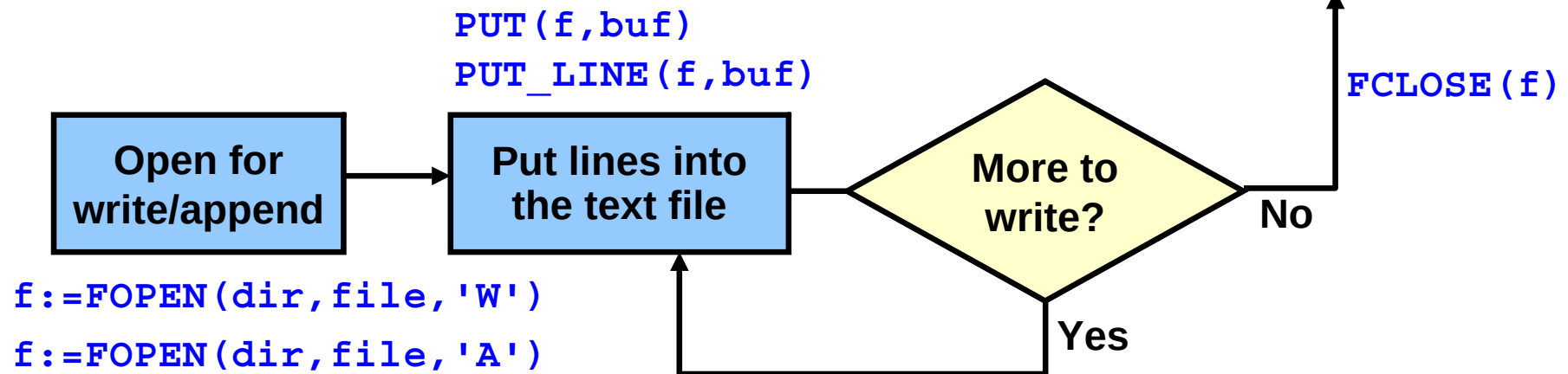


File Processing Using the UTL_FILE Package: Overview

Reading a file



Writing or appending to a file



Using the Available Declared Exceptions in the UTL_FILE Package

Exception Name	Description
INVALID_PATH	File location invalid
INVALID_MODE	The <code>open_mode</code> parameter in <code>FOPEN</code> is invalid
INVALID_FILEHANDLE	File handle invalid
INVALID_OPERATION	File could not be opened or operated on as requested
READ_ERROR	Operating system error occurred during the read operation
WRITE_ERROR	Operating system error occurred during the write operation
INTERNAL_ERROR	Unspecified PL/SQL error

FOPEN and IS_OPEN Functions: Example

- This FOPEN function opens a file for input or output.

```
FUNCTION FOPEN (p_location  IN VARCHAR2,  
                p_filename  IN VARCHAR2,  
                p_open_mode IN VARCHAR2)  
RETURN UTL_FILE.FILE_TYPE;
```

- The IS_OPEN function determines whether a file handle refers to an open file.

```
FUNCTION IS_OPEN (p_file IN FILE_TYPE)  
RETURN BOOLEAN;
```

Using UTL_FILE: Example

```
CREATE OR REPLACE PROCEDURE sal_status(  
  p_dir IN VARCHAR2, p_filename IN VARCHAR2) IS  
  f_file UTL_FILE.FILE_TYPE;  
  CURSOR cur_emp IS  
    SELECT last_name, salary, department_id  
    FROM employees ORDER BY department_id;  
  v_newdeptno employees.department_id%TYPE;  
  v_olddeptno employees.department_id%TYPE := 0;  
BEGIN  
  f_file:= UTL_FILE.FOPEN (p_dir, p_filename, 'W');  
  UTL_FILE.PUT_LINE(f_file,  
    'REPORT: GENERATED ON ' || SYSDATE);  
  UTL_FILE.NEW_LINE (f_file);  
  . . .
```

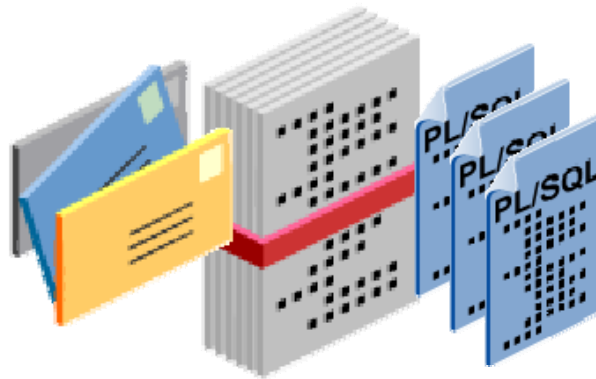

Using UTL_FILE: Example

```
. . .
FOR emp_rec IN cur_emp LOOP
    IF emp_rec.department_id <> v_olddeptno THEN
        UTL_FILE.PUT_LINE (f_file,
            'DEPARTMENT: ' || emp_rec.department_id);
        UTL_FILE.NEW_LINE (f_file);
    END IF;
    UTL_FILE.PUT_LINE (f_file,
        '  EMPLOYEE: ' || emp_rec.last_name ||
        '  earns: ' || emp_rec.salary);
    v_olddeptno := emp_rec.department_id;
    UTL_FILE.NEW_LINE (f_file);
END LOOP;
UTL_FILE.PUT_LINE(f_file, '*** END OF REPORT ***');
UTL_FILE.FCLOSE (f_file);
EXCEPTION
    WHEN UTL_FILE.INVALID_FILEHANDLE THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid File.');
```

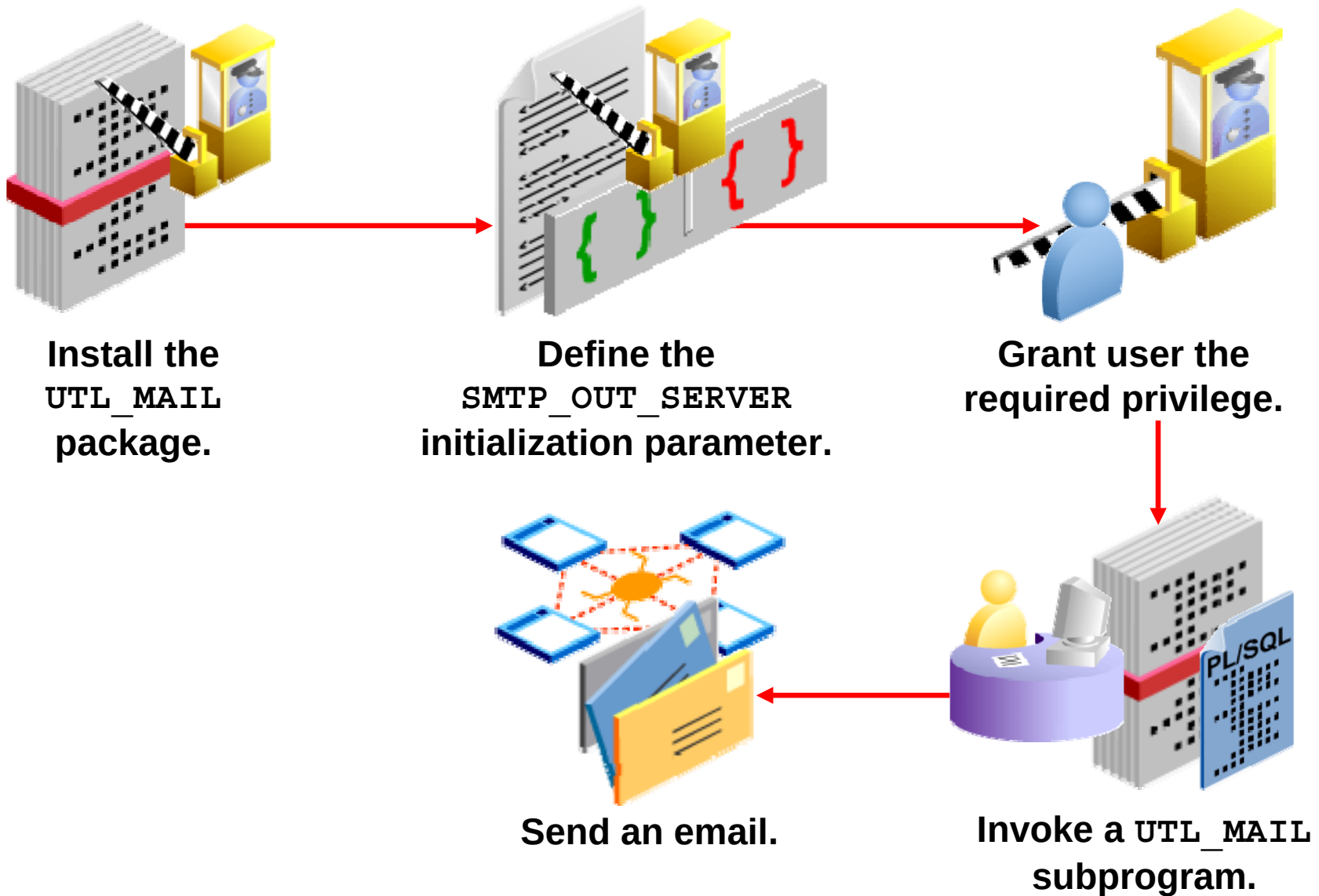
```
    WHEN UTL_FILE.WRITE_ERROR THEN
        RAISE_APPLICATION_ERROR (-20002, 'Unable to write to file');
END sal_status;
/
```

What Is the UTL_MAIL Package?

- A utility for managing email
- Requires the setting of the SMTP_OUT_SERVER database initialization parameter
- Provides the following procedures:
 - SEND for messages without attachments
 - SEND_ATTACH_RAW for messages with binary attachments
 - SEND_ATTACH_VARCHAR2 for messages with text attachments



Setting Up and Using the UTL_MAIL: Overview



Summary of UTL_MAIL Subprograms

Subprogram	Description
SEND procedure	Packages an email message, locates SMTP information, and delivers the message to the SMTP server for forwarding to the recipients
SEND_ATTACH_RAW Procedure	Represents the SEND procedure overloaded for RAW attachments
SEND_ATTACH_VARCHAR2 Procedure	Represents the SEND procedure overloaded for VARCHAR2 attachments

Installing and Using UTL_MAIL

- As SYSDBA, using SQL Developer or SQL*Plus:
 - Install the UTL_MAIL package

```
@?/rdbms/admin/utlmail.sql  
@?/rdbms/admin/prvtmail.plb
```

- Set the SMTP_OUT_SERVER

```
ALTER SYSTEM SET SMTP_OUT_SERVER='smtp.server.com'  
SCOPE=SPFILE
```

- As a developer, invoke a UTL_MAIL procedure:

```
BEGIN  
  UTL_MAIL.SEND('otn@oracle.com','user@oracle.com',  
    message => 'For latest downloads visit OTN',  
    subject => 'OTN Newsletter');  
END;
```

The SEND Procedure Syntax

Packages an email message into the appropriate format, locates SMTP information, and delivers the message to the SMTP server for forwarding to the recipients.

```
UTL_MAIL.SEND (  
    sender      IN      VARCHAR2 CHARACTER SET ANY_CS,  
    recipients  IN      VARCHAR2 CHARACTER SET ANY_CS,  
    cc          IN      VARCHAR2 CHARACTER SET ANY_CS  
                DEFAULT NULL,  
    bcc         IN      VARCHAR2 CHARACTER SET ANY_CS  
                DEFAULT NULL,  
    subject     IN      VARCHAR2 CHARACTER SET ANY_CS  
                DEFAULT NULL,  
    message     IN      VARCHAR2 CHARACTER SET ANY_CS,  
    mime_type   IN      VARCHAR2  
                DEFAULT 'text/plain; charset=us-ascii',  
    priority    IN      PLS_INTEGER DEFAULT NULL);
```

The SEND_ATTACH_RAW Procedure

This procedure is the SEND procedure overloaded for RAW attachments.

```
UTL_MAIL.SEND_ATTACH_RAW (  
    sender          IN      VARCHAR2 CHARACTER SET ANY_CS,  
    recipients      IN      VARCHAR2 CHARACTER SET ANY_CS,  
    cc              IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    bcc             IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    subject         IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    message         IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    mime_type       IN      VARCHAR2 DEFAULT CHARACTER SET ANY_CS  
                        DEFAULT 'text/plain; charset=us-ascii',  
    priority        IN      PLS_INTEGER DEFAULT 3,  
    attachment      IN      RAW,  
    att_inline      IN      BOOLEAN DEFAULT TRUE,  
    att_mime_type   IN      VARCHAR2 CHARACTER SET ANY_CS  
                        DEFAULT 'text/plain; charset=us-ascii',  
    att_filename    IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL);
```

Sending Email with a Binary Attachment: Example

```
CREATE OR REPLACE PROCEDURE send_mail_logo IS
BEGIN
    UTL_MAIL.SEND_ATTACH_RAW(
        sender => 'me@oracle.com',
        recipients => 'you@somewhere.net',
        message =>
            '<HTML><BODY>See attachment</BODY></HTML>',
        subject => 'Oracle Logo',
        mime_type => 'text/html'
        attachment => get_image('oracle.gif'),
        att_inline => true,
        att_mime_type => 'image/gif',
        att_filename => 'oralego.gif');
END;
/
```


The SEND_ATTACH_VARCHAR2 Procedure

This procedure is the SEND procedure overloaded for VARCHAR2 attachments.

```
UTL_MAIL.SEND_ATTACH_VARCHAR2 (  
    sender            IN      VARCHAR2 CHARACTER SET ANY_CS,  
    recipients        IN      VARCHAR2 CHARACTER SET ANY_CS,  
    cc                IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    bcc               IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    subject           IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    message           IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
    mime_type         IN      VARCHAR2 CHARACTER SET ANY_CS  
                        DEFAULT 'text/plain; charset=us-ascii',  
    priority          IN      PLS_INTEGER DEFAULT 3,  
    attachment        IN      VARCHAR2 CHARACTER SET ANY_CS,  
    att_inline        IN      BOOLEAN DEFAULT TRUE,  
    att_mime_type     IN      VARCHAR2 CHARACTER SET ANY_CS  
                        DEFAULT 'text/plain; charset=us-ascii',  
    att_filename      IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL);
```

Sending Email with a Text Attachment: Example

```
CREATE OR REPLACE PROCEDURE send_mail_file IS
BEGIN
    UTL_MAIL.SEND_ATTACH_VARCHAR2 (
        sender => 'me@oracle.com',
        recipients => 'you@somewhere.net',
        message =>
            '<HTML><BODY>See attachment</BODY></HTML>',
        subject => 'Oracle Notes',
        mime_type => 'text/html'
        attachment => get_file('notes.txt'),
        att_inline => false,
        att_mime_type => 'text/plain',
        att_filename => 'notes.txt');
END;
/
```

Quiz

The Oracle-supplied UTL_FILE package is used to access text files in the operating system of the database server. The database provides read and write access to specific operating system directories.

1. True
2. False

Summary

In this lesson, you should have learned:

- How the `DBMS_OUTPUT` package works
- How to use `UTL_FILE` to direct output to operating system files
- About the main features of `UTL_MAIL`

Practice 6: Overview

This practice covers how to use `UTL_FILE` to generate a text report.



Using Dynamic SQL

Objectives

After completing this lesson, you should be able to do the following:

- Describe the execution flow of SQL statements
- Build and execute SQL statements dynamically using Native Dynamic SQL (NDS)
- Identify situations when you must use the DBMS_SQL package instead of NDS to build and execute SQL statements dynamically

Lesson Agenda

- Using Native Dynamic SQL (NDS)
- Using the `DBMS_SQL` package

Execution Flow of SQL

- All SQL statements go through some or all of the following stages:
 - Parse
 - Bind
 - Execute
 - Fetch
- Some stages may not be relevant for all statements:
 - The fetch phase is applicable to queries.
 - For embedded SQL statements such as `SELECT`, `DML`, `MERGE`, `COMMIT`, `SAVEPOINT`, and `ROLLBACK`, the parse and bind phases are done at compile time.
 - For dynamic SQL statements, all phases are performed at run time.

Working With Dynamic SQL

Use dynamic SQL to create a SQL statement whose structure may change during run time. Dynamic SQL:

- Is constructed and stored as a character string, string variable, or string expression within the application.
- Is a SQL statement with varying column data, or different conditions with or without placeholders (bind variables).
- Enables DDL, DCL, or session-control statements to be written and executed from PL/SQL.
- Is executed with Native Dynamic SQL statements or the `DBMS_SQL` package.

Using Dynamic SQL

- Use dynamic SQL when the full text of the dynamic SQL statement is unknown until run time; therefore, its syntax is checked at *run time* rather than at *compile time*.
- Use dynamic SQL when one of the following items is unknown at precompile time:
 - Text of the SQL statement such as commands, clauses, and so on
 - The number and data types of host variables
 - References to database objects such as tables, columns, indexes, sequences, usernames, and views
- Use dynamic SQL to make your PL/SQL programs more general and flexible.

Native Dynamic SQL (NDS)

- Provides native support for dynamic SQL directly in the PL/SQL language.
- Provides the ability to execute SQL statements whose structure is unknown until execution time.
- If the dynamic SQL statement is a `SELECT` statement that returns multiple rows, NDS gives you the following choices:
 - Use the `EXECUTE IMMEDIATE` statement with the `BULK COLLECT INTO` clause
 - Use the `OPEN-FOR`, `FETCH`, and `CLOSE` statements
- In Oracle Database 11g, NDS supports statements larger than 32 KB by accepting a `CLOB` argument.

Using the EXECUTE IMMEDIATE Statement

Use the EXECUTE IMMEDIATE statement for NDS or PL/SQL anonymous blocks:

```
EXECUTE IMMEDIATE dynamic_string  
  [INTO {define_variable  
        [, define_variable] ... | record}]  
  [USING [IN|OUT|IN OUT] bind_argument  
        [, [IN|OUT|IN OUT] bind_argument] ... ];
```

- INTO is used for single-row queries and specifies the variables or records into which column values are retrieved.
- USING is used to hold all bind arguments. The default parameter mode is IN.

Available Methods for Using NDS

Method #	SQL Statement Type	NDS SQL Statements Used
Method 1	<i>Non-query without</i> host variables	EXECUTE IMMEDIATE without the USING and INTO clauses
Method 2	<i>Non-query with</i> known number of input host variables	EXECUTE IMMEDIATE with a USING clause
Method 3	<i>Query with known</i> number of select-list items and input host variables	EXECUTE IMMEDIATE with the USING and INTO clauses
Method 4	<i>Query with unknown</i> number of select-list items or input host variables	Use the DBMS_SQL package

Dynamic SQL with a DDL Statement: Examples

```
-- Create a table using dynamic SQL

CREATE OR REPLACE PROCEDURE create_table(
  p_table_name VARCHAR2, p_col_specs  VARCHAR2) IS
BEGIN
  EXECUTE IMMEDIATE 'CREATE TABLE ' || p_table_name ||
                    ' (' || p_col_specs || ')';
END;
/
```

```
-- Call the procedure

BEGIN
  create_table('EMPLOYEE_NAMES',
    'id NUMBER(4) PRIMARY KEY, name VARCHAR2(40)');
END;
/
```

Dynamic SQL with DML Statements

```
-- Delete rows from any table:
CREATE FUNCTION del_rows(p_table_name VARCHAR2)
RETURN NUMBER IS
BEGIN
    EXECUTE IMMEDIATE 'DELETE FROM ' || p_table_name;
    RETURN SQL%ROWCOUNT;
END;
/
BEGIN DBMS_OUTPUT.PUT_LINE(
    del_rows('EMPLOYEE_NAMES') || ' rows deleted. ');
END;
/
```

```
-- Insert a row into a table with two columns:
CREATE PROCEDURE add_row(p_table_name VARCHAR2,
    p_id NUMBER, p_name VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE 'INSERT INTO ' || p_table_name ||
        ' VALUES (:1, :2)' USING p_id, p_name;
END;
```


Dynamic SQL with a Single-Row Query: Example

```
CREATE FUNCTION get_emp(p_emp_id NUMBER)
RETURN employees%ROWTYPE IS
    v_stmt VARCHAR2(200);
    v_emprec employees%ROWTYPE;
BEGIN
    v_stmt := 'SELECT * FROM employees ' ||
              'WHERE employee_id = :p_emp_id';
    EXECUTE IMMEDIATE v_stmt INTO v_emprec USING p_emp_id;
    RETURN v_emprec;
END;
/
DECLARE
    v_emprec employees%ROWTYPE := get_emp(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Emp: ' || v_emprec.last_name);
END;
/
```

Dynamic SQL with a Multirow Query: Example

```
-- Use OPEN-FOR, FETCH, and CLOSE processing:

CREATE PROCEDURE list_employees( p_deptid NUMBER ) IS
  TYPE emp_refcsr_type IS REF CURSOR;
  cur_emp emp_refcsr_type;
  rec_emp employees%ROWTYPE;
  v_stmt varchar2(200) := 'SELECT * FROM employees';
BEGIN
  IF p_deptid IS NULL THEN OPEN cur_emp FOR v_stmt;
  ELSE
    v_stmt := v_stmt || ' WHERE department id = :id';
    OPEN cur_emp FOR v_stmt USING p_deptid;
  END IF;
  LOOP
    FETCH cur_emp INTO rec_emp;
    EXIT WHEN cur_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(rec_emp.department_id ||
                        ' ' || rec_emp.last_name);
  END LOOP;
  CLOSE cur_emp;
END;
```

1

2

Declaring Cursor Variables

```
-- Declare two cursor types as REF CURSORS:

CREATE OR REPLACE FUNCTION process_data
RETURN employees%ROWTYPE IS
    TYPE cur_ref_type      IS REF CURSOR; -- weak ref. cursor
    TYPE cur_ref_emp_type IS REF CURSOR; -- weak ref. cursor
    . . .

-- Declare a cursor variable using the cursor type:
    v_dept_csr cur_ref_type;
    v_emp_csr  cur_ref_emp_type;

BEGIN
    OPEN v_dept_csr FOR SELECT * FROM departments;
    OPEN v_emp_csr  FOR SELECT * FROM employees;

-- Then use as normal cursors
    . . .

END;
```

Executing a PL/SQL Anonymous Block Dynamically

```
CREATE FUNCTION annual_sal( p_emp_id NUMBER)
RETURN NUMBER IS
  v_plsql varchar2(200) :=
    'DECLARE ' ||
    ' rec_emp employees%ROWTYPE; ' ||
    'BEGIN ' ||
    ' rec_emp := get_emp(:empid); ' ||
    ' :res := rec_emp.salary * 12; ' ||
    'END;';
  v_result NUMBER;
BEGIN
  EXECUTE IMMEDIATE v_plsql
    USING IN p_emp_id, OUT v_result;
  RETURN v_result;
END;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(annual_sal(100))
```

The diagram illustrates the execution of the PL/SQL code. Red boxes highlight the following elements:

- `p_emp_id` in the function signature.
- `:empid` in the `get_emp` call.
- `:res` in the assignment statement.
- `p_emp_id` and `v_result` in the `EXECUTE IMMEDIATE` statement.
- The final function call `annual_sal(100)`.

Green circles with numbers indicate the following steps:

- Step 1: The `EXECUTE IMMEDIATE` statement is executed, passing `p_emp_id` and `v_result` as input and output parameters.
- Step 2: The `get_emp` function is called with `:empid` as input, and the result is stored in `:res`.

Using Native Dynamic SQL to Compile PL/SQL Code

Compile PL/SQL code with the ALTER statement:

- ALTER PROCEDURE name COMPILE
- ALTER FUNCTION name COMPILE
- ALTER PACKAGE name COMPILE SPECIFICATION
- ALTER PACKAGE name COMPILE BODY

```
CREATE PROCEDURE compile_plsql(p_name VARCHAR2,  
    p_plsql_type VARCHAR2, p_options VARCHAR2 := NULL) IS  
    v_stmt varchar2(200) := 'ALTER ' || p_plsql_type ||  
        ' ' || p_name || ' COMPILE';  
  
BEGIN  
    IF p_options IS NOT NULL THEN  
        v_stmt := v_stmt || ' ' || p_options;  
    END IF;  
    EXECUTE IMMEDIATE v_stmt;  
END;/
```

Lesson Agenda

- Using Native Dynamic SQL (NDS)
- Using the DBMS_SQL package

Using the DBMS_SQL Package

- The DBMS_SQL package is used to write dynamic SQL in stored procedures and to parse DDL statements.
- You must use the DBMS_SQL package to execute a dynamic SQL statement that has an unknown number of input or output variables, also known as Method 4.
- In most cases, NDS is easier to use and performs better than DBMS_SQL except when dealing with Method 4.
- For example, you must use the DBMS_SQL package in the following situations:
 - You do not know the `SELECT` list at compile time
 - You do not know how many columns a `SELECT` statement will return, or what their data types will be

Using the DBMS_SQL Package Subprograms

Examples of the package procedures and functions:

- OPEN_CURSOR
- PARSE
- BIND_VARIABLE
- EXECUTE
- FETCH_ROWS
- CLOSE_CURSOR

Using DBMS_SQL with a DML Statement: Deleting Rows

```
CREATE OR REPLACE FUNCTION delete_all_rows
  (p_table_name  VARCHAR2) RETURN NUMBER IS
  v_cur_id      INTEGER;
  v_rows_del    NUMBER;
BEGIN
  v_cur_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(v_cur_id,
    'DELETE FROM ' || p_table_name, DBMS_SQL.NATIVE);
  v_rows_del := DBMS_SQL.EXECUTE (v_cur_id);
  DBMS_SQL.CLOSE_CURSOR(v_cur_id);
  RETURN v_rows_del;
END;
/
```

```
CREATE TABLE temp_emp AS SELECT * FROM employees;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Rows Deleted: ' ||
delete_all_rows('temp_emp'));
END;
/
```

Using DBMS_SQL with a Parameterized DML Statement

```
CREATE PROCEDURE insert_row (p_table_name VARCHAR2,  
    p_id VARCHAR2, p_name VARCHAR2, p_region NUMBER) IS  
    v_cur_id          INTEGER;  
    v_stmt            VARCHAR2(200);  
    v_rows_added      NUMBER;  
BEGIN  
    v_stmt := 'INSERT INTO ' || p_table_name ||  
              ' VALUES (:cid, :cname, :rid)';  
    v_cur_id := DBMS_SQL.OPEN_CURSOR;  
    DBMS_SQL.PARSE(v_cur_id, v_stmt, DBMS_SQL.NATIVE);  
    DBMS_SQL.BIND_VARIABLE(v_cur_id, ':cid', p_id);  
    DBMS_SQL.BIND_VARIABLE(v_cur_id, ':cname', p_name);  
    DBMS_SQL.BIND_VARIABLE(v_cur_id, ':rid', p_region);  
    rows_added := DBMS_SQL.EXECUTE(v_cur_id);  
    DBMS_SQL.CLOSE_CURSOR(v_cur_id);  
    DBMS_OUTPUT.PUT_LINE(v_rows_added || ' row added');  
END; /
```

Dynamic SQL Functional Completeness

For functional completeness, interoperability between native dynamic SQL and `DBMS_SQL` is supported in Oracle Database 11g:

- SQL statements larger than 32 KB are allowed in native dynamic SQL.
- `DBMS_SQL.PARSE()` is overloaded for CLOBs.
- A `REF CURSOR` can be converted to a `DBMS_SQL` cursor and vice versa to support interoperability.
- `DBMS_SQL` supports the full range of data types including collections and object types.

Quiz

The full text of the dynamic SQL statement might be unknown until run time; therefore, its syntax is checked at *run time* rather than at *compile time*.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Describe the execution flow of SQL statements
- Build and execute SQL statements dynamically using Native Dynamic SQL (NDS)
- Identify situations when you must use the `DBMS_SQL` package instead of NDS to build and execute SQL statements dynamically

Practice 7 Overview: Using Native Dynamic SQL

This practice covers the following topics:

- Creating a package that uses Native Dynamic SQL to create or drop a table and to populate, modify, and delete rows from a table
- Creating a package that compiles the PL/SQL code in your schema

8

Design Considerations for PL/SQL Code

Objectives

After completing this lesson, you should be able to do the following:

- Create standard constants and exceptions
- Write and call local subprograms
- Control the run-time privileges of a subprogram
- Perform autonomous transactions
- Pass parameters by reference using a `NOCOPY` hint
- Use the `PARALLEL ENABLE` hint for optimization
- Use the cross-session PL/SQL function result cache
- Use the `DETERMINISTIC` clause with functions
- Use bulk binding and the `RETURNING` clause with DML

Lesson Agenda

- Standardizing constants and exceptions, using local subprograms, controlling the run-time privileges of a subprogram, and performing autonomous transactions
- Using the `NOCOPY` and the `PARALLEL ENABLE` hints, the cross-session PL/SQL function result cache, and the `DETERMINISTIC` clause
- Using bulk binding and the `RETURNING` clause with DML

Standardizing Constants and Exceptions

Constants and exceptions are typically implemented using a bodiless package (that is, a package specification).

- Standardizing helps to:
 - Develop programs that are consistent
 - Promote a higher degree of code reuse
 - Ease code maintenance
 - Implement company standards across entire applications
- Start with standardization of:
 - Exception names
 - Constant definitions

Standardizing Exceptions

Create a standardized error-handling package that includes all named and programmer-defined exceptions to be used in the application.

```
CREATE OR REPLACE PACKAGE error_pkg IS
    e_fk_err      EXCEPTION;
    e_seq_nbr_err EXCEPTION;
    PRAGMA EXCEPTION_INIT (e_fk_err, -2292);
    PRAGMA EXCEPTION_INIT (e_seq_nbr_err, -2277);
    ...
END error_pkg;
/
```

Standardizing Exception Handling

Consider writing a subprogram for common exception handling to:

- Display errors based on `SQLCODE` and `SQLERRM` values for exceptions
- Track run-time errors easily by using parameters in your code to identify:
 - The procedure in which the error occurred
 - The location (line number) of the error
 - `RAISE_APPLICATION_ERROR` using stack trace capabilities, with the third argument set to `TRUE`

Standardizing Constants

For programs that use local variables whose values should not change:

- Convert the variables to constants to reduce maintenance and debugging
- Create one central package specification and place all constants in it

```
CREATE OR REPLACE PACKAGE constant_pkg IS  
    c_order_received CONSTANT VARCHAR(2) := 'OR';  
    c_order_shipped   CONSTANT VARCHAR(2) := 'OS';  
    c_min_sal         CONSTANT NUMBER(3)  := 900;  
END constant_pkg;
```

Local Subprograms

A local subprogram is a PROCEDURE or FUNCTION defined at the end of the declarative section.

```
CREATE PROCEDURE employee_sal(p_id NUMBER) IS
    v_emp employees%ROWTYPE;
    FUNCTION tax(p_salary VARCHAR2) RETURN NUMBER IS
    BEGIN
        RETURN p_salary * 0.825;
    END tax;
BEGIN
    SELECT * INTO v_emp
    FROM EMPLOYEES WHERE employee_id = p_id;
    DBMS_OUTPUT.PUT_LINE('Tax: ' || tax(v_emp.salary));
END;
/
EXECUTE employee_sal(100)
```

```
PROCEDURE employee_sal(p_id Compiled.
anonymous block completed
Tax: 19800
```

Definer's Rights Versus Invoker's Rights

Definer's rights:

- Used prior to Oracle8i
- Programs execute with the privileges of the creating user.
- User does not require privileges on underlying objects that the procedure accesses. User requires privilege only to execute a procedure.

Invoker's rights:

- Introduced in Oracle8i
- Programs execute with the privileges of the calling user.
- User requires privileges on the underlying objects that the procedure accesses.

Specifying Invoker's Rights: Setting AUTHID to CURRENT_USER

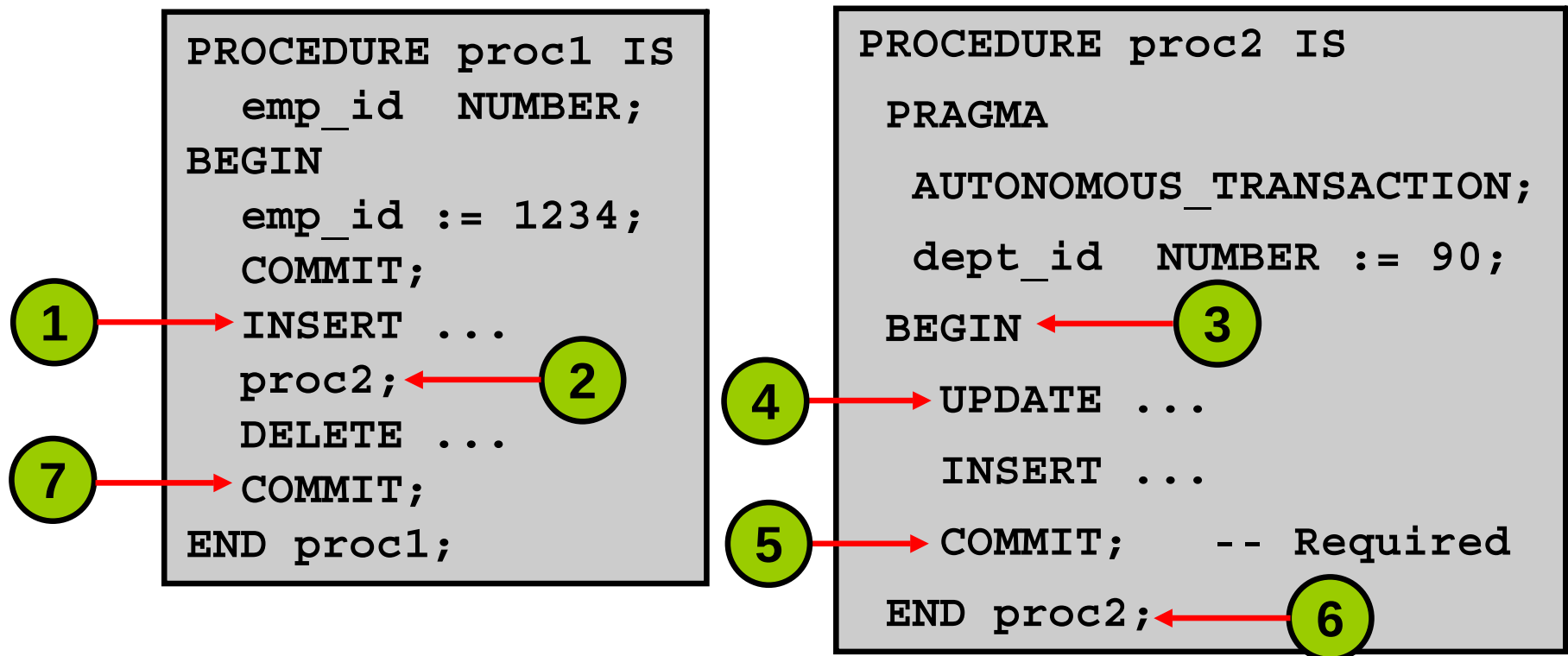
```
CREATE OR REPLACE PROCEDURE add_dept(  
    p_id NUMBER, p_name VARCHAR2) AUTHID CURRENT_USER IS  
BEGIN  
    INSERT INTO departments  
    VALUES (p_id, p_name, NULL, NULL);  
END;
```

When used with stand-alone functions, procedures, or packages:

- Names used in queries, DML, Native Dynamic SQL, and DBMS_SQL package are resolved in the invoker's schema
- Calls to other packages, functions, and procedures are resolved in the definer's schema

Autonomous Transactions

- Are independent transactions started by another main transaction
- Are specified with PRAGMA AUTONOMOUS_TRANSACTION



Features of Autonomous Transactions

- Are independent of the main transaction
- Suspend the calling transaction until the autonomous transactions are completed
- Are not nested transactions
- Do not roll back if the main transaction rolls back
- Enable the changes to become visible to other transactions upon a commit
- Are started and ended by individual subprograms and not by nested or anonymous PL/SQL blocks

Using Autonomous Transactions: Example

```
PROCEDURE bank_trans(p_cardnbr NUMBER, p_loc NUMBER) IS
BEGIN
    log_usage(p_cardnbr, p_loc);
    INSERT INTO txn VALUES (9001, 1000,...);
END bank_trans;
```

```
PROCEDURE log_usage (p_card_id NUMBER, p_loc NUMBER)
IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO usage          -- usage is an existing table
    VALUES (p_card_id, p_loc);
    COMMIT;
END log_usage;
```

Lesson Agenda

- Standardizing constants and exceptions, using local subprograms, controlling the run-time privileges of a subprogram, and performing autonomous transactions
- Using the `NOCOPY` and the `PARALLEL ENABLE` hints, the cross-session PL/SQL function result cache, and the `DETERMINISTIC` clause
- Using bulk binding and the `RETURNING` clause with DML

Using the NOCOPY Hint

- Allows the PL/SQL compiler to pass OUT and IN OUT parameters by reference rather than by value
- Enhances performance by reducing overhead when passing parameters

```
DECLARE
  TYPE      rec_emp_type IS TABLE OF employees%ROWTYPE;
  rec_emp   rec_emp_type;
  PROCEDURE populate(p_tab IN OUT NOCOPY emptabtype) IS
    BEGIN
      . . .
    END;
BEGIN
  populate(rec_emp);
END;
/
```

Effects of the NOCOPY Hint

- If the subprogram exits with an exception that is not handled:
 - You cannot rely on the values of the actual parameters passed to a NOCOPY parameter
 - Any incomplete modifications are not “rolled back”
- The remote procedure call (RPC) protocol enables you to pass parameters only by value.

When Does the PL/SQL Compiler Ignore the NOCOPY Hint?

The NOCOPY hint has no effect if:

- The actual parameter:
 - Is an element of an index-by table
 - Is constrained (for example, by scale or NOT NULL)
 - And formal parameter are records, where one or both records were declared by using %ROWTYPE or %TYPE, and constraints on corresponding fields in the records differ
 - Requires an implicit data type conversion
- The subprogram is involved in an external or remote procedure call

Using the `PARALLEL_ENABLE` Hint

- Can be used in functions as an optimization hint
- Indicates that a function can be used in a parallelized query or parallelized DML statement

```
CREATE OR REPLACE FUNCTION f2 (p_p1 NUMBER)
  RETURN NUMBER PARALLEL_ENABLE IS
BEGIN
  RETURN p_p1 * 2;
END f2;
```

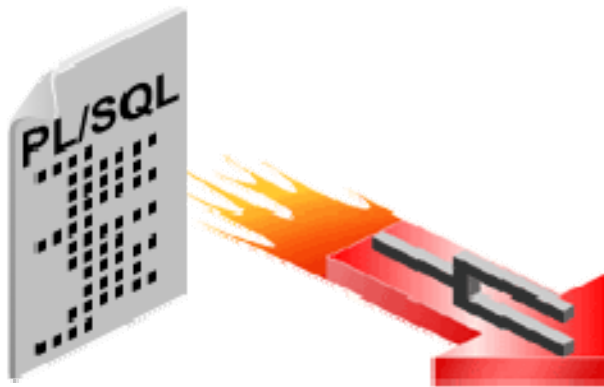

Using the Cross-Session PL/SQL Function Result Cache

- Each time a result-cached PL/SQL function is called with different parameter values, those parameters and their results are stored in cache.
- The function result cache is stored in a shared global area (SGA), making it available to any session that runs your application.
- Subsequent calls to the same function with the same parameters uses the result from cache.
- Performance and scalability are improved.
- This feature is used with functions that are called frequently and dependent on information that changes infrequently.

Enabling Result-Caching for a Function

You can make a function result-cached as follows:

- Include the `RESULT_CACHE` clause in the following:
 - The function declaration
 - The function definition
- Include an optional `RELIES_ON` clause to specify any tables or views on which the function results depend.



Declaring and Defining a Result-Cached Function: Example

```
CREATE OR REPLACE FUNCTION emp_hire_date (p_emp_id  
    NUMBER) RETURN VARCHAR  
    RESULT_CACHE RELIES_ON (employees) IS  
    v_date_hired DATE;  
BEGIN  
    SELECT hire_date INTO v_date_hired  
    FROM HR.Employees  
    WHERE Employee_ID = p_emp_ID;  
    RETURN to_char(v_date_hired);  
END;
```

Using the DETERMINISTIC Clause with Functions

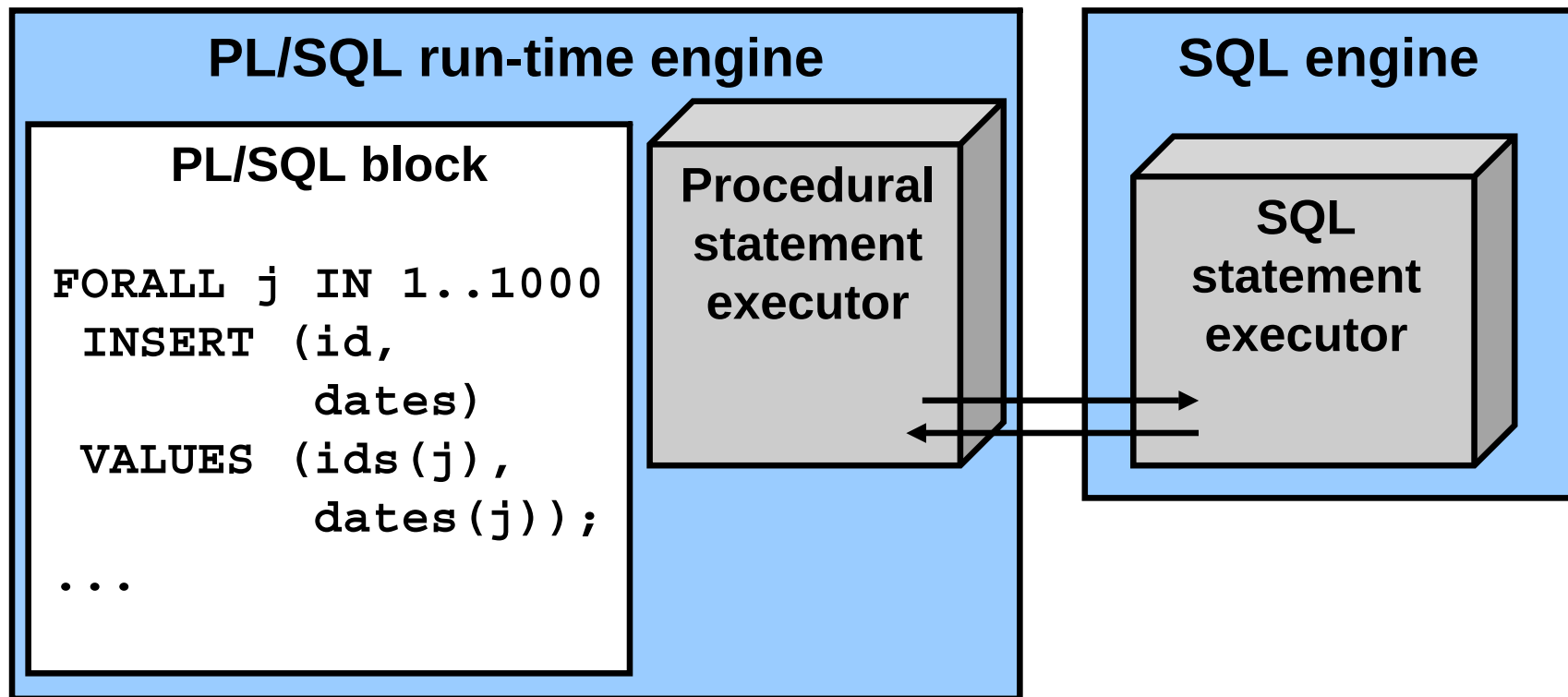
- Specify DETERMINISTIC to indicate that the function returns the same result value whenever it is called with the same values for its arguments.
- This helps the optimizer avoid redundant function calls.
- If a function was called previously with the same arguments, the optimizer can elect to use the previous result.
- Do not specify DETERMINISTIC for a function whose result depends on the state of session variables or schema objects.

Lesson Agenda

- Standardizing constants and exceptions, using local subprograms, controlling the run-time privileges of a subprogram, and performing autonomous transactions
- Using the `NOCOPY` and the `PARALLEL ENABLE` hints, the cross-session PL/SQL function result cache, and the `DETERMINISTIC` clause
- Using bulk binding and the `RETURNING` clause with DML

Bulk Binding

Binds whole arrays of values in a single operation, rather than using a loop to perform a `FETCH`, `INSERT`, `UPDATE`, and `DELETE` operation multiple times



Using Bulk Binding: Syntax and Keywords

- The `FORALL` keyword instructs the *PL/SQL engine* to bulk bind input collections before sending them to the SQL engine.

```
FORALL index IN lower_bound .. upper_bound  
  [SAVE EXCEPTIONS]  
  sql_statement;
```

- The `BULK COLLECT` keyword instructs the *SQL engine* to bulk bind output collections before returning them to the PL/SQL engine.

```
... BULK COLLECT INTO  
    collection_name[,collection_name] ...
```

Bulk Binding FORALL: Example

```
CREATE PROCEDURE raise_salary(p_percent NUMBER) IS
  TYPE numlist_type IS TABLE OF NUMBER
    INDEX BY BINARY_INTEGER;
  v_id  numlist_type; -- collection
BEGIN
  v_id(1) := 100; v_id(2) := 102; v_id(3) := 104; v_id(4) := 110;
  -- bulk-bind the PL/SQL table
  FORALL i IN v_id.FIRST .. v_id.LAST
    UPDATE employees
      SET salary = (1 + p_percent/100) * salary
      WHERE employee_id = v_id(i);
END;
/
```

```
EXECUTE raise_salary(10)
```

```
PL/SQL procedure successfully completed.
```


Using BULK COLLECT INTO with Queries

The SELECT statement has been enhanced to support the BULK COLLECT INTO syntax.

```
CREATE PROCEDURE get_departments(p_loc NUMBER) IS
  TYPE dept_tab_type IS
    TABLE OF departments%ROWTYPE;
  v_depts dept_tab_type;
BEGIN
  SELECT * BULK COLLECT INTO v_depts
  FROM departments
  WHERE location_id = p_loc;
  FOR i IN 1 .. v_depts.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_depts(i).department_id
      || ' ' || v_depts(i).department_name);
  END LOOP;
END;
```

Using BULK COLLECT INTO with Cursors

The FETCH statement has been enhanced to support the BULK COLLECT INTO syntax.

```
CREATE PROCEDURE get_departments(p_loc NUMBER) IS
  CURSOR cur_dept IS
    SELECT * FROM departments
    WHERE location_id = p_loc;
  TYPE dept_tab_type IS TABLE OF cur_dept%ROWTYPE;
  v_depts dept_tab_type;
BEGIN
  OPEN cur_dept;
  FETCH cur_dept BULK COLLECT INTO v_depts;
  CLOSE cur_dept;
  FOR i IN 1 .. v_depts.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_depts(i).department_id
      || ' ' || v_depts(i).department_name);
  END LOOP;
END;
```

Using BULK COLLECT INTO with a RETURNING Clause

```
CREATE PROCEDURE raise_salary(p_rate NUMBER) IS
  TYPE emplist_type IS TABLE OF NUMBER;
  TYPE numlist_type IS TABLE OF employees.salary%TYPE
    INDEX BY BINARY_INTEGER;
  v_emp_ids  emplist_type :=
    emplist_type(100,101,102,104);
  v_new_sals numlist_type;
BEGIN
  FORALL i IN v_emp_ids.FIRST .. v_emp_ids.LAST
    UPDATE employees
      SET commission_pct = p_rate * salary
      WHERE employee_id = v_emp_ids(i)
      RETURNING salary BULK COLLECT INTO v_new_sals;
  FOR i IN 1 .. v_new_sals.COUNT LOOP ...
END;
```

FORALL Support for Sparse Collections

```
-- The new INDICES OF syntax allows the bound arrays  
-- themselves to be sparse.
```

```
FORALL index_name IN INDICES OF sparse_array_name  
    BETWEEN LOWER_BOUND AND UPPER_BOUND -- optional  
    SAVE EXCEPTIONS -- optional, but recommended  
    INSERT INTO table_name VALUES  
    sparse_array(index_name);
```

```
-- The new VALUES OF syntax lets you indicate a subset  
-- of the binding arrays.
```

```
FORALL index_name IN VALUES OF index_array_name  
    SAVE EXCEPTIONS -- optional, but recommended  
    INSERT INTO table_name VALUES  
    binding_array_name(index_name);
```

Using Bulk Binds in Sparse Collections

The typical application for this feature is an order entry and order processing system where:

- Users enter orders through the Web
- Orders are placed in a staging table before validation
- Data is later validated based on complex business rules (usually implemented programmatically using PL/SQL)
- Invalid orders are separated from valid ones
- Valid orders are inserted into a permanent table for processing

Using Bulk Bind with Index Array

```
CREATE OR REPLACE PROCEDURE ins_emp2 AS
  TYPE emptab_type IS TABLE OF employees%ROWTYPE;
  v_emp emptab_type;
  TYPE values_of_tab_type IS TABLE OF PLS_INTEGER
    INDEX BY PLS_INTEGER;
  v_num      values_of_tab_type;
  . . .
BEGIN
  . . .
  FORALL k IN VALUES OF v_num
    INSERT INTO new_employees VALUES v_emp(k);
END;
```

Using the RETURNING Clause

- Improves performance by returning column values with INSERT, UPDATE, and DELETE statements
- Eliminates the need for a SELECT statement

```
CREATE PROCEDURE update_salary(p_emp_id NUMBER) IS
    v_name      employees.last_name%TYPE;
    v_new_sal    employees.salary%TYPE;
BEGIN
    UPDATE employees
        SET salary = salary * 1.1
        WHERE employee_id = p_emp_id
        RETURNING last_name, salary INTO name, v_new_sal;
END update_salary;
/
```

Quiz

The `NOCOPY` hint allows the PL/SQL compiler to pass `OUT` and `IN OUT` parameters by reference rather than by value. This enhances performance by reducing overhead when passing parameters

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Create standard constants and exceptions
- Write and call local subprograms
- Control the run-time privileges of a subprogram
- Perform autonomous transactions
- Pass parameters by reference using a `NOCOPY` hint
- Use the `PARALLEL ENABLE` hint for optimization
- Use the cross-session PL/SQL function result cache
- Use the `DETERMINISTIC` clause with functions
- Use bulk binding and the `RETURNING` clause with DML

Practice 8: Overview

This practice covers the following topics:

- Creating a package that uses bulk fetch operations
- Creating a local subprogram to perform an autonomous transaction to audit a business operation



Creating Triggers

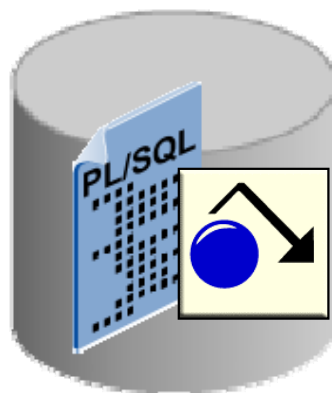
Objectives

After completing this lesson, you should be able to do the following:

- Describe database triggers and their uses
- Describe the different types of triggers
- Create database triggers
- Describe database trigger-firing rules
- Remove database triggers
- Display trigger information

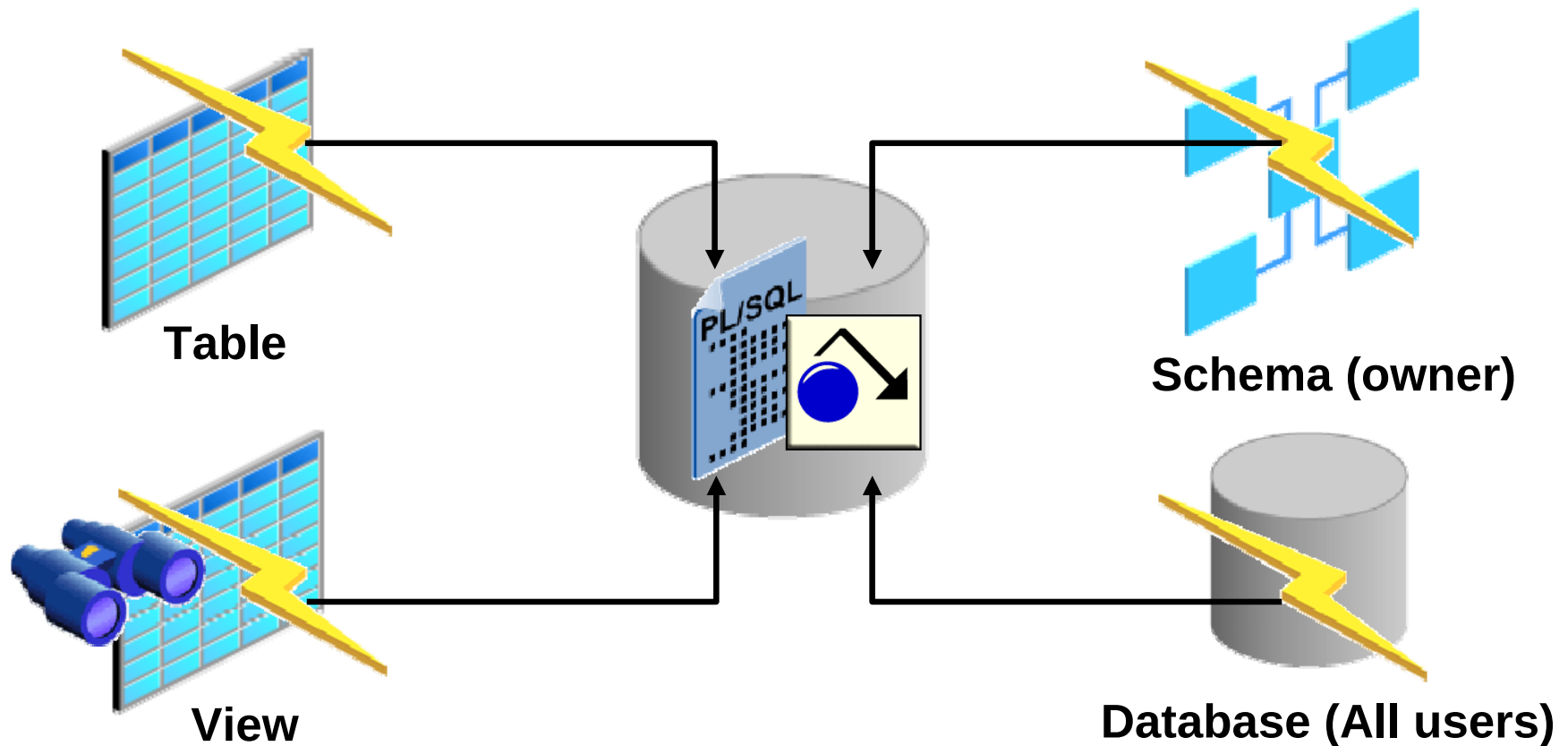
What Are Triggers?

- A trigger is a PL/SQL block that is stored in the database and fired (executed) in response to a specified event.
- The Oracle database automatically executes a trigger when specified conditions occur.



Defining Triggers

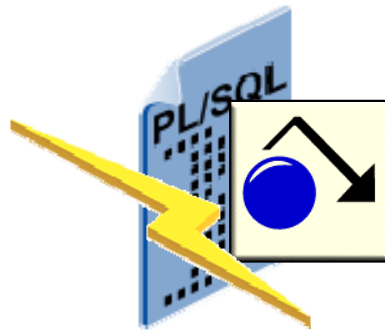
A trigger can be defined on the table, view, schema (schema owner), or database (all users).



Trigger Event Types

You can write triggers that fire whenever one of the following operations occurs in the database:

- A database manipulation (DML) statement (DELETE, INSERT, or UPDATE).
- A database definition (DDL) statement (CREATE, ALTER, or DROP).
- A database operation such as SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN.

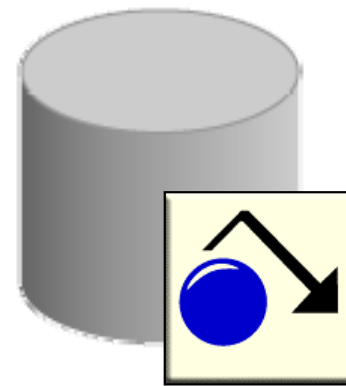


Application and Database Triggers

- Database trigger (covered in this course):
 - Fires whenever a DML, a DLL, or system event occurs on a schema or database
- Application trigger:
 - Fires whenever an event occurs within a particular application



Application Trigger



Database Trigger

Business Application Scenarios for Implementing Triggers

You can use triggers for:

- Security
- Auditing
- Data integrity
- Referential integrity
- Table replication
- Computing derived data automatically
- Event logging

Available Trigger Types

- Simple DML triggers
 - BEFORE
 - AFTER
 - INSTEAD OF
- Compound triggers
- Non-DML triggers
 - DDL event triggers
 - Database event triggers

Trigger Event Types and Body

- A trigger event type determines which DML statement causes the trigger to execute. The possible events are:
 - INSERT
 - UPDATE [OF column]
 - DELETE
- A trigger body determines what action is performed and is a PL/SQL block or a CALL to a procedure

Creating DML Triggers Using the CREATE TRIGGER Statement

```
CREATE [OR REPLACE] TRIGGER trigger_name
timing -- when to fire the trigger
event1 [OR event2 OR event3]
ON object_name
[REFERENCING OLD AS old / NEW AS new]
FOR EACH ROW -- default is statement level trigger
WHEN (condition)]
DECLARE]
BEGIN
... trigger_body -- executable statements
[EXCEPTION . . .]
END [trigger_name];
```

timing = BEFORE | AFTER | INSTEAD OF

event = INSERT | DELETE | UPDATE | UPDATE OF *column_list*

Specifying the Trigger Firing (Timing)

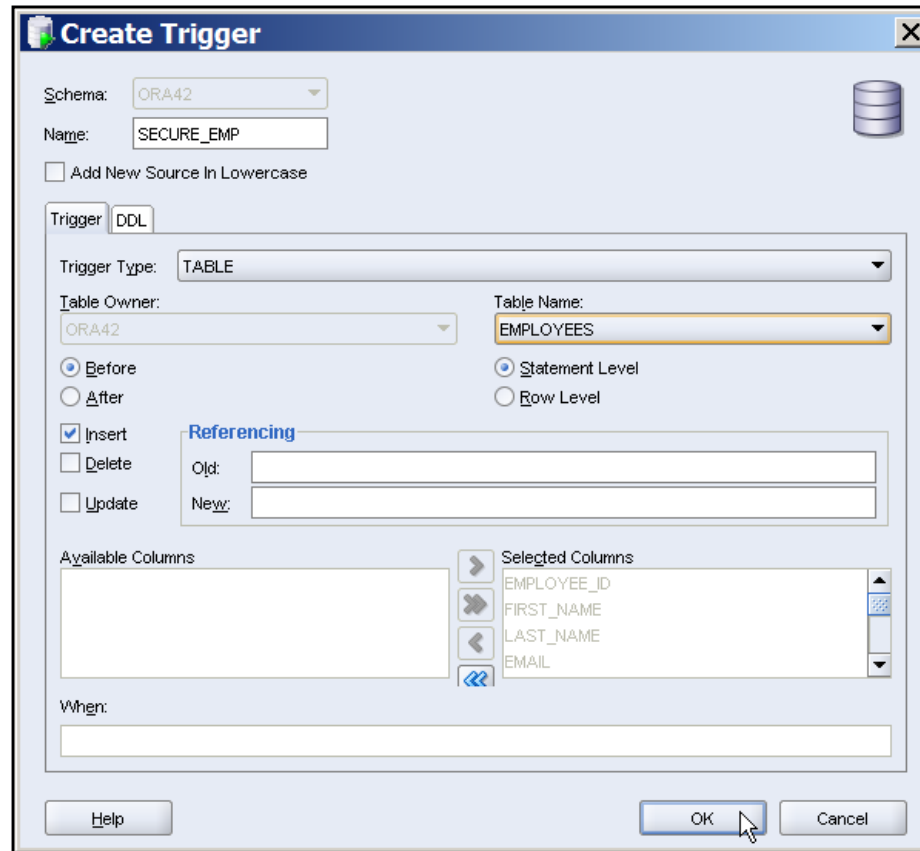
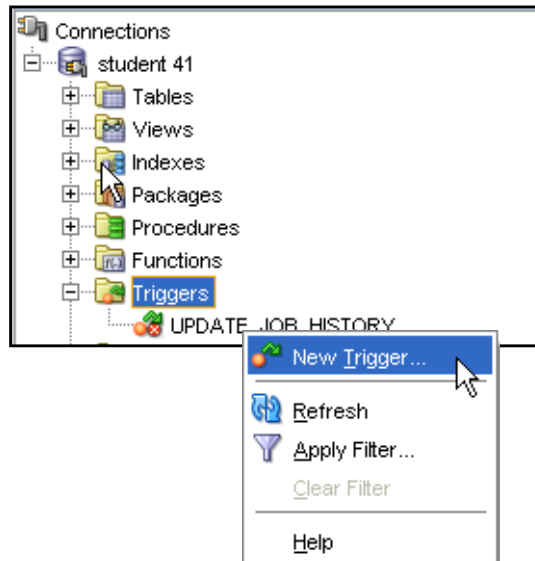
You can specify the trigger timing as to whether to run the trigger's action before or after the triggering statement:

- **BEFORE:** Executes the trigger body before the triggering DML event on a table.
- **AFTER:** Execute the trigger body after the triggering DML event on a table.
- **INSTEAD OF:** Execute the trigger body instead of the triggering statement. This is used for views that are not otherwise modifiable.

Statement-Level Triggers Versus Row-Level Triggers

Statement-Level Triggers	Row-Level Triggers
Is the default when creating a trigger	Use the <code>FOR EACH ROW</code> clause when creating a trigger.
Fires once for the triggering event	Fires once for each row affected by the triggering event
Fires once even if no rows are affected	Does not fire if the triggering event does not affect any rows

Creating DML Triggers Using SQL Developer



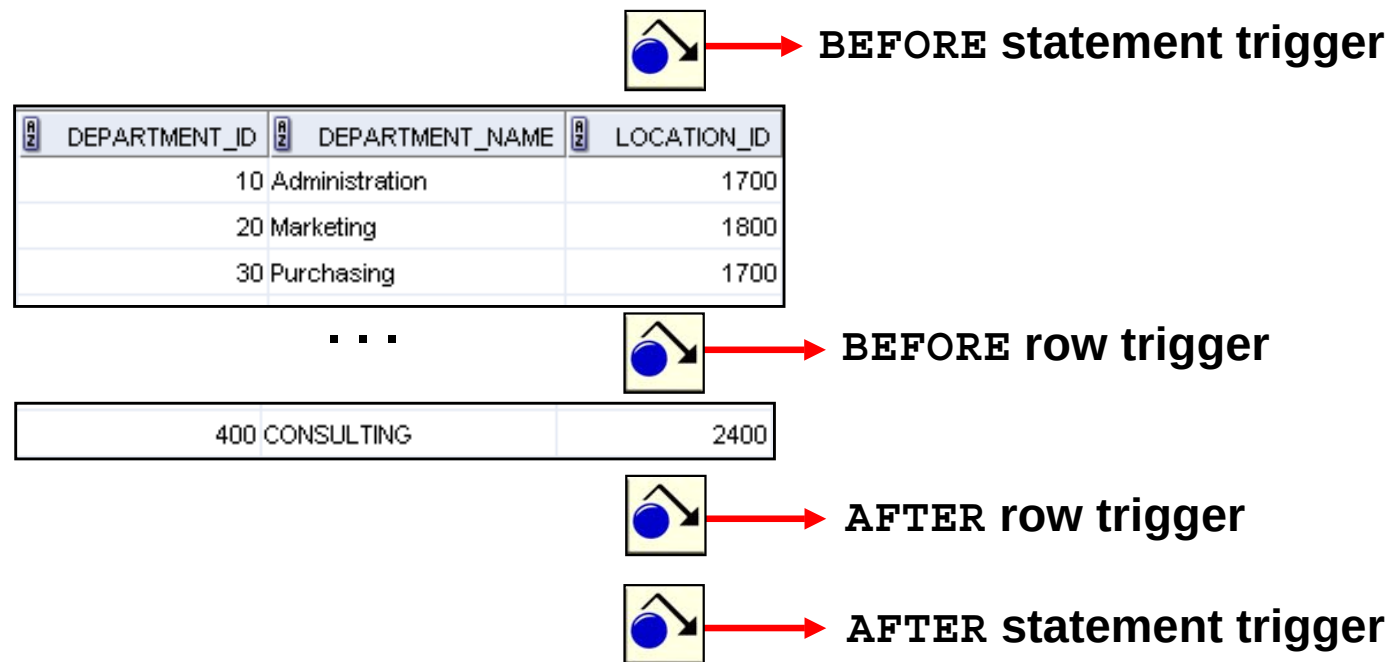
```
CREATE OR REPLACE TRIGGER secure_emp
BEFORE INSERT ON employees BEGIN
  IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
     (TO_CHAR(SYSDATE,'HH24:MI') NOT BETWEEN '08:00' AND '18:00') THEN
    RAISE_APPLICATION_ERROR(-20500, 'You may insert' ||
      ' into EMPLOYEES table only during ' || ' business hours.');
```

```
END IF;
END;
```

Trigger-Firing Sequence: Single-Row Manipulation

Use the following firing sequence for a trigger on a table when a single row is manipulated:

```
INSERT INTO departments  
  (department_id, department_name, location_id)  
VALUES (400, 'CONSULTING', 2400);
```



Trigger-Firing Sequence: Multirow Manipulation

Use the following firing sequence for a trigger on a table when many rows are manipulated:

```
UPDATE employees  
  SET salary = salary * 1.1  
  WHERE department_id = 30;
```



BEFORE statement trigger

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
114	Raphaely	30
115	Khoo	30
116	Baida	30
117	Tobias	30
118	Himuro	30
119	Colmenares	30

BEFORE row trigger

AFTER row trigger

...

BEFORE row trigger

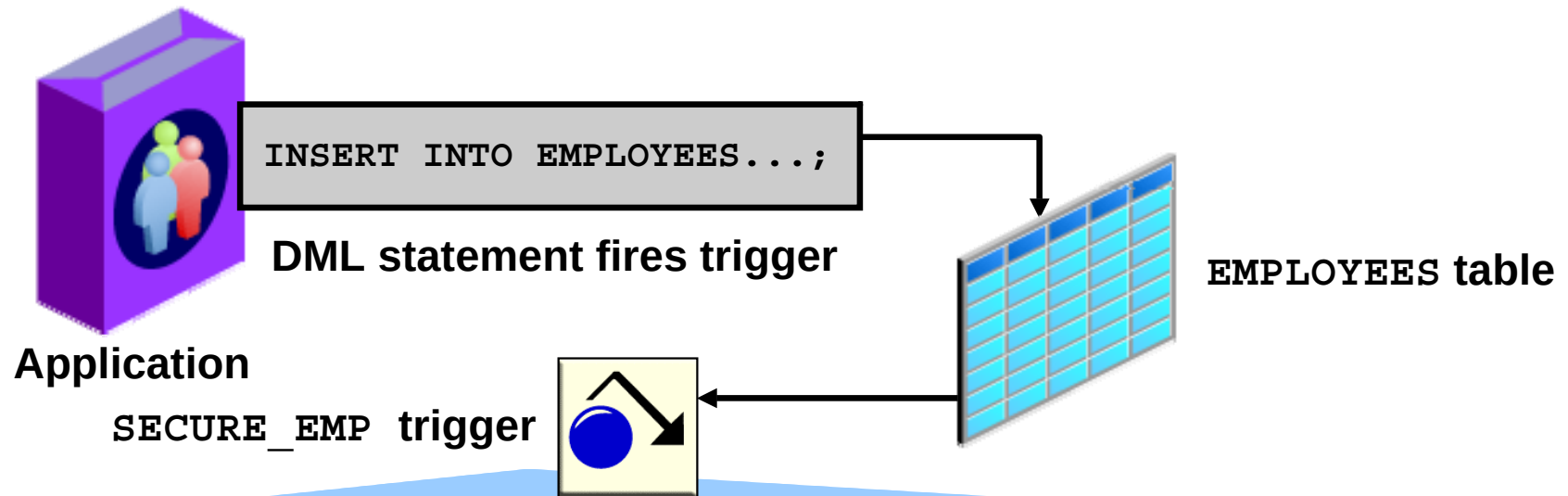
AFTER row trigger

...



AFTER statement trigger

Creating a DML Statement Trigger Example: SECURE_EMP



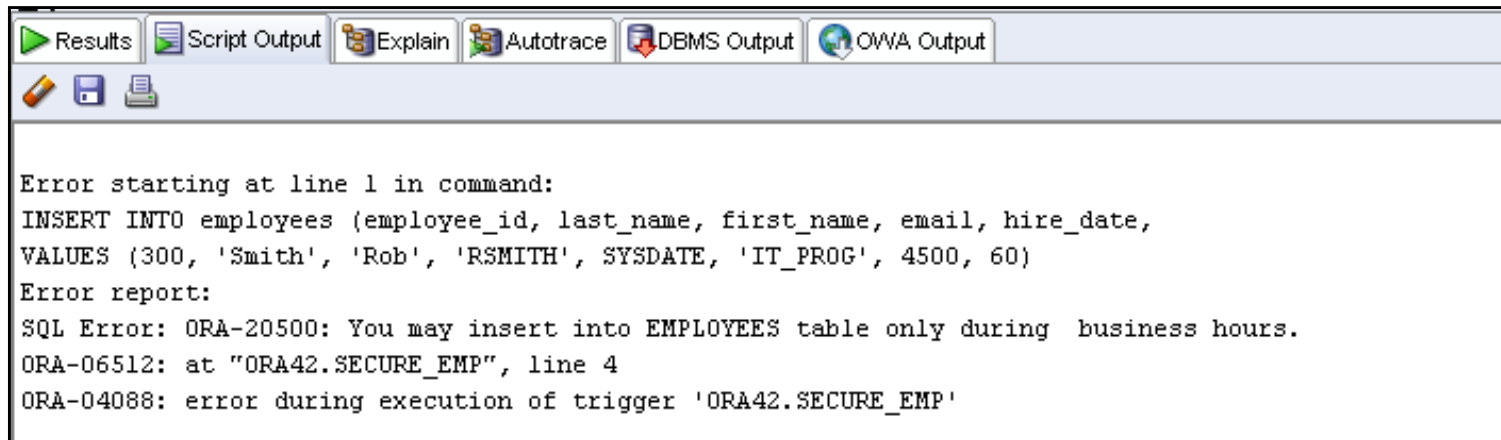
```
CREATE OR REPLACE TRIGGER secure_emp
BEFORE INSERT ON employees
BEGIN
  IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
    (TO_CHAR(SYSDATE,'HH24:MI')
      NOT BETWEEN '08:00' AND '18:00') THEN
    RAISE_APPLICATION_ERROR(-20500, 'You may insert'
      || ' into EMPLOYEES table only during '
      || ' normal business hours.');
```

END IF;

END;

Testing Trigger SECURE_EMP

```
INSERT INTO employees (employee_id, last_name,  
    first_name, email, hire_date,  
    job_id, salary, department_id)  
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE,  
    'IT_PROG', 4500, 60);
```



The screenshot shows a window with a toolbar at the top containing icons for Results, Script Output, Explain, Autotrace, DBMS Output, and OWA Output. Below the toolbar, the main text area displays the following error message:

```
Error starting at line 1 in command:  
INSERT INTO employees (employee_id, last_name, first_name, email, hire_date,  
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE, 'IT_PROG', 4500, 60)  
Error report:  
SQL Error: ORA-20500: You may insert into EMPLOYEES table only during business hours.  
ORA-06512: at "ORA42.SECURE_EMP", line 4  
ORA-04088: error during execution of trigger 'ORA42.SECURE_EMP'
```

Using Conditional Predicates

```
CREATE OR REPLACE TRIGGER secure_emp BEFORE
INSERT OR UPDATE OR DELETE ON employees
BEGIN
    IF (TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN')) OR
       (TO_CHAR(SYSDATE, 'HH24')
        NOT BETWEEN '08' AND '18') THEN
        IF DELETING THEN RAISE_APPLICATION_ERROR(
            -20502, 'You may delete from EMPLOYEES table' ||
            'only during normal business hours. ');
        ELSIF INSERTING THEN RAISE_APPLICATION_ERROR(
            -20500, 'You may insert into EMPLOYEES table' ||
            'only during normal business hours. ');
        ELSIF UPDATING ('SALARY') THEN
            RAISE_APPLICATION_ERROR(-20503, 'You may ' ||
            'update SALARY only normal during business hours. ');
        ELSE RAISE_APPLICATION_ERROR(-20504, 'You may' ||
            ' update EMPLOYEES table only during' ||
            ' normal business hours. ');
    END IF;
END IF;
END;
```

Creating a DML Row Trigger

```
CREATE OR REPLACE TRIGGER restrict_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF NOT (:NEW.job_id IN ('AD_PRES', 'AD_VP'))
        AND :NEW.salary > 15000 THEN
        RAISE_APPLICATION_ERROR (-20202,
            'Employee cannot earn more than $15,000.');
```

END IF;

END; /

```
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell';
```

```
Error starting at line 1 in command:
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell'
Error report:
SQL Error: ORA-20202: Employee cannot earn more than $15,000.
ORA-06512: at "ORA62.RESTRICT_SALARY", line 4
ORA-04088: error during execution of trigger 'ORA62.RESTRICT_SALARY'
```

Using OLD and NEW Qualifiers

- When a row-level trigger fires, the PL/SQL run-time engine creates and populates two data structures:
 - OLD: Stores the original values of the record processed by the trigger
 - NEW: Contains the new values
- NEW and OLD have the same structure as a record declared using the %ROWTYPE on the table to which the trigger is attached.

Data Operations	Old Value	New Value
INSERT	NULL	Inserted value
UPDATE	Value before update	Value after update
DELETE	Value before delete	NULL

Using OLD and NEW Qualifiers: Example

```
CREATE OR REPLACE TRIGGER audit_emp_values
AFTER DELETE OR INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_emp(user_name, time_stamp, id,
        old_last_name, new_last_name, old_title,
        new_title, old_salary, new_salary)
    VALUES (USER, SYSDATE, :OLD.employee_id,
        :OLD.last_name, :NEW.last_name, :OLD.job_id,
        :NEW.job_id, :OLD.salary, :NEW.salary);
END;
/
```

Using OLD and NEW Qualifiers: Example Using AUDIT_EMP

```

INSERT INTO employees (employee_id, last_name, job_id,
salary, email, hire_date)
VALUES (999, 'Temp emp', 'SA_REP', 6000, 'TEMPEMP',
TRUNC(SYSDATE));
/
UPDATE employees
  SET salary = 7000, last_name = 'Smith'
  WHERE employee_id = 999;
/
SELECT *
FROM audit_emp;

```

	 USER_NAME	 TIME_STAMP	 ID	 OLD_LAST_NAME	 NEW_LAST_NAME	 OLD_TITLE	 NEW_TITLE	 OLD_SALARY	 NEW_SALARY
1	ORA62	27-JUN-07	(null)	(null)	Temp emp	(null)	SA_REP	(null)	6000
2	ORA62	27-JUN-07	999	Temp emp	Smith	SA_REP	SA_REP	6000	7000

Using the WHEN Clause to Fire a Row Trigger Based on a Condition

```
CREATE OR REPLACE TRIGGER derive_commission_pct
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
WHEN (NEW.job_id = 'SA_REP')
BEGIN
    IF INSERTING THEN
        :NEW.commission_pct := 0;
    ELSIF :OLD.commission_pct IS NULL THEN
        :NEW.commission_pct := 0;
    ELSE
        :NEW.commission_pct := :OLD.commission_pct+0.05;
    END IF;
END;
/
```

Summary of the Trigger Execution Model

1. Execute all BEFORE STATEMENT triggers.
2. Loop *for each row* affected by the SQL statement:
 - a. Execute all BEFORE ROW triggers *for that row*.
 - b. Execute the DML statement and perform integrity constraint checking *for that row*.
 - c. Execute all AFTER ROW triggers *for that row*.
3. Execute all AFTER STATEMENT triggers.

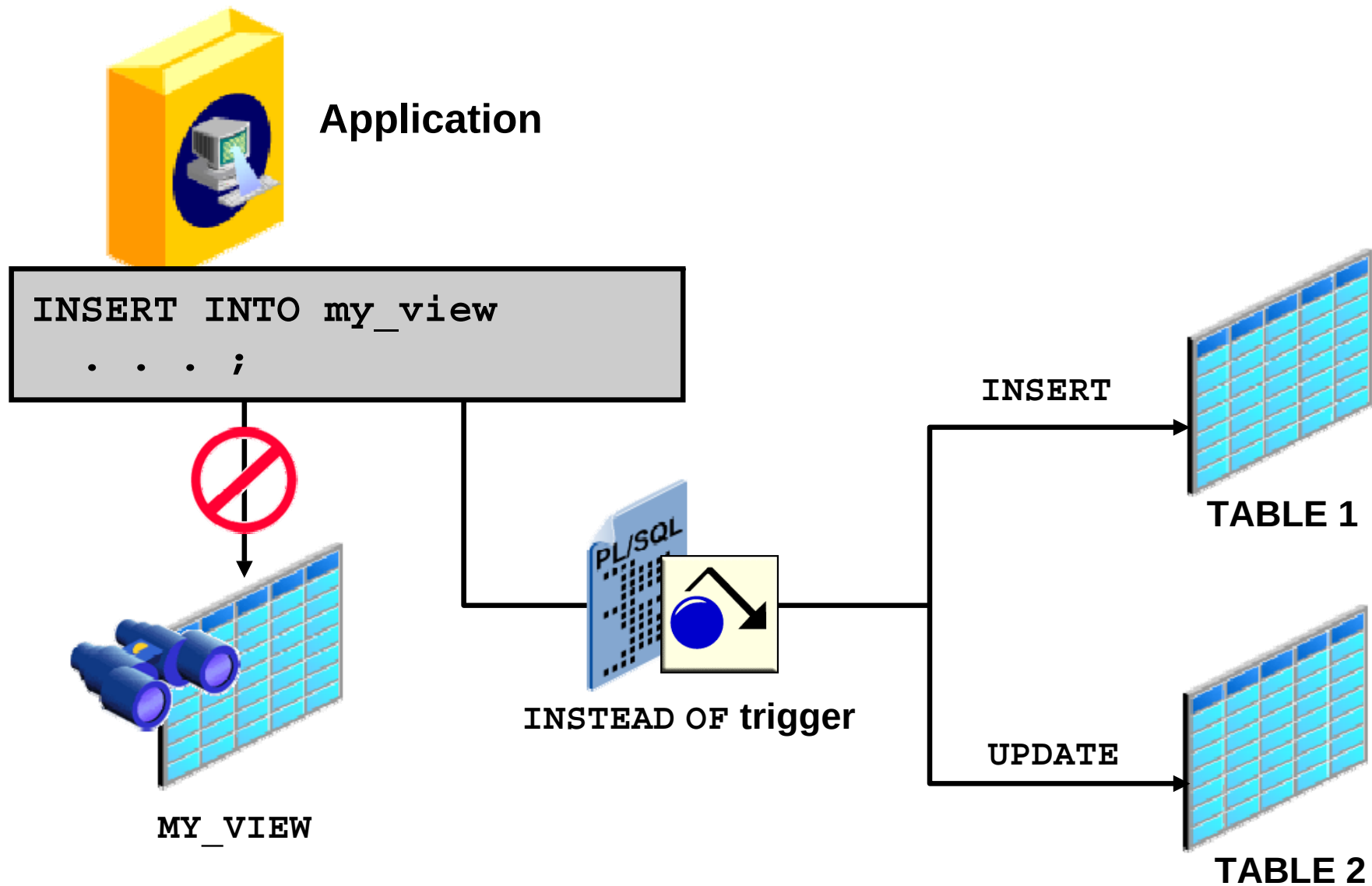
Implementing an Integrity Constraint with an After Trigger

```
-- Integrity constraint violation error -2992 raised.  
UPDATE employees SET department_id = 999  
WHERE employee_id = 170;
```

```
CREATE OR REPLACE TRIGGER employee_dept_fk_trg  
AFTER UPDATE OF department_id  
ON employees FOR EACH ROW  
BEGIN  
    INSERT INTO departments VALUES(:new.department_id,  
                                     'Dept ' || :new.department_id, NULL, NULL);  
EXCEPTION  
    WHEN DUP_VAL_ON_INDEX THEN  
        NULL; -- mask exception if department exists  
END; /
```

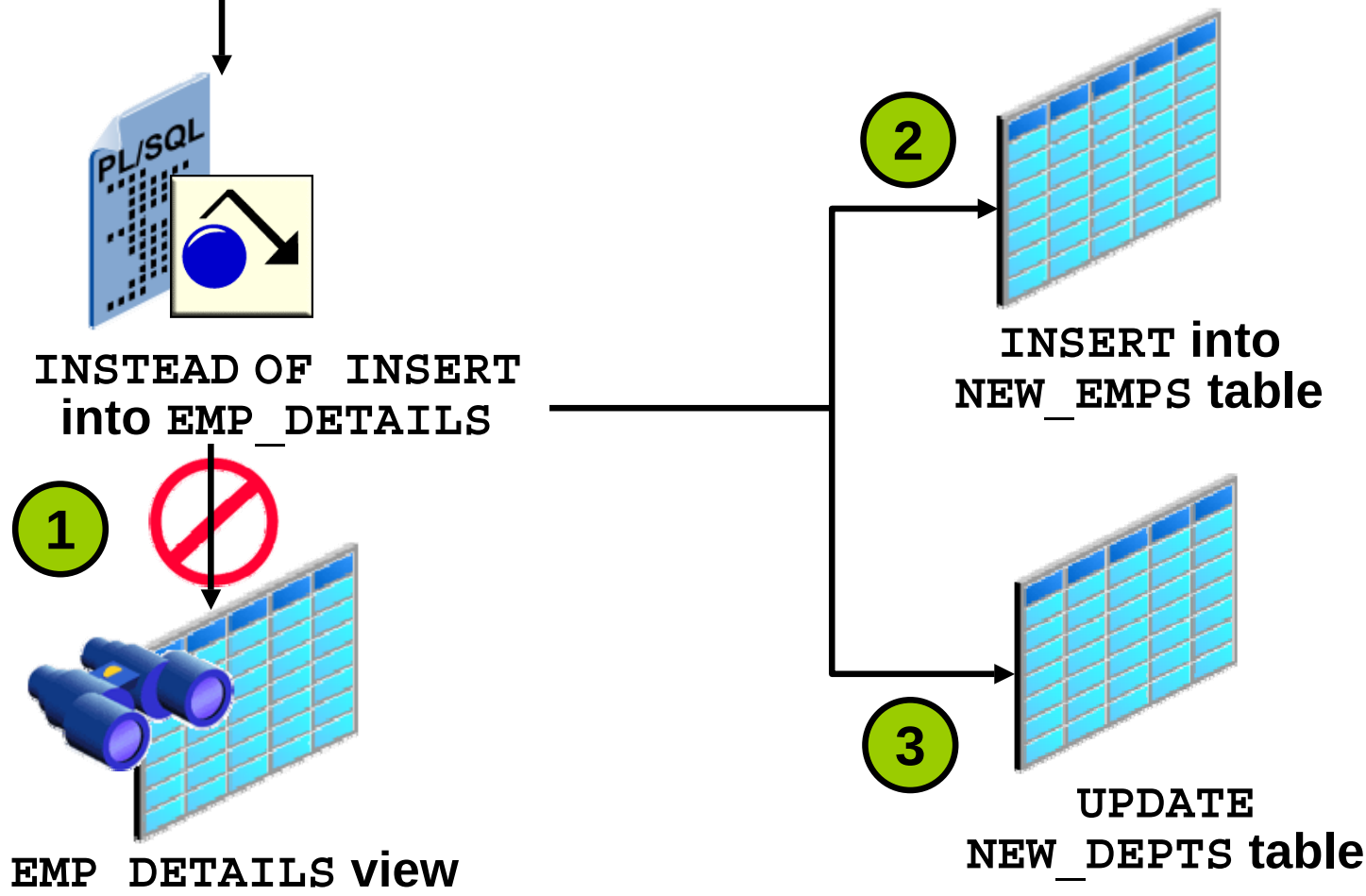
```
-- Successful after trigger is fired  
UPDATE employees SET department_id = 999  
WHERE employee_id = 170;
```

INSTEAD OF Triggers



Creating an INSTEAD OF Trigger: Example

```
INSERT INTO emp_details  
VALUES (9001, 'ABBOTT', 3000, 10, 'Administration');
```



Creating an INSTEAD OF Trigger to Perform DML on Complex Views

```
CREATE TABLE new_emps AS
  SELECT employee_id, last_name, salary, department_id
     FROM employees;
```

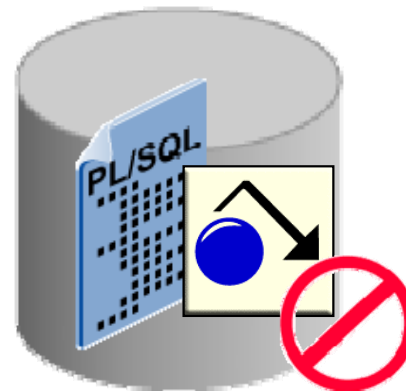
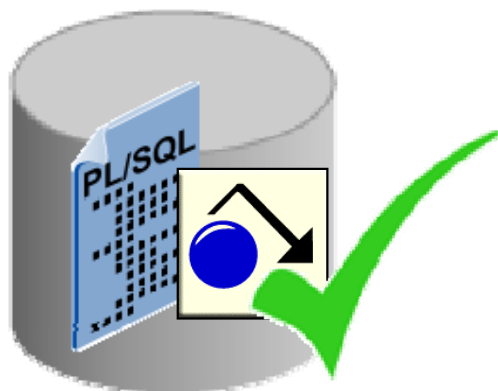
```
CREATE TABLE new_depts AS
  SELECT d.department_id, d.department_name,
         sum(e.salary) dept_sal
     FROM employees e, departments d
    WHERE e.department_id = d.department_id;
```

```
CREATE VIEW emp_details AS
  SELECT e.employee_id, e.last_name, e.salary,
         e.department_id, d.department_name
     FROM employees e, departments d
    WHERE e.department_id = d.department_id
 GROUP BY d.department_id, d.department_name;
```

The Status of a Trigger

A trigger is in either of two distinct modes:

- Enabled: The trigger runs its trigger action if a triggering statement is issued and the trigger restriction (if any) evaluates to true (default).
- Disabled: The trigger does not run its trigger action, even if a triggering statement is issued and the trigger restriction (if any) would evaluate to true.



Creating a Disabled Trigger

- Before Oracle Database 11g, if you created a trigger whose body had a PL/SQL compilation error, then DML to the table failed.
- In Oracle Database 11g, you can create a disabled trigger and then enable it only when you know it will be compiled successfully.

```
CREATE OR REPLACE TRIGGER mytrg
  BEFORE INSERT ON mytable FOR EACH ROW
  DISABLE
BEGIN
  :New.ID := my_seq.Nextval;
  . . .
END;
/
```


Managing Triggers Using the ALTER and DROP SQL Statements

```
-- Disable or reenable a database trigger:
```

```
ALTER TRIGGER trigger_name DISABLE | ENABLE;
```

```
-- Disable or reenable all triggers for a table:
```

```
ALTER TABLE table_name DISABLE | ENABLE ALL TRIGGERS;
```

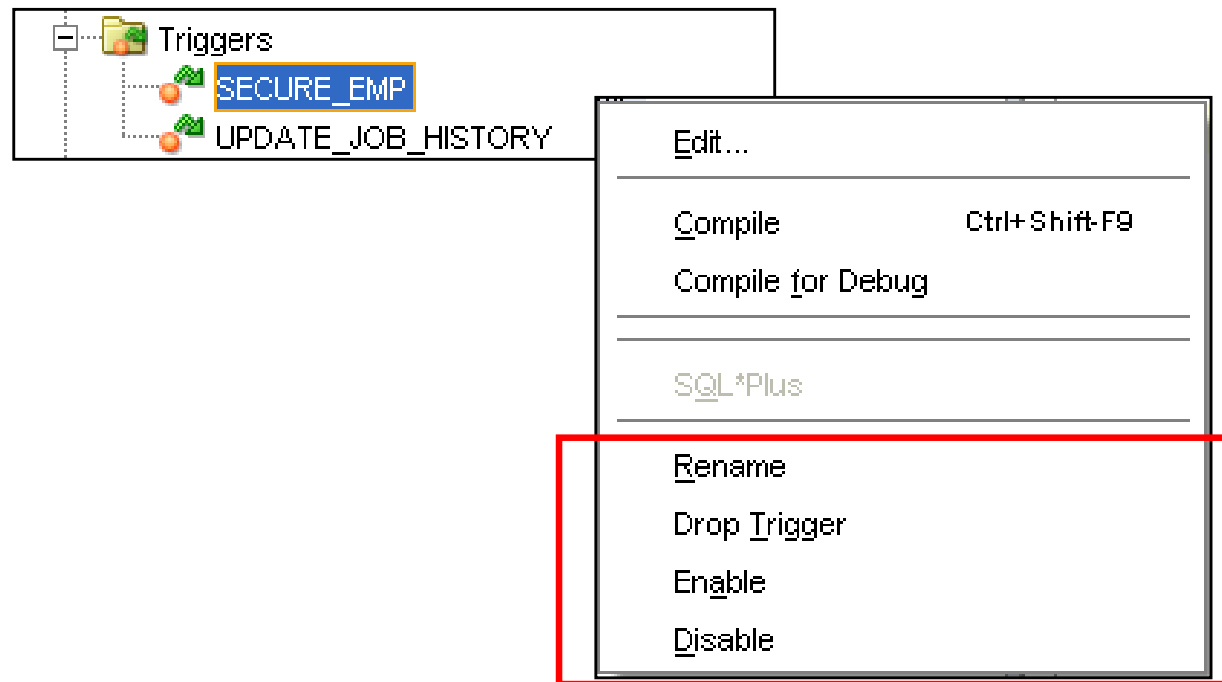
```
-- Recompile a trigger for a table:
```

```
ALTER TRIGGER trigger_name COMPILE;
```

```
-- Remove a trigger from the database:
```

```
DROP TRIGGER trigger_name;
```

Managing Triggers Using SQL Developer



Testing Triggers

- Test each triggering data operation, as well as non-triggering data operations.
- Test each case of the `WHEN` clause.
- Cause the trigger to fire directly from a basic data operation, as well as indirectly from a procedure.
- Test the effect of the trigger on other triggers.
- Test the effect of other triggers on the trigger.

Viewing Trigger Information

You can view the following trigger information:

Data Dictionary View	Description
USER_OBJECTS	Displays object information
USER/ALL/DBA_TRIGGERS	Displays trigger information
USER_ERRORS	Displays PL/SQL syntax errors for a trigger

Using USER_TRIGGERS

```
DESCRIBE user_triggers
```

Name	Null	Type

TRIGGER_NAME		VARCHAR2(30)
TRIGGER_TYPE		VARCHAR2(16)
TRIGGERING_EVENT		VARCHAR2(227)
TABLE_OWNER		VARCHAR2(30)
BASE_OBJECT_TYPE		VARCHAR2(16)
TABLE_NAME		VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
REFERENCING_NAMES		VARCHAR2(128)
WHEN_CLAUSE		VARCHAR2(4000)
STATUS		VARCHAR2(8)
DESCRIPTION		VARCHAR2(4000)
ACTION_TYPE		VARCHAR2(11)
TRIGGER_BODY		LONG()
CROSSEDITION		VARCHAR2(7)
14 rows selected		

```
SELECT trigger_type, trigger_body
FROM user_triggers
WHERE trigger_name = 'SECURE_EMP';
```

Quiz

A triggering event can be one or more of the following:

1. An INSERT, UPDATE, or DELETE statement on a specific table (or view, in some cases)
2. A CREATE, ALTER, or DROP statement on any schema object
3. A database startup or instance shutdown
4. A specific error message or any error message
5. A user logon or logoff

Summary

In this lesson, you should have learned how to:

- Create database triggers that are invoked by DML operations
- Create statement and row trigger types
- Use database trigger-firing rules
- Enable, disable, and manage database triggers
- Develop a strategy for testing triggers
- Remove database triggers

Practice 9 Overview: Creating Statement and Row Triggers

This practice covers the following topics:

- Creating row triggers
- Creating a statement trigger
- Calling procedures from a trigger

10

Creating Compound, DDL, and Event Database Triggers

Objectives

After completing this lesson, you should be able to do the following:

- Describe compound triggers
- Describe mutating tables
- Create triggers on DDL statements
- Create triggers on system events
- Display information about triggers

What Is a Compound Trigger?

A single trigger on a table that allows you to specify actions for each of the following four timing points:

- Before the firing statement
- Before each row that the firing statement affects
- After each row that the firing statement affects
- After the firing statement

Working with Compound Triggers

- The compound trigger body supports a common PL/SQL state that the code for each timing point can access.
- The compound trigger common state is:
 - Established when the triggering statement starts
 - Destroyed when the triggering statement completes
- A compound trigger has a declaration section and a section for each of its timing points.

The Benefits of Using a Compound Trigger

You can use compound triggers to:

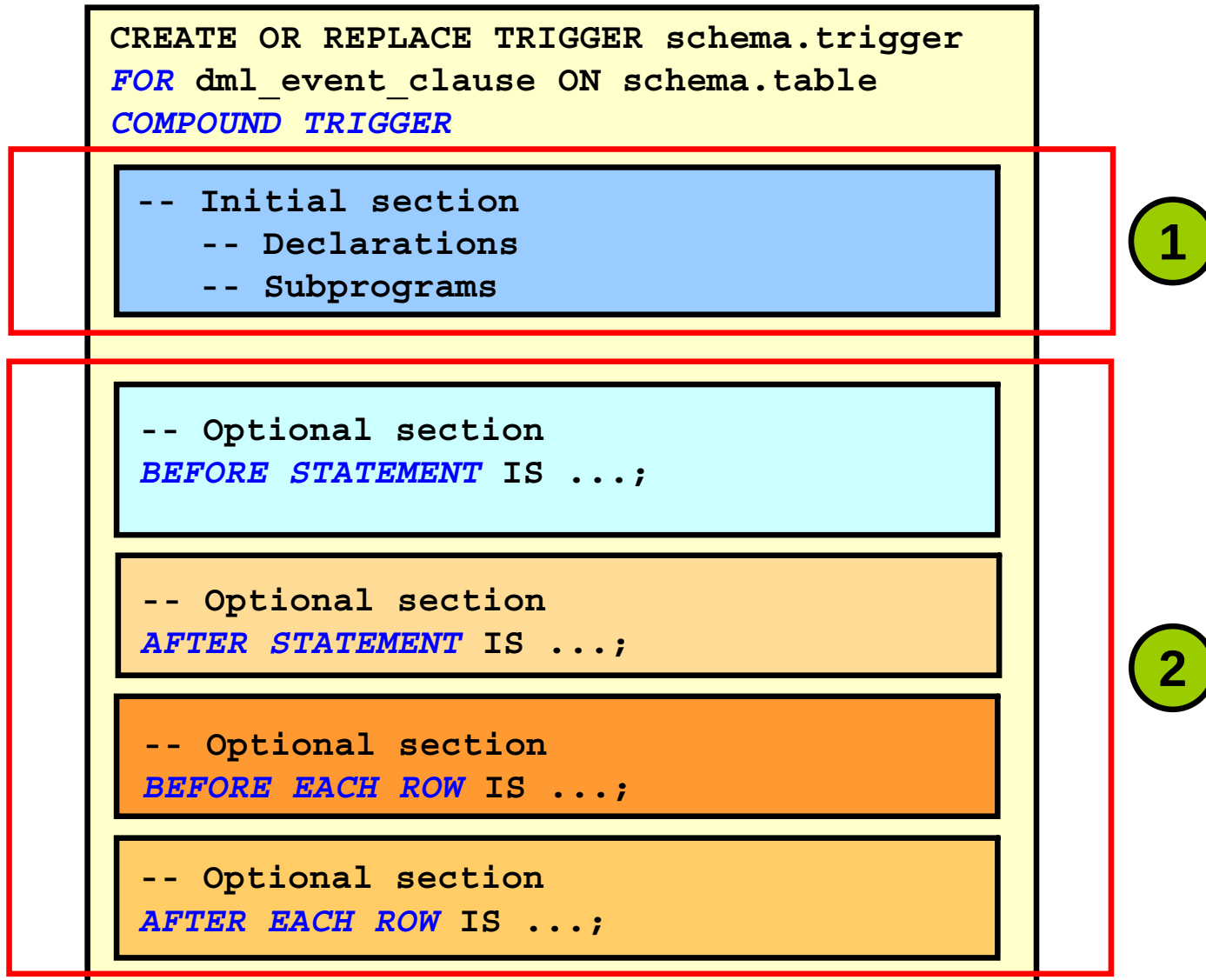
- Program an approach where you want the actions you implement for the various timing points to share common data.
- Accumulate rows destined for a second table so that you can periodically bulk-insert them
- Avoid the mutating-table error (ORA-04091) by allowing rows destined for a second table to accumulate and then bulk-inserting them

Timing-Point Sections of a Table Compound Trigger

A compound trigger defined on a table has one or more of the following timing-point sections. Timing-point sections must appear in the order shown in the table.

Timing Point	Compound Trigger Section
Before the triggering statement executes	BEFORE statement
After the triggering statement executes	AFTER statement
Before each row that the triggering statement affects	BEFORE EACH ROW
After each row that the triggering statement affects	AFTER EACH ROW

Compound Trigger Structure for Tables



Compound Trigger Structure for Views

```
CREATE OR REPLACE TRIGGER
schema.trigger

FOR dml_event_clause ON schema.view

COMPOUND TRIGGER

-- Initial section
-- Declarations
-- Subprograms

-- Optional section (exclusive)
INSTEAD OF EACH ROW IS
...;
```


Compound Trigger Restrictions

- A compound trigger must be a DML trigger and defined on either a table or a view.
- The body of a compound trigger must be compound trigger block, written in PL/SQL.
- A compound trigger body cannot have an initialization block; therefore, it cannot have an exception section.
- An exception that occurs in one section must be handled in that section. It cannot transfer control to another section.
- :OLD and :NEW cannot appear in the declaration, BEFORE STATEMENT, or the AFTER STATEMENT sections.
- Only the BEFORE EACH ROW section can change the value of :NEW.
- The firing order of compound triggers is not guaranteed unless you use the FOLLOWS clause.

Trigger Restrictions on Mutating Tables

- A mutating table is:
 - A table that is being modified by an `UPDATE`, `DELETE`, or `INSERT` statement, or
 - A table that might be updated by the effects of a `DELETE CASCADE` constraint
- The session that issued the triggering statement cannot query or modify a mutating table.
- This restriction prevents a trigger from seeing an inconsistent set of data.
- This restriction applies to all triggers that use the `FOR EACH ROW` clause.
- Views being modified in the `INSTEAD OF` triggers are not considered mutating.

Mutating Table: Example

```
CREATE OR REPLACE TRIGGER check_salary
  BEFORE INSERT OR UPDATE OF salary, job_id
  ON employees
  FOR EACH ROW
  WHEN (NEW.job_id <> 'AD_PRES')
DECLARE
  v_minsalary employees.salary%TYPE;
  v_maxsalary employees.salary%TYPE;
BEGIN
  SELECT MIN(salary), MAX(salary)
    INTO   v_minsalary, v_maxsalary
  FROM   employees
  WHERE  job_id = :NEW.job_id;
  IF :NEW.salary < v_minsalary OR :NEW.salary > v_maxsalary THEN
    RAISE_APPLICATION_ERROR(-20505, 'Out of range');
  END IF;
END;
/
```

Mutating Table: Example

```
UPDATE employees
SET salary = 3400
WHERE last_name = 'Stiles';
```

```
TRIGGER check_salary Compiled.
```

```
Error starting at line 1 in command:
```

```
UPDATE employees
  SET salary = 3400
  WHERE last_name = 'Stiles'
```

```
Error report:
```

```
SQL Error: ORA-04091: table ORA42.EMPLOYEES is mutating, trigger/function may not see it
```

```
ORA-06512: at "ORA42.CHECK_SALARY", line 5
```

```
ORA-04088: error during execution of trigger 'ORA42.CHECK_SALARY'
```

```
04091. 00000 - "table %s.%s is mutating, trigger/function may not see it"
```

```
*Cause:      A trigger (or a user defined plsql function that is referenced in
              this statement) attempted to look at (or modify) a table that was
              in the middle of being modified by the statement which fired it.
```

```
*Action:     Rewrite the trigger (or function) so it does not read that table.
```

Using a Compound Trigger to Resolve the Mutating Table Error

```
CREATE OR REPLACE TRIGGER check_salary
  FOR INSERT OR UPDATE OF salary, job_id
  ON employees
  WHEN (NEW.job_id <> 'AD_PRES')
  COMPOUND TRIGGER

  TYPE salaries_t          IS TABLE OF employees.salary%TYPE;
  min_salaries              salaries_t;
  max_salaries              salaries_t;

  TYPE department_ids_t    IS TABLE OF employees.department_id%TYPE;
  department_ids           department_ids_t;

  TYPE department_salaries_t IS TABLE OF employees.salary%TYPE
                           INDEX BY VARCHAR2(80);
  department_min_salaries  department_salaries_t;
  department_max_salaries  department_salaries_t;

  -- example continues on next slide
```

Using a Compound Trigger to Resolve the Mutating Table Error

```
. . .  
BEFORE STATEMENT IS  
BEGIN  
    SELECT MIN(salary), MAX(salary), NVL(department_id, -1)  
        BULK COLLECT INTO min_salaries, max_salaries, department_ids  
    FROM      employees  
    GROUP BY department_id;  
    FOR j IN 1..department_ids.COUNT() LOOP  
        department_min_salaries(department_ids(j)) := min_salaries(j);  
        department_max_salaries(department_ids(j)) := max_salaries(j);  
    END LOOP;  
END BEFORE STATEMENT;  
  
AFTER EACH ROW IS  
BEGIN  
    IF :NEW.salary < department_min_salaries(:NEW.department_id)  
        OR :NEW.salary > department_max_salaries(:NEW.department_id) THEN  
        RAISE_APPLICATION_ERROR(-20505, 'New Salary is out of acceptable  
                                         range');  
    END IF;  
END AFTER EACH ROW;  
END check_salary;
```

Comparing Database Triggers to Stored Procedures

Triggers	Procedures
Defined with <code>CREATE TRIGGER</code>	Defined with <code>CREATE PROCEDURE</code>
Data dictionary contains source code in <code>USER_TRIGGERS</code>	Data dictionary contains source code in <code>USER_SOURCE</code>
Implicitly invoked by DML	Explicitly invoked
<code>COMMIT</code> , <code>SAVEPOINT</code> , and <code>ROLLBACK</code> are not allowed	<code>COMMIT</code> , <code>SAVEPOINT</code> , and <code>ROLLBACK</code> are allowed

Comparing Database Triggers to Oracle Forms Triggers

Database Trigger	Forms Builder Trigger
Executed by actions from any database tool or application	Executed only within a particular Forms Builder application
Always triggered by a SQL DML, DDL, or a certain database event	Triggered by navigating from field to field, pressing a key, or many other actions
Can be a statement or row trigger	Can be a statement or row trigger
Upon failure, causes the triggering statement to roll back	Upon failure, causes the cursor to freeze and the entire transaction may roll back
Fires independently of, and in addition to, Forms Builder triggers	Fires independently of, and in addition to, database triggers
Executes under the security domain of the author of the trigger	Executes under the security domain of the Forms Builder user

Creating Triggers on DDL Statements

```
CREATE [OR REPLACE] TRIGGER trigger_name
BEFORE | AFTER -- Timing
[ddl_event1 [OR ddl_event2 OR ...]]
ON {DATABASE | SCHEMA}
trigger_body
```

Sample DDL Events	Fires When
CREATE	Any database object is created using the CREATE command.
ALTER	Any database object is altered using the ALTER command.
DROP	Any database object is dropped using the DROP command.

Creating Database-Event Triggers

- Triggering user event:
 - CREATE, ALTER, or DROP
 - Logging on or off
- Triggering database or system event:
 - Shutting down or starting up the database
 - A specific error (or any error) being raised

Creating Triggers on System Events

```
CREATE [OR REPLACE] TRIGGER trigger_name
BEFORE | AFTER -- timing
[database_event1 [OR database_event2 OR ...]]
ON {DATABASE | SCHEMA}
trigger_body
```

Database Event	Triggers Fires When
AFTER SERVERERROR	An Oracle error is raised
AFTER LOGON	A user logs on to the database
BEFORE LOGOFF	A user logs off the database
AFTER STARTUP	The database is opened
BEFORE SHUTDOWN	The database is shut down normally

LOGON and LOGOFF Triggers: Example

```
-- Create the log_trig_table shown in the notes page  
-- first
```

```
CREATE OR REPLACE TRIGGER logon_trig  
AFTER LOGON ON SCHEMA  
BEGIN  
    INSERT INTO log_trig_table(user_id,log_date,action)  
    VALUES (USER, SYSDATE, 'Logging on');  
END;  
/
```

```
CREATE OR REPLACE TRIGGER logoff_trig  
BEFORE LOGOFF ON SCHEMA  
BEGIN  
    INSERT INTO log_trig_table(user_id,log_date,action)  
    VALUES (USER, SYSDATE, 'Logging off');  
END;  
/
```

CALL Statements in Triggers

```
CREATE [OR REPLACE] TRIGGER trigger_name
timing
event1 [OR event2 OR event3]
ON table_name
[REFERENCING OLD AS old | NEW AS new]
[FOR EACH ROW]
[WHEN condition]
CALL procedure_name
/
```

```
CREATE OR REPLACE PROCEDURE log_execution IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('log_exection: Employee Inserted');
END;
/
CREATE OR REPLACE TRIGGER log_employee
BEFORE INSERT ON EMPLOYEES
CALL log_execution -- no semicolon needed
/
```

Benefits of Database-Event Triggers

- Improved data security:
 - Provide enhanced and complex security checks
 - Provide enhanced and complex auditing
- Improved data integrity:
 - Enforce dynamic data integrity constraints
 - Enforce complex referential integrity constraints
 - Ensure that related operations are performed together implicitly

System Privileges Required to Manage Triggers

The following system privileges are required to manage triggers:

- The `CREATE/ALTER/DROP (ANY) TRIGGER` privilege that enables you to create a trigger in any schema
- The `ADMINISTER DATABASE TRIGGER` privilege that enables you to create a trigger on `DATABASE`
- The `EXECUTE` privilege (if your trigger refers to any objects that are not in your schema)

Guidelines for Designing Triggers

- You can design triggers to:
 - Perform related actions
 - Centralize global operations
- You must not design triggers:
 - Where functionality is already built into the Oracle server
 - That duplicate other triggers
- You can create stored procedures and invoke them in a trigger, if the PL/SQL code is very lengthy.
- Excessive use of triggers can result in complex interdependencies, which may be difficult to maintain in large applications.

Quiz

A trigger:

1. Defined with a `CREATE TRIGGER` statement
2. Data dictionary contains source code in `USER_TRIGGERS`
3. Explicitly invoked
4. Implicitly invoked by DML
5. `COMMIT`, `SAVEPOINT`, and `ROLLBACK` are not allowed

Summary

In this lesson, you should have learned how to:

- Describe compound triggers
- Describe mutating tables
- Create triggers on DDL statements
- Create triggers on system events
- Display information about triggers

Practice 10: Overview

This practice covers the following topics:

- Creating advanced triggers to manage data integrity rules
- Creating triggers that cause a mutating table exception
- Creating triggers that use package state to solve the mutating table problem

11

Using the PL/SQL Compiler

Objectives

After completing this lesson, you should be able to do the following:

- Describe the new PL/SQL compiler and features
- Use the new PL/SQL compiler initialization parameters
- Use the new PL/SQL compile time warnings

Lesson Agenda

- Describing the new PL/SQL compiler and features
- Using the new `PLSQL_CODE_TYPE`, `PLSQL_DEBUG`, and `PLSQL_OPTIMIZE_LEVEL` PL/SQL compilation initialization parameters
- Using the new PL/SQL compile time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package routines

Using the PL/SQL Compiler

- PL/SQL uses a new optimizing compiler that can rearrange code for better performance.
- PL/SQL performance is improved across the board.
- Most improvements are automatic.
- The default optimization level improves performance for a broad range of PL/SQL operations.



Changes in the PL/SQL Compiler

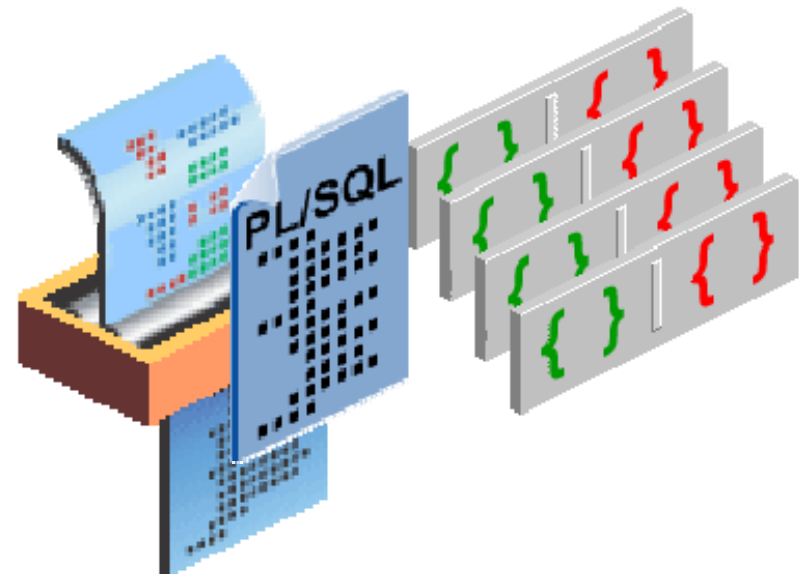
- Elimination of compiler-generated temporary operands
- Computation of some operations during compilation
- Reuse of some expression values
- Simplification or elimination of some branches and dead code elimination
- Avoidance of library calls by direct execution in the PL/SQL virtual machine of some operations
- All cursors correctly finalized upon exit from a cursor loop or a declare block
- Elimination of computations whose only effect is, as a side effect, to raise an exception

Lesson Agenda

- Describing the new PL/SQL compiler and features
- Using the new `PLSQL_CODE_TYPE`, `PLSQL_DEBUG`, and `PLSQL_OPTIMIZE_LEVEL` PL/SQL compilation initialization parameters
- Using the new PL/SQL compile time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package routines

Initialization Parameters for PL/SQL Compilation

- `PLSQL_CODE_TYPE`
- `PLSQL_OPTIMIZE_LEVEL`
- `PLSQL_CCFLAGS`
- `PLSQL_WARNINGS`



Using the Initialization Parameters for PL/SQL Compilation

- PLSQL_CODE_TYPE: Specifies the compilation mode for PL/SQL library units

```
PLSQL_CODE_TYPE = { INTERPRETED | NATIVE }
```

- PLSQL_OPTIMIZE_LEVEL: Specifies the optimization level to be used to compile PL/SQL library units

```
PLSQL_OPTIMIZE_LEVEL = { 0 | 1 | 2 | 3 }
```

The New Compiler Settings Since Oracle 10g

Compiler Option	Description
PLSQL_CODE_TYPE	Specifies the compilation mode for PL/SQL library units.
PLSQL_OPTIMIZE_LEVEL	Specifies the optimization level to be used to compile PL/SQL library units.
PLSQL_WARNINGS	Enables or disables the reporting of warning messages by the PL/SQL compiler.
PLSQL_CCFLAGS	Controls conditional compilation of each PL/SQL library unit independently.

In general, for the fastest performance, use the following setting:

```
PLSQL_CODE_TYPE = NATIVE  
PLSQL_OPTIMIZE_LEVEL = 2
```

Displaying the PL/SQL Initialization Parameters

Use the USER | ALL | DBA_PLSQL_OBJECT_SETTINGS data dictionary views to display the settings for a PL/SQL object:

```
DESCRIBE USER_PLSQL_OBJECT_SETTINGS
```

Name	Null	Type

NAME	NOT NULL	VARCHAR2(30)
TYPE		VARCHAR2(12)
PLSQL_OPTIMIZE_LEVEL		NUMBER
PLSQL_CODE_TYPE		VARCHAR2(4000)
PLSQL_DEBUG		VARCHAR2(4000)
PLSQL_WARNINGS		VARCHAR2(4000)
NLS_LENGTH_SEMANTICS		VARCHAR2(4000)
PLSQL_CCFLAGS		VARCHAR2(4000)
PLSCOPE_SETTINGS		VARCHAR2(4000)
9 rows selected		

Displaying and Setting the PL/SQL Initialization Parameters

```
SELECT name, type, plsql_code_type AS CODE_TYPE,  
       plsql_optimize_level AS OPT_LVL  
FROM   user_plsql_object_settings;
```

...

AUDIT_EMP_VALUES	TRIGGER	INTERPRETED
ANNUAL_SAL	FUNCTION	INTERPRETED
ADD_ROW	PROCEDURE	INTERPRETED
ADD_JOB_HISTORY	PROCEDURE	INTERPRETED
ADD_DEPT	PROCEDURE	INTERPRETED
ADD_DEPARTMENT_NOEX	PROCEDURE	INTERPRETED
ADD_DEPARTMENT	PROCEDURE	INTERPRETED
ADD_COL	PROCEDURE	INTERPRETED

...

- Set the compiler initialization parameter's value using the ALTER SYSTEM or ALTER SESSION statements.
- The parameters' values are accessed when the CREATE OR REPLACE statement is executed.

Changing PL/SQL Initialization Parameters: Example

```
ALTER SESSION SET PLSQL_OPTIMIZE_LEVEL = 1;  
ALTER SESSION SET PLSQL_CODE_TYPE = 'NATIVE';
```

```
ALTER SESSION SET succeeded.  
ALTER SESSION SET succeeded.
```

```
-- code displayed in the notes page  
CREATE OR REPLACE PROCEDURE add_job_history  
. . .
```

. . .

AUDIT_EMP_VALUES	TRIGGER	INTERPRETED
ANNUAL_SAL	FUNCTION	INTERPRETED
ADD_ROW	PROCEDURE	INTERPRETED
ADD_JOB_HISTORY	PROCEDURE	NATIVE
ADD_DEPT	PROCEDURE	INTERPRETED
ADD_DEPARTMENT_NOEX	PROCEDURE	INTERPRETED
ADD_DEPARTMENT	PROCEDURE	INTERPRETED
ADD_COL	PROCEDURE	INTERPRETED

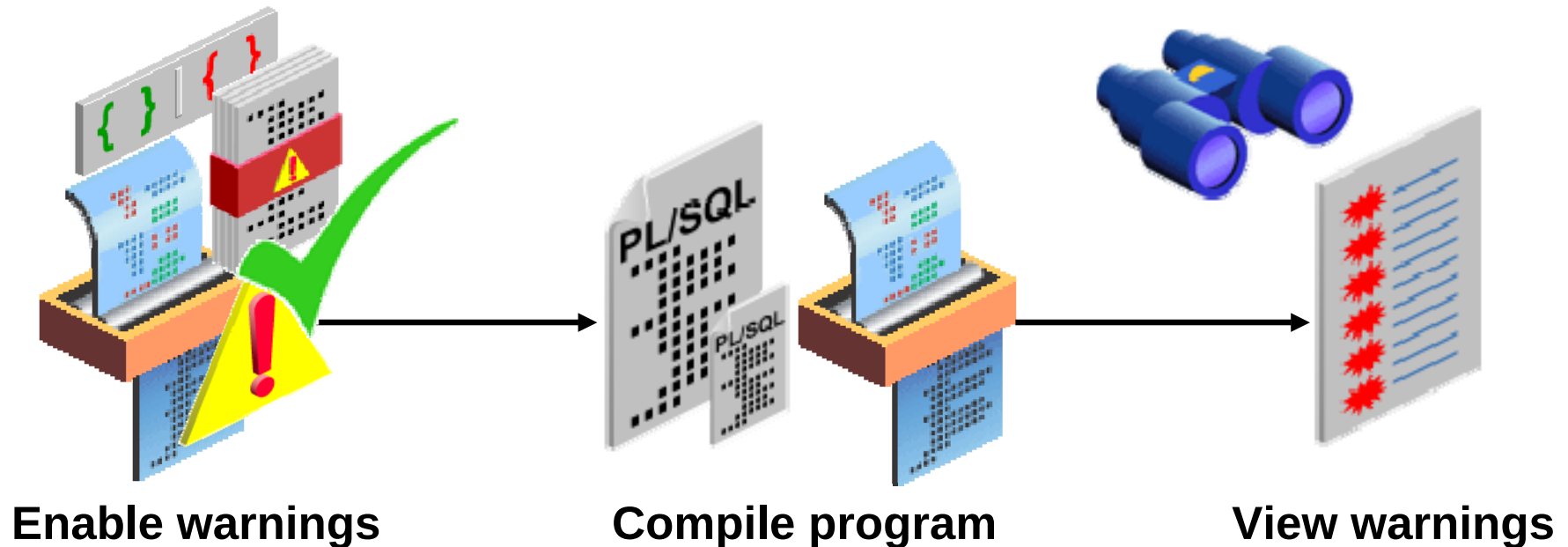
. . .

Lesson Agenda

- Describing the new PL/SQL compiler and features
- Using the new `PLSQL_CODE_TYPE`, `PLSQL_DEBUG`, and `PLSQL_OPTIMIZE_LEVEL` PL/SQL compilation initialization parameters
- Using the new PL/SQL compile time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package routines

Overview of PL/SQL Compile-Time Warnings for Subprograms

Starting with Oracle 10g, the PL/SQL compiler has been enhanced to produce warnings for subprograms.

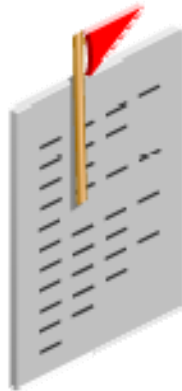


Benefits of Compiler Warnings

- Make programs more robust and avoid problems at run time
- Identify potential performance problems
- Identify factors that produce undefined results



Categories of PL/SQL Compile-Time Warning Messages



SEVERE



PERFORMANCE



INFORMATIONAL

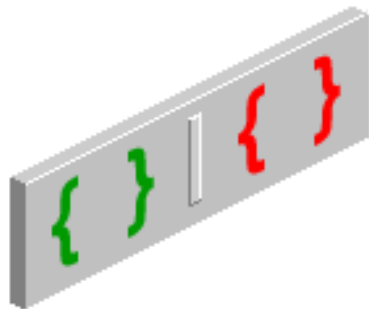


ALL

Setting the Warning Messages Levels

You can set warning levels using one of the following methods:

- Declaratively:
 - Using the `PLSQL_WARNINGS` initialization parameter
- Programmatically:
 - Using the `DBMS_WARNING` package



PLSQL_WARNINGS
initialization parameter



DBMS_WARNING
package

Setting Compiler Warning Levels: Using PLSQL_WARNINGS

```
ALTER [SESSION|SYSTEM]
PLSQL_WARNINGS = 'value_clause1' [ , 'value_clause2']...
```

```
value_clause = Qualifier Value : Modifier Value
```

```
Qualifier Value = { ENABLE | DISABLE | ERROR }
```

```
Modifier Value =
```

```
{ ALL | SEVERE | INFORMATIONAL | PERFORMANCE |  
{ integer | (integer [, integer ] ...) } }
```

Setting Compiler Warning Levels: Using PLSQL_WARNINGS, Examples

```
ALTER SESSION  
SET plsql_warnings = 'enable:severe',  
                    'enable:performance',  
                    'disable:informational';
```

```
ALTER SESSION  succeeded.
```

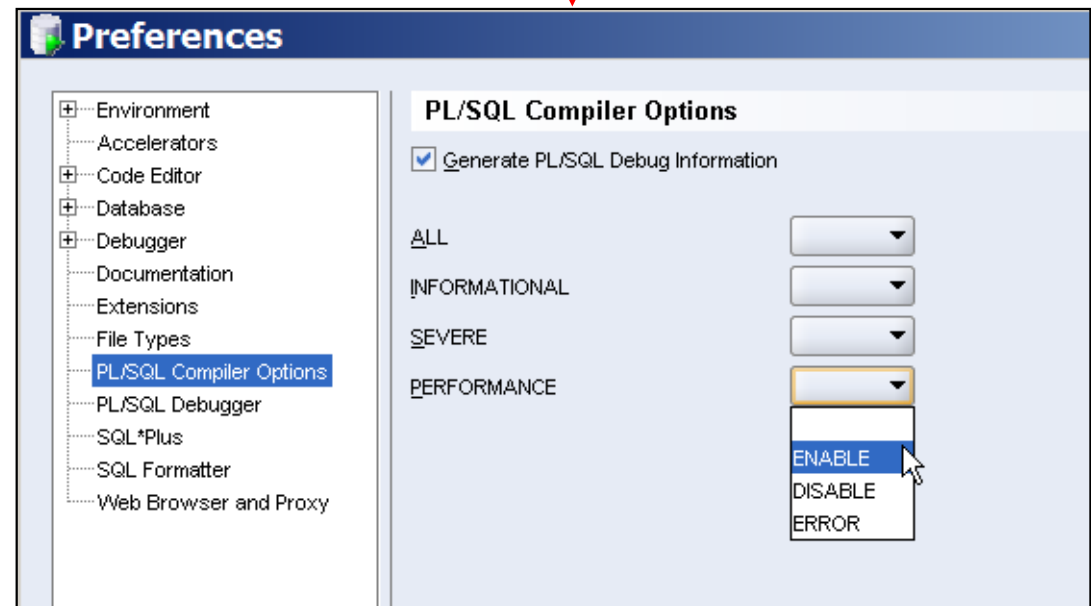
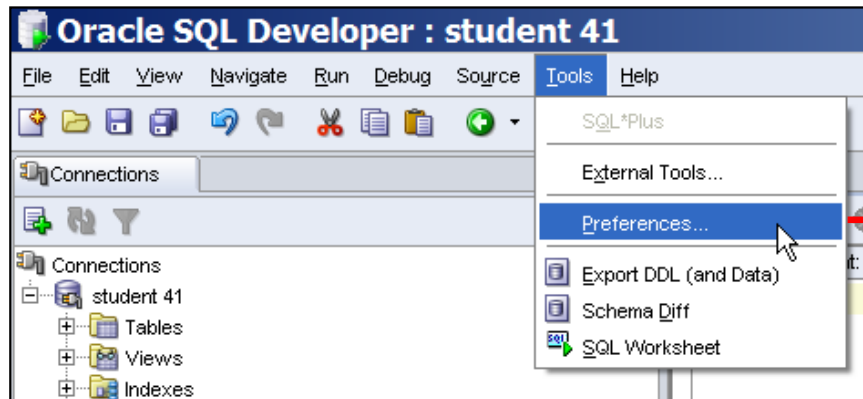
```
ALTER SESSION  
SET plsql_warnings = 'enable:severe';
```

```
ALTER SESSION  succeeded.
```

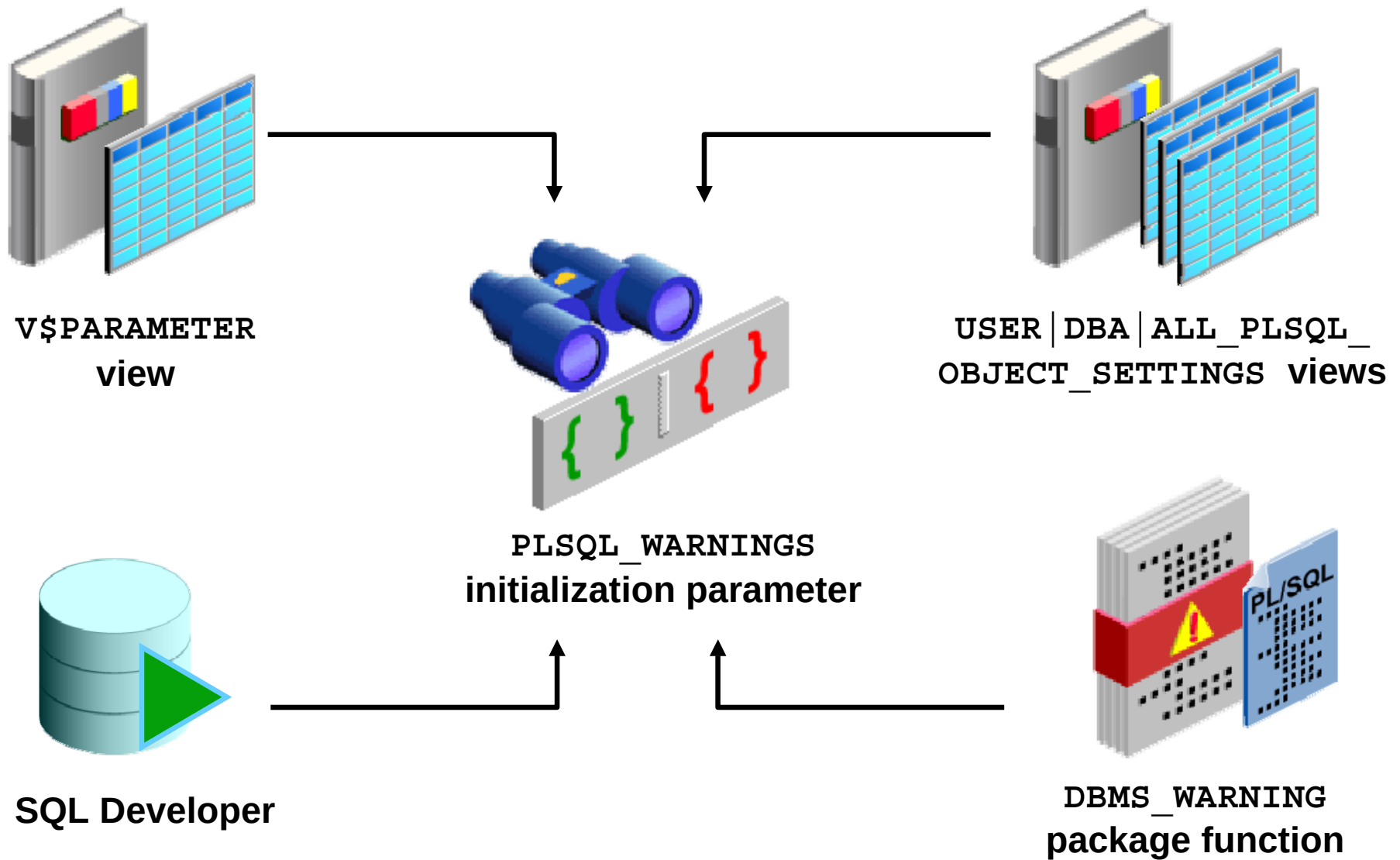
```
ALTER SESSION SET PLSQL_WARNINGS='ENABLE:SEVERE',  
                'DISABLE:PERFORMANCE' , 'ERROR:05003';
```

```
ALTER SESSION SET succeeded.
```

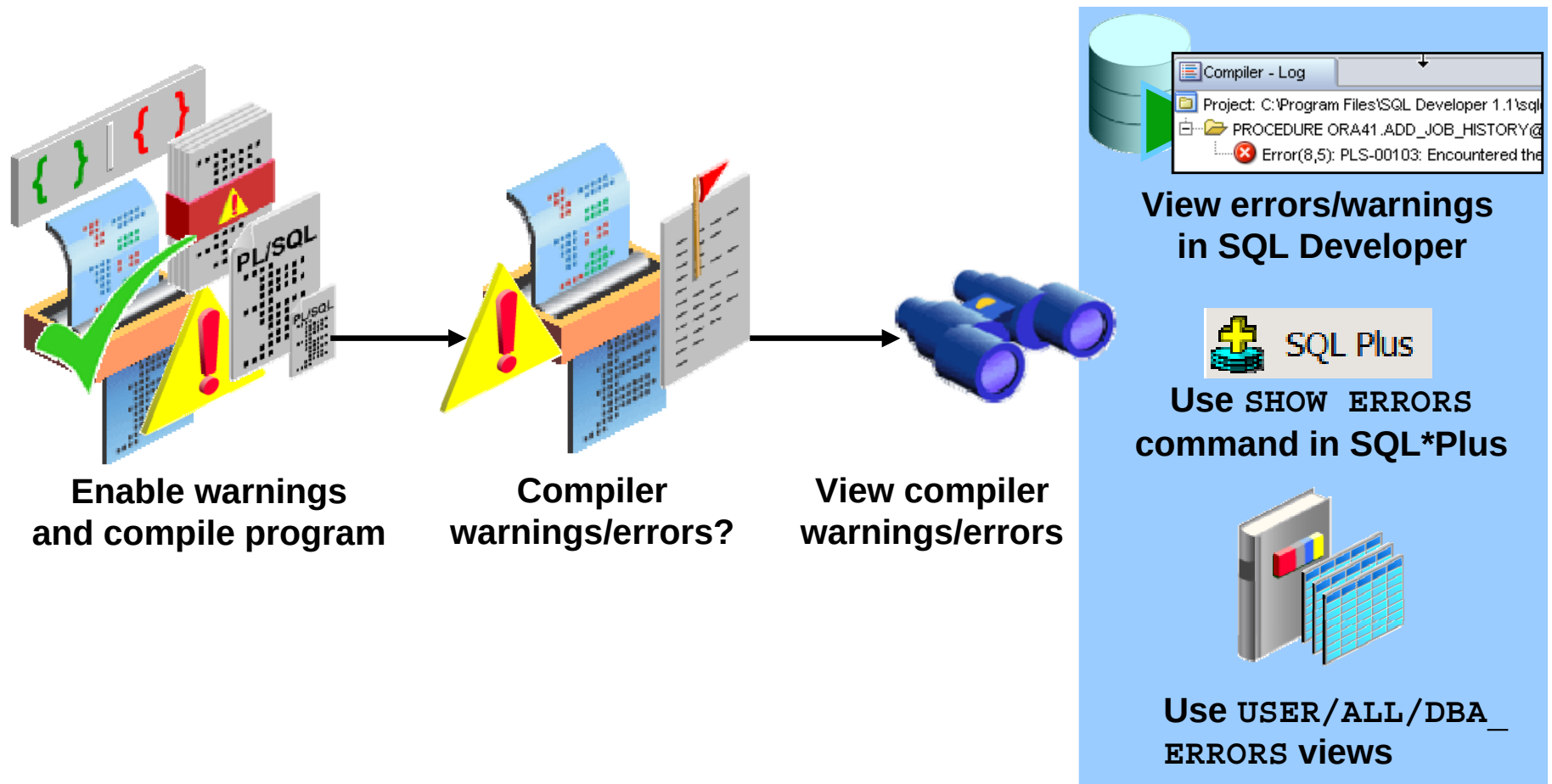
Setting Compiler Warning Levels: Using PLSQL_WARNINGS in SQL Developer



Viewing the Current Setting of PLSQL_WARNINGS



Viewing the Compiler Warnings: Using SQL Developer, SQL*Plus, or Data Dictionary Views



SQL*Plus Warning Messages: Example

```
CREATE OR REPLACE PROCEDURE bad_proc(p_out ...) IS
BEGIN
  . . .;
END;
/
```

SP2-0804: Procedure created with compilation warnings.

```
SHOW ERRORS;
```

Errors for PROCEDURE BAD_PROC:

LINE/COL	ERROR
-----	-----
6/24	PLW-07203: parameter 'p_out' may benefit from use of the NOCOPY compiler hint

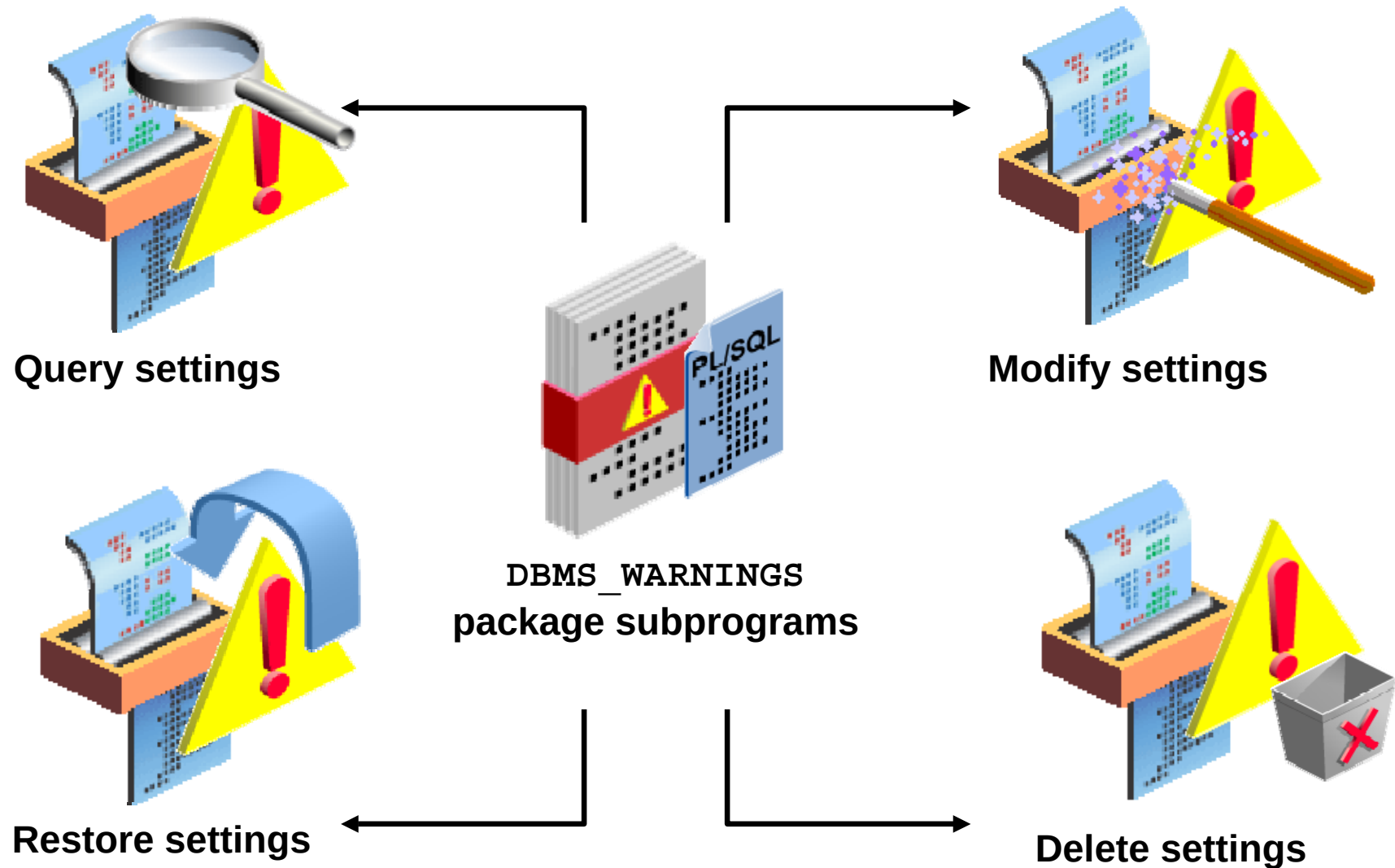
Guidelines for Using PLSQL_WARNINGS

- The settings for the PLSQL_WARNINGS parameter are stored along with each compiled subprogram.
- If you recompile the subprogram using one of the following statements, the current settings for that session are used:
 - CREATE OR REPLACE
 - ALTER ... COMPILE
- If you recompile the subprogram using the ALTER ... COMPILE statement with the REUSE SETTINGS clause, the original setting stored with the program is used.

Lesson Agenda

- Describing the new PL/SQL compiler and features
- Using the New `PLSQL_CODE_TYPE`, `PLSQL_DEBUG`, and `PLSQL_OPTIMIZE_LEVEL` PL/SQL Compilation Initialization Parameters
- Using the New PL/SQL Compile Time Warnings:
 - Using the `PLSQL_WARNING` Initialization Parameter
 - Using the `DBMS_WARNING` Package Routines

Setting Compiler Warning Levels: Using the DBMS_WARNING Package



Using the DBMS_WARNING Package Subprograms

Scenario	Subprograms to Use
Set warnings	ADD_WARNING_SETTING_CAT (procedure) ADD_WARNING_SETTING_NUM (procedure)
Query warnings	GET_WARNING_SETTING_CAT (function) GET_WARNING_SETTING_NUM (function) GET_WARNING_SETTING_STRING (function)
Replace warnings	SET_WARNING_SETTING_STRING (procedure)
Get the warnings' categories names	GET_CATEGORY (function)

The DBMS_WARNING Procedures: Syntax, Parameters, and Allowed Values

```
EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_CAT (-  
    warning_category      IN      VARCHAR2,  
    warning_value         IN      VARCHAR2,  
    scope                 IN      VARCHAR2);
```

```
EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_NUM (-  
    warning_number        IN      NUMBER,  
    warning_value         IN      VARCHAR2,  
    scope                 IN      VARCHAR2);
```

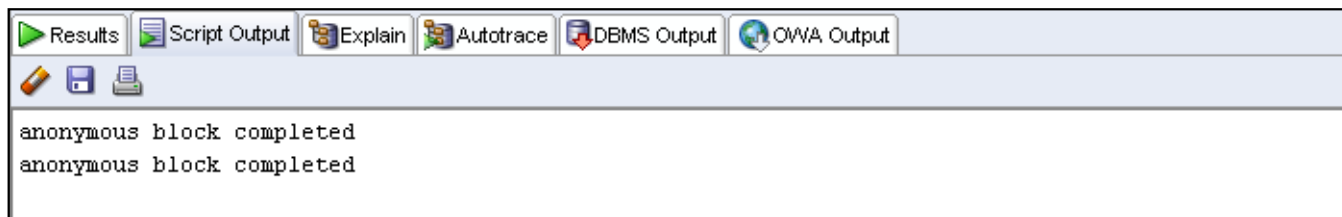
```
EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING (-  
    warning_value         IN      VARCHAR2,  
    scope                 IN      VARCHAR2);
```

The DBMS_WARNING Procedures: Example

```
-- Establish the following warning setting string in the
-- current session:
-- ENABLE:INFORMATIONAL,
-- DISABLE:PERFORMANCE,
-- ENABLE:SEVERE
```

```
EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING(-
    'ENABLE:ALL', 'SESSION');
```

```
EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_CAT(-
    'PERFORMANCE', 'DISABLE', 'SESSION');
```



The DBMS_WARNING Functions: Syntax, Parameters, and Allowed Values

```
DBMS_WARNING.GET_WARNING_SETTING_CAT (-  
    warning_category IN VARCHAR2) RETURN warning_value;
```

```
DBMS_WARNING.GET_WARNING_SETTING_NUM (-  
    warning_number IN NUMBER) RETURN warning_value;
```

```
DBMS_WARNING.GET_WARNING_SETTING_STRING  
    RETURN pls_integer;
```

```
DBMS_WARNING.GET_CATEGORY (-  
    warning_number IN pls_integer) RETURN VARCHAR2;
```

The DBMS_WARNING Functions: Example

```
-- Determine the current session warning settings

EXECUTE DBMS_OUTPUT.PUT_LINE( -
DBMS_WARNING.GET_WARNING_SETTING_STRING);
```

```
anonymous block completed
ENABLE: INFORMATIONAL,DISABLE: PERFORMANCE,ENABLE: SEVERE
```

```
-- Determine the category for warning message number
-- PLW-07203

EXECUTE DBMS_OUTPUT.PUT_LINE( -
DBMS_WARNING.GET_CATEGORY(7203));
```

```
anonymous block completed
PERFORMANCE
```

Using DBMS_WARNING: Example

```
CREATE OR REPLACE PROCEDURE compile_code(p_pkg_name VARCHAR2) IS
    v_warn_value    VARCHAR2(200);
    v_compile_stmt  VARCHAR2(200) :=
        'ALTER PACKAGE ' || p_pkg_name || ' COMPILE';

BEGIN
    v_warn_value := DBMS_WARNING.GET_WARNING_SETTING_STRING;
    DBMS_OUTPUT.PUT_LINE('Current warning settings: ' ||
        v_warn_value);
    DBMS_WARNING.ADD_WARNING_SETTING_CAT(
        'PERFORMANCE', 'DISABLE', 'SESSION');
    DBMS_OUTPUT.PUT_LINE('Modified warning settings: ' ||
        DBMS_WARNING.GET_WARNING_SETTING_STRING);
    EXECUTE IMMEDIATE v_compile_stmt;
    DBMS_WARNING.SET_WARNING_SETTING_STRING(v_warn_value,
        'SESSION');
    DBMS_OUTPUT.PUT_LINE('Restored warning settings: ' ||
        DBMS_WARNING.GET_WARNING_SETTING_STRING);
END;
/
```

Using DBMS_WARNING: Example

```
EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING(-  
  'ENABLE:ALL', 'SESSION');
```

```
anonymous block completed
```

```
@code_11_36_bs.sql
```

```
PROCEDURE compile_code(p_pkg_name Compiled.
```

```
@code_11_37_cs.sql -- compiles the DEPT_PKG package  
EXECUTE compile_code('DEPT_PKG');
```

Current warning settings: ENABLE:ALL

Modified warning settings:

ENABLE:INFORMATIONAL,DISABLE:PERFORMANCE,ENABLE:SEVERE

Restored warning settings: ENABLE:ALL

```
anonymous block completed  
Current warning settings: ENABLE:ALL  
Modified warning settings: ENABLE:INFORMATIONAL,DISABLE:PERFORMANCE,ENABLE:SEVERE  
Restored warning settings: ENABLE:ALL
```

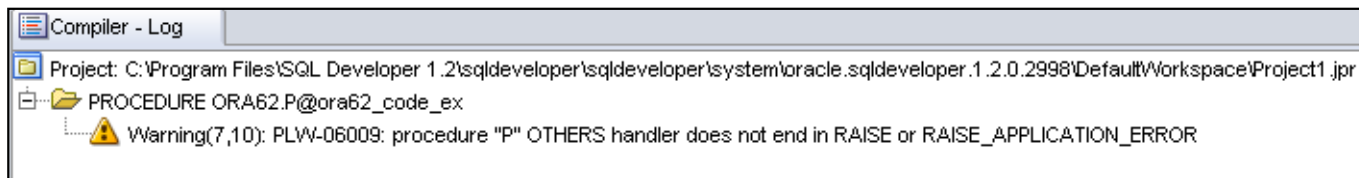
Using the New PLW 06009 Warning Message

- A new PLW warning is available in Oracle Database 11g.
- This warning indicates that the OTHERS handler of your PL/SQL subroutine can exit without executing:
 - Some form of RAISE, or
 - A call to the standard procedure
`RAISE_APPLICATION_ERROR`
- A good programming practice suggests that OTHERS handlers must always pass an exception upward.



The New PLW 06009 Warning: Example

```
-- You must create table t shown in the notes section first.
CREATE OR REPLACE PROCEDURE p(i IN VARCHAR2)
IS
BEGIN
    INSERT INTO t(col_a) VALUES (i);
EXCEPTION
    WHEN OTHERS THEN null;
END p;
/
ALTER PROCEDURE P COMPILE
PLSQL_warnings = 'enable:all' REUSE SETTINGS;
```



```
SELECT *
FROM user_errors
WHERE name = 'P'
```

Results

Script Output

Explain

Autotrace

DBMS Output

OWA Output

Results:

	NAME	TYPE	SEQUENCE	LINE	POSITION	TEXT	ATTRIBUTE	MESSAGE_NUMBER
1	P	PROCEDURE	1	6		10 PLW-06009: procedure "P" OTHERS handler does not end in RAISE or RAISE_APPLICATION_ERROR	WARNING	6009

Quiz

The categories of PL/SQL compile-time warning messages are:

1. SEVERE
2. PERFORMANCE
3. INFORMATIONAL
4. All
5. CRITICAL

Summary

In this lesson, you should have learned how to:

- Describe the new PL/SQL compiler and features
- Use the new PL/SQL compiler initialization parameters
- Use the new PL/SQL compile time warnings

Practice 11: Overview

This practice covers the following topics:

- Displaying the compiler initialization parameters
- Enabling native compilation for your session and compiling a procedure
- Disabling the compiler warnings, and then restoring the original session-warning settings
- Identifying the categories for some compiler-warning message numbers

12

Managing PL/SQL Code

Objectives

After completing this lesson, you should be able to do the following:

- Describe and use conditional compilation
- Hide PL/SQL source code using dynamic obfuscation and the Wrap utility

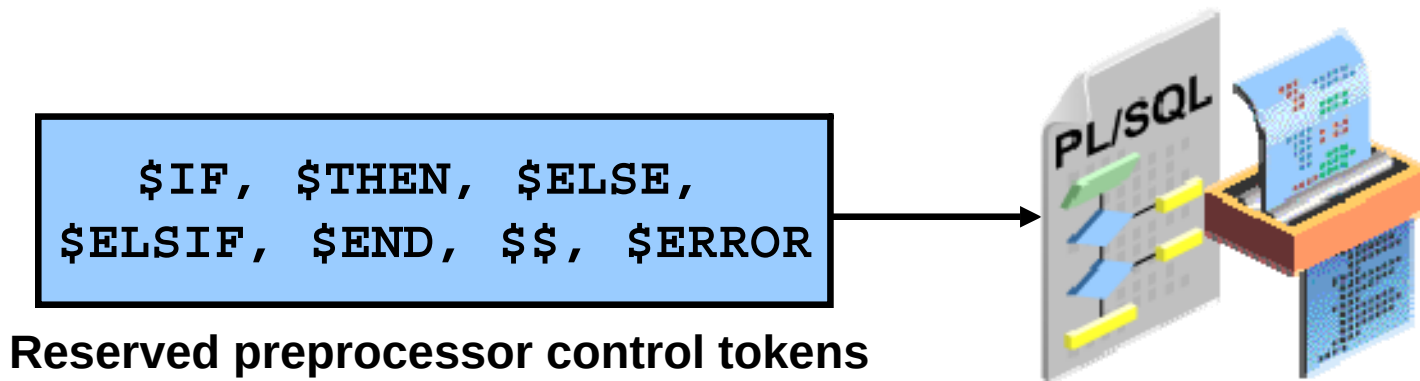
Lesson Agenda

- Using conditional compilation
- Obfuscating PL/SQL code

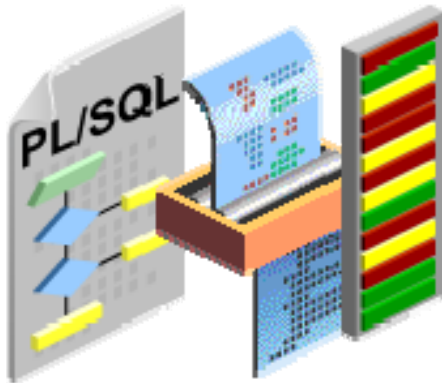
What Is Conditional Compilation?

Enables you to customize the functionality in a PL/SQL application without removing any source code:

- Utilize the latest functionality with the latest database release or disable the new features to run the application against an older release of the database.
- Activate debugging or tracing functionality in the development environment and hide that functionality in the application while it runs at a production site.



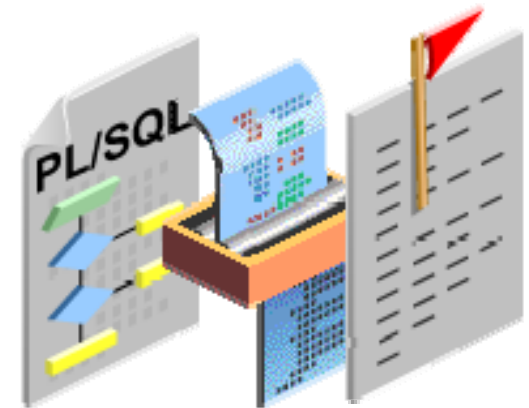
How Does Conditional Compilation Work?



Selection directives:
Use the `$IF` token.



Inquiry directives:
Use the `$$` token.



Error directives:
Use the `$ERROR` token.



**DBMS_PREPROCESSOR
package**



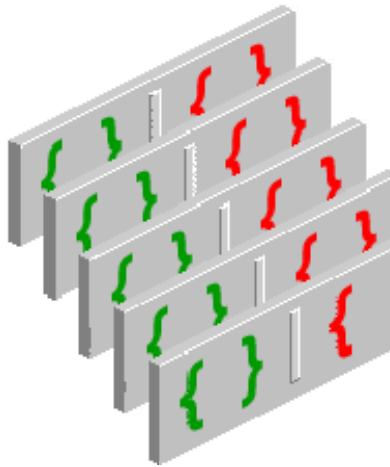
**DBMS_DB_VERSION
package**

Using Selection Directives

```
$IF <Boolean-expression> $THEN Text
$ELSEIF <Boolean-expression> $THEN Text
. . .
$ELSE Text
$END
```

```
DECLARE
CURSOR cur IS SELECT employee_id FROM
employees WHERE
$IF myapp_tax_package.new_tax_code $THEN
    salary > 20000;
$ELSE
    salary > 50000;
$END
BEGIN
    OPEN cur;
. . .
END;
```

Using Predefined and User-Defined Inquiry Directives



```
PLSQL_CCFLAGS  
PLSQL_CODE_TYPE  
PLSQL_DEBUG  
PLSQL_OPTIMIZE_LEVEL  
PLSQL_WARNINGS  
NLS_LENGTH_SEMANTICS  
PLSQL_LINE  
PLSQL_UNIT
```

Predefined inquiry directives

```
PLSQL_CCFLAGS = 'plsql_ccflags:true,debug:true,debug:0';
```

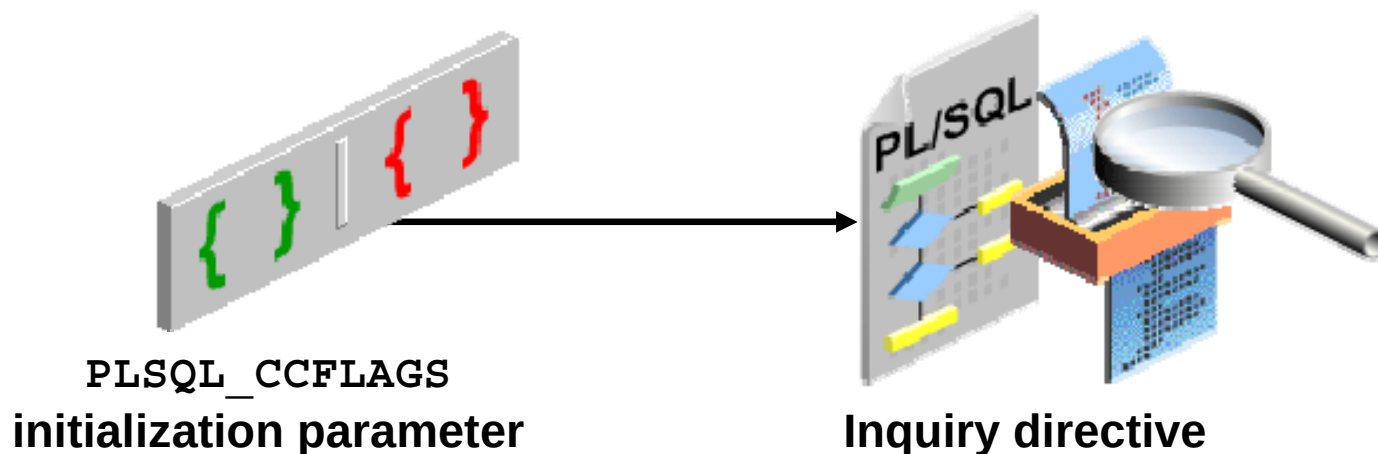
User-defined inquiry directives

The PLSQL_CCFLAGS Parameter and the Inquiry Directive

Use the PLSQL_CCFLAGS parameter to control conditional compilation of each PL/SQL library unit independently.

```
PLSQL_CCFLAGS = '<v1>:<c1>,<v2>:<c2>,...,<vn>:<cn>'
```

```
ALTER SESSION SET  
PLSQL_CCFLAGS = 'plsql_ccflags:true, debug:true, debug:0';
```



Displaying the PLSQL_CCFLAGS Initialization Parameter Setting

```
SELECT name, type, plsql_ccflags
FROM   user_plsql_object_settings
```

Results:

	NAME	TYPE	PLSQL_CCFLAGS
1	DEPT_PKG	PACKAGE	(null)
2	DEPT_PKG	PACKAGE BODY	(null)
3	TAXES_PKG	PACKAGE	(null)
4	TAXES_PKG	PACKAGE BODY	(null)
5	EMP_PKG	PACKAGE	(null)
6	EMP_PKG	PACKAGE BODY	(null)
7	SECURE_DML	PROCEDURE	(null)
8	SECURE_EMPLOYEES	TRIGGER	(null)
9	ADD_JOB_HISTORY	PROCEDURE	plsql_ccflags:true, debug:true, debug:0
10	UPDATE_JOB_HISTORY	TRIGGER	(null)

...

The PLSQL_CCFLAGS Parameter and the Inquiry Directive: Example

```
ALTER SESSION SET PLSQL_CCFLAGS = 'Tracing:true';
CREATE OR REPLACE PROCEDURE P IS
BEGIN
  $IF $$tracing $THEN
    DBMS_OUTPUT.PUT_LINE ('TRACING');
  $END
END P;
```

```
ALTER SESSION SET succeeded.
PROCEDURE P Compiled.
```

```
SELECT name, plsql_ccflags
FROM USER_PLSQL_OBJECT_SETTINGS
WHERE name = 'P';
```

Results:

	NAME	PLSQL_CCFLAGS
1	P	Tracing:true

Using Conditional Compilation Error Directives to Raise User-Defined Errors

```
$ERROR varchar2_static_expression $END
```

```
ALTER SESSION SET Plsql_CCFlags = ' Trace_Level:3 '  
/ CREATE PROCEDURE P IS  
BEGIN  
    $IF      $$Trace_Level = 0 $THEN ...;  
    $ELSIF   $$Trace_Level = 1 $THEN ...;  
    $ELSIF   $$Trace_Level = 2 $THEN ...;  
    $else $error 'Bad: ' || $$Trace_Level $END  
$END  
END P; /
```

```
SHOW ERRORS
```

```
Errors for PROCEDURE P:
```

```
LINE/COL ERROR
```

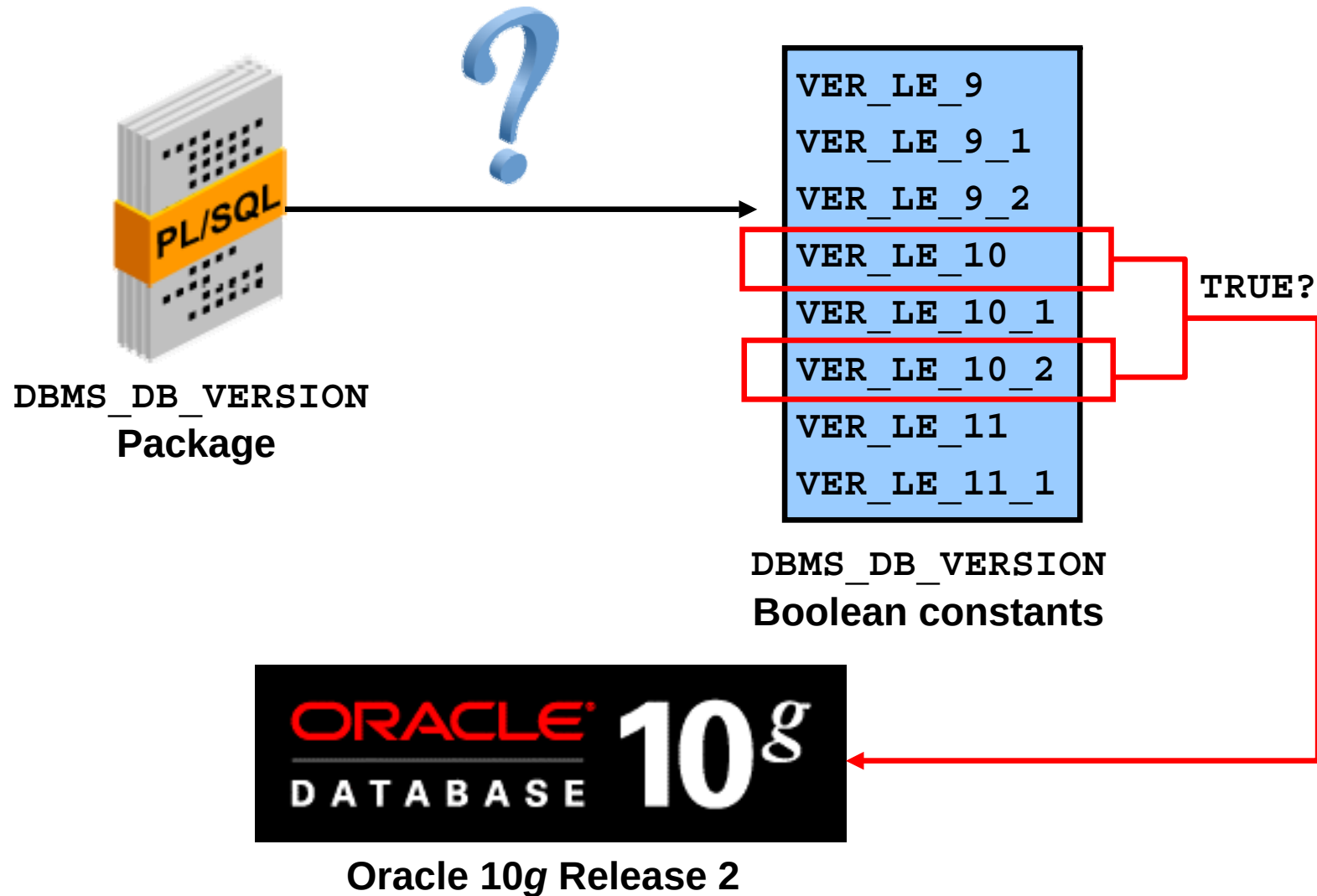
```
-----  
6/9          PLS-00179: $ERROR: Bad: 3
```

Using Static Expressions with Conditional Compilation

- Boolean static expressions:
 - TRUE, FALSE, NULL, IS NULL, IS NOT NULL
 - > , < , >= , <= , = , <> , NOT, AND, OR
- PLS_INTEGER static expressions:
 - -2147483648 to 2147483647, NULL
- VARCHAR2 static expressions include:
 - ||, NULL, TO_CHAR
- Static constants:

```
static_constant CONSTANT datatype := static_expression;
```

The DBMS_DB_VERSION Package: Boolean Constants



The DBMS_DB_VERSION Package Constants

Name	Value	Description
VERSION	10	Current version.
RELEASE	2	Current release.
VER_LE_9	FALSE	Version <= 9.
VER_LE_9_1	FALSE	Version <= 9 and release <= 1.
VER_LE_9_2	FALSE	Version <= 9 and release <= 2.
VER_LE_10	TRUE	Version <= 10.
VER_LE_10_1	FALSE	Version <= 10 and release <= 1.
VER_LE_10_2	TRUE	Version <=10 and release <= 2.
VER_LE_11	FALSE	Version <= 11.
VER_LE_11_1	TRUE	Version <=11 and release <= 1.

Using Conditional Compilation with Database Versions: Example

```
ALTER SESSION SET PLSQL_CCFLAGS = 'my_debug:FALSE, my_tracing:FALSE';
CREATE PACKAGE my_pkg AS
  SUBTYPE my_real IS
    -- Check the database version, if >= 10g, use BINARY_DOUBLE data type,
    -- else use NUMBER data type
    $IF DBMS_DB_VERSION.VERSION < 10 $THEN      NUMBER;
    $ELSE    BINARY_DOUBLE;
    $END
  my_pi my_real; my_e my_real;
END my_pkg;
/
CREATE PACKAGE BODY my_pkg AS
BEGIN
  $IF DBMS_DB_VERSION.VERSION < 10 $THEN
    my_pi := 3.14016408289008292431940027343666863227;
    my_e  := 2.71828182845904523536028747135266249775;
  $ELSE
    my_pi := 3.14016408289008292431940027343666863227d;
    my_e  := 2.71828182845904523536028747135266249775d;
  $END
END my_pkg;
/
```


Using Conditional Compilation with Database Versions: Example

```
CREATE OR REPLACE PROCEDURE circle_area(p_radius my_pkg.my_real) IS
  v_my_area my_pkg.my_real;
  v_my_datatype VARCHAR2(30);
BEGIN
  v_my_area := my_pkg.my_pi * p_radius;
  DBMS_OUTPUT.PUT_LINE('Radius: ' || TO_CHAR(p_radius)
                      || ' Area: ' || TO_CHAR(v_my_area) );
  $IF $$my_debug $THEN -- if my_debug is TRUE, run some debugging code
    SELECT DATA_TYPE INTO v_my_datatype FROM USER_ARGUMENTS
      WHERE OBJECT_NAME = 'CIRCLE_AREA' AND ARGUMENT_NAME = 'P_RADIUS';
    DBMS_OUTPUT.PUT_LINE('Datatype of the RADIUS argument is: ' ||
      v_my_datatype);
  $END
END; /
```

```
PROCEDURE circle_area(p_radius Compiled.
```

```
CALL circle_area(50); -- Using Oracle Database 11g Release 1
```

```
CALL circle_area(50) succeeded.
Radius: 5.0E+001 Area: 1.5700820414450413E+002
```

Using DBMS_PREPROCESSOR Procedures to Print or Retrieve Source Text

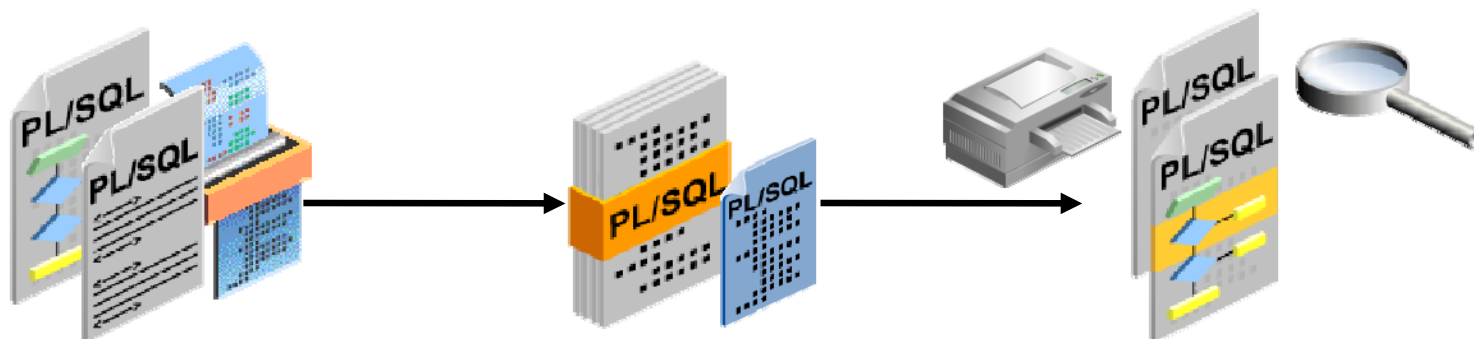
```
-- Substitute ORA62 with your user account as needed
CALL
  DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE('PACKAGE'
    , 'ORA62', 'MY_PKG');
```

```
CALL DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE('PACKAGE', succeeded.
PACKAGE my_pkg AS
  SUBTYPE my_real IS

-- Check the database version, if >= 10g, use BINARY_DOUBLE data type,
-- else use NUMBER data type

      BINARY_DOUBLE;

  my_pi my_real; my_e my_real;
END my_pkg;
```



Lesson Agenda

- Using conditional compilation
- Obfuscating PL/SQL code

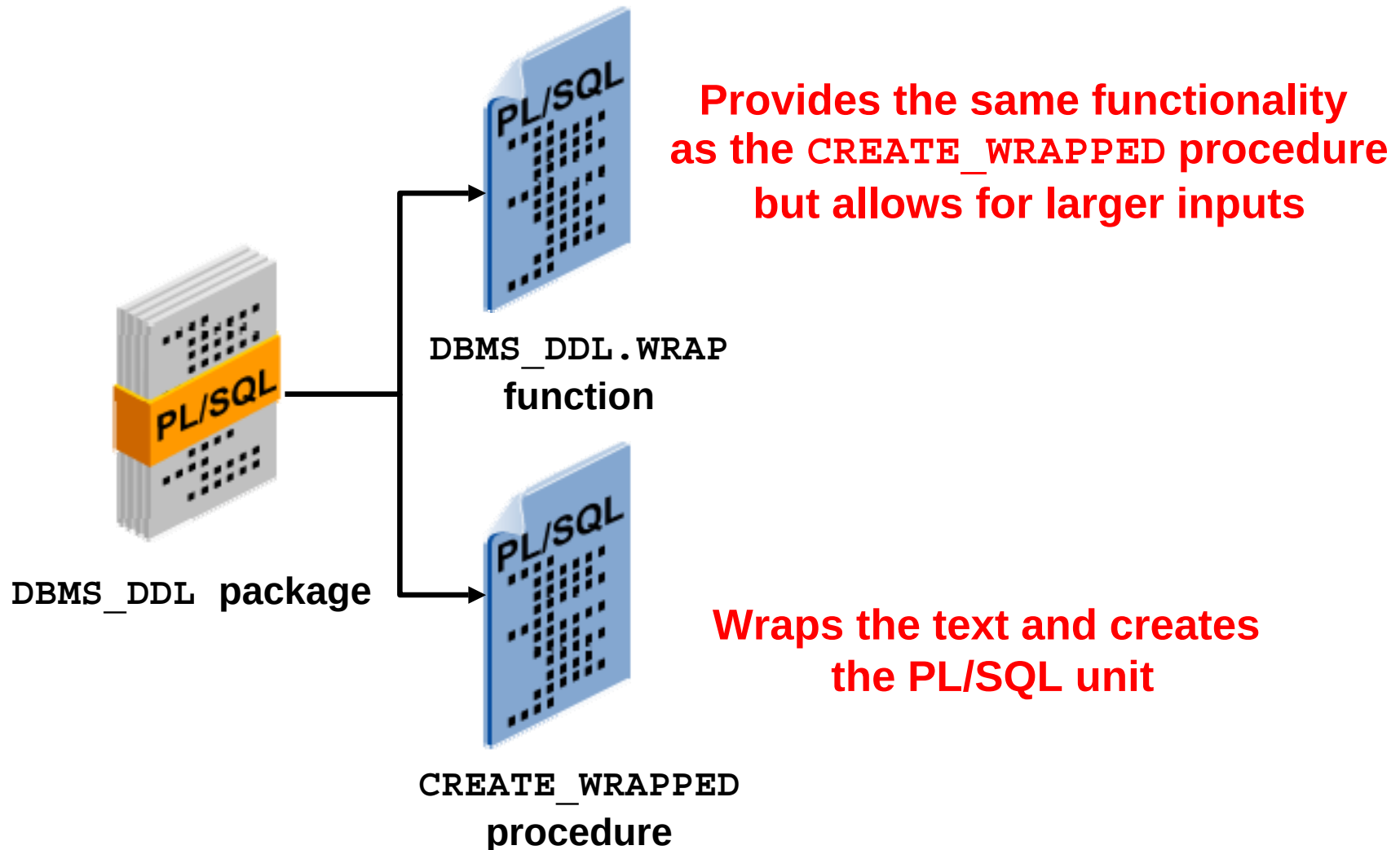
What Is Obfuscation?

- Obfuscation (or wrapping) of a PL/SQL unit is the process of hiding the PL/SQL source code.
- Wrapping can be done with the wrap utility and DBMS_DDL subprograms.
- The wrap utility is run from the command line and it processes an input SQL file, such as a SQL*Plus installation script.
- The DBMS_DDL subprograms wrap a single PL/SQL unit, such as a single CREATE PROCEDURE command, that has been generated dynamically.

Benefits of Obfuscating

- It prevents others from seeing your source code.
- Your source code is not visible through the `USER_SOURCE`, `ALL_SOURCE`, or `DBA_SOURCE` data dictionary views.
- SQL*Plus can process the obfuscated source files.
- The `Import` and `Export` utilities accept wrapped files.

What's New in Dynamic Obfuscating Since Oracle 10g?



Nonobfuscated PL/SQL Code: Example

```
BEGIN -- The ALL_SOURCE view family shows source code
EXECUTE IMMEDIATE '
  CREATE OR REPLACE PROCEDURE P1 IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE (''I am not wrapped'');
  END P1;
';
END;
/
CALL p1();
```

```
anonymous block completed
CALL p1() succeeded.
I'm not wrapped
```

```
SELECT text FROM user_source
WHERE name = 'P1' ORDER BY line;
```

	TEXT
1	PROCEDURE P1 IS
2	BEGIN
3	DBMS_OUTPUT.PUT_LINE ('I am not wrapped');
4	END P1;
5	

Obfuscated PL/SQL Code: Example

```
BEGIN -- ALL_SOURCE view family obfuscates source code
  DBMS_DDL.CREATE_WRAPPED ( '
    CREATE OR REPLACE PROCEDURE P1 IS
    BEGIN
      DBMS_OUTPUT.PUT_LINE (''I am wrapped now'');
    END P1;
  ' );
END;
/
CALL p1();
```

```
anonymous block completed
call pl() succeeded.
I am wrapped now
```

```
SELECT text FROM user_source
WHERE name = 'P1' ORDER BY line;
```

```
TEXT
-----
PROCEDURE P1 wrapped
a0000000
b2
abcd
```

...

Dynamic Obfuscation: Example

```
SET SERVEROUTPUT ON

DECLARE
c_code CONSTANT VARCHAR2(32767) :=
' CREATE OR REPLACE PROCEDURE new_proc AS
  v_VDATE  DATE;
  BEGIN
    v_VDATE := SYSDATE;
    DBMS_OUTPUT.PUT_LINE(v_VDATE) ;
  END; ' ;
BEGIN
  DBMS_DDL.CREATE_WRAPPED (c_CODE) ;
END;
/
```

The PL/SQL Wrapper Utility

- The PL/SQL wrapper is a stand-alone utility that hides application internals by converting PL/SQL source code into portable object code.
- Wrapping has the following features:
 - Platform independence
 - Dynamic loading
 - Dynamic binding
 - Dependency checking
 - Normal importing and exporting when invoked

Running the Wrapper Utility

```
WRAP INAME=input_file_name [ONAME=output_file_name]
```

- Do not use spaces around the equal signs.
- The `INAME` argument is required.
- The default extension for the input file is `.sql`, unless it is specified with the name.
- The `ONAME` argument is optional.
- The default extension for output file is `.plb`, unless specified with the `ONAME` argument.

Examples

```
WRAP INAME=demo_04_hello.sql  
WRAP INAME=demo_04_hello  
WRAP INAME=demo_04_hello.sql ONAME=demo_04_hello.plb
```

Results of Wrapping

```
-- Original PL/SQL source code in input file:
```

```
CREATE PACKAGE banking IS
  min_bal := 100;
  no_funds EXCEPTION;
  ...
END banking;
/
```

```
-- Wrapped code in output file:
```

```
CREATE PACKAGE banking
  wrapped
012abc463e ...
/
```

Guidelines for Wrapping

- You must wrap only the package body, not the package specification.
- The wrapper can detect syntactic errors but cannot detect semantic errors.
- The output file should not be edited. You maintain the original source code and wrap again as required.
- To ensure that all the important parts of your source code are obfuscated, view the wrapped file in a text editor before distributing it.

DBMS_DDL Package Versus the Wrap Utility

Functionality	DBMS_DDL	Wrap Utility
Code obfuscation	Yes	Yes
Dynamic Obfuscation	Yes	No
Obfuscate multiple programs at a time	No	Yes

Quiz

Conditional compilation enables you to customize the functionality in a PL/SQL application without removing any source code.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Describe and use conditional compilation
- Hide PL/SQL source code using dynamic obfuscation and the Wrap utility

Practice 12: Overview

This practice covers the following topics:

- Creating a package and a procedure that uses conditional compilation
- Using the appropriate package to retrieve the postprocessed source text of the PL/SQL unit
- Obfuscating some PL/SQL code

13

Managing Dependencies

Objectives

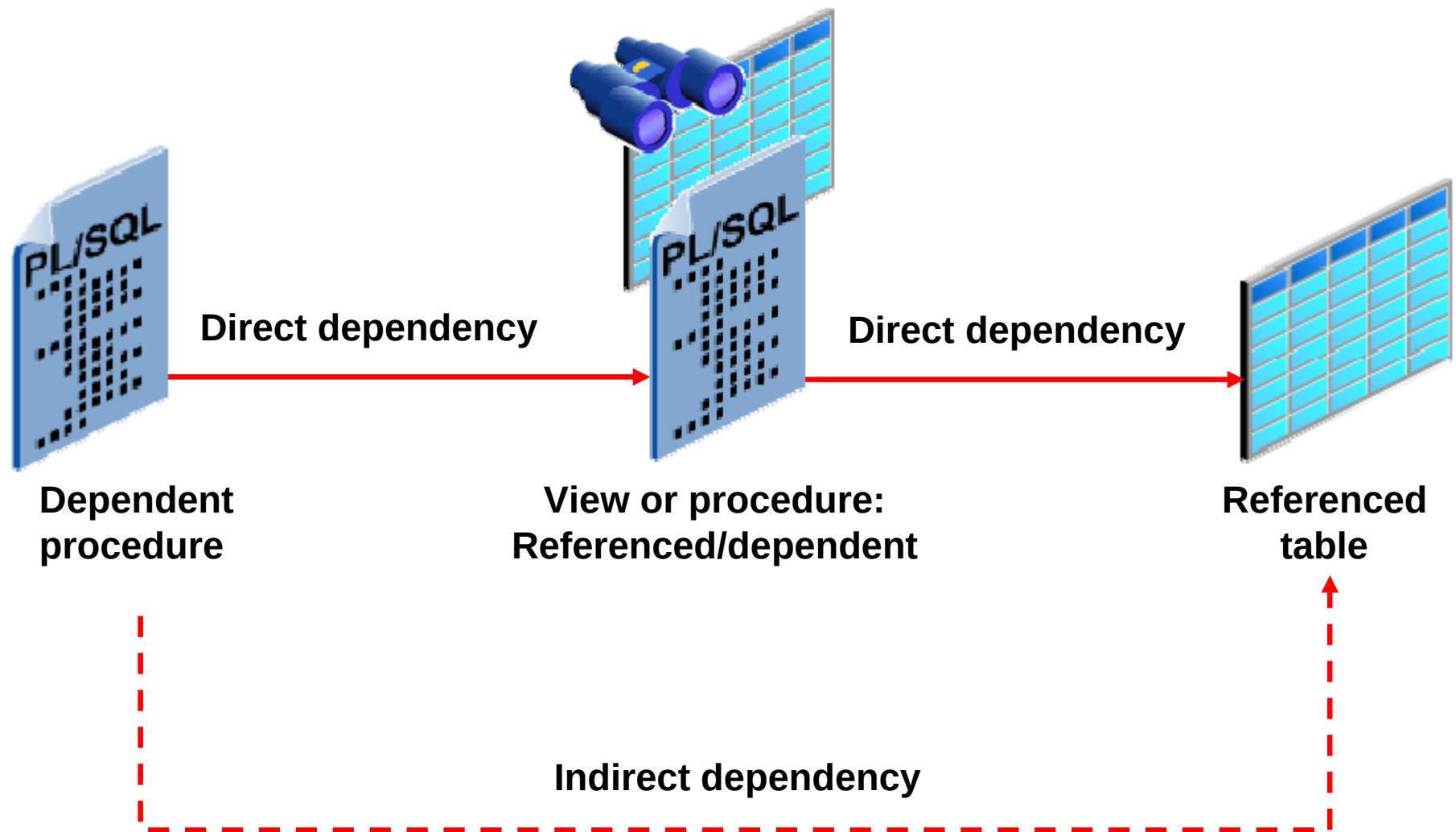
After completing this lesson, you should be able to do the following:

- Track procedural dependencies
- Predict the effect of changing a database object on procedures and functions
- Manage procedural dependencies

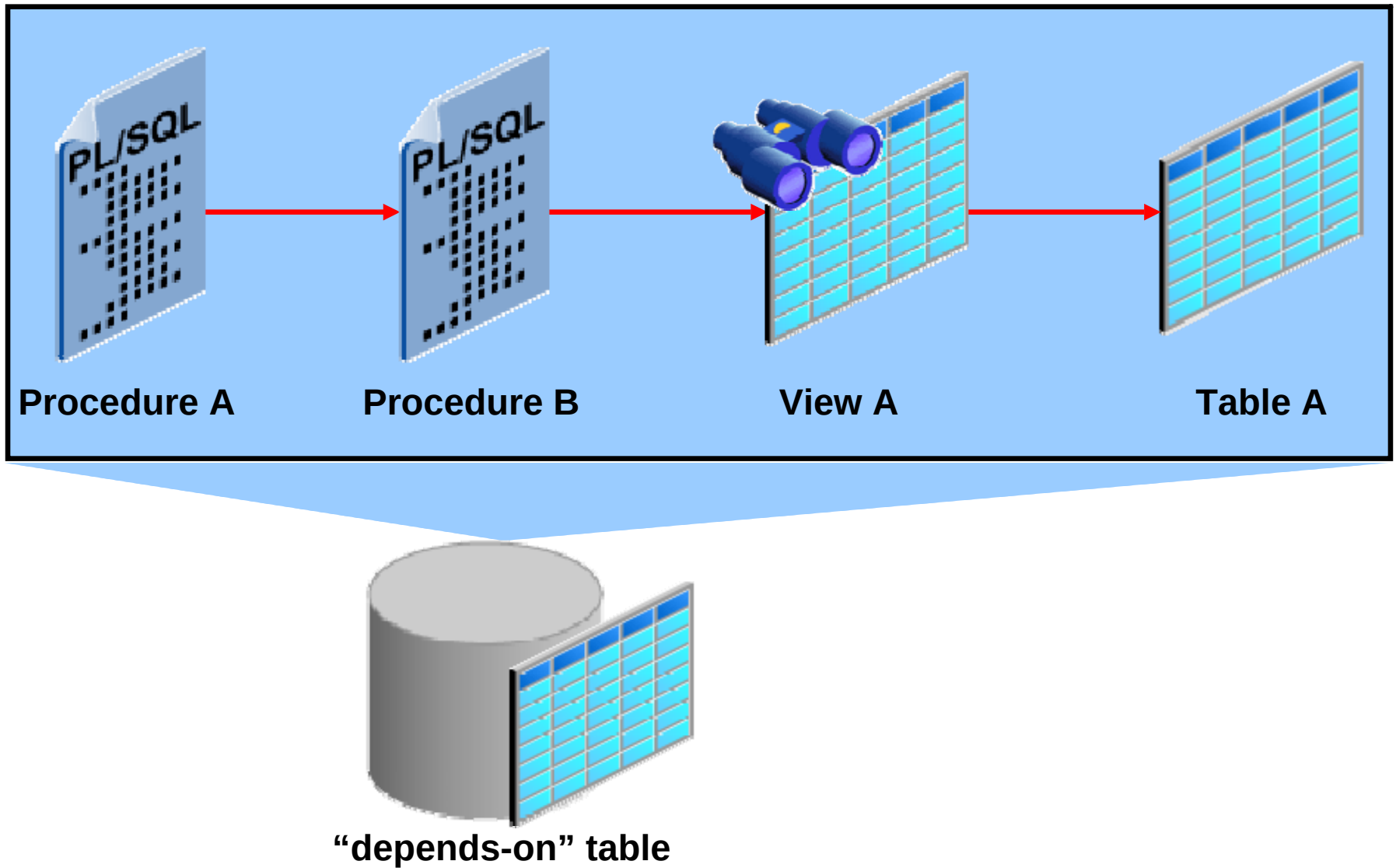
Overview of Schema Object Dependencies

Object Type	Can Be Dependent or Referenced
Package body	Dependent only
Package specification	Both
Sequence	Referenced only
Subprogram	Both
Synonym	Both
Table	Both
Trigger	Both
User-defined object	Both
User-defined collection	Both
View	Both

Dependencies



Direct Local Dependencies



Querying Direct Object Dependencies: Using the USER_DEPENDENCIES View

```
DESC User_dependencies
Name                               Null    Type
-----
NAME                               NOT NULL VARCHAR2(30)
TYPE                               VARCHAR2(17)
REFERENCED_OWNER                   VARCHAR2(30)
REFERENCED_NAME                    VARCHAR2(64)
REFERENCED_TYPE                    VARCHAR2(17)
REFERENCED_LINK_NAME              VARCHAR2(128)
SCHEMAID                          NUMBER
DEPENDENCY_TYPE                   VARCHAR2(4)

8 rows selected
```

```
SELECT name, type, referenced_name, referenced_type
FROM   user_dependencies
WHERE  referenced_name IN ( 'EMPLOYEES', 'EMP_VW' );
```

Results:				
	NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE
1	QUERY_EMP	PROCEDURE	EMPLOYEES	TABLE
2	DML_CALL_SQL	FUNCTION	EMPLOYEES	TABLE
3	QUERY_CALL_SQL	FUNCTION	EMPLOYEES	TABLE
4	COMM_PKG	PACKAGE BODY	EMPLOYEES	TABLE
5	CURS_PKG	PACKAGE BODY	EMPLOYEES	TABLE
6	EMP_PKG	PACKAGE	EMPLOYEES	TABLE
7	EMP_PKG	PACKAGE BODY	EMPLOYEES	TABLE

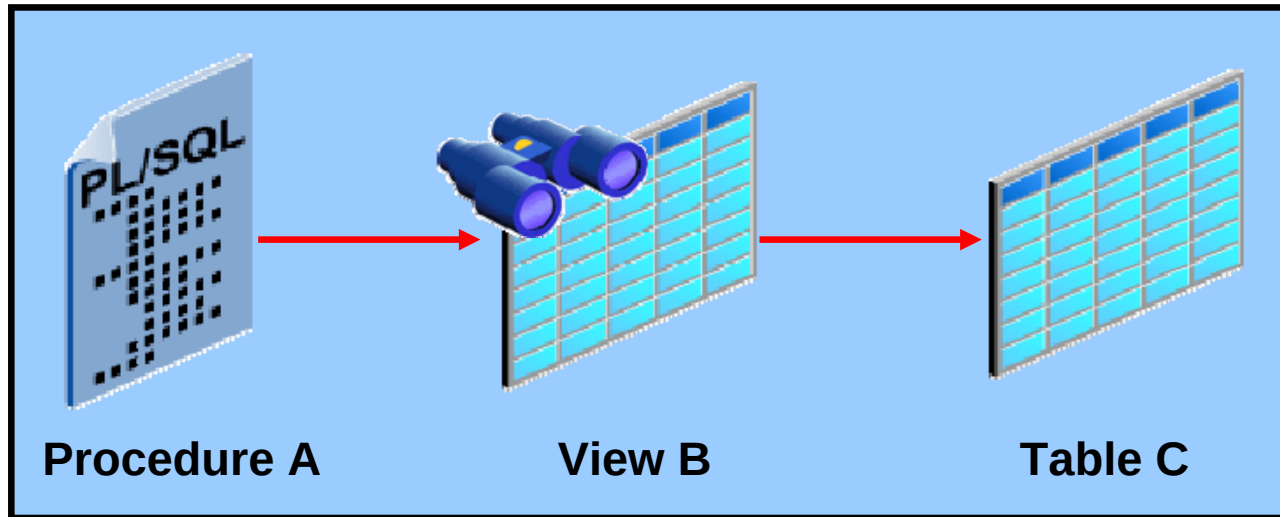
...

Querying an Object's Status

Every database object has one of the following status values:

Status	Description
VALID	The object was successfully compiled, using the current definition in the data dictionary.
COMPILED WITH ERRORS	The most recent attempt to compile the object produced errors.
INVALID	The object is marked invalid because an object that it references has changed. (Only a dependent object can be invalid.)
UNAUTHORIZED	An access privilege on a referenced object was revoked. (Only a dependent object can be unauthorized.)

Invalidation of Dependent Objects



- Procedure A is a direct dependent of View B. View B is a direct dependent of Table C. Procedure A is an indirect dependent of Table C.
- Direct dependents are invalidated only by changes to the referenced object that affect them.
- Indirect dependents can be invalidated by changes to the reference object that do not affect them.

Schema Object Change That Invalidates Some Dependents: Example

```
CREATE VIEW commissioned AS  
SELECT first_name, last_name, commission_pct FROM employees  
WHERE commission_pct > 0.00;
```

```
CREATE VIEW six_figure_salary AS  
SELECT * FROM employees  
WHERE salary >= 100000;
```

```
SELECT object_name, status  
FROM user_objects  
WHERE object_type = 'VIEW';
```

Results:

	OBJECT_NAME	STATUS
1	EMP_DETAILS	VALID
2	COMMISSIONED	VALID
3	SIX_FIGURE_SALARY	VALID
4	EMP_DETAILS_VIEW	VALID

Schema Object Change That Invalidates Some Dependents: Example

```
ALTER TABLE employees MODIFY email VARCHAR2(50);
```

```
SELECT object_name, status  
FROM user_objects  
WHERE object_type = 'VIEW';
```

Results:

	OBJECT_NAME	STATUS
1	EMP_DETAILS	VALID
2	COMMISSIONED	VALID
3	SIX_FIGURE_SALARY	INVALID
4	EMP_DETAILS_VIEW	VALID

Displaying Direct and Indirect Dependencies

1. Run the `utldtree.sql` script that creates the objects that enable you to display the direct and indirect dependencies.

```
@D:/Labs/plpu/labs/utldtree.sql
```

2. Execute the `DEPTREE_FILL` procedure.

```
EXECUTE deptree_fill('TABLE', 'ORA61', 'EMPLOYEES')
```

Displaying Dependencies Using the DEPTREE View

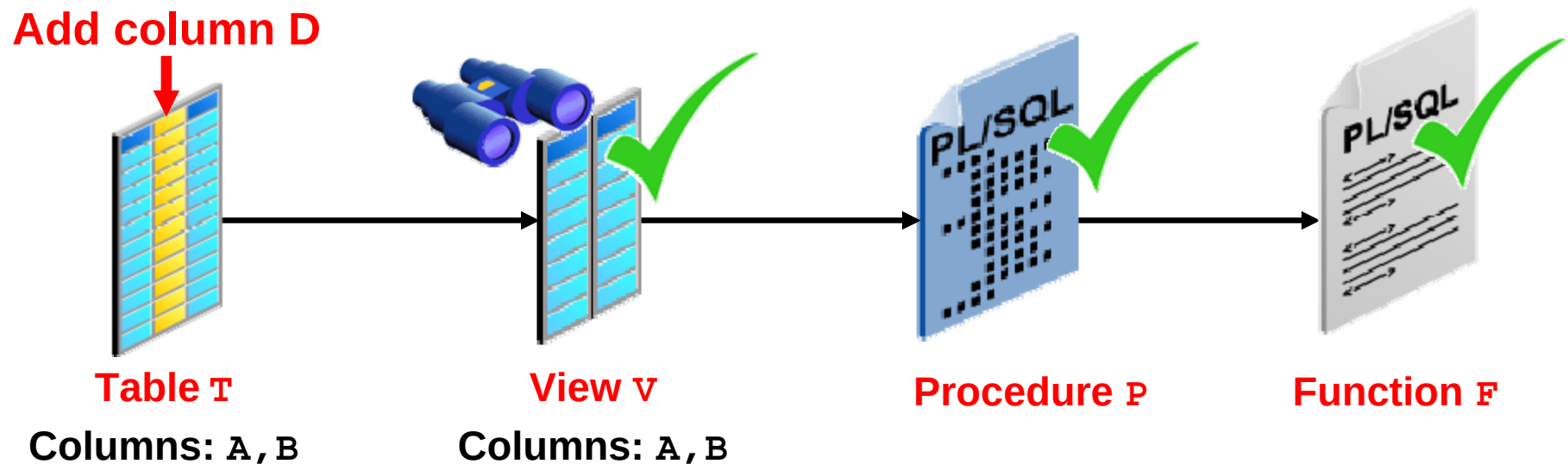
```
SELECT    nested_level, type, name
FROM      deptree
ORDER BY  seq#;
```

NESTED_LEVEL	TYPE	NAME
0	TABLE	EMPLOYEES
1	VIEW	EMP_DETAILS_VIEW
1	TRIGGER	SECURE_EMPLOYEES
1	TRIGGER	UPDATE_JOB_HISTORY
1	PROCEDURE	RAISE_SALARY
2	PROCEDURE	PROCESS_EMPLOYEES
1	PROCEDURE	QUERY_EMP
1	PROCEDURE	PROCESS_EMPLOYEES
1	FUNCTION	DML_CALL_SQL
1	FUNCTION	QUERY_CALL_SQL
1	PACKAGE BODY	COMM_PKG
1	PACKAGE BODY	CURS_PKG
1	PACKAGE	EMP_PKG
2	PACKAGE BODY	EMP_PKG
1	PACKAGE BODY	EMP_PKG
1	PROCEDURE	SAL_STATUS

...

More Precise Dependency Metadata in Oracle Database 11g

- Before 11g, adding column D to table T invalidated the dependent objects.
- Oracle Database 11g records additional, finer-grained dependency management:
 - Adding column D to table T does not impact view v and does not invalidate the dependent objects



Fine-Grained Dependency Management

- In Oracle Database 11g, dependencies are now tracked at the level of *element within unit*.
- Element-based dependency tracking covers the following:
 - Dependency of a single-table view on its base table
 - Dependency of a PL/SQL program unit (package specification, package body, or subprogram) on the following:
 - Other PL/SQL program units
 - Tables
 - Views



Fine-Grained Dependency Management: Example 1

```
CREATE TABLE t2 (col_a NUMBER, col_b NUMBER, col_c NUMBER);  
CREATE VIEW v AS SELECT col_a, col_b FROM t2;
```

```
SELECT ud.name, ud.type, ud.referenced_name,  
       ud.referenced_type, uo.status  
FROM user_dependencies ud, user_objects uo  
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:

NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1 V	VIEW	T2	TABLE	VALID

```
ALTER TABLE t2 ADD (col_d VARCHAR2(20));
```

```
SELECT ud.name, ud.type, ud.referenced_name,  
       ud.referenced_type, uo.status  
FROM user_dependencies ud, user_objects uo  
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:

NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1 V	VIEW	T2	TABLE	VALID

Fine-Grained Dependency Management: Example 1

```
ALTER TABLE t2 MODIFY (col_a VARCHAR2(20));  
SELECT ud.name, ud.referenced_name, ud.referenced_type,  
       uo.status  
FROM user_dependencies ud, user_objects uo  
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:

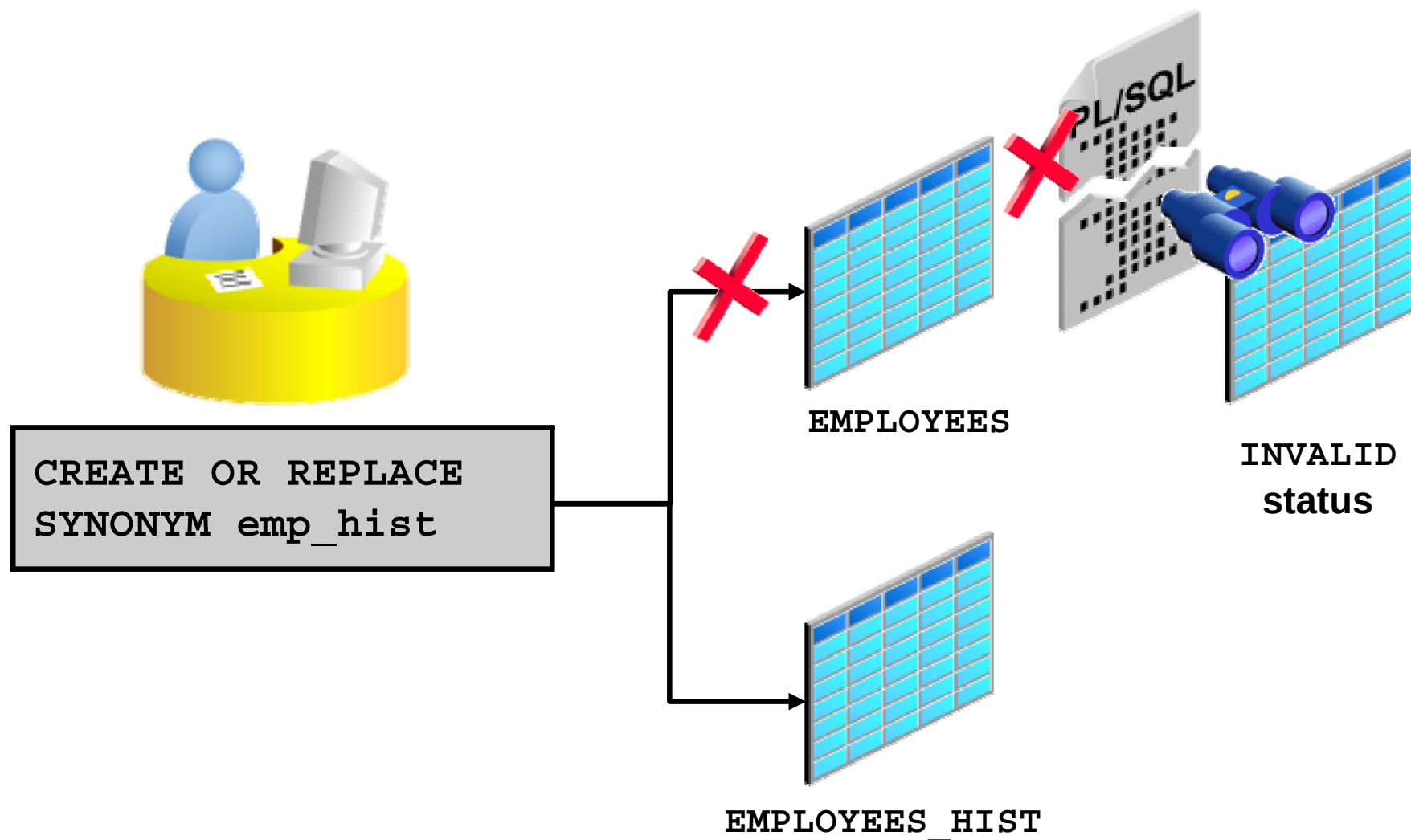
NAME	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1 V	T2	TABLE	INVALID

Fine-Grained Dependency Management: Example 2

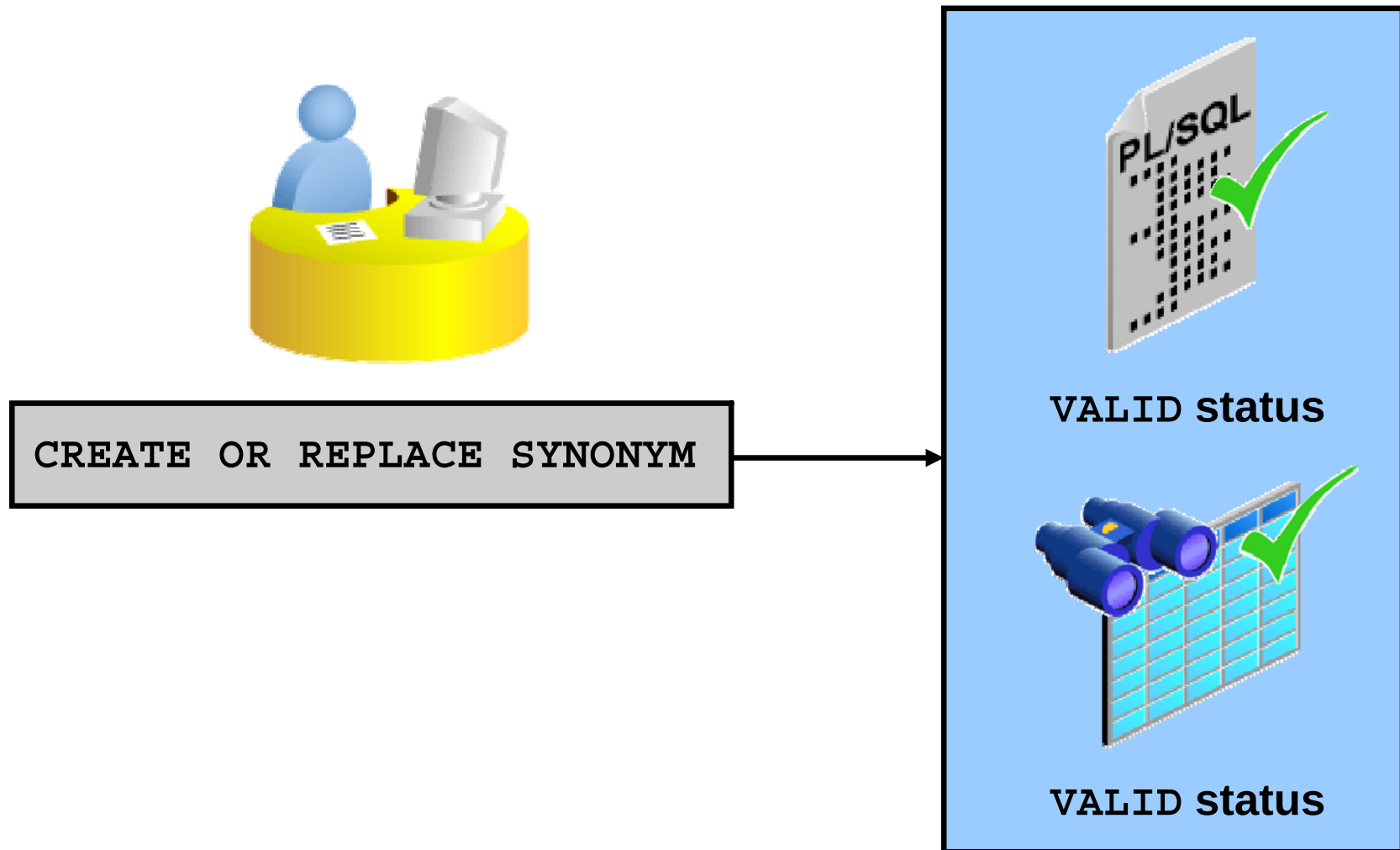
```
CREATE PACKAGE pkg IS
  PROCEDURE proc_1;
END pkg;
/
CREATE OR REPLACE PROCEDURE p IS
BEGIN
  pkg.proc_1();
END p;
/
CREATE OR REPLACE PACKAGE pkg
IS
  PROCEDURE proc_1;
  PROCEDURE unheard_of;
END pkg;
/
```

```
PACKAGE pkg Compiled.
PROCEDURE p Compiled.
PACKAGE pkg Compiled.
```

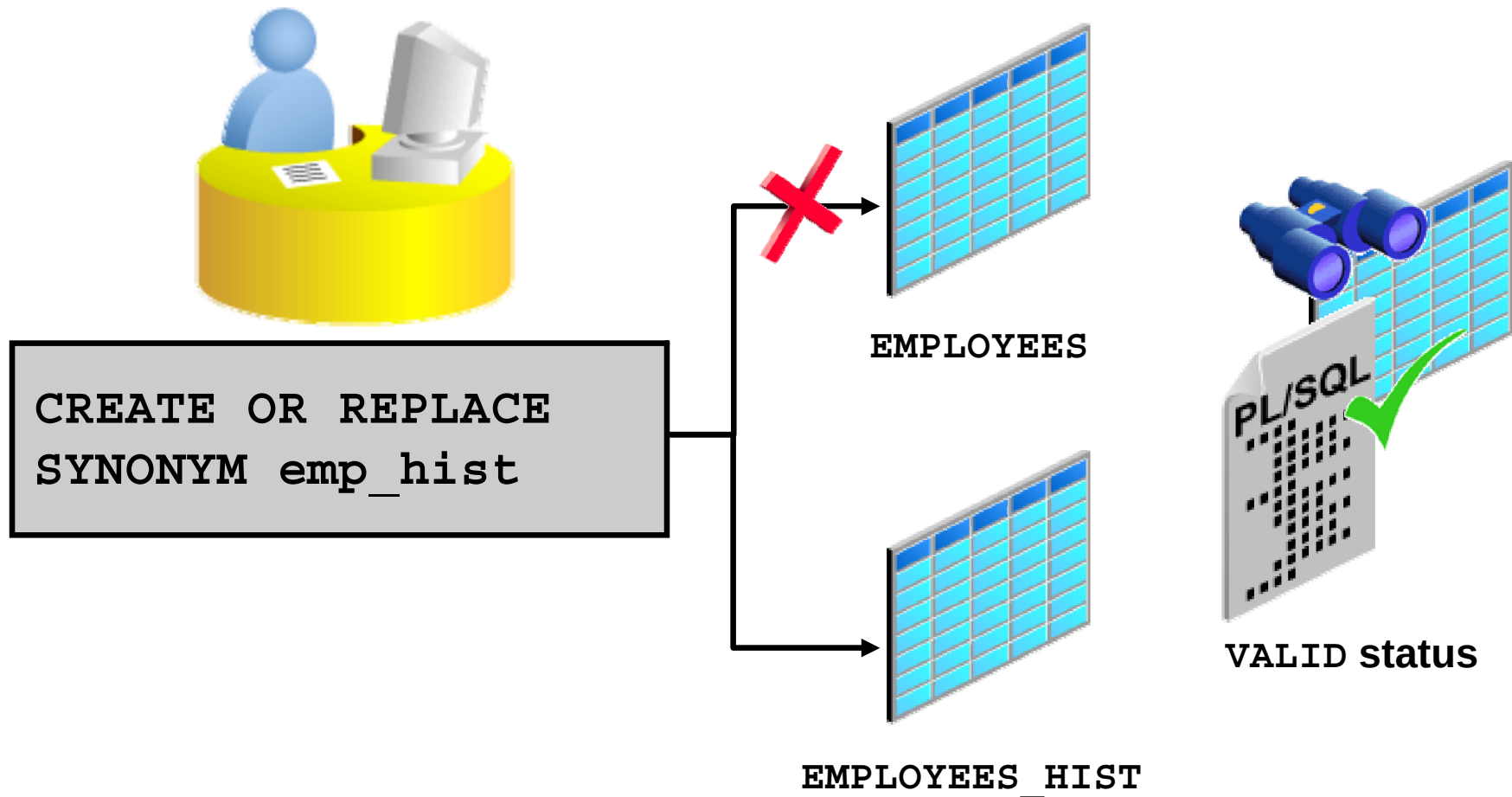
Impact of Redefining Synonyms Before Oracle Database 10g



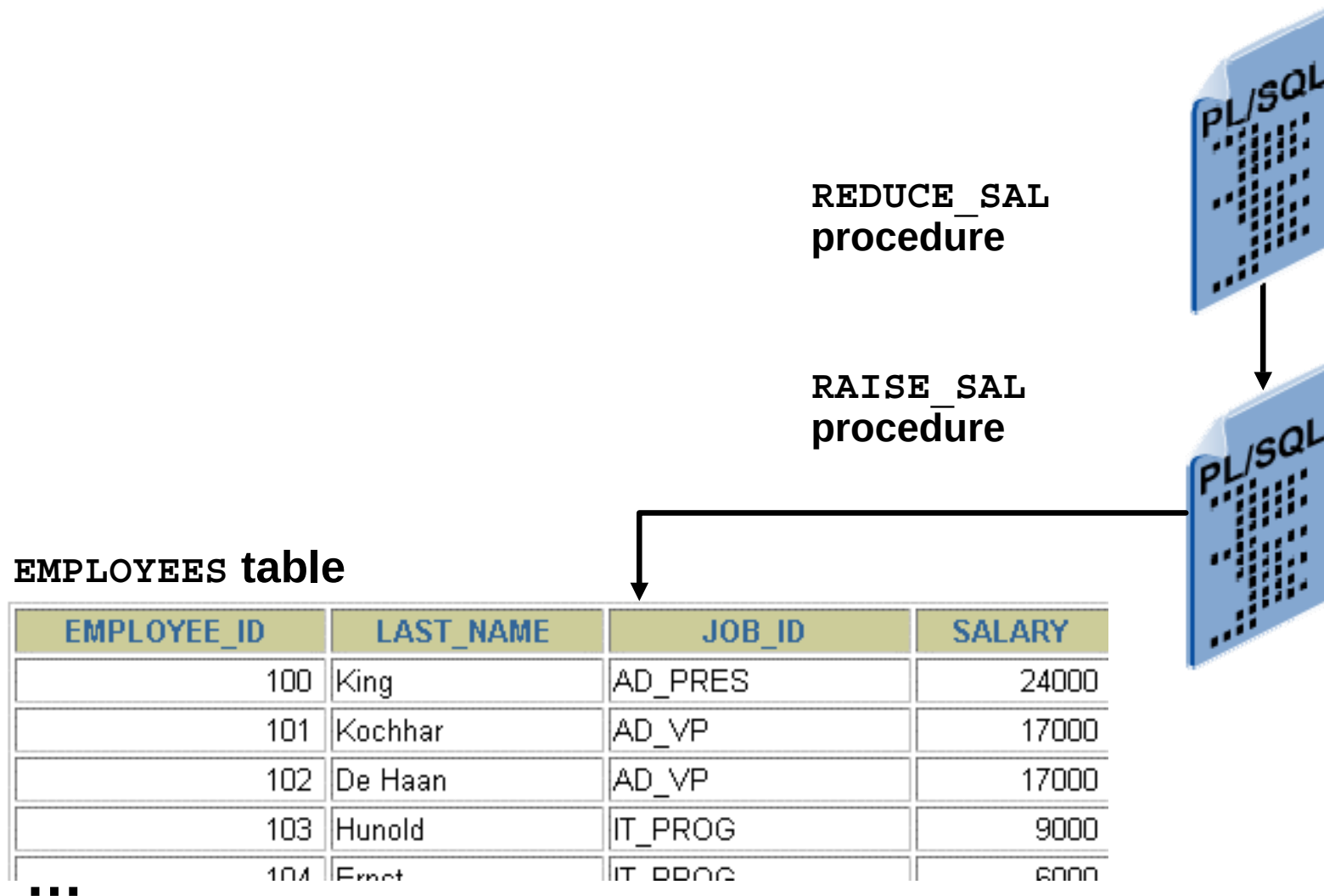
Changes to Synonym Dependencies Starting with Oracle Database 10g



Maintaining Valid PL/SQL Program Units and Views

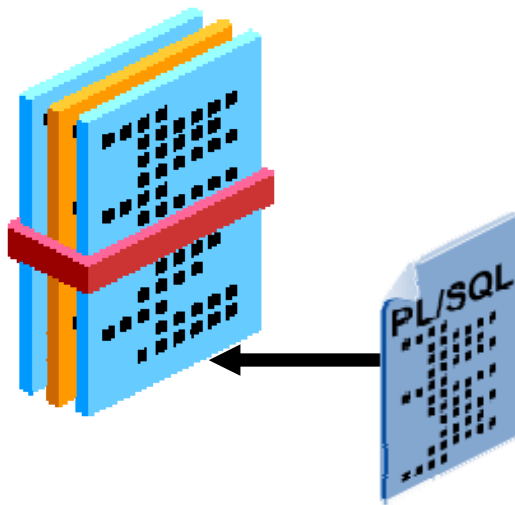


Another Scenario of Local Dependencies

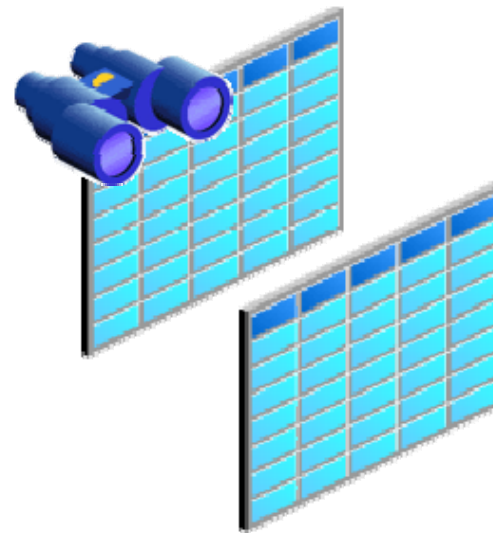


Guidelines for Reducing Invalidation

To reduce invalidation of dependent objects:



Add new items to the end of the package

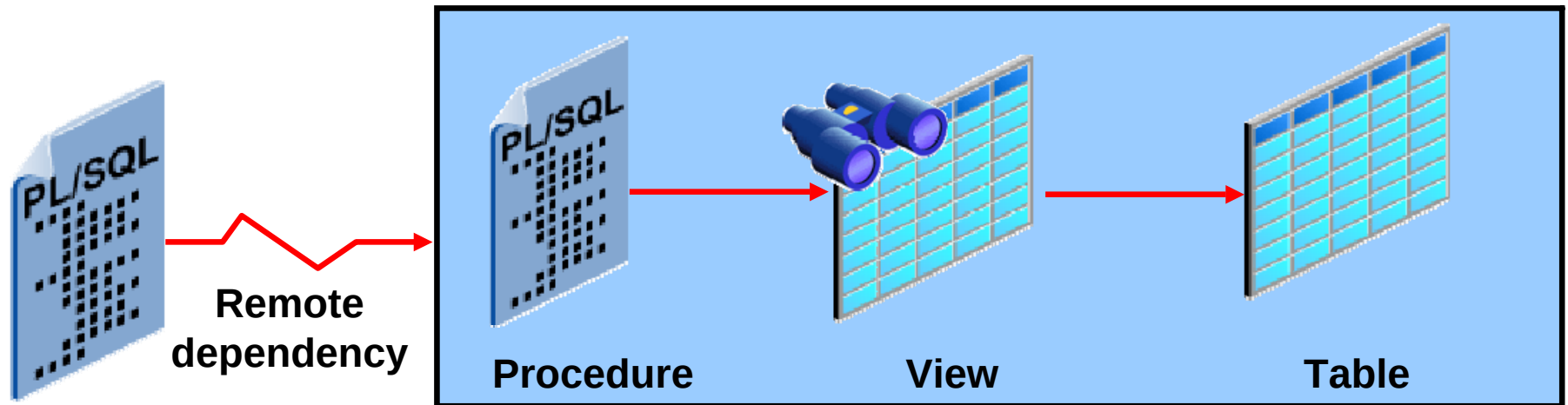


Reference each table through a view

Object Revalidation

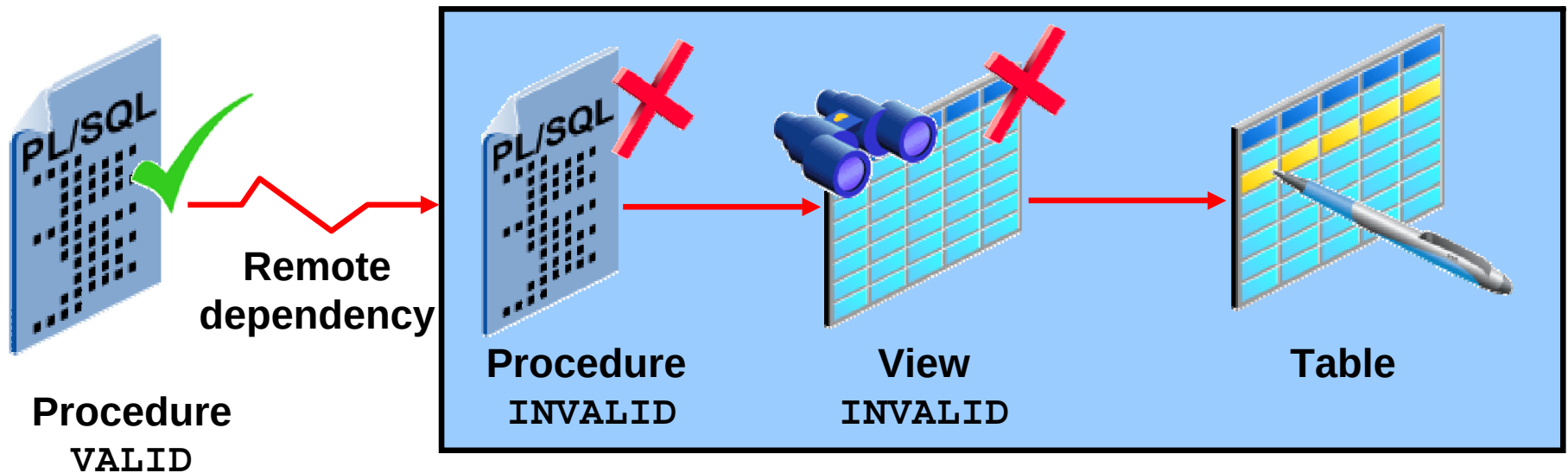
- An object that is not valid when it is referenced must be validated before it can be used.
- Validation occurs automatically when an object is referenced; it does not require explicit user action.
- If an object is not valid, its status is either `COMPILED WITH ERRORS`, `UNAUTHORIZED`, or `INVALID`.

Remote Dependencies



Procedure

Local and remote references



Procedure
VALID

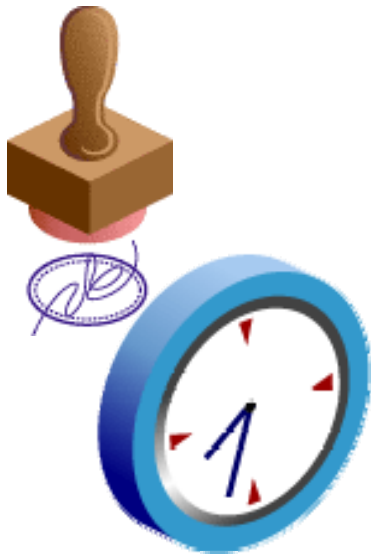
Procedure
INVALID

View
INVALID

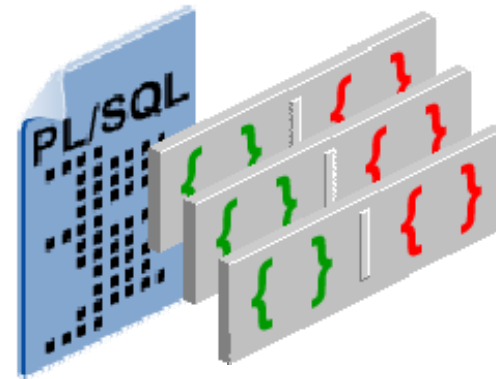
Table

Concepts of Remote Dependencies

Remote dependencies are governed by the mode that is chosen by the user:



TIMESTAMP checking

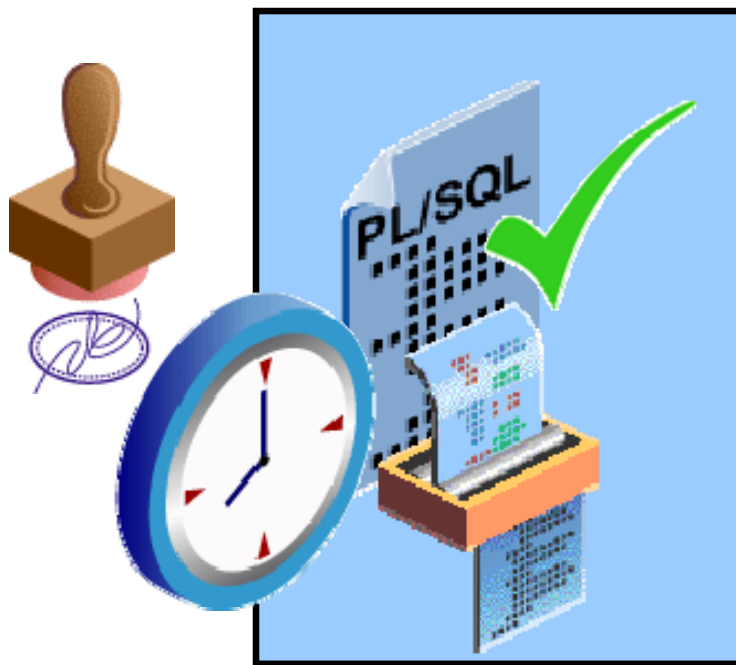


SIGNATURE checking

Setting the REMOTE_DEPENDENCIES_MODE Parameter

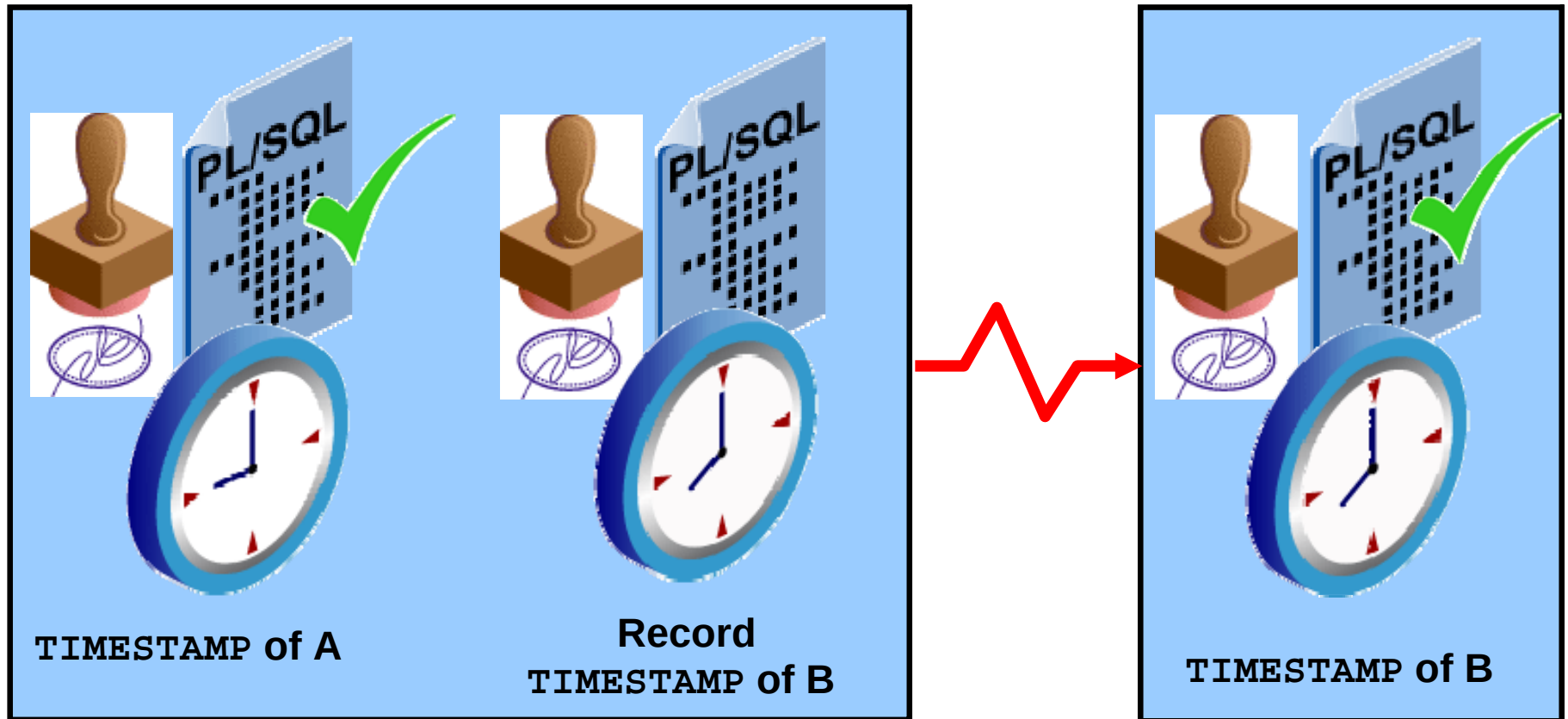
- As an `init.ora` parameter:
`REMOTE_DEPENDENCIES_MODE = value`
- At the system level:
`ALTER SYSTEM SET REMOTE_DEPENDENCIES_MODE = value`
- At the session level:
`ALTER SESSION SET REMOTE_DEPENDENCIES_MODE = value`

Remote Procedure B Compiles at 8:00 AM



**Remote procedure B:
Compiles and is `VALID`
at 8:00 AM**

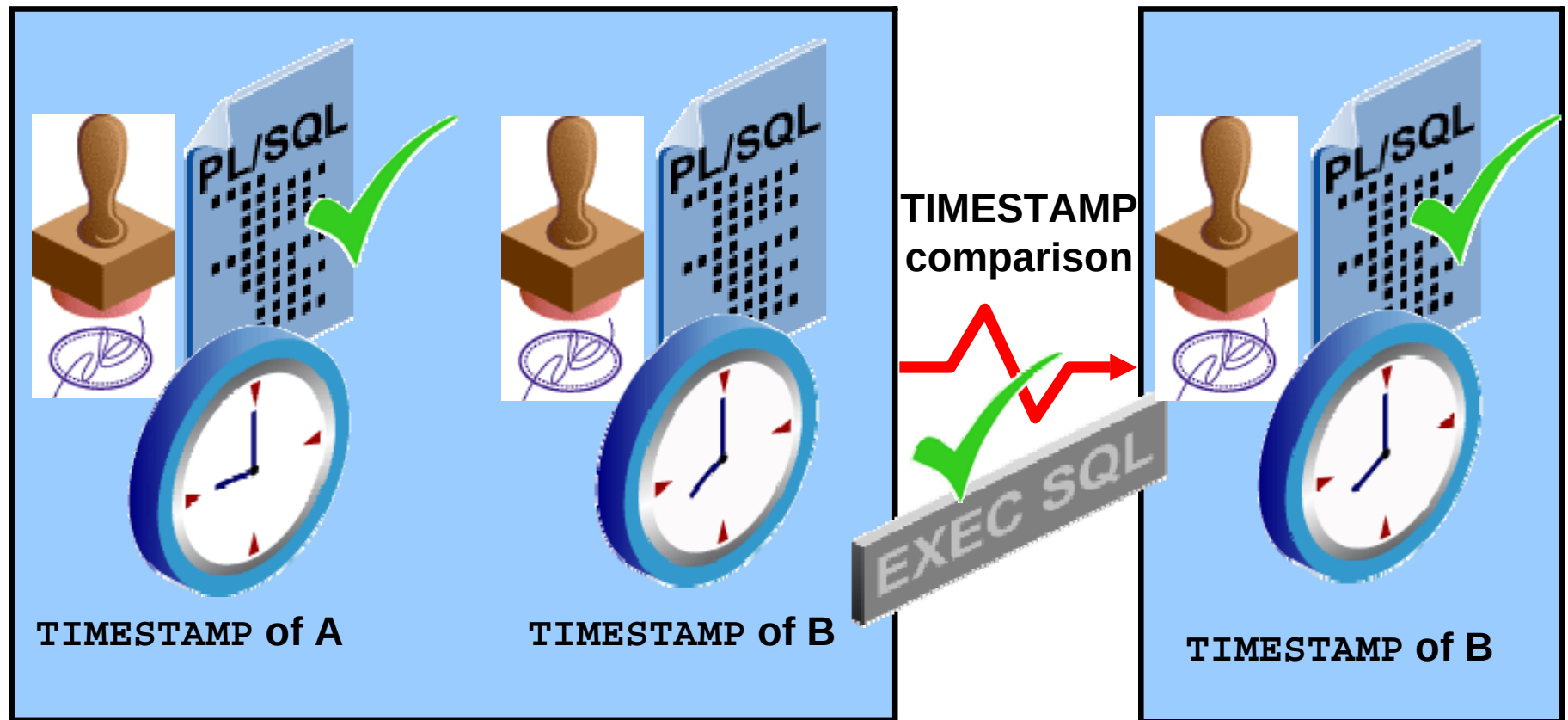
Local Procedure A Compiles at 9:00 AM



Local procedure A: VALID

Remote procedure
B: VALID

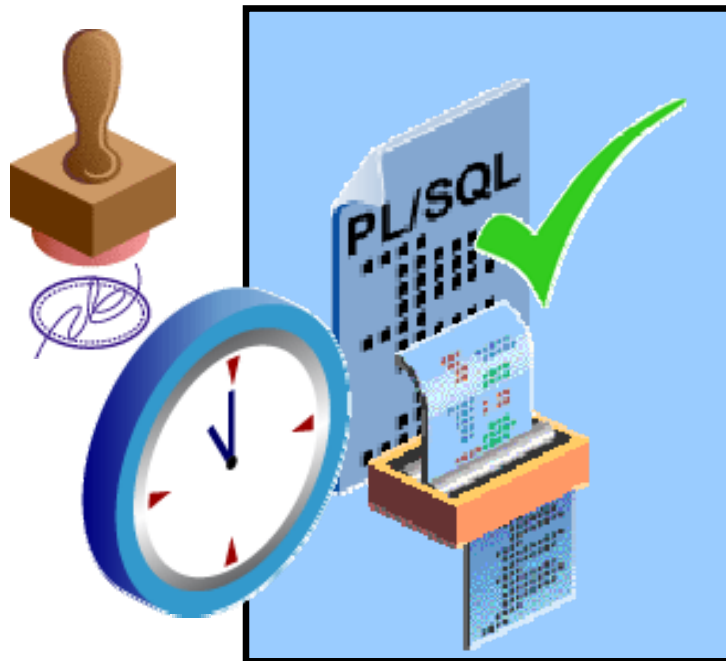
Execute Procedure A



Local procedure A: VALID

Remote procedure B: VALID

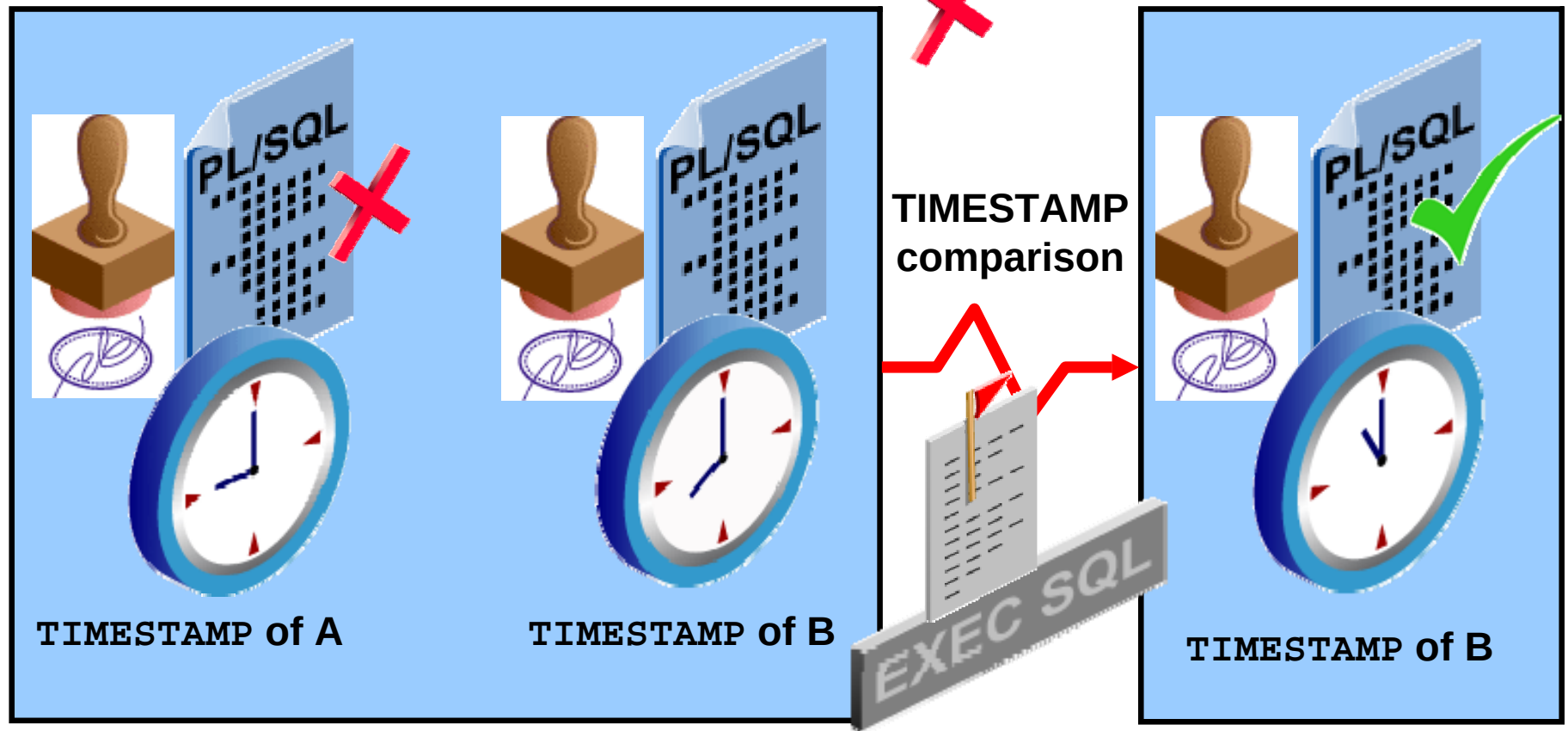
Remote Procedure B Recompiled at 11:00 AM



**Remote procedure B:
Recompiles and is VALID
at 11:00 AM**

Execute Procedure A

Saved **TIMESTAMP** of B **!=** **COMPILE TIME** of B



Local procedure A: INVALID

Remote procedure
B: VALID

Signature Mode

- The signature of a procedure is:
 - The name of the procedure
 - The data types of the parameters
 - The modes of the parameters
- The signature of the remote procedure is saved in the local procedure.
- When executing a dependent procedure, the signature of the referenced remote procedure is compared.

Recompiling a PL/SQL Program Unit

Recompilation:

- Is handled automatically through implicit run-time recompilation
- Is handled through explicit recompilation with the ALTER statement

```
ALTER PROCEDURE [SCHEMA.] procedure_name COMPILE;
```

```
ALTER FUNCTION [SCHEMA.] function_name COMPILE;
```

```
ALTER PACKAGE [SCHEMA.] package_name  
COMPILE [PACKAGE | SPECIFICATION | BODY];
```

```
ALTER TRIGGER trigger_name [COMPILE [DEBUG]];
```

Unsuccessful Recompilation

Recompiling dependent procedures and functions is unsuccessful when:

- The referenced object is dropped or renamed
- The data type of the referenced column is changed
- The referenced column is dropped
- A referenced view is replaced by a view with different columns
- The parameter list of a referenced procedure is modified

Successful Recompilation

Recompiling dependent procedures and functions is successful if:

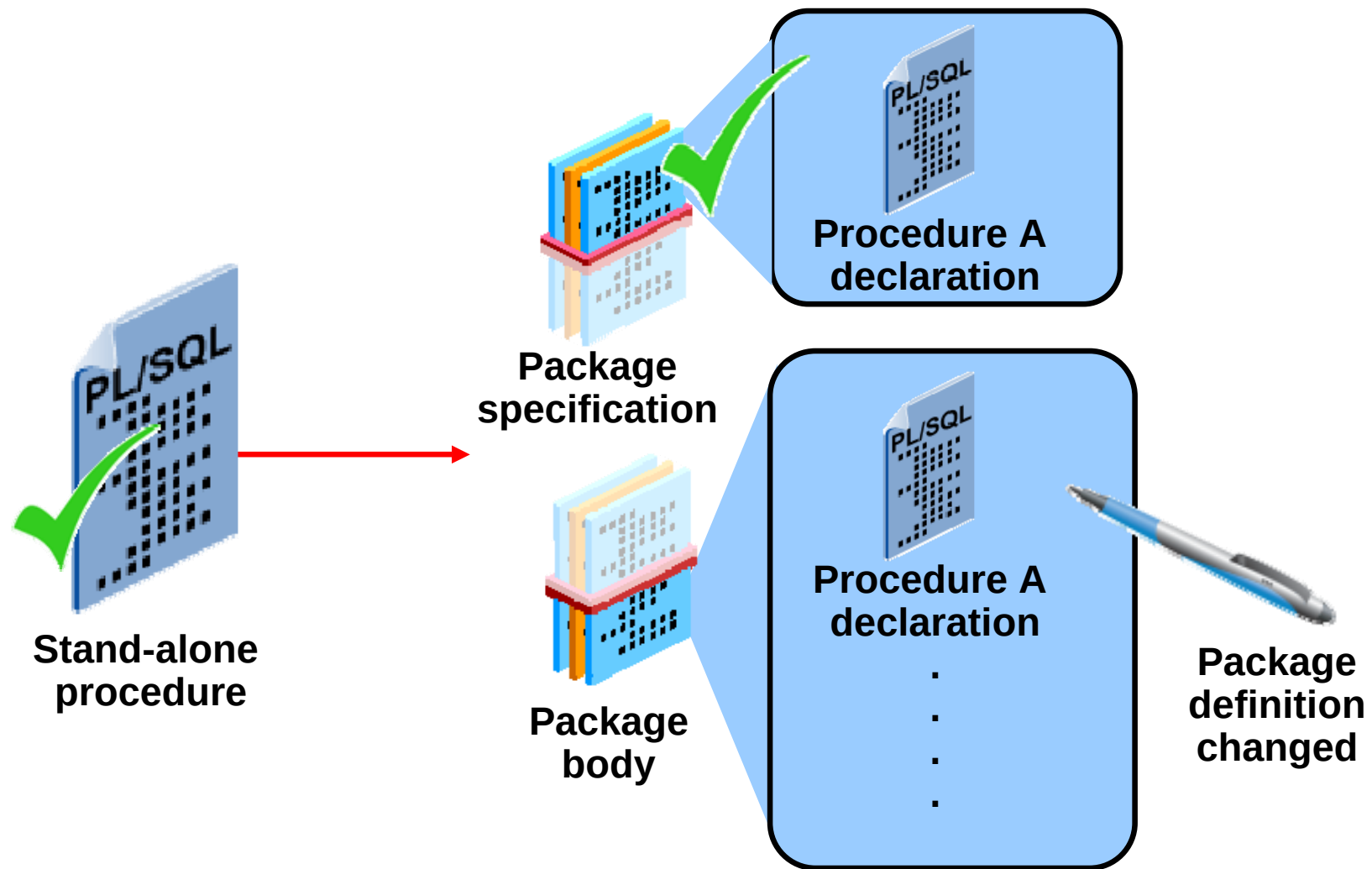
- The referenced table has new columns
- The data type of referenced columns has not changed
- A private table is dropped, but a public table that has the same name and structure exists
- The PL/SQL body of a referenced procedure has been modified and recompiled successfully

Recompiling Procedures

Minimize dependency failures by:

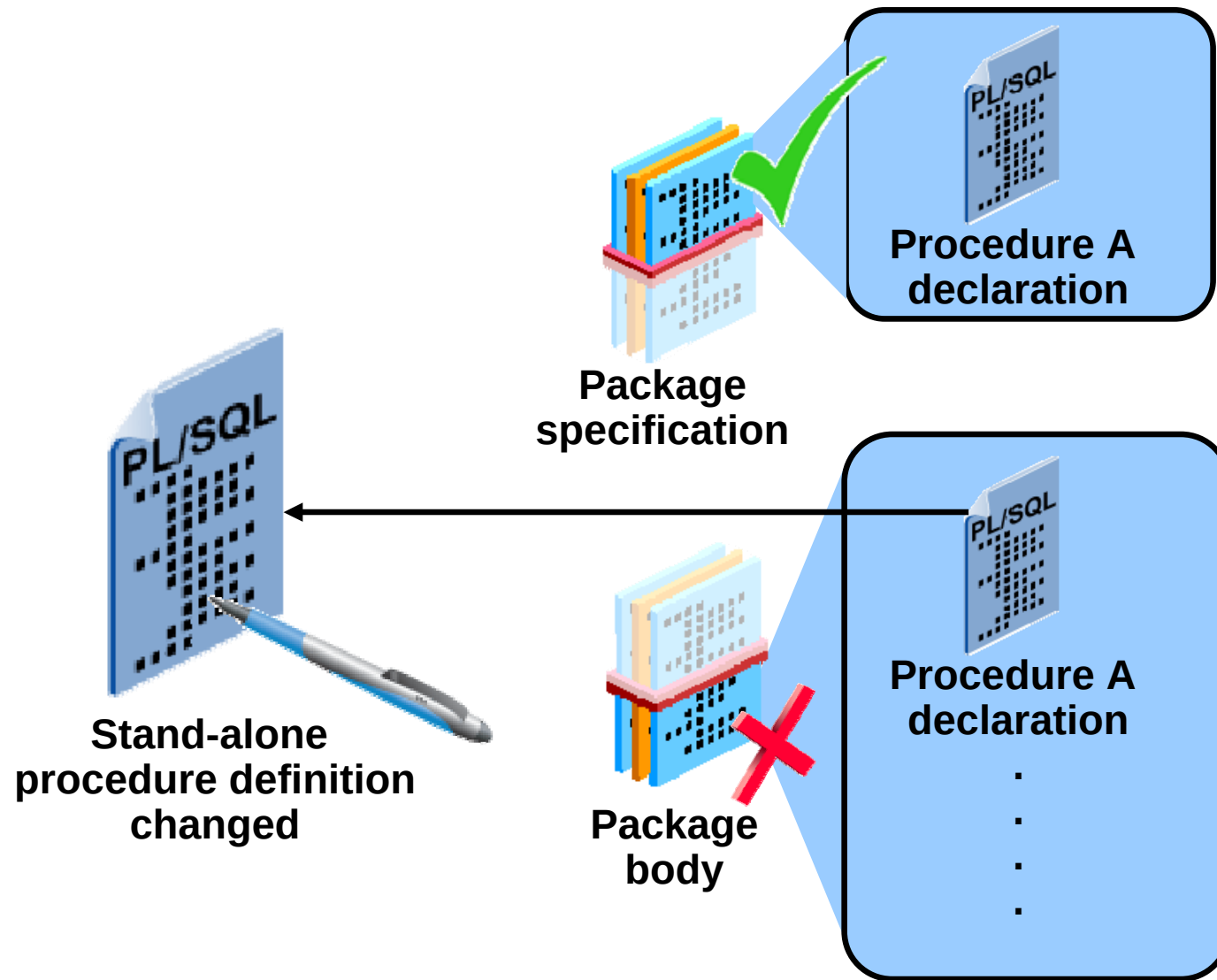
- Declaring records with the `%ROWTYPE` attribute
- Declaring variables with the `%TYPE` attribute
- Querying with the `SELECT *` notation
- Including a column list with `INSERT` statements

Packages and Dependencies: Subprogram References the Package



Packages and Dependencies:

Package Subprogram References Procedure



Quiz

You can display direct and indirect dependencies by running the `utldtree.sql` script, populating the `DEPTREE_TEMPTAB` table with information for a particular referenced object, and querying the `DEPTREE` or `IDETREE` views.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Track procedural dependencies
- Predict the effect of changing a database object on procedures and functions
- Manage procedural dependencies

Practice 13 Overview: Managing Dependencies in Your Schema

This practice covers the following topics:

- Using `DEPTREE_FILL` and `IDETREE` to view dependencies
- Recompiling procedures, functions, and packages



Using SQL Developer

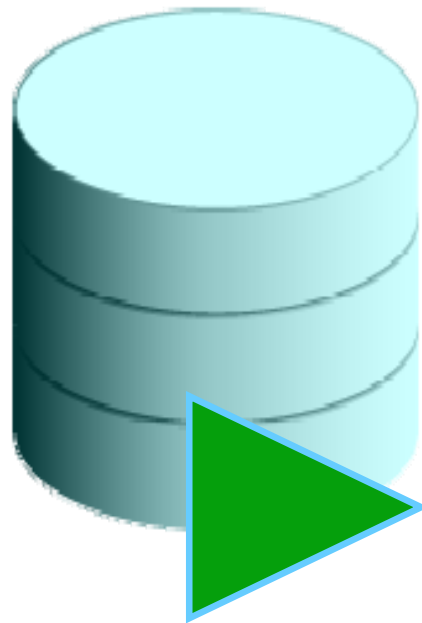
Objectives

After completing this appendix, you should be able to do the following:

- List the key features of Oracle SQL Developer
- Install Oracle SQL Developer 1.2.1
- Identify menu items of Oracle SQL Developer
- Create a database connection
- Manage database objects
- Use SQL Worksheet
- Save and Run SQL scripts
- Create and save reports
- Install and use Oracle SQL Developer 1.5.3

What Is Oracle SQL Developer?

- Oracle SQL Developer is a graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.



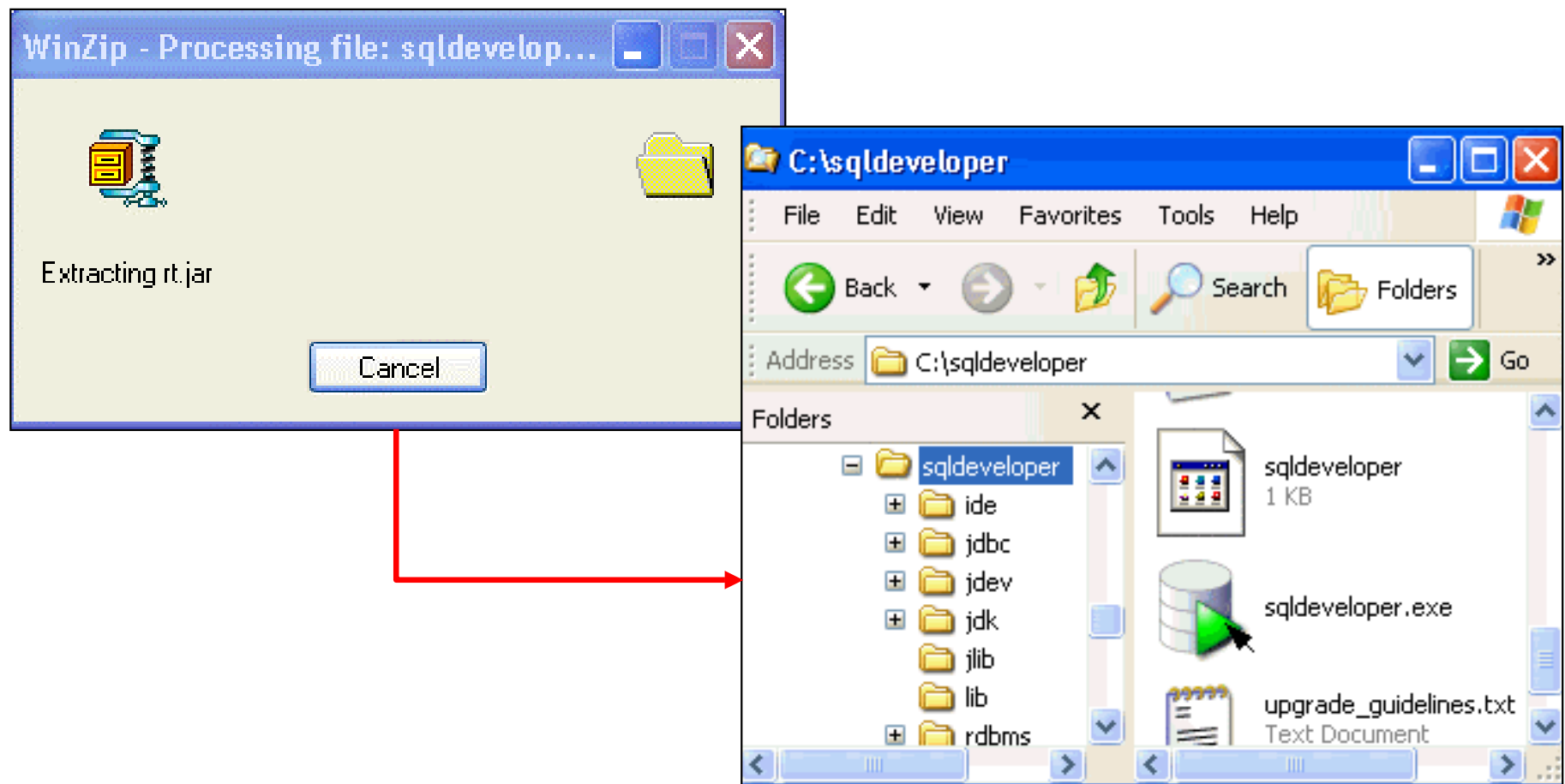
SQL Developer

Specifications of SQL Developer

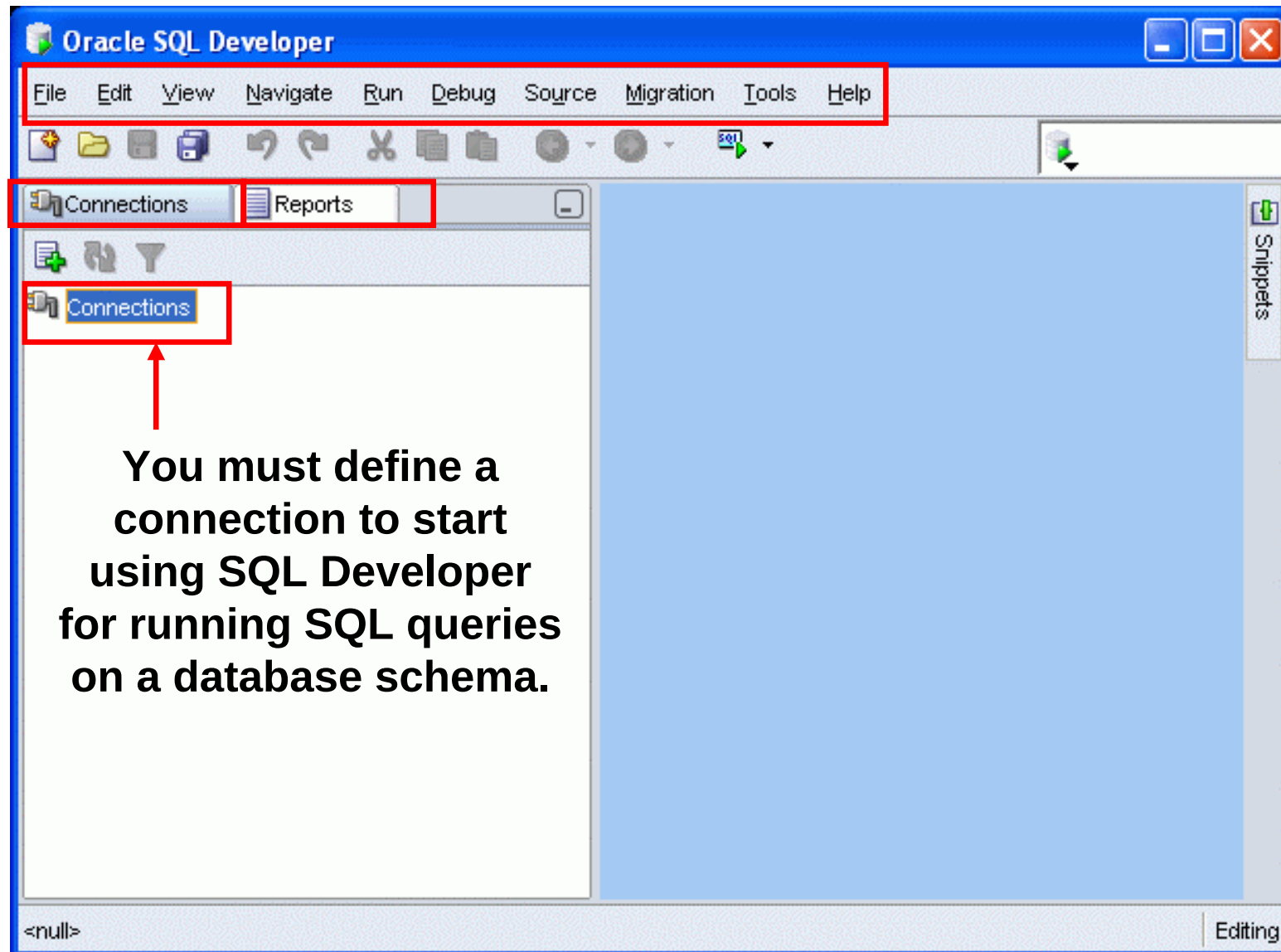
- Developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Default connectivity by using the JDBC Thin driver
- Does not require an installer
 - Unzip the downloaded SQL Developer kit and double-click `sqldeveloper.exe` to start SQL Developer.
- Connects to Oracle Database version 9.2.0.1 and later
- Freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html
- Needs JDK 1.5 installed on your system that can be downloaded from the following link:
 - http://java.sun.com/javase/downloads/index_jdk5.jsp

Installing SQL Developer

Download the Oracle SQL Developer kit and unzip into any directory on your machine.



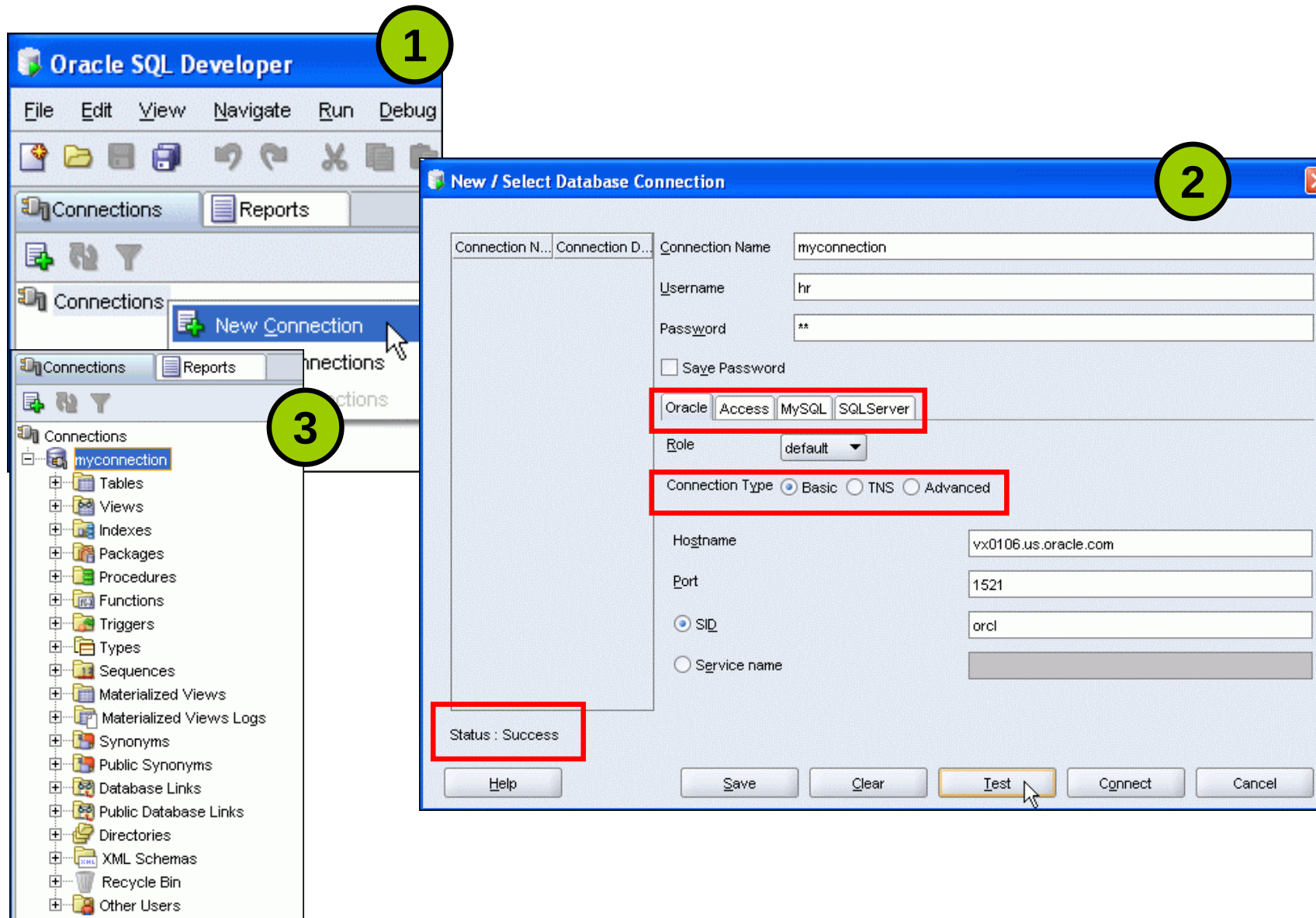
SQL Developer 1.2 Interface



Creating a Database Connection

- You must have at least one database connection to use SQL Developer.
- You can create and test connections for:
 - Multiple databases
 - Multiple schemas
- SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.
- You can export connections to an Extensible Markup Language (XML) file.
- Each additional database connection created is listed in the Connections Navigator hierarchy.

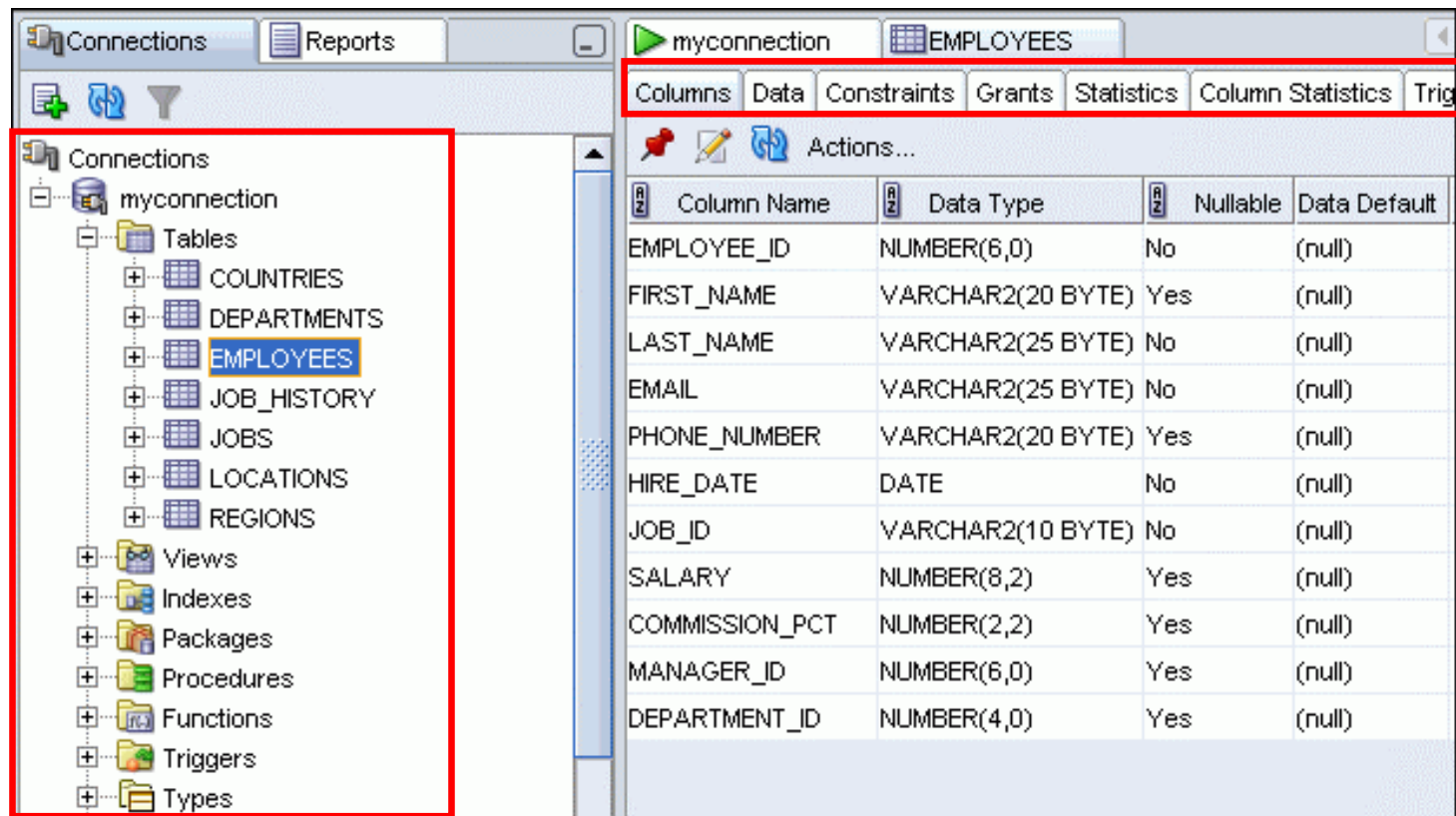
Creating a Database Connection



Browsing Database Objects

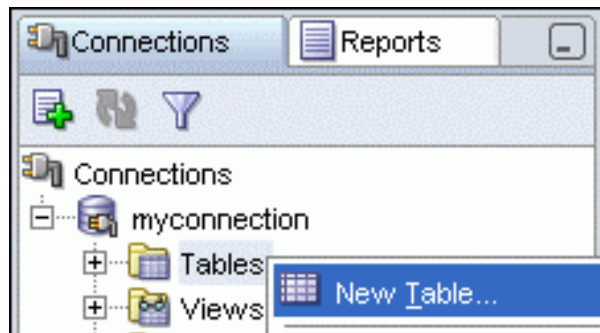
Use the Connections Navigator to:

- Browse through many objects in a database schema
- Review the definitions of objects at a glance



Creating a Schema Object

- SQL Developer supports the creation of any schema object by:
 - Executing a SQL statement in SQL Worksheet
 - Using the context menu
- Edit the objects by using an edit dialog or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.



Creating a New Table: Example

Create Table

Schema:

Name:

Table Type: ☒ Normal ☐ External ☐ Index Organized ☐ Temporary (Transaction) ☐ Temporary (Session)

☒ Advanced

Columns

- Primary Key
- Unique Constraints
- Foreign Keys
- Check Constraints
- Indexes
- Column Sequences
- Storage Options
- Lob Parameters
- Partitioning
 - Partition Definitions
 - Subpartition Templates
- Comment
- DDL

Columns:

- ID
- FIRST_NAME
- LAST_NAME
- RELATION
- BIRTHDATE

Column Properties

Name:

Datatype: ☒ Simple ☐ Complex

Type:

Precision:

Scale:

Default:

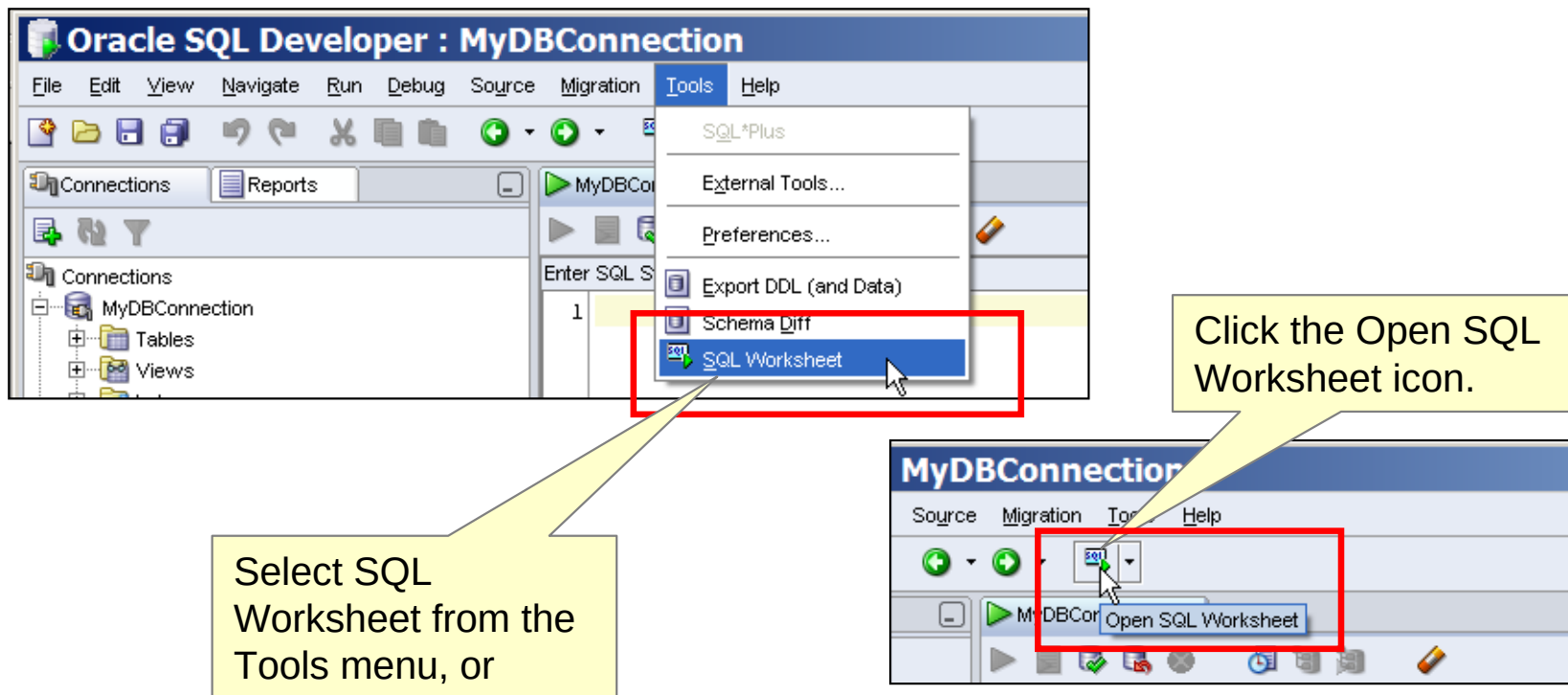
☒ Cannot be NULL

Comment:

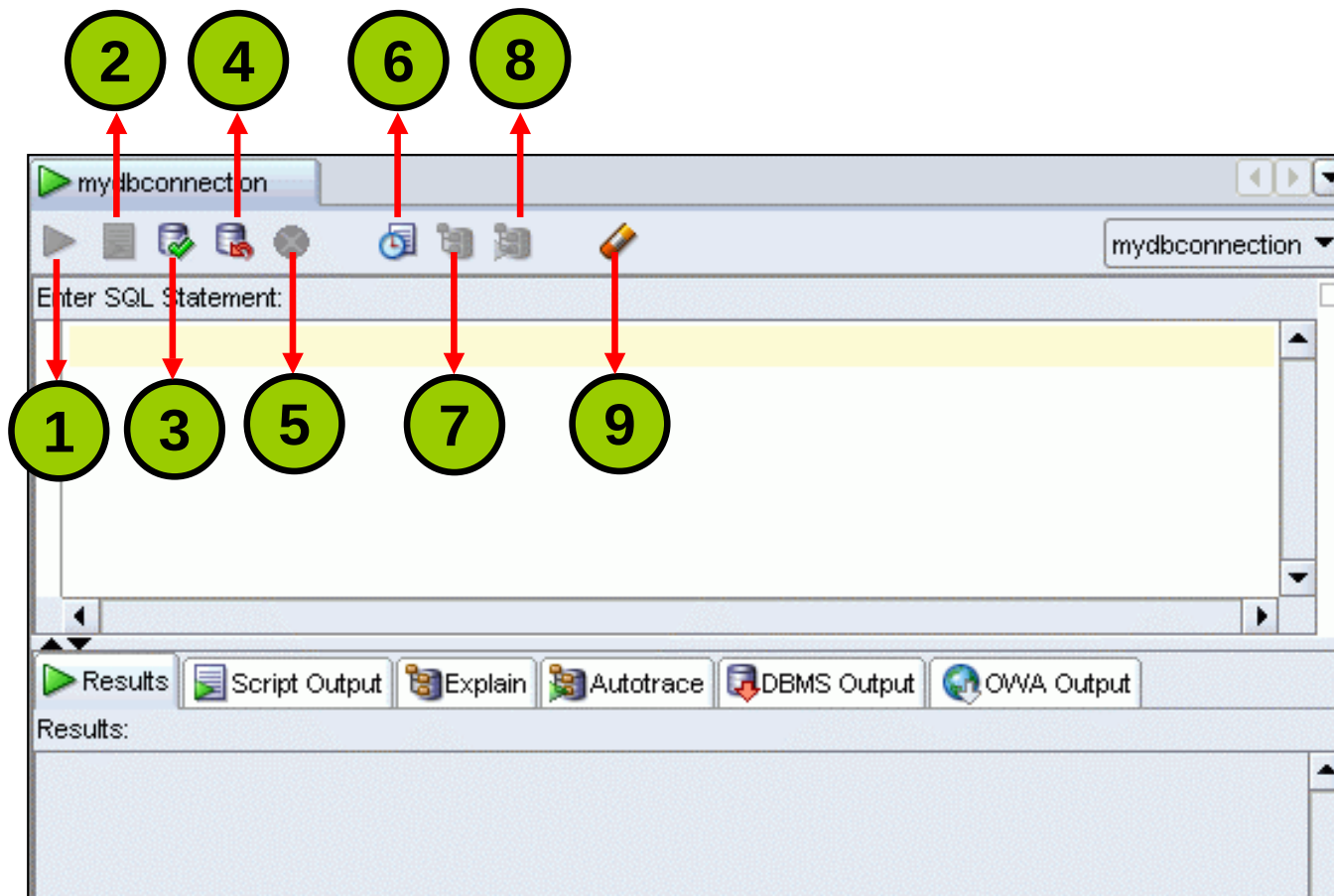
Help OK Cancel

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.

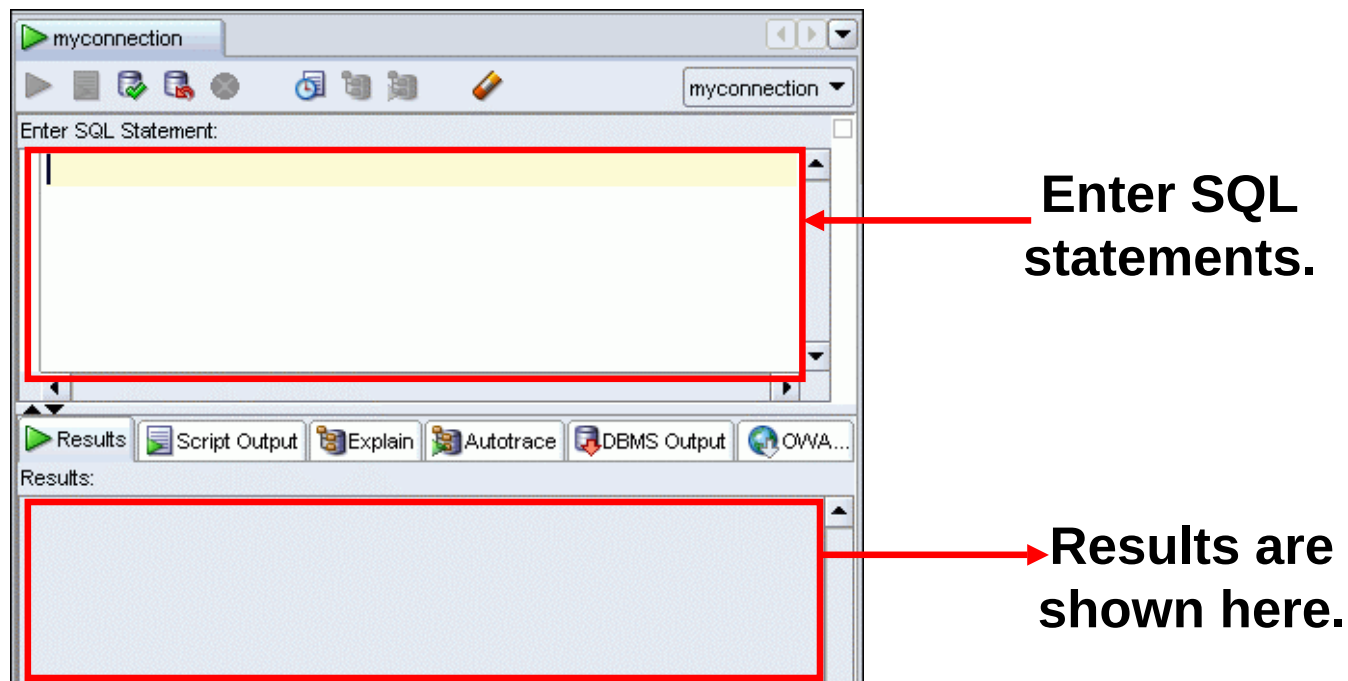


Using the SQL Worksheet



Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

Use the Enter SQL Statement box to enter single or multiple SQL statements.

View the results on the Script Output tabbed page.

The screenshot displays the Oracle SQL Developer interface. At the top, a tab labeled 'MyDBConnection' is active. Below it, a toolbar contains icons for running, saving, and other database operations. A status bar indicates '2.03304029 seconds'. The main text area, titled 'Enter SQL Statement:', contains two SQL queries. The first query is 'SELECT last_name, salary FROM employees WHERE salary > 10000;'. The second query is 'SELECT last_name "Name", salary*12 "Annual Salary" FROM employees;'. Below the text area, a toolbar includes tabs for 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. The 'Script Output' tab is selected, showing the results of the executed queries. The first query's results are displayed as a table with two columns: 'last_name' and 'salary'. The second query's results are displayed as a table with two columns: 'Name' and 'Annual Salary'.

last_name	salary
Ozer	11500
Abel	11000

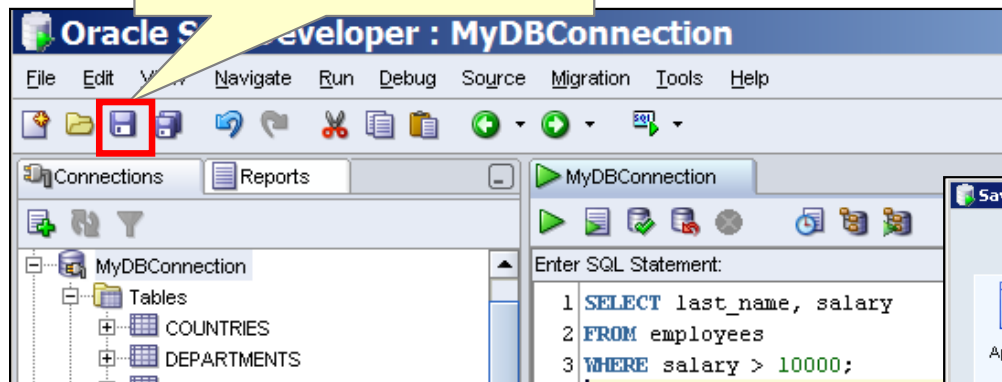
15 rows selected

Name	Annual Salary
OConnell	31200
Grant	31200
Whalen	52800

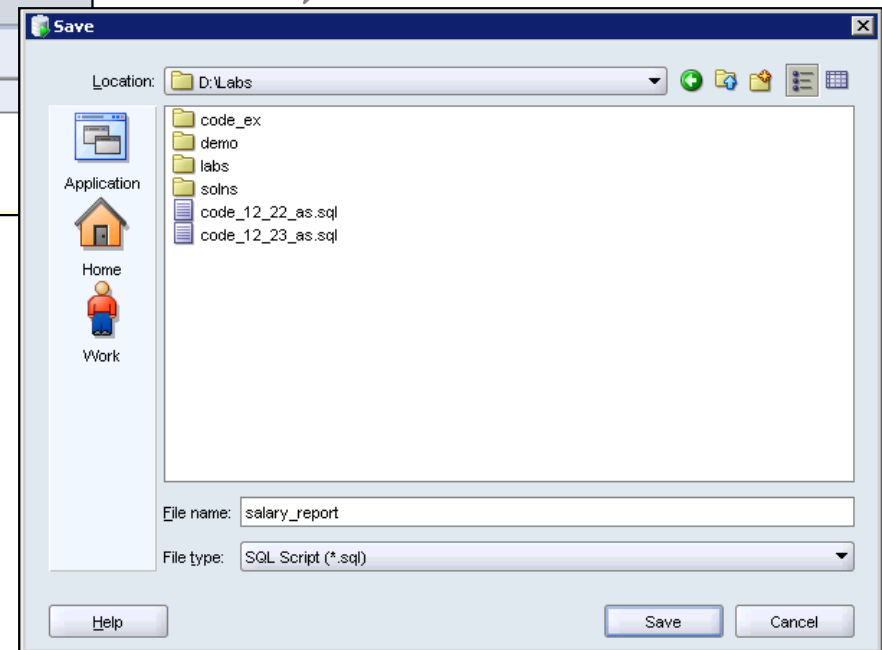
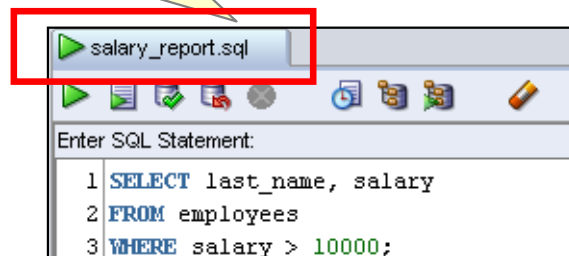
Saving SQL Scripts

Click the Save icon to save your SQL statement to a file.

Enter a file name and identify a location to save the file, and click Save.



The contents of the saved file are visible and editable in your SQL Worksheet window.



Executing Saved Script Files: Method 1

Right-click in the SQL Worksheet area, and select Open File from the shortcut menu.

Select (or navigate to) the script file that you want to open.

Click Open.

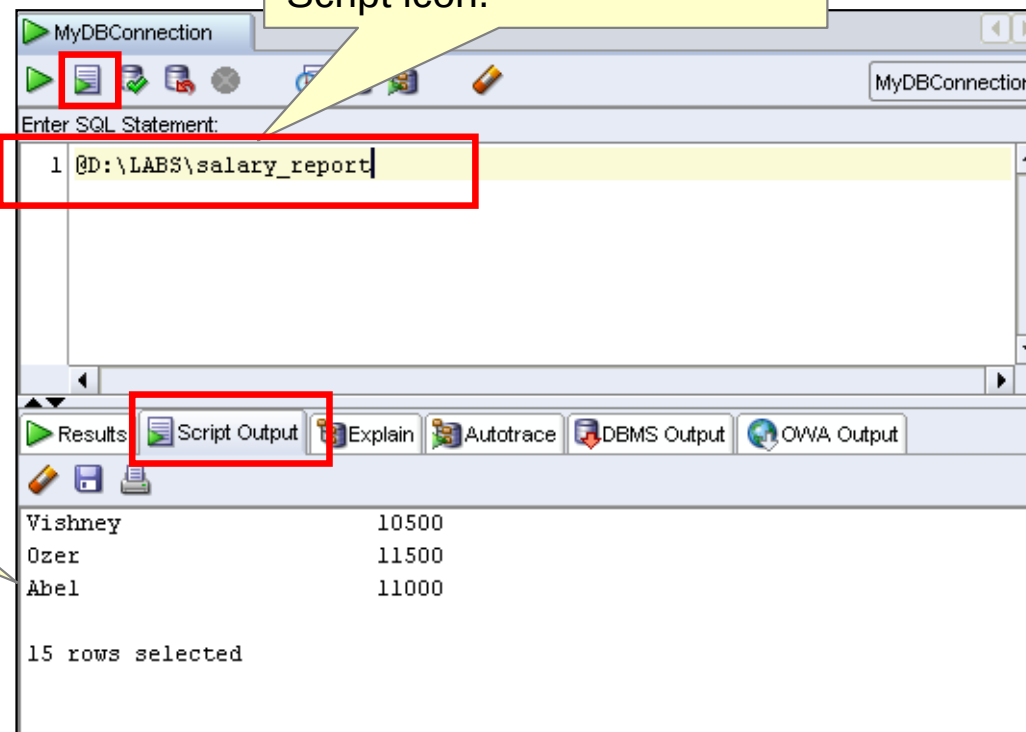
To run the code, click the Run Script (F5) icon.

```
CREATE PROCEDURE getemp IS -- header
emp_id employees.employee_id%type;
lname employees.last_name%type;
BEGIN
emp_id := 100;
SELECT last_name INTO lname
FROM EMPLOYEES
WHERE employee_id = emp_id;
DBMS_OUTPUT.PUT_LINE('Last name: '||lname);
END;
```

Executing Saved Script Files: Method 2

Use the @ command followed by the location and name of the file you want to execute, and click the Run Script icon.

The output from the script is displayed on the Script Output tabbed page.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

The screenshot illustrates the workflow for executing SQL statements in Oracle SQL Developer. It features three main components: the 'Enter SQL Statement' window, the 'Results' window, and the 'Script Output' window. The 'Enter SQL Statement' window contains the SQL query: `SELECT employee_id, last_name FROM employees;`. The 'Results' window displays the query results in a table format. The 'Script Output' window shows the execution log. Red boxes and green circles with labels (F9, F5) highlight the execution steps: clicking the 'Execute' button (F9) in the 'Enter SQL Statement' window, clicking the 'Script Output' button (F5) in the 'Script Output' window, and clicking the 'Results' button (F9) in the 'Results' window.

Enter SQL Statement:

```
SELECT employee_id, last_name
FROM employees;
```

Results:

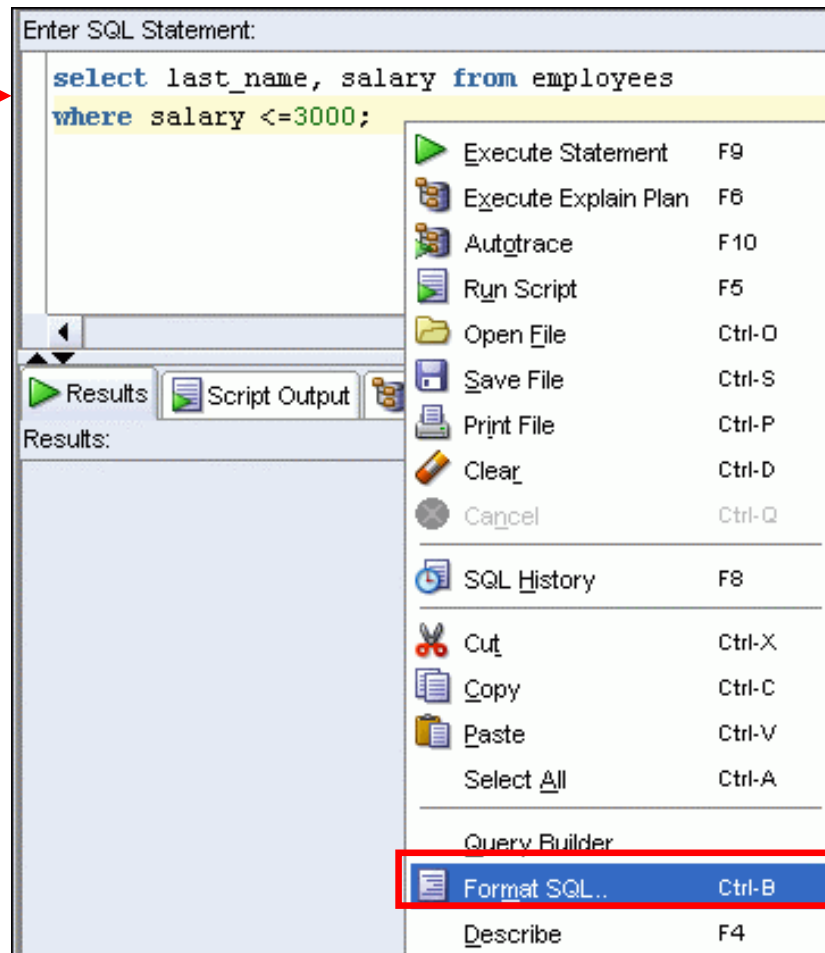
	EMPLOYEE_ID	LAST_NAME
1	100	King
2	101	Kochhar
3	102	De Haan
4	103	Hunold
5	104	Ernst

Script Output:

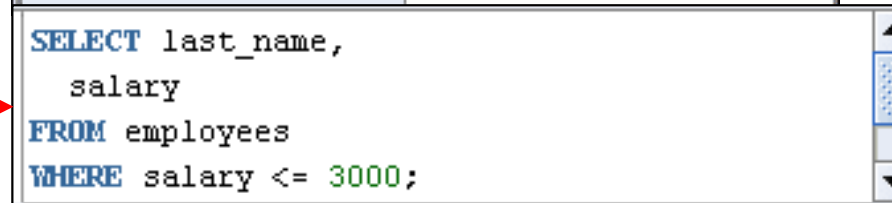
EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
105	Austin

Formatting the SQL Code

Before
formatting

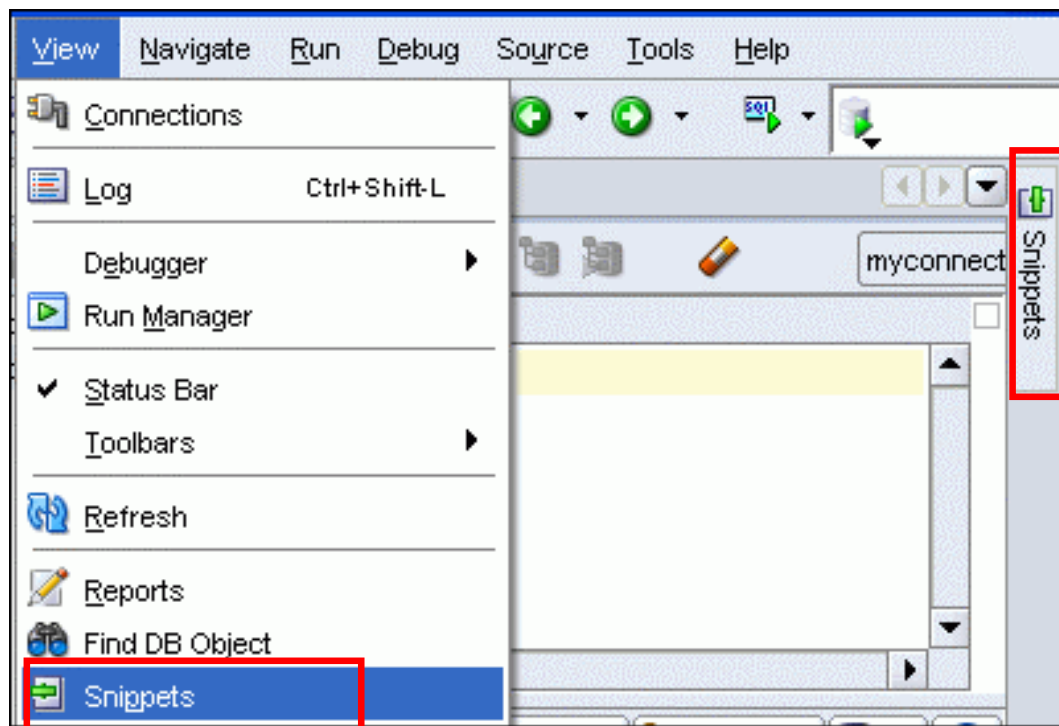


After
formatting

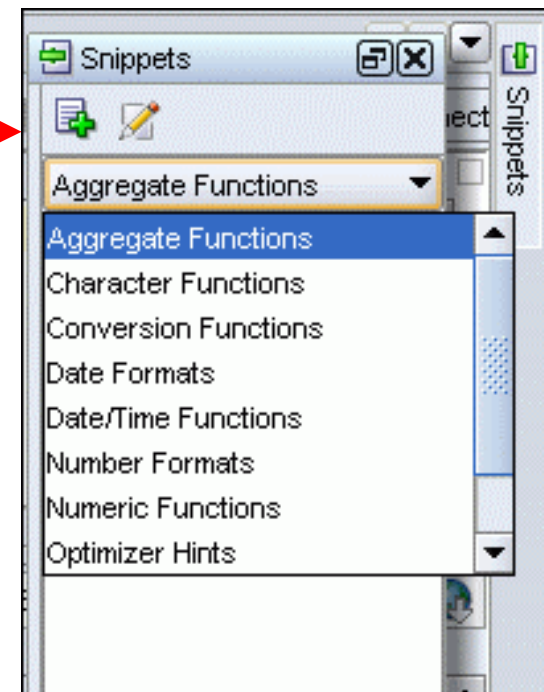


Using Snippets

Snippets are code fragments that may be just syntax or examples.

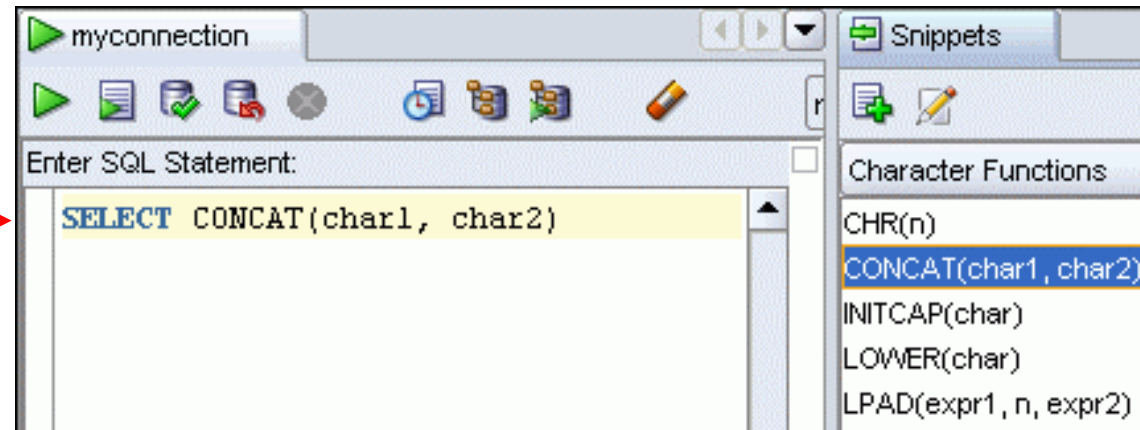


When you place your cursor here, it shows the Snippets window. From the drop-down list, you can select the functions category you want.

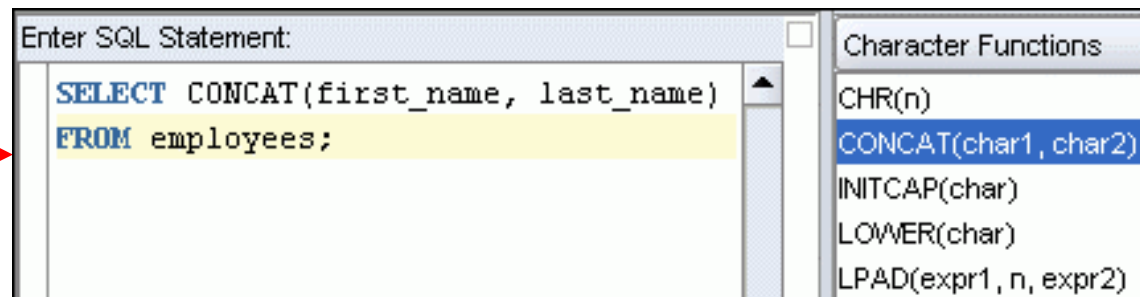


Using Snippets: Example

Inserting a snippet

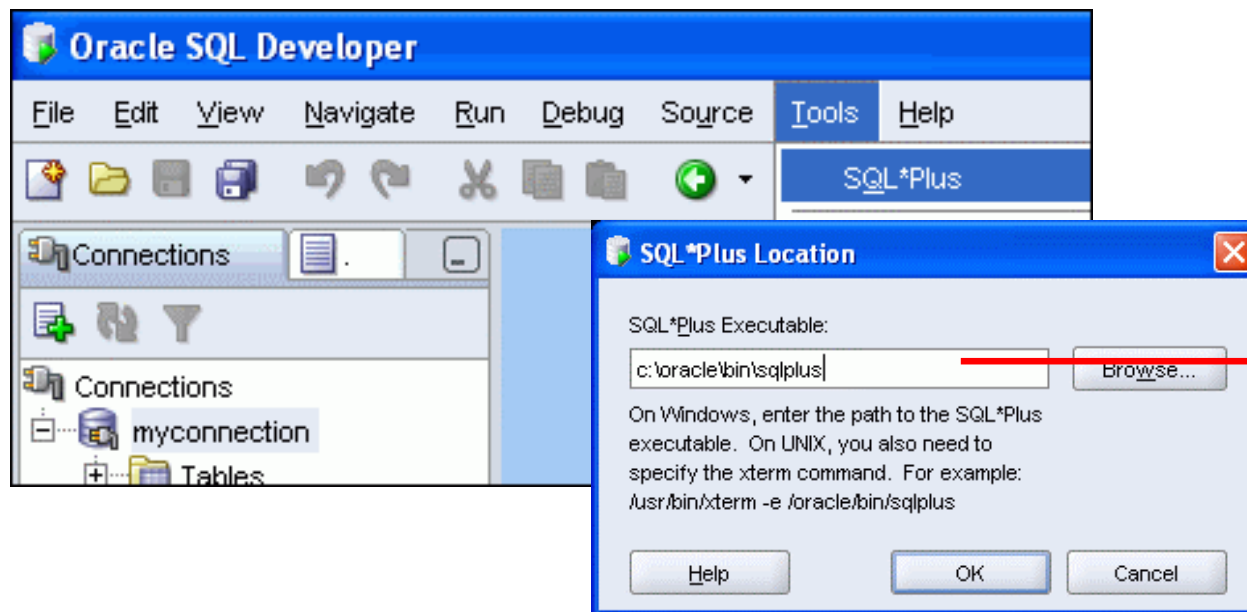


Editing the snippet



Using SQL*Plus

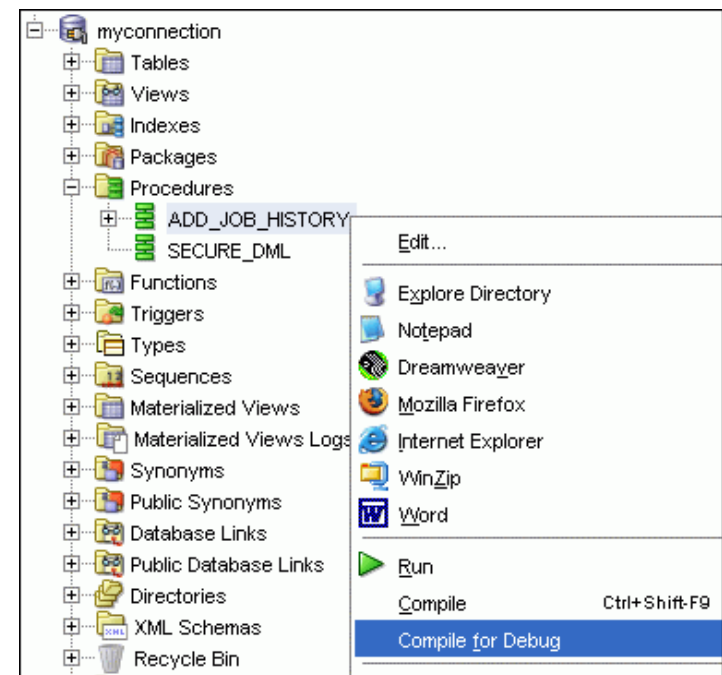
- You can invoke the SQL*Plus command-line interface from SQL Developer.
- Close all the SQL Worksheets to enable the SQL*Plus menu option.



Provide the location of the sqlplus.exe file only the first time you invoke SQL*Plus.

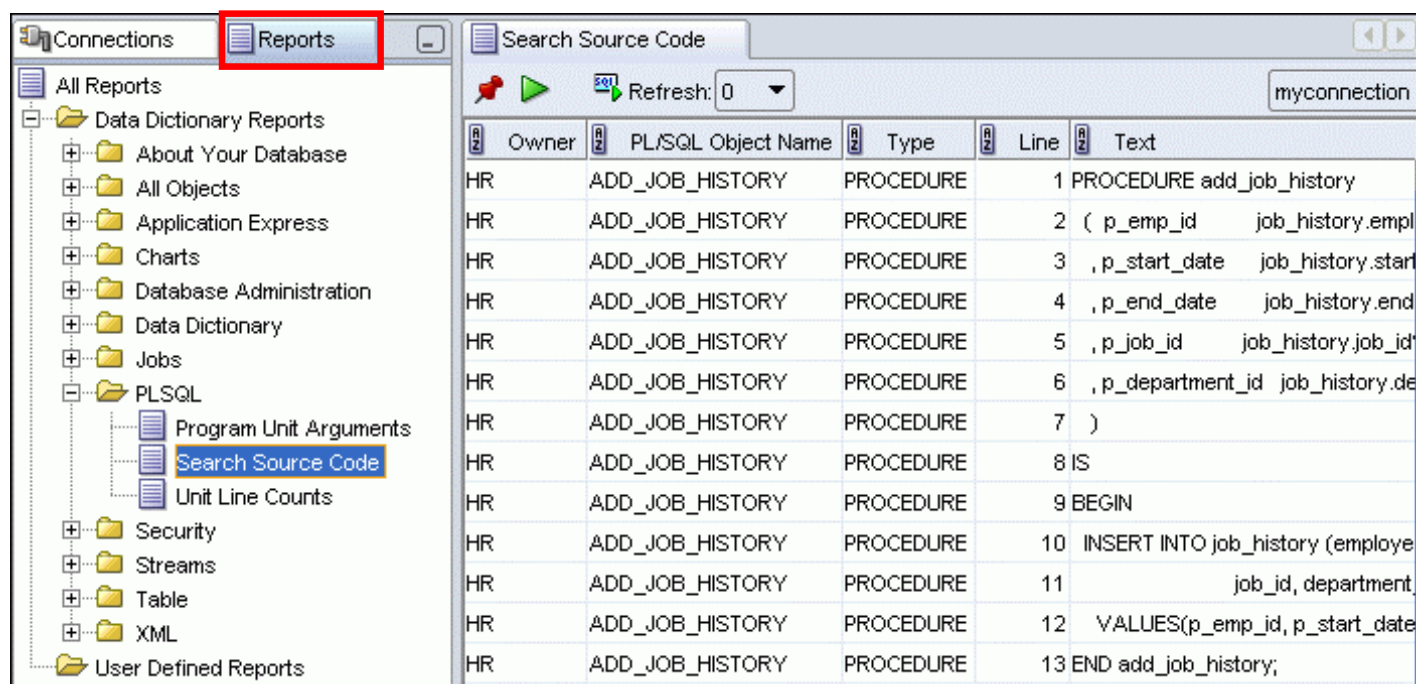
Debugging Procedures and Functions

- Use SQL Developer to debug PL/SQL functions and procedures.
- Use the Compile for Debug option to perform a PL/SQL compilation so that the procedure can be debugged.
- Use Debug menu options to set breakpoints, and to perform step into, step over tasks.



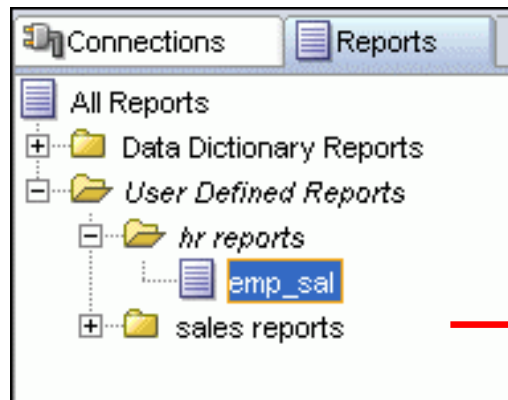
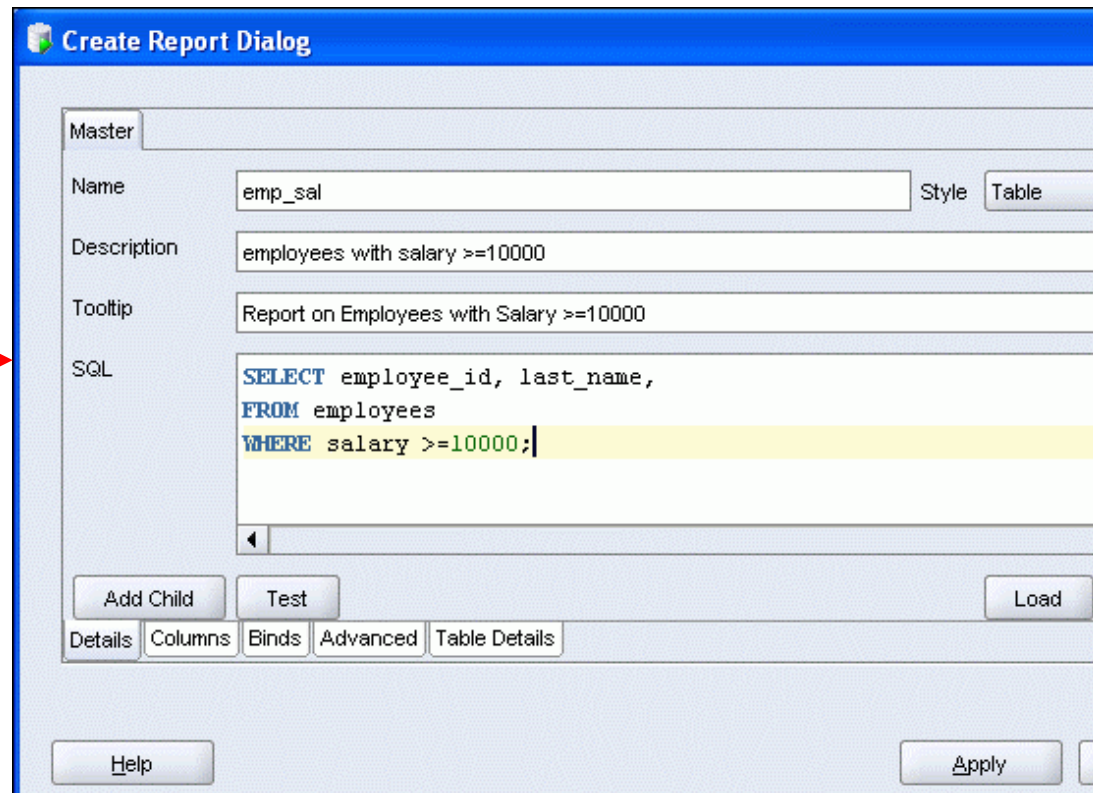
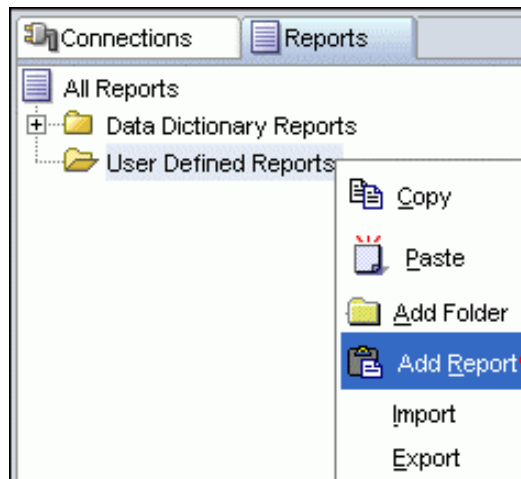
Database Reporting

SQL Developer provides a number of predefined reports about the database and its objects.



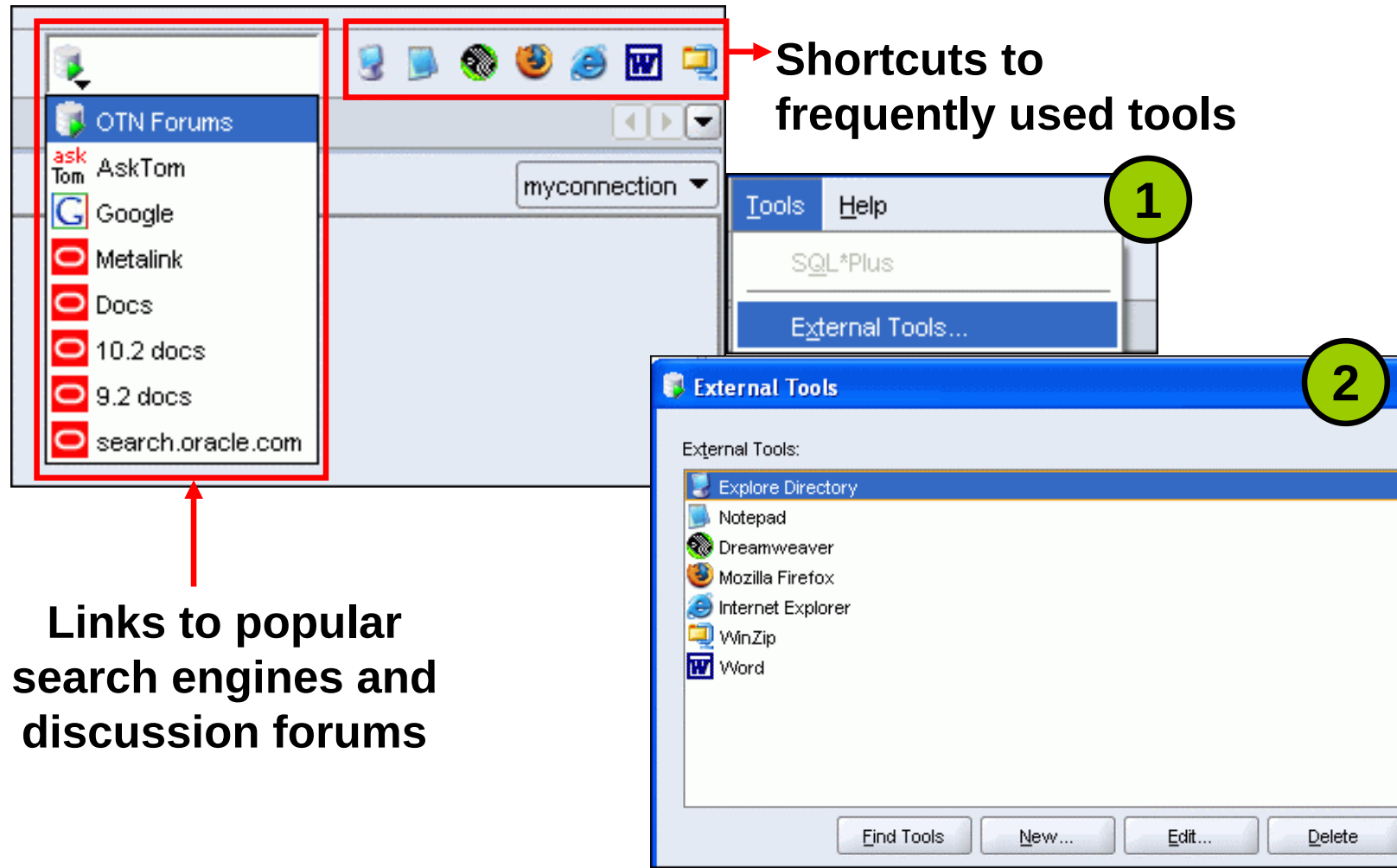
Creating a User-Defined Report

Create and save user-defined reports for repeated use.



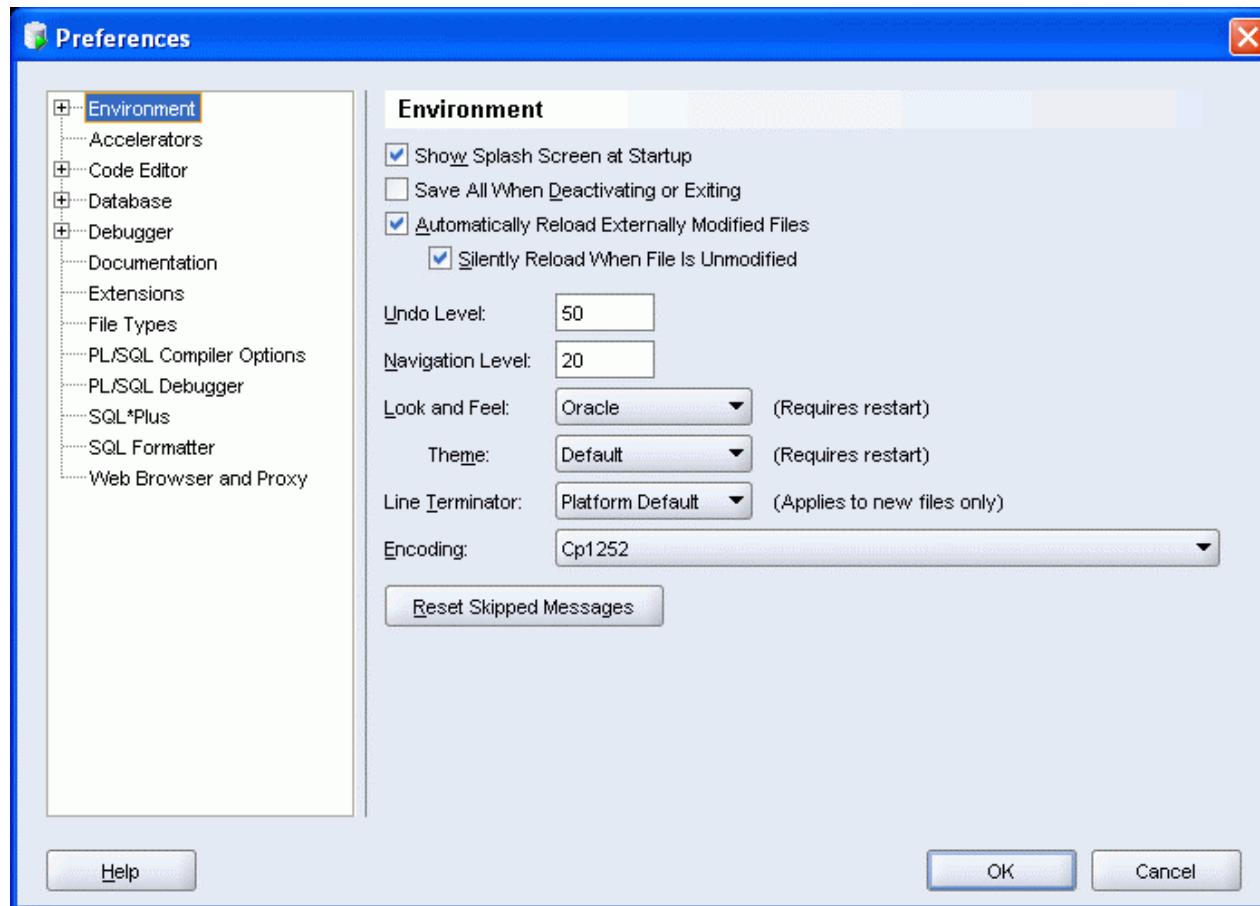
Organize reports in folders.

Search Engines and External Tools



Setting Preferences

- Customize the SQL Developer interface and environment.
- In the Tools menu, select Preferences.

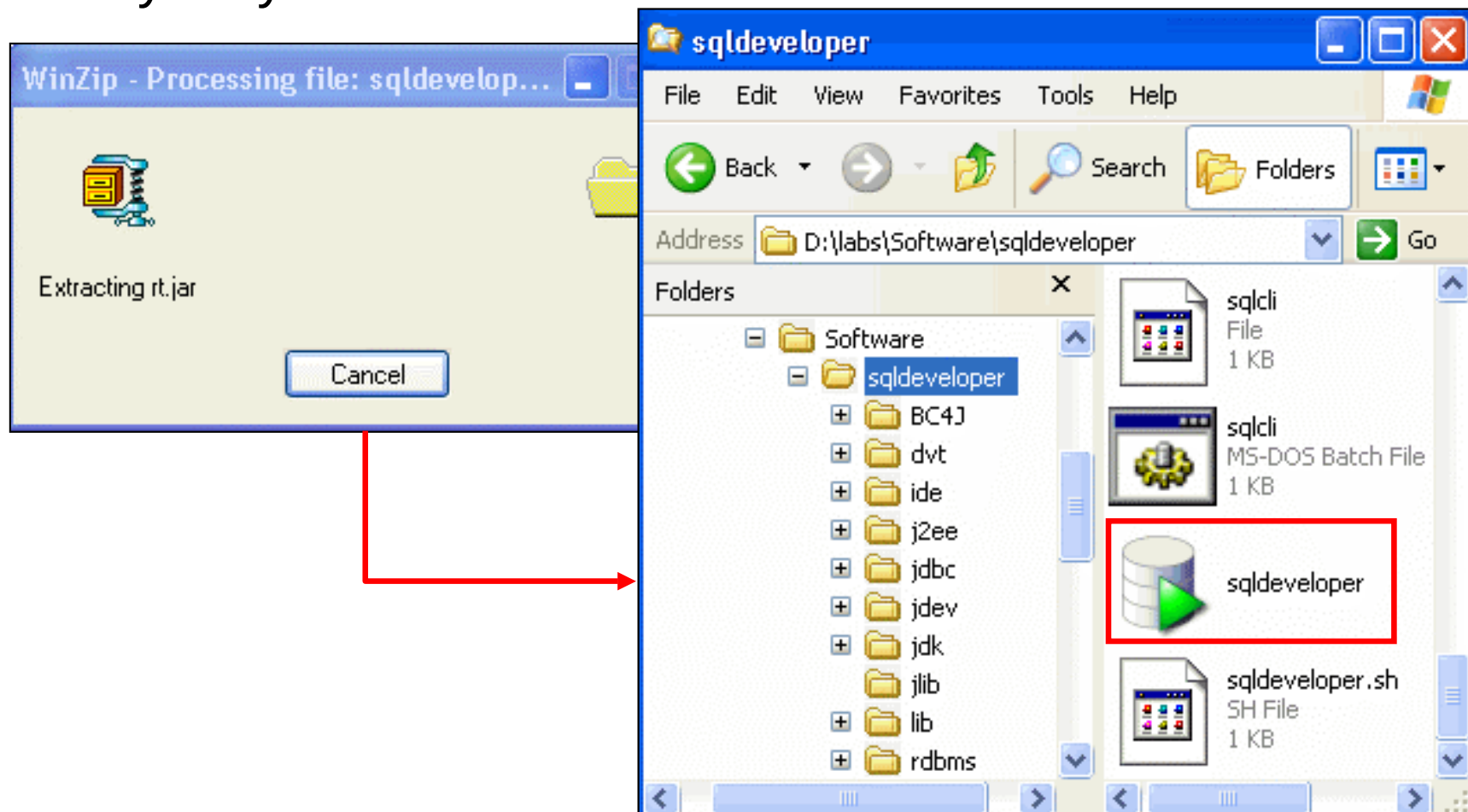


Specifications of SQL Developer 1.5.3

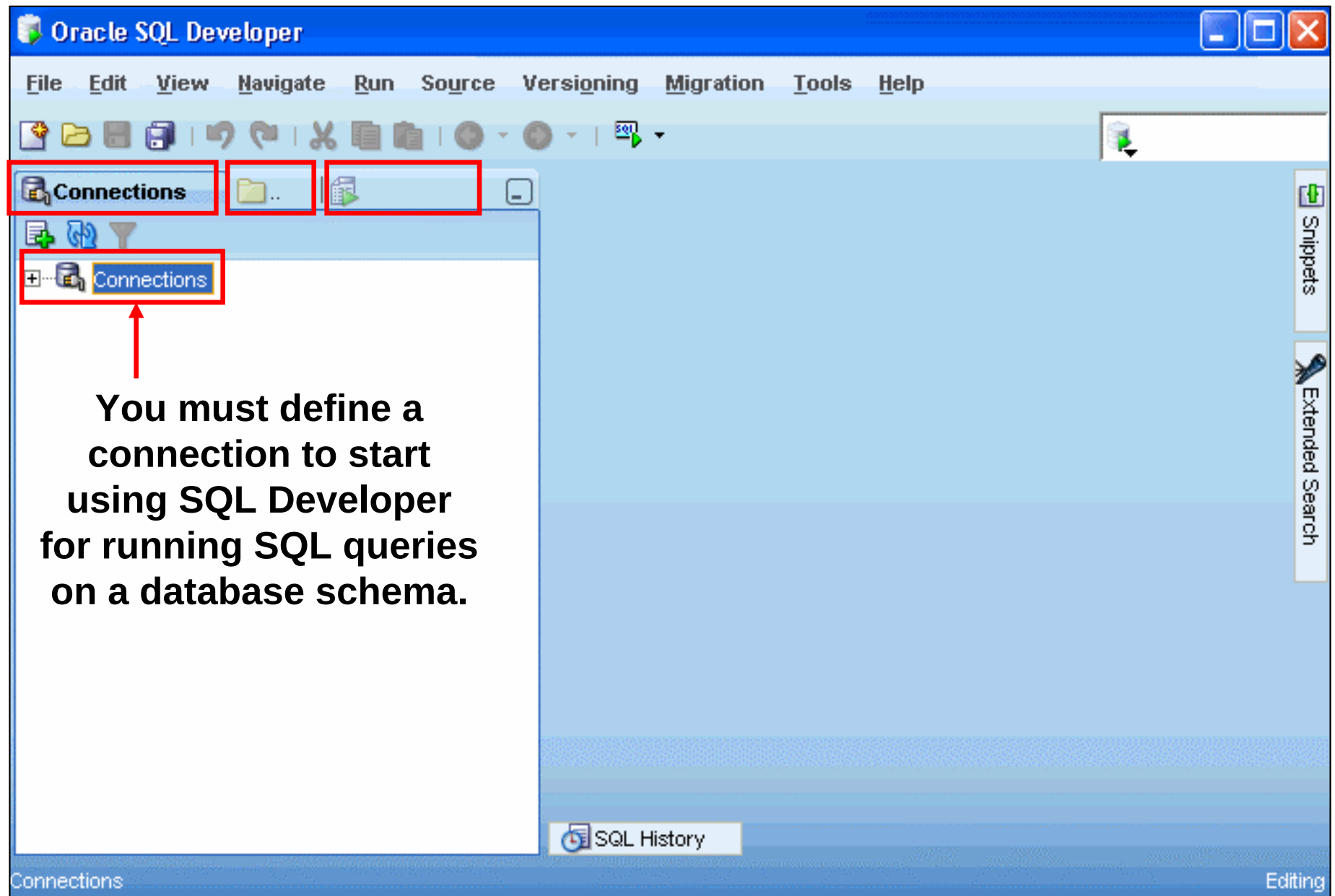
- SQL Developer 1.5.3 is the first translation release, and is a patch to Oracle SQL Developer 1.5.
- New feature list is available at:
 - http://www.oracle.com/technology/products/database/sql_developer/files/newFeatures_v15.html
- Supports Windows, Linux, and Mac OS X platforms
- To install, unzip the downloaded SQL Developer kit, which includes the required minimum JDK (JDK1.5.0_06).
- To start, double-click `sqldeveloper.exe`
- Connects to Oracle Database version 9.2.0.1 and later
- Freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html

Installing SQL Developer 1.5.3

Download the Oracle SQL Developer kit and unzip into any directory on your machine.



SQL Developer 1.5.3 Interface



Summary

In this appendix, you should have learned how to use SQL Developer to do the following:

- Browse, create, and edit database objects
- Execute SQL statements and scripts in SQL Worksheet
- Create and save custom reports

D

Review of PL/SQL

Block Structure for Anonymous PL/SQL Blocks

- DECLARE (optional)
 - Declare PL/SQL objects to be used within this block.
- BEGIN (mandatory)
 - Define the executable statements.
- EXCEPTION (optional)
 - Define the actions that take place if an error or exception arises.
- END; (mandatory)

Declaring PL/SQL Variables

- Syntax:

```
identifier [CONSTANT] datatype [NOT NULL]
  [:= | DEFAULT expr];
```

- Examples:

```
Declare
  v_hiredate      DATE;
  v_deptno        NUMBER(2) NOT NULL := 10;
  v_location      VARCHAR2(13) := 'Atlanta';
  c_comm          CONSTANT NUMBER := 1400;
  v_count         BINARY_INTEGER := 0;
  v_valid         BOOLEAN NOT NULL := TRUE;
```

Declaring Variables with the %TYPE Attribute: Examples

```
...  
    v_ename                employees.last_name%TYPE;  
    v_balance              NUMBER(7,2);  
    v_min_balance          v_balance%TYPE := 10;  
...
```

Creating a PL/SQL Record

- Declare variables to store the name, job, and salary of a new employee.

```
...  
  TYPE emp_record_type IS RECORD  
    (ename      VARCHAR2(25),  
     job        VARCHAR2(10),  
     sal        NUMBER(8,2));  
  emp_record    emp_record_type;  
...
```

%ROWTYPE Attribute: Examples

- Declare a variable to store the same information about a department as is stored in the DEPARTMENTS table.

```
dept_record    departments%ROWTYPE;
```

- Declare a variable to store the same information about an employee as is stored in the EMPLOYEES table.

```
emp_record    employees%ROWTYPE;
```


Creating a PL/SQL Table

```
DECLARE
  TYPE ename_table_type IS TABLE OF
    employees.last_name%TYPE
    INDEX BY BINARY_INTEGER;
  TYPE hiredate_table_type IS TABLE OF DATE
    INDEX BY BINARY_INTEGER;
  ename_table      ename_table_type;
  hiredate_table   hiredate_table_type;
BEGIN
  ename_table(1) := 'CAMERON';
  hiredate_table(8) := SYSDATE + 7;
  IF ename_table.EXISTS(1) THEN
    INSERT INTO ...
  ...
END;
```

SELECT Statements in PL/SQL: Example

The INTO clause is mandatory.

```
DECLARE
  v_deptid  NUMBER(4);
  v_loc     NUMBER(4);
BEGIN
  SELECT    department_id, location_id
  INTO      v_deptid, v_loc
  FROM      departments
  WHERE     department_name = 'Sales';
  ...
END;
```

Inserting Data: Example

Add new employee information to the EMPLOYEES table.

```
DECLARE
  v_empid  employees.employee_id%TYPE;
BEGIN
  SELECT  employees_seq.NEXTVAL
  INTO    v_empno
  FROM    dual;
  INSERT INTO  employees(employee_id, last_name,
                        job_id, department_id)
  VALUES(v_empid, 'HARDING', 'PU_CLERK', 30);
END;
```

Updating Data: Example

Increase the salary of all employees in the EMPLOYEES table who are purchasing clerks.

```
DECLARE
    v_sal_increase    employees.salary%TYPE := 2000;
BEGIN
    UPDATE    employees
    SET        salary = salary + v_sal_increase
    WHERE      job_id = 'PU_CLERK';
END;
```

Deleting Data: Example

Delete rows that belong to department 190 from the EMPLOYEES table.

```
DECLARE
    v_deptid    employees.department_id%TYPE := 190;
BEGIN
    DELETE FROM employees
    WHERE department_id = v_deptid;
END;
```

COMMIT and ROLLBACK Statements

- Initiate a transaction with the first DML command to follow a COMMIT or ROLLBACK statement.
- Use COMMIT and ROLLBACK SQL statements to terminate a transaction explicitly.

SQL Cursor Attributes

You can use SQL cursor attributes to test the outcome of your SQL statements.

SQL Cursor Attributes	Description
SQL%ROWCOUNT	Number of rows affected by the most recent SQL statement (an integer value)
SQL%FOUND	Boolean attribute that evaluates to TRUE if the most recent SQL statement affects one or more rows
SQL%NOTFOUND	Boolean attribute that evaluates to TRUE if the most recent SQL statement does not affect any rows
SQL%ISOPEN	Boolean attribute that always evaluates to FALSE because PL/SQL closes implicit cursors immediately after they are executed

IF, THEN, and ELSIF Statements: Example

For a given value entered, return a calculated value.

```
. . .  
IF v_start > 100 THEN  
    v_start := 2 * v_start;  
ELSIF v_start >= 50 THEN  
    v_start := 0.5 * v_start;  
ELSE  
    v_start := 0.1 * v_start;  
END IF;  
. . .
```


Basic Loop: Example

```
DECLARE
  v_ordid      order_items.order_id%TYPE := 101;
  v_counter    NUMBER(2) := 1;
BEGIN
  LOOP
    INSERT INTO order_items(order_id,line_item_id)
    VALUES(v_ordid, v_counter);
    v_counter := v_counter + 1;
    EXIT WHEN v_counter > 10;
  END LOOP;
END;
```

FOR Loop: Example

Insert the first 10 new line items for order number 101.

```
DECLARE
    v_ordid      order_items.order_id%TYPE := 101;
BEGIN
    FOR i IN 1..10 LOOP
        INSERT INTO order_items(order_id,line_item_id)
        VALUES(v_ordid, i);
    END LOOP;
END;
```

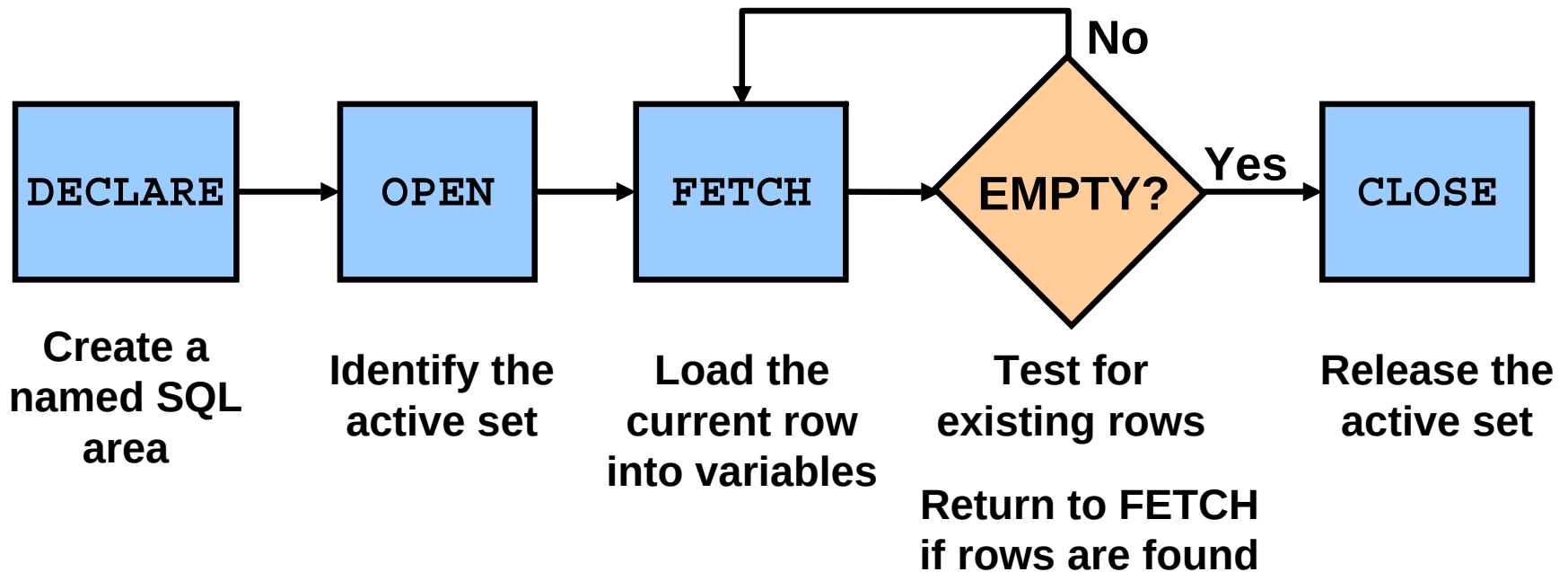
WHILE Loop: Example

```
ACCEPT p_price PROMPT 'Enter the price of the item: '
ACCEPT p_itemtot -
  PROMPT 'Enter the maximum total for purchase of item: '
DECLARE
...
v_qty          NUMBER(8) := 1;
v_running_total NUMBER(7,2) := 0;

BEGIN
...
  WHILE v_running_total < &p_itemtot LOOP
    ...
    v_qty := v_qty + 1;
    v_running_total := v_qty * &p_price;
  END LOOP;
...

```

Controlling Explicit Cursors



Declaring the Cursor: Example

```
DECLARE
  CURSOR c1 IS
    SELECT employee_id, last_name
    FROM   employees;

  CURSOR c2 IS
    SELECT *
    FROM   departments
    WHERE  department_id = 10;
BEGIN
  ...
```

Opening the Cursor

```
OPEN cursor_name;
```

- Open the cursor to execute the query and identify the active set.
- If the query returns no rows, no exception is raised.
- Use cursor attributes to test the outcome after a fetch.

Fetching Data from the Cursor: Examples

```
FETCH c1 INTO v_empid, v_ename;
```

```
...  
OPEN defined_cursor;  
LOOP  
    FETCH defined_cursor INTO defined_variables  
    EXIT WHEN ...;  
    ...  
    -- Process the retrieved data  
    ...  
END;
```

Closing the Cursor

```
CLOSE    cursor_name;
```

- Close the cursor after completing the processing of the rows.
- Reopen the cursor, if required.
- Do not attempt to fetch data from a cursor after it has been closed.

Explicit Cursor Attributes

Obtain status information about a cursor.

Attribute	Type	Description
ISOPEN	BOOLEAN	Evaluates to TRUE if the cursor is open
%NOTFOUND	BOOLEAN	Evaluates to TRUE if the most recent fetch does not return a row
%FOUND	BOOLEAN	Evaluates to TRUE if the most recent fetch returns a row; complement of %NOTFOUND
%ROWCOUNT	NUMBER	Evaluates to the total number of rows returned so far

Cursor FOR Loops: Example

Retrieve employees one by one until there are no more left.

```
DECLARE
  CURSOR c1 IS
    SELECT employee_id, last_name
    FROM   employees;
BEGIN
  FOR emp_record IN c1 LOOP
    -- implicit open and implicit fetch occur
    IF emp_record.employee_id = 134 THEN
      ...
    END LOOP; -- implicit close occurs
END;
```

FOR UPDATE Clause: Example

Retrieve the orders for amounts over \$1,000 that were processed today.

```
DECLARE
  CURSOR c1 IS
    SELECT customer_id, order_id
    FROM   orders
    WHERE  order_date = SYSDATE
          AND order_total > 1000.00
    ORDER BY customer_id
    FOR UPDATE NOWAIT;
```

WHERE CURRENT OF Clause: Example

```
DECLARE
  CURSOR c1 IS
    SELECT salary FROM employees
    FOR UPDATE OF salary NOWAIT;
BEGIN
  ...
  FOR emp_record IN c1 LOOP
    UPDATE ...
      WHERE CURRENT OF c1;
    ...
  END LOOP;
  COMMIT;
END;
```

Trapping Predefined Oracle Server Errors

- Reference the standard name in the exception-handling routine.
- Sample predefined exceptions:
 - NO_DATA_FOUND
 - TOO_MANY_ROWS
 - INVALID_CURSOR
 - ZERO_DIVIDE
 - DUP_VAL_ON_INDEX

Trapping Predefined Oracle Server Errors: Example

```
BEGIN  SELECT ... COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    statement1;
    statement2;
  WHEN TOO_MANY_ROWS THEN
    statement1;
  WHEN OTHERS THEN
    statement1;
    statement2;
    statement3;
END;
```

Non-Predefined Error

Trap for Oracle server error number –2292, which is an integrity constraint violation.

```
DECLARE
  e_products_invalid EXCEPTION;
  PRAGMA EXCEPTION_INIT (
    e_products_invalid, -2292);
  v_message VARCHAR2(50);
BEGIN
  . . .
EXCEPTION
  WHEN e_products_invalid THEN
    :g_message := 'Product ID
    specified is not valid.';
  . . .
END;
```

User-Defined Exceptions: Example

```
[DECLARE]
  e_amount_remaining EXCEPTION; ①
. . .
BEGIN ②
. . .
  RAISE e_amount_remaining;
. . .

EXCEPTION ③
  WHEN e_amount_remaining THEN
    :g_message := 'There is still an amount
                  in stock.';
. . .
END;
```


RAISE_APPLICATION_ERROR Procedure

```
raise_application_error (error_number,  
    message[, {TRUE | FALSE}]);
```

- Enables you to issue user-defined error messages from stored subprograms
- Is called from an executing stored subprogram only

RAISE_APPLICATION_ERROR Procedure

- Is used in two different places:
 - Executable section
 - Exception section
- Returns error conditions to the user in a manner consistent with other Oracle server errors

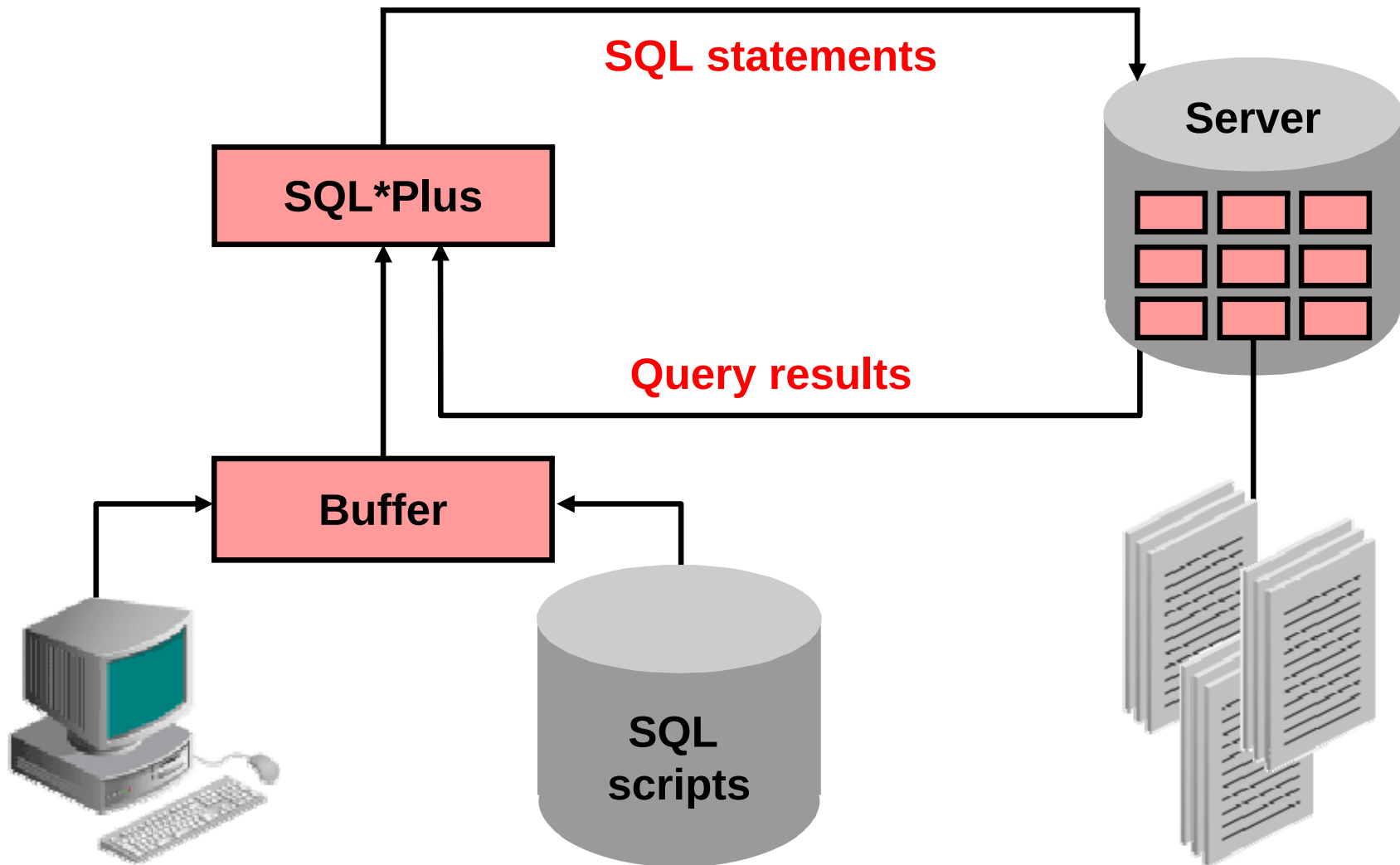
Using SQL*Plus

Objectives

After completing this appendix, you should be able to do the following:

- Log in to SQL*Plus
- Edit SQL commands
- Format output using SQL*Plus commands
- Interact with script files

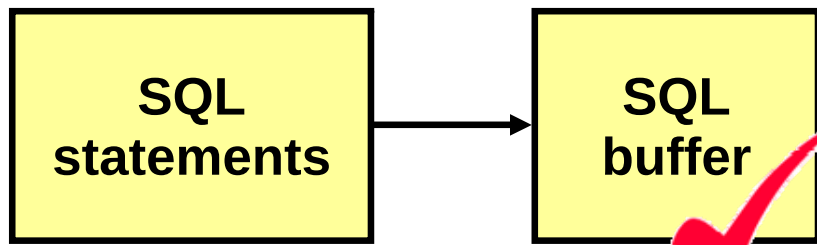
SQL and SQL*Plus Interaction



SQL Statements Versus SQL*Plus Commands

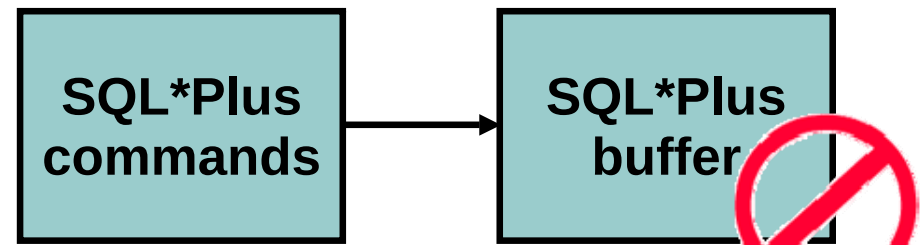
SQL

- A language
- ANSI-standard
- Keywords cannot be abbreviated
- Statements manipulate data and table definitions in the database



SQL*Plus

- An environment
- Oracle-proprietary
- Keywords can be abbreviated
- Commands do not allow manipulation of values in the database



Overview of SQL*Plus

- Log in to SQL*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from file to buffer to edit.

Logging In to SQL*Plus: Available Methods

1

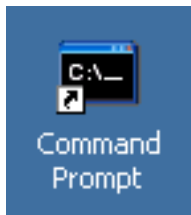


```
CA sqlplus

SQL*Plus: Release 11.1.0.5.0 - Beta on Fri Jun 29 07:03:28 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.

SQL> connect ora62/ora62@orcl
Connected.
SQL>
```

2



```
CA Command Prompt - sqlplus ora62/ora62@orcl

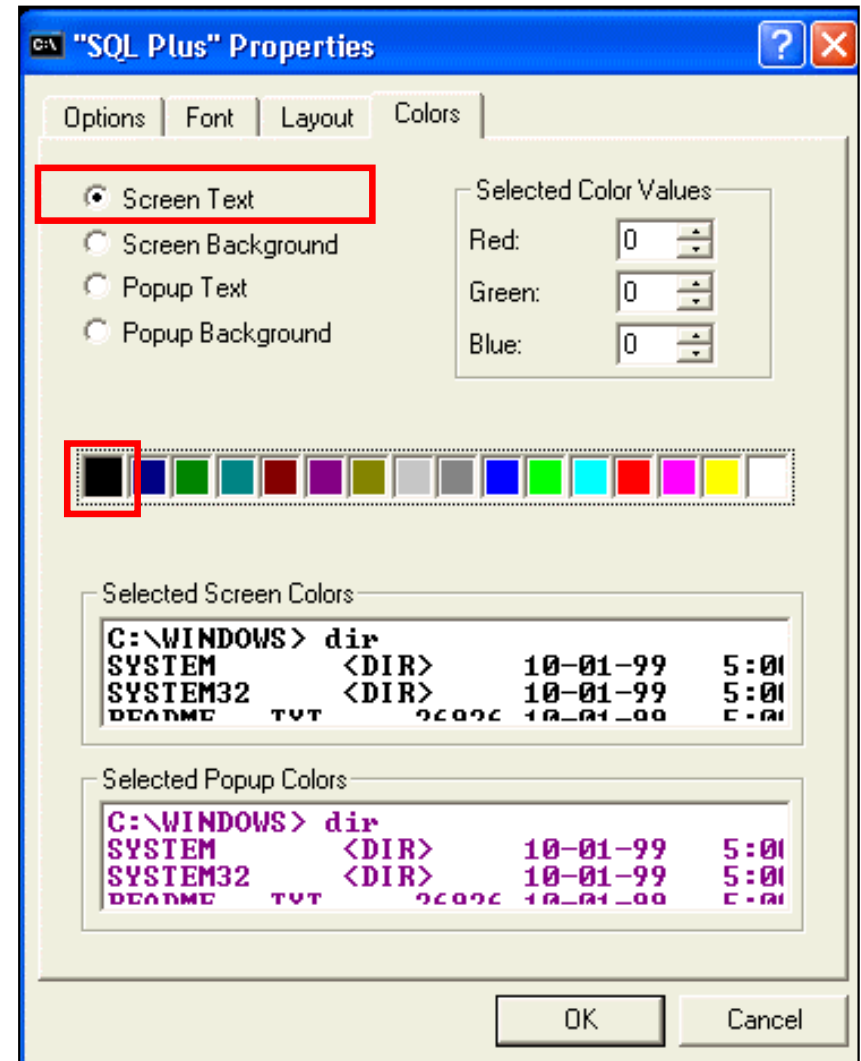
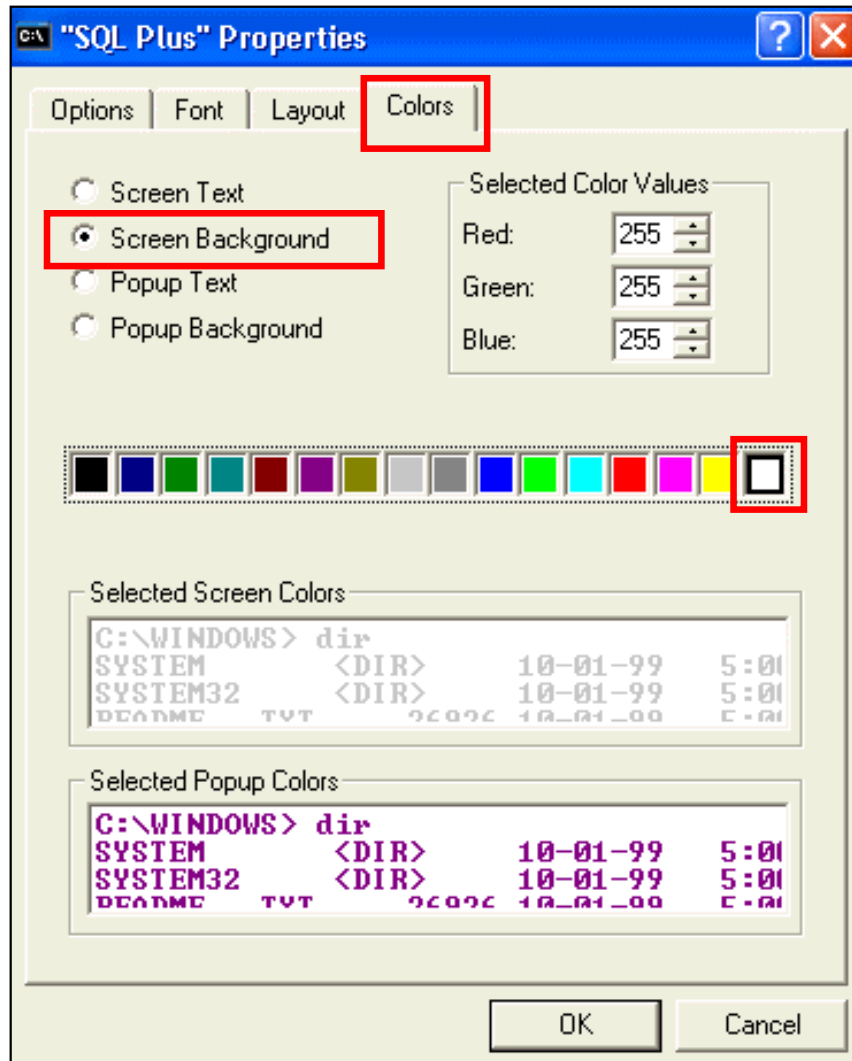
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

D:\WINNT\system32>cd \app\administrator\product\11.1.0\client_1\bin
D:\app\Administrator\product\11.1.0\client_1\BIN>sqlplus ora62/ora62@orcl

SQL*Plus: Release 11.1.0.5.0 - Beta on Fri Jun 29 07:25:01 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.

Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.5.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```


Customizing the SQL*Plus Environment



Displaying Table Structure

Use the SQL*Plus DESCRIBE command to display the structure of a table:

```
DESC[RIBE] tablename
```

Displaying Table Structure

```
DESCRIBE departments
```

Name	Null?	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

SQL*Plus Editing Commands

- `A[PPEND] text`
- `C[HANGE] / old / new`
- `C[HANGE] / text /`
- `CL[EAR] BUFF[ER]`
- `DEL`
- `DEL n`
- `DEL m n`

SQL*Plus Editing Commands

- I [NPUT]
- I [NPUT] *text*
- L [IST]
- L [IST] *n*
- L [IST] *m n*
- R [UN]
- *n*
- *n text*
- 0 *text*

Using LIST, n, and APPEND

```
LIST
  1  SELECT last_name
  2* FROM    employees
```

```
1
  1* SELECT last_name
```

```
A , job_id
  1* SELECT last_name, job_id
```

```
LIST
  1  SELECT last_name, job_id
  2* FROM    employees
```

Using the CHANGE Command

```
LIST  
1* SELECT * from employees
```

```
c/employees/departments  
1* SELECT * from departments
```

```
LIST  
1* SELECT * from departments
```

SQL*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`

Using the SAVE, START, and EDIT Commands

```
LIST
```

```
1  SELECT last_name, manager_id, department_id
2*  FROM employees
```

```
SAVE my_query
```

```
Created file my_query
```

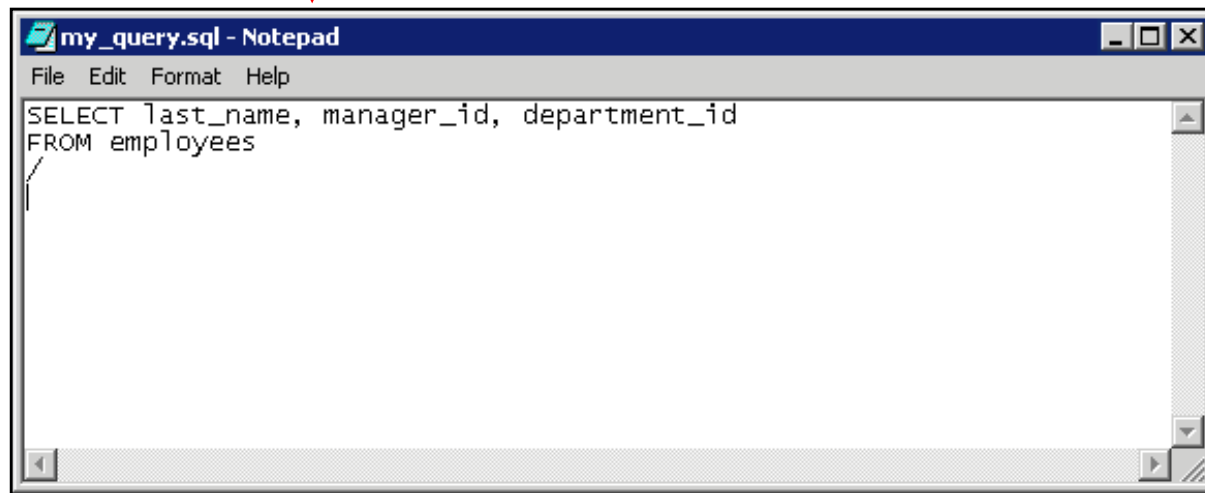
```
START my_query
```

LAST_NAME	MANAGER_ID	DEPARTMENT_ID
-----	-----	-----
King		90
Kochhar	100	90
...		

107 rows selected.

Using the SAVE, START, and EDIT Commands

EDIT my_query



SQL*Plus Enhancements Since Oracle Database 10g

- Changes to the `SET SERVEROUT [PUT]` command
- White space support in file and path names in Windows
- Three new predefined SQL*Plus variables
- The new `RECYCLEBIN` clause of the `SHOW` command
- The new `APPEND`, `CREATE`, and `REPLACE` extensions to the `SPOOL` command
- New error messages for the `COPY` command
- Change in the `DESCRIBE` command behavior
- New `PAGESIZE` default
- New `SQLPLUS` program compatibility option
- Execution statistics information in the `AUTOTRACE` command report

Changes to the SERVEROUTPUT Command

- Use the SET SERVEROUT [PUT] command to control whether to display the output of stored procedures or PL/SQL blocks in SQL*Plus.
- The DBMS_OUTPUT line length limit is increased from 255 bytes to 32,767 bytes.
- The default size is now unlimited.
- Resources are not preallocated when SERVEROUTPUT is set.
- Because there is no performance penalty, use UNLIMITED unless you want to conserve physical memory.

```
SET SERVEROUT [PUT] {ON | OFF} [SIZE {n | UNL[IMITED]}]  
[FOR [MAT] {WRA [PPED] | WOR [D_WAPPED] | TRU [NCATED] }]
```

White Space Support in File and Path Names in Windows

- In Windows, white space can be included in file names and paths.
- Examples of where white space can be used:
 - START, @, @@, RUN, SPOOL, SAVE, and EDIT commands
- To reference files or paths containing spaces, enclose the name or path in double quotation marks.

Examples

```
SAVE "Monthly Report.sql"  
START "Monthly Report.sql"
```

Predefined SQL*Plus Variables

Variable Name	Contains
<code>__CONNECT__IDENTIFIER</code>	Connection identifier used to make connection, where available
<code>__DATE</code>	Current date, or a user-defined fixed string
<code>__EDITOR</code>	Specifies the editor used by the <code>EDIT</code> command
<code>__O__VERSION</code>	Current version of the installed Oracle Database
<code>O__RELEASE</code>	Full release number of the installed Oracle Database
<code>__PRIVILEGE</code>	Privilege level of the current connection
<code>__SQLPLUS__RELEASE</code>	Full release number of installed SQL*Plus component
<code>__USER</code>	Username used to make connection

Using the New Predefined SQL*Plus Variables: Examples

```
-- Change the SQL*Plus prompt to display the connection  
-- identifier
```

```
SQL> SET SQLPROMPT '_CONNECT_IDENTIFIER > '  
orcl >
```

```
-- view the predefined value of the _SQLPLUS_RELEASE  
-- substitution variable
```

```
orcl > DEFINE _SQLPLUS_RELEASE  
DEFINE _SQLPLUS_RELEASE = "1002000100" (CHAR)
```

```
-- View the user name connected to the current  
-- connection.
```

```
orcl > DEFINE _USER  
DEFINE _USER = "HR" (CHAR)
```

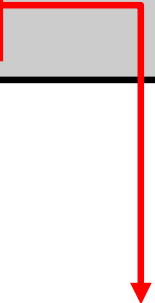
The SHOW Command and the New RECYCLEBIN Clause

```
SHOW RECYC[LEBIN] [original_name]
SELECT * FROM USER_RECYCLEBIN
desc user_recyclebin;
Name                Null?      Type
-----
OBJECT_NAME         NOT NULL  VARCHAR2 (30)
ORIGINAL_NAME                          VARCHAR2 (32)
OPERATION                        VARCHAR2 (9)
TYPE                        VARCHAR2 (25)
TS_NAME                        VARCHAR2 (30)
CREATETIME                        VARCHAR2 (19)
DROPTIME                        VARCHAR2 (19)
DROPSCN                        NUMBER
PARTITION_NAME        VARCHAR2 (32)
CAN_UNDROP                        VARCHAR2 (3)
CAN_PURGE                        VARCHAR2 (3)
RELATED                NOT NULL  NUMBER
BASE_OBJECT            NOT NULL  NUMBER
PURGE_OBJECT          NOT NULL  NUMBER
SPACE                        NUMBER
```


The SHOW Command and the RECYCLEBIN Clause: Example

```
DROP TABLE test;  
Table dropped.
```

```
SHOW recyclebin
```



ORIGINAL NAME	RECYCLEBIN NAME	OBJECT TYPE	DROP TIME
TEST	BIN\$SefY+qPKSY6mvU8eDT1r+A== \$0	TABLE	2006-07-06:11:12:00

```
SQL>
```

Using the SQL*Plus SPOOL Command

```
SPO[OL] [file_name[.ext] [CRE[ATE] | REP[LACE] |  
APP[END]] | OFF | OUT]
```

Option	Description
<code>file_name[.ext]</code>	Spools output to the specified file name
<code>CRE[ATE]</code>	Creates a new file with the name specified
<code>REP[LACE]</code>	Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file.
<code>APP[END]</code>	Adds the contents of the buffer to the end of the file you specify
<code>OFF</code>	Stops spooling
<code>OUT</code>	Stops spooling and sends the file to your computer's standard (default) printer

Using the SQL*Plus SPOOL Command: Examples

```
-- Record the output in the new file DIARY using the  
-- default file extension.
```

```
SPOOL DIARY CREATE
```

```
-- Append the output to the existing file DIARY.
```

```
SPOOL DIARY APPEND
```

```
-- Record the output to the file DIARY, overwriting the  
-- existing content
```

```
SPOOL DIARY REPLACE
```

```
-- Stop spooling and print the file on your default printer.
```

```
SPOOL OUT
```

The COPY Command: New Error Messages

CPY-0002 Illegal or missing APPEND, CREATE, INSERT, or REPLACE option

CPY-0003 Internal Error: logical host number out of Range

CPY-0004 Source and destination table and column names don't match

CPY-0005 Source and destination column attributes don't Match

CPY-0006 Select list has more columns than destination Table

CPY-0007 Select list has fewer columns than destination table

The COPY Command: New Error Messages

CPY-0008 More column list names than columns in the destination table

CPY-0009 Fewer column list names than columns in the destination table

CPY-0012 Datatype cannot be copied

Change in the DESCRIBE Command Behavior

- Prior to Oracle Database 10g, using DESCRIBE on an invalidated object failed with the error:
 - `ORA-24372: invalid object for describe`
- The DESCRIBE command continued to fail even if the object had since been validated.
- Starting with Oracle Database 10g, the DESCRIBE command now automatically validates the object and continues if the validation is successful.

The SET PAGES [IZE] Command

- It sets the number of rows on each page of the output in SQL*Plus.
- The default PAGESIZE has changed from 24 to 14.
- You can set PAGESIZE to zero to suppress all headings, page breaks, titles, the initial blank line, and other formatting information.

```
SET PAGES [IZE] {14 | n}
```

The SQLPLUS Program and the Compatibility Option

Sets the value of the SQLPLUSCOMPATIBILITY system variable to the SQL*Plus release specified by x.y[.z]

```
SQLPLUS -C[OMPATIBILITY] {x.y[.z]}
```

```
-- x is the version number  
-- y is the release number  
-- z is the update number
```

```
SQLPLUS -C 10.2.0
```


Using the AUTOTRACE Command

- It displays a report after the successful execution of SQL DML statements such as SELECT, INSERT, UPDATE or DELETE.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]]  
[STAT[ISTICS]]
```

```
SET AUTOTRACE ON  
-- The AUTOTRACE report includes both the optimizer  
-- execution path and the SQL statement execution  
-- statistics.
```

Displaying a Plan Table Using the DBMS_XPLAN.DISPLAY Package Function

```
-- Execute an explain plan command on a SELECT
-- statement

EXPLAIN PLAN FOR
SELECT * FROM emp e, dept d
  WHERE e.deptno = d.deptno
  AND e.ename='benoit';

-- Display the plan using the DBMS_XPLAN.DISPLAY table
-- function

SET LINESIZE 130
SET PAGESIZE 0
SELECT * FROM table(DBMS_XPLAN.DISPLAY);
```

Summary

In this appendix, you should have learned how to use SQL*Plus as an environment to do the following:

- Execute SQL statements
- Edit SQL statements
- Format output
- Interact with script files



Studies for Implementing Triggers

Objectives

After completing this lesson, you should be able to do the following:

- Enhance database security with triggers
- Enforce data integrity with DML triggers
- Maintain referential integrity using triggers
- Use triggers to replicate data between tables
- Use triggers to automate computation of derived data
- Provide event-logging capabilities using triggers

Controlling Security Within the Server

Using database security with the GRANT statement.

```
GRANT SELECT, INSERT, UPDATE, DELETE
  ON      employees
  TO      clerk;                -- database role
GRANT clerk TO scott;
```

Controlling Security with a Database Trigger

```
CREATE OR REPLACE TRIGGER secure_emp
  BEFORE INSERT OR UPDATE OR DELETE ON employees
DECLARE
dummy PLS_INTEGER;
BEGIN
  IF (TO_CHAR (SYSDATE, 'DY') IN ('SAT','SUN')) THEN
    RAISE_APPLICATION_ERROR(-20506,'You may only
      change data during normal business hours.');
```

```
  END IF;
  SELECT COUNT(*) INTO dummy FROM holiday
  WHERE holiday_date = TRUNC (SYSDATE);
  IF dummy > 0 THEN
    RAISE_APPLICATION_ERROR(-20507,
      'You may not change data on a holiday.');
```

```
  END IF;
END;
/
```

Enforcing Data Integrity Within the Server

```
ALTER TABLE employees ADD  
  CONSTRAINT ck_salary CHECK (salary >= 500);
```

Table altered.

Protecting Data Integrity with a Trigger

```
CREATE OR REPLACE TRIGGER check_salary
  BEFORE UPDATE OF salary ON employees
  FOR EACH ROW
  WHEN (NEW.salary < OLD.salary)
BEGIN
  RAISE_APPLICATION_ERROR (-20508,
    'Do not decrease salary.');
```

END;
/

Enforcing Referential Integrity Within the Server

```
ALTER TABLE employees
  ADD CONSTRAINT emp_deptno_fk
  FOREIGN KEY (department_id)
    REFERENCES departments(department_id)
  ON DELETE CASCADE;
```

Protecting Referential Integrity with a Trigger

```
CREATE OR REPLACE TRIGGER cascade_updates
  AFTER UPDATE OF department_id ON departments
  FOR EACH ROW
BEGIN
  UPDATE employees
    SET employees.department_id=:NEW.department_id
    WHERE employees.department_id=:OLD.department_id;
  UPDATE job_history
    SET department_id=:NEW.department_id
    WHERE department_id=:OLD.department_id;
END;
/
```

Replicating a Table Within the Server

```
CREATE MATERIALIZED VIEW emp_copy  
  NEXT sysdate + 7  
  AS SELECT * FROM employees@ny;
```

Replicating a Table with a Trigger

```
CREATE OR REPLACE TRIGGER emp_replica
BEFORE INSERT OR UPDATE ON employees FOR EACH ROW
BEGIN /* Proceed if user initiates data operation,
      NOT through the cascading trigger.*/
  IF INSERTING THEN
    IF :NEW.flag IS NULL THEN
      INSERT INTO employees@sf
      VALUES (:new.employee_id,...,'B');
      :NEW.flag := 'A';
    END IF;
  ELSE /* Updating. */
    IF :NEW.flag = :OLD.flag THEN
      UPDATE employees@sf
      SET ename=:NEW.last_name,...,flag=:NEW.flag
      WHERE employee_id = :NEW.employee_id;
    END IF;
    IF :OLD.flag = 'A' THEN :NEW.flag := 'B';
    ELSE :NEW.flag := 'A';
  END IF;
END IF;
END;
```

Computing Derived Data Within the Server

```
UPDATE departments
  SET total_sal=(SELECT SUM(salary)
                  FROM employees
                  WHERE employees.department_id =
                      departments.department_id);
```

Computing Derived Values with a Trigger

```
CREATE PROCEDURE increment_salary
  (id NUMBER, new_sal NUMBER) IS
BEGIN
  UPDATE departments
  SET    total_sal = NVL (total_sal, 0) + new_sal
  WHERE  department_id = id;
END increment_salary;
```

```
CREATE OR REPLACE TRIGGER compute_salary
AFTER INSERT OR UPDATE OF salary OR DELETE
ON employees FOR EACH ROW
BEGIN
  IF DELETING THEN      increment_salary(
    :OLD.department_id, (-1* :OLD.salary));
  ELSIF UPDATING THEN   increment_salary(
    :NEW.department_id, (:NEW.salary - :OLD.salary));
  ELSE                  increment_salary(
    :NEW.department_id, :NEW.salary); -- INSERT
  END IF;
END;
```

Logging Events with a Trigger

```
CREATE OR REPLACE TRIGGER notify_reorder_rep
BEFORE UPDATE OF quantity_on_hand, reorder_point
ON inventories FOR EACH ROW
DECLARE
  dsc product_descriptions.product_description%TYPE;
  msg_text VARCHAR2(2000);
BEGIN
  IF :NEW.quantity_on_hand <=
    :NEW.reorder_point THEN
    SELECT product_description INTO dsc
    FROM product_descriptions
    WHERE product_id = :NEW.product_id;
    msg_text := 'ALERT: INVENTORY LOW ORDER:' ||
      'Yours,' || CHR(10) || user || '.' || CHR(10);
  ELSIF :OLD.quantity_on_hand >=
    :NEW.quantity_on_hand THEN
    msg_text := 'Product #' || ... CHR(10);
  END IF;
  UTL_MAIL.SEND('inv@oracle.com', 'ord@oracle.com',
    message=>msg_text, subject=>'Inventory Notice');
END;
```


Summary

In this lesson, you should have learned how to:

- Enhance database security with triggers
- Enforce data integrity with DML triggers
- Maintain referential integrity using triggers
- Use triggers to replicate data between tables
- Use triggers to automate computation of derived data
- Provide event-logging capabilities using triggers



Using the DBMS_SCHEDULER and HTP Packages

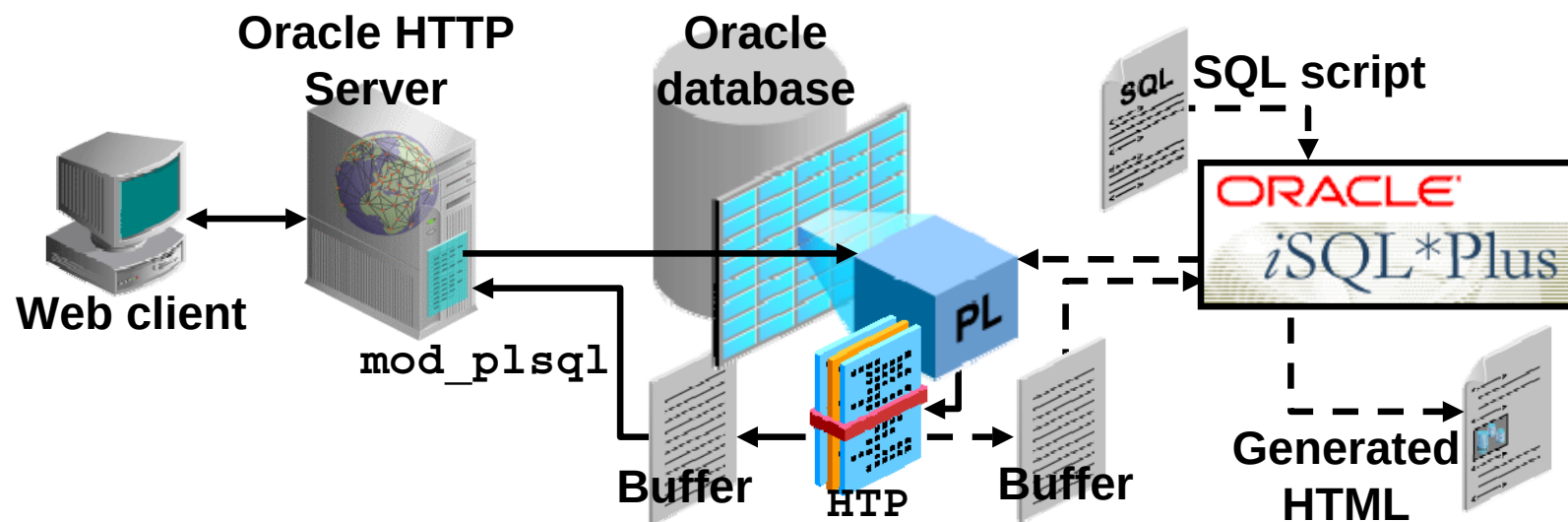
Objectives

After completing this lesson, you should be able to do the following:

- Use the `HTP` package to generate a simple Web page
- Call the `DBMS_SCHEDULER` package to schedule PL/SQL code for execution

Generating Web Pages with the HTP Package

- The HTP package procedures generate HTML tags.
- The HTP package is used to generate HTML documents dynamically and can be invoked from:
 - A browser using Oracle HTTP Server and PL/SQL Gateway (mod_plsql) services
 - An SQL*Plus script to display HTML output



Using the HTTP Package Procedures

- Generate one or more HTML tags. For example:

```
http.bold('Hello');           -- <B>Hello</B>
http.print('Hi <B>World</B>'); -- Hi <B>World</B>
```

- Are used to create a well-formed HTML document:

```
BEGIN                                -- Generates:
  http.htmlOpen;      ----->
  http.headOpen;      ----->
  http.title('Welcome'); -->
  http.headClose;     ----->
  http.bodyOpen;      ----->
  http.print('My home page');
  http.bodyClose;     ----->
  http.htmlClose;     ----->
END;
```

```
<HTML>
<HEAD>
<TITLE>Welcome</TITLE>
</HEAD>
<BODY>
My home page
</BODY>
</HTML>
```

Creating an HTML File with SQL*Plus

To create an HTML file with SQL*Plus, perform the following steps:

1. Create a SQL script with the following commands:

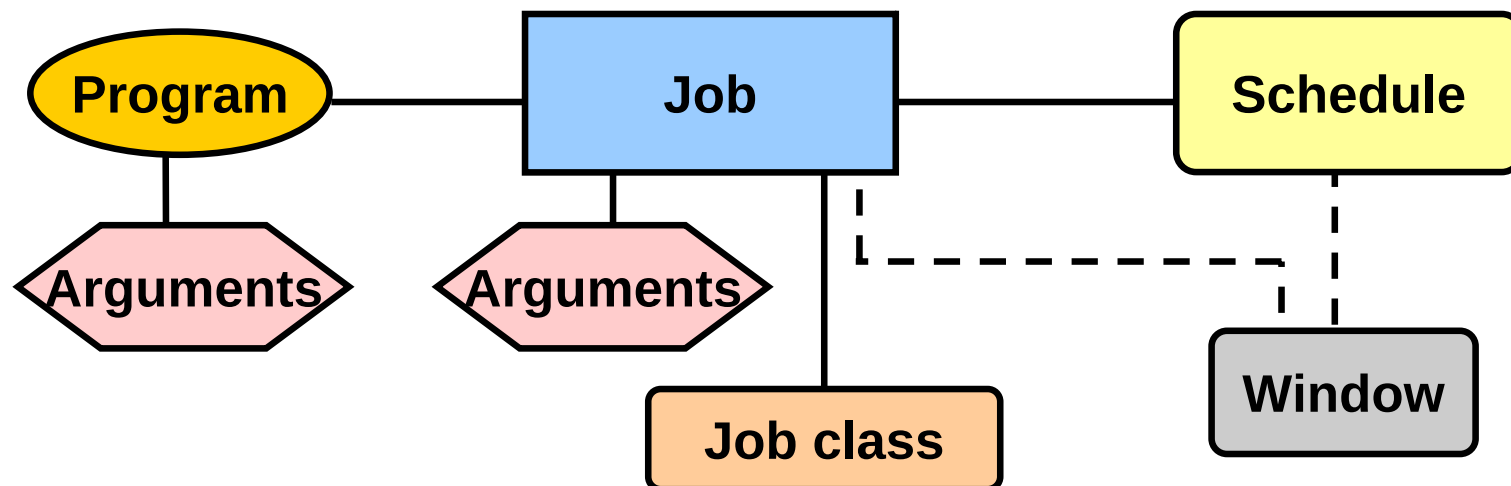
```
SET SERVEROUTPUT ON
ACCEPT procname PROMPT "Procedure: "
EXECUTE &procname
EXECUTE owa_util.showpage
UNDEFINE proc
```

2. Load and execute the script in SQL*Plus, supplying values for substitution variables.
3. Select, copy, and paste the HTML text that is generated in the browser to an HTML file.
4. Open the HTML file in a browser.

The DBMS_SCHEDULER Package

The database Scheduler comprises several components to enable jobs to be run. Use the DBMS_SCHEDULER package to create each job with:

- A unique job name
- A program (“what” should be executed)
- A schedule (“when” it should run)



Creating a Job

- A job can be created in several ways by using a combination of inline parameters, named Programs, and named Schedules.
- You can create a job with the `CREATE_JOB` procedure by:
 - Using inline information with the “what” and the schedule specified as parameters
 - Using a named (saved) program and specifying the schedule inline
 - Specifying what should be done inline and using a named Schedule
 - Using named Program and Schedule components

Creating a Job with Inline Parameters

Specify the type of code, code, start time, and frequency of the job to be run in the arguments of the `CREATE_JOB` procedure.

```
-- Schedule a PL/SQL block every hour:

BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name => 'JOB_NAME',
    job_type => 'PLSQL_BLOCK',
    job_action => 'BEGIN ...; END;',
    start_date => SYSTIMESTAMP,
    repeat_interval=>'FREQUENCY=HOURLY;INTERVAL=1',
    enabled => TRUE);
END;
/
```

Creating a Job Using a Program

- Use CREATE_PROGRAM to create a program:

```
BEGIN
  DBMS_SCHEDULER.CREATE_PROGRAM(
    program_name => 'PROG_NAME',
    program_type => 'PLSQL_BLOCK',
    program_action => 'BEGIN ...; END;');
END;
```

- Use overloaded CREATE_JOB procedure with its program_name parameter:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    program_name => 'PROG_NAME',
    start_date => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY',
    enabled => TRUE);
END;
```

Creating a Job for a Program with Arguments

- Create a program:

```
DBMS_SCHEDULER.CREATE_PROGRAM(  
  program_name => 'PROG_NAME',  
  program_type => 'STORED PROCEDURE',  
  program_action => 'EMP_REPORT');
```

- Define an argument:

```
DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(  
  program_name => 'PROG_NAME',  
  argument_name => 'DEPT_ID',  
  argument_position=> 1, argument_type=> 'NUMBER',  
  default_value => '50');
```

- Create a job specifying the number of arguments:

```
DBMS_SCHEDULER.CREATE_JOB('JOB_NAME', program_name  
  => 'PROG_NAME', start_date => SYSTIMESTAMP,  
  repeat_interval => 'FREQ=DAILY',  
  number_of_arguments => 1, enabled => TRUE);
```

Creating a Job Using a Schedule

- Use CREATE_SCHEDULE to create a schedule:

```
BEGIN
  DBMS_SCHEDULER.CREATE_SCHEDULE('SCHED_NAME',
    start_date => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY',
    end_date => SYSTIMESTAMP +15);
END;
```

- Use CREATE_JOB by referencing the schedule in the schedule_name parameter:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    schedule_name => 'SCHED_NAME',
    job_type => 'PLSQL_BLOCK',
    job_action => 'BEGIN ...; END;',
    enabled => TRUE);
END;
```

Setting the Repeat Interval for a Job

- Using a calendaring expression:

```
repeat_interval=> 'FREQ=HOURLY; INTERVAL=4 '  
repeat_interval=> 'FREQ=DAILY '  
repeat_interval=> 'FREQ=MINUTELY; INTERVAL=15 '  
repeat_interval=> 'FREQ=YEARLY;  
                  BYMONTH=MAR, JUN, SEP, DEC;  
                  BYMONTHDAY=15 '
```

- Using a PL/SQL expression:

```
repeat_interval=> 'SYSDATE + 36/24 '  
repeat_interval=> 'SYSDATE + 1 '  
repeat_interval=> 'SYSDATE + 15/(24*60) '
```

Creating a Job Using a Named Program and Schedule

- Create a named program called `PROG_NAME` by using the `CREATE_PROGRAM` procedure.
- Create a named schedule called `SCHED_NAME` by using the `CREATE_SCHEDULE` procedure.
- Create a job referencing the named program and schedule:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    program_name => 'PROG_NAME',
    schedule_name => 'SCHED_NAME',
    enabled => TRUE);
END;
/
```

Managing Jobs

- Run a job:

```
DBMS_SCHEDULER.RUN_JOB ( 'SCHEMA.JOB_NAME' ) ;
```

- Stop a job:

```
DBMS_SCHEDULER.STOP_JOB ( 'SCHEMA.JOB_NAME' ) ;
```

- Drop a job even if it is currently running:

```
DBMS_SCHEDULER.DROP_JOB ( 'JOB_NAME' , TRUE ) ;
```

Data Dictionary Views

- [DBA | ALL | USER] _SCHEDULER_JOBS
- [DBA | ALL | USER] _SCHEDULER_RUNNING_JOBS
- [DBA | ALL] _SCHEDULER_JOB_CLASSES
- [DBA | ALL | USER] _SCHEDULER_JOB_LOG
- [DBA | ALL | USER] _SCHEDULER_JOB_RUN_DETAILS
- [DBA | ALL | USER] _SCHEDULER_PROGRAMS

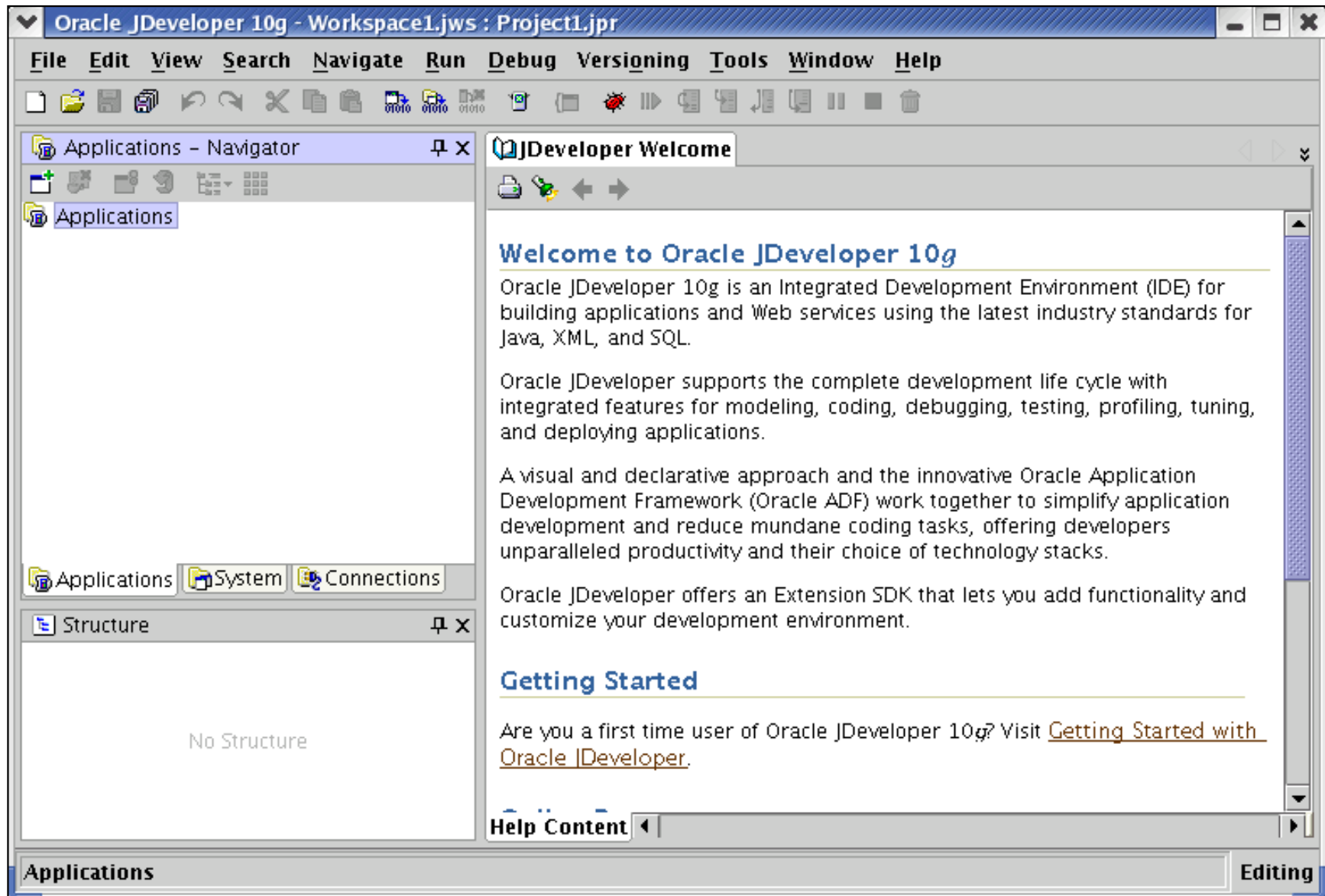
Summary

In this lesson, you should have learned how to:

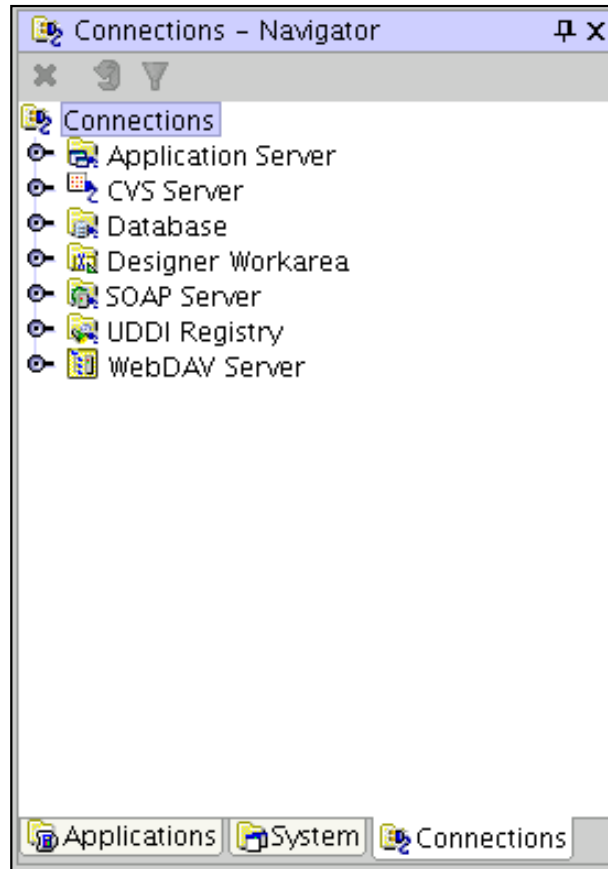
- Use the `HTP` package to generate a simple Web page
- Call the `DBMS_SCHEDULER` package to schedule PL/SQL code for execution

Review of JDeveloper

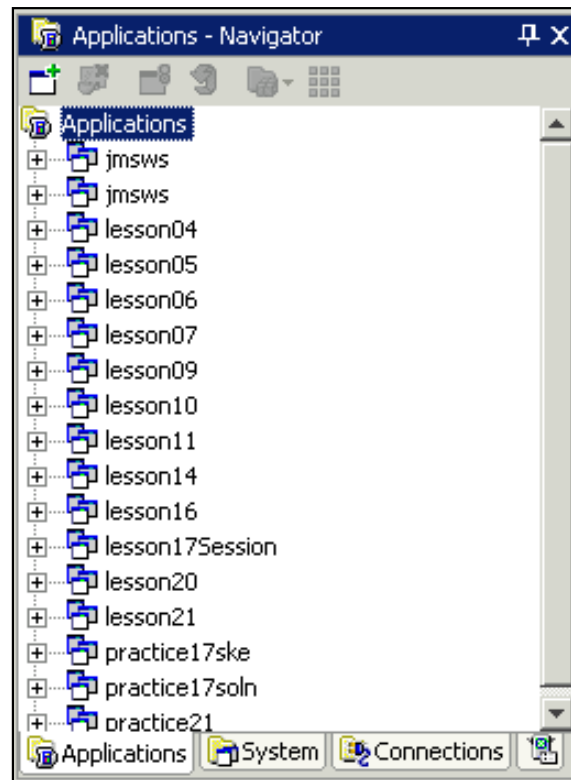
JDeveloper



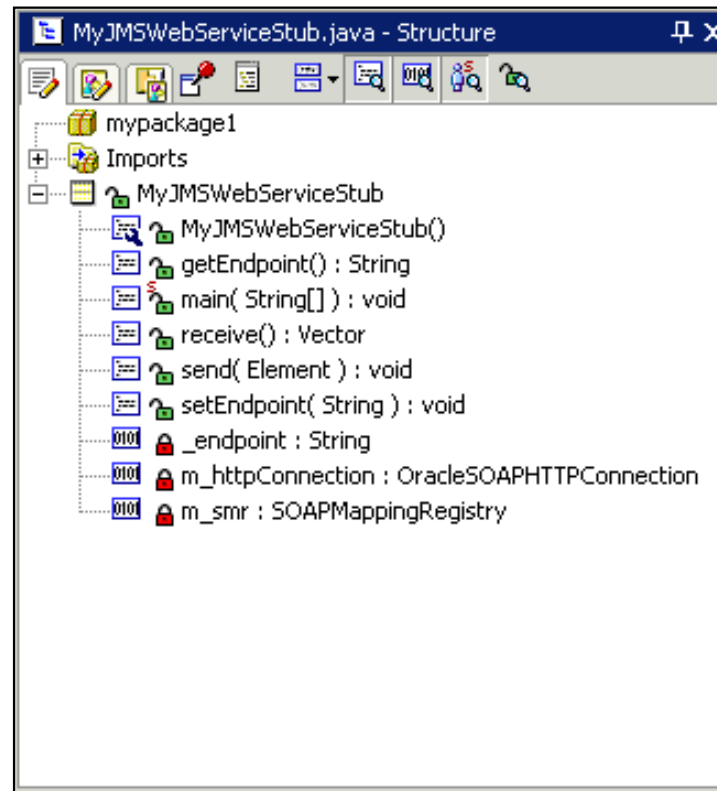
Connection Navigator



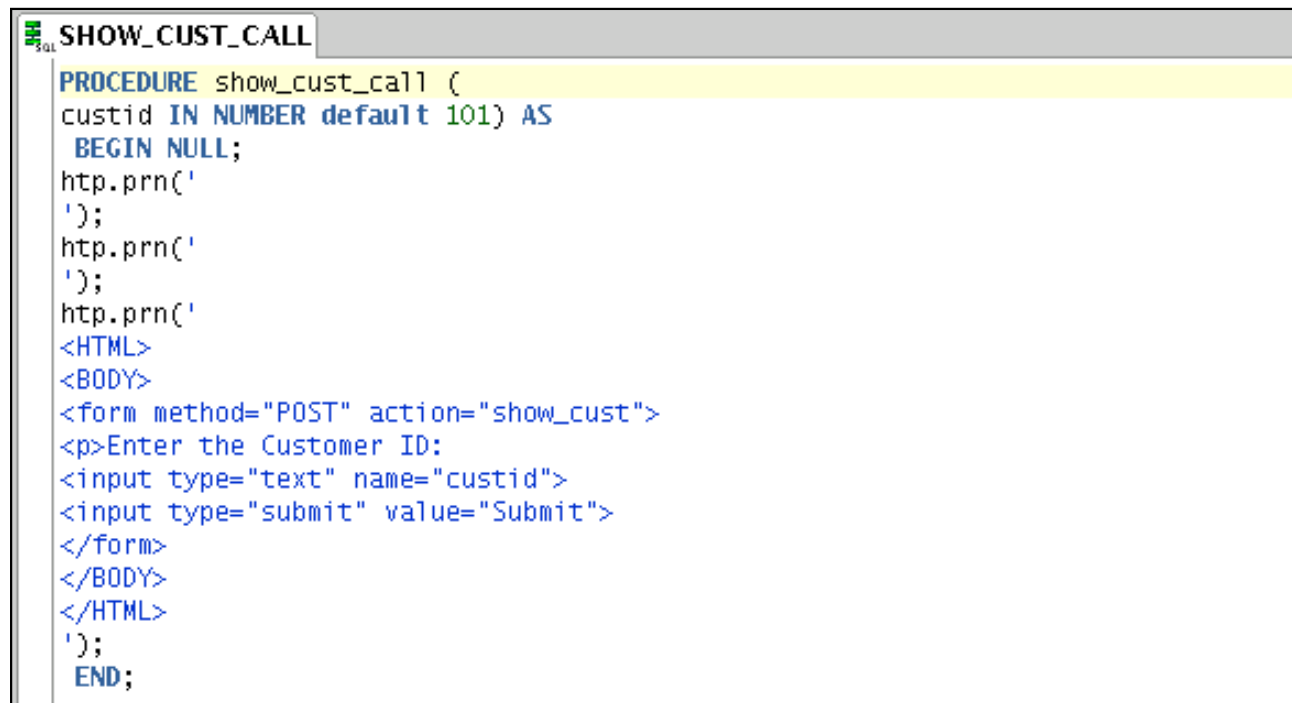
Application Navigator



Structure Window



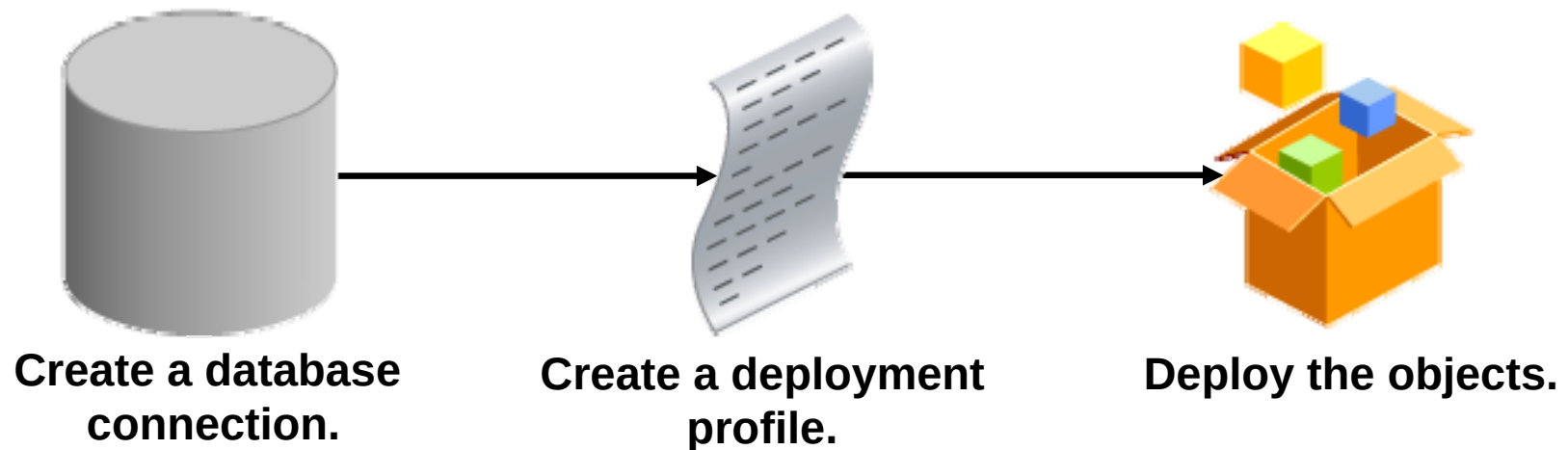
Editor Window



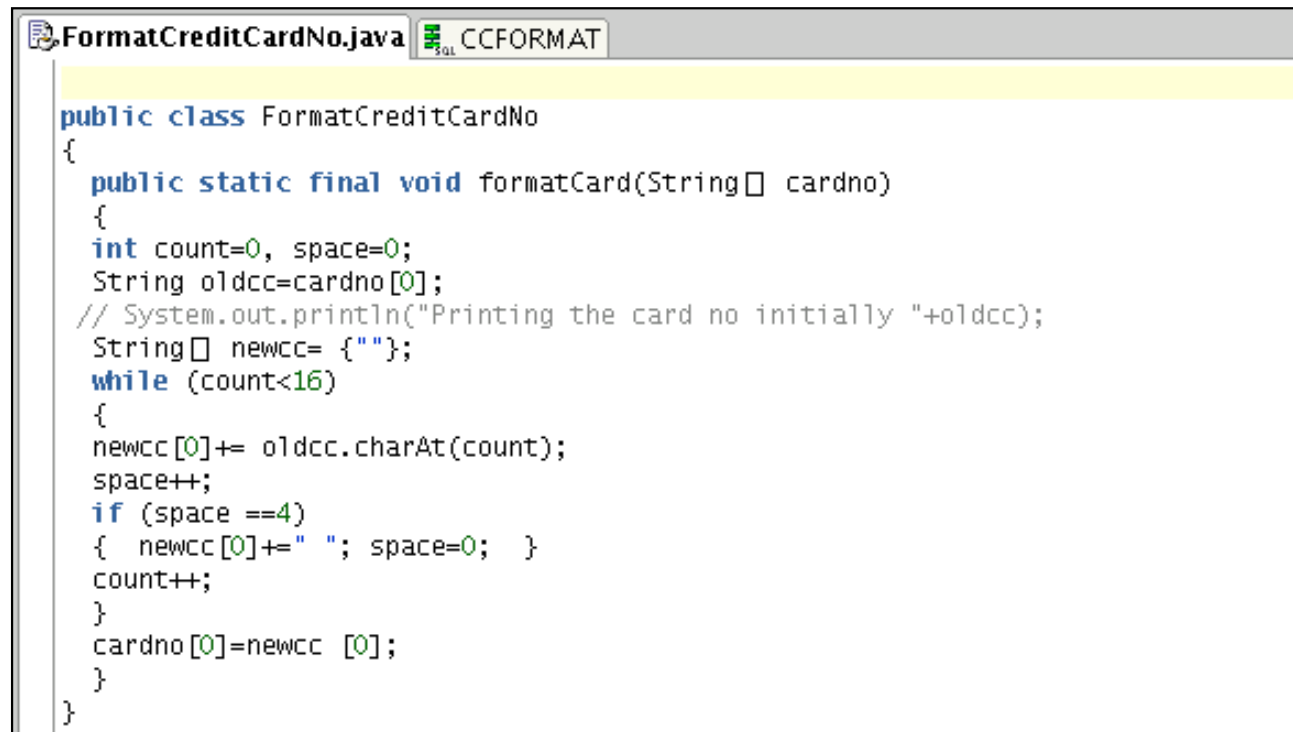
```
SHOW_CUST_CALL
PROCEDURE show_cust_call (
custid IN NUMBER default 101) AS
BEGIN NULL;
http.prn('
');
http.prn('
');
http.prn('
<HTML>
<BODY>
<form method="POST" action="show_cust">
<p>Enter the Customer ID:
<input type="text" name="custid">
<input type="submit" value="Submit">
</form>
</BODY>
</HTML>
');
END;
```

Deploying Java Stored Procedures

Before deploying Java stored procedures, perform the following steps:



Publishing Java to PL/SQL



```
public class FormatCreditCardNo
{
    public static final void formatCard(String[] cardno)
    {
        int count=0, space=0;
        String oldcc=cardno[0];
        // System.out.println("Printing the card no initially "+oldcc);
        String[] newcc= {" "};
        while (count<16)
        {
            newcc[0] += oldcc.charAt(count);
            space++;
            if (space ==4)
            { newcc[0] += " "; space=0; }
            count++;
        }
        cardno[0]=newcc [0];
    }
}
```



```
PROCEDURE ccformat (x IN OUT varchar2)
AS LANGUAGE JAVA
NAME 'FormatCreditCardNo.formatCard(java.lang.String[])';
```

Creating Program Units

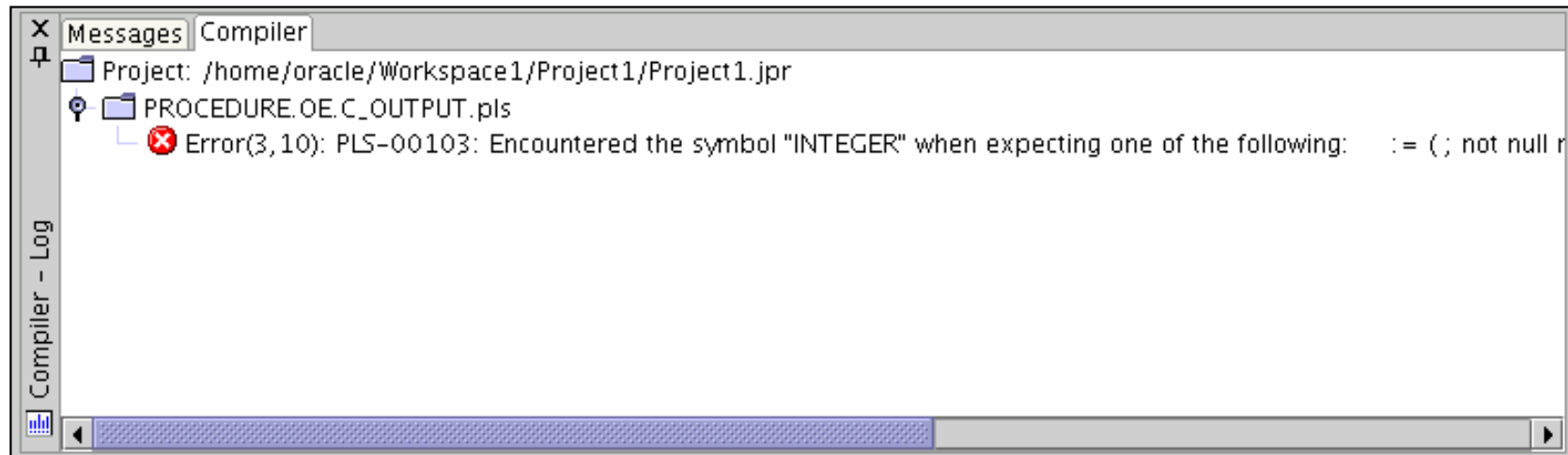


A screenshot of a SQL editor window. The title bar shows a small icon and the text "TEST_JDEV". The editor area contains the following SQL code:

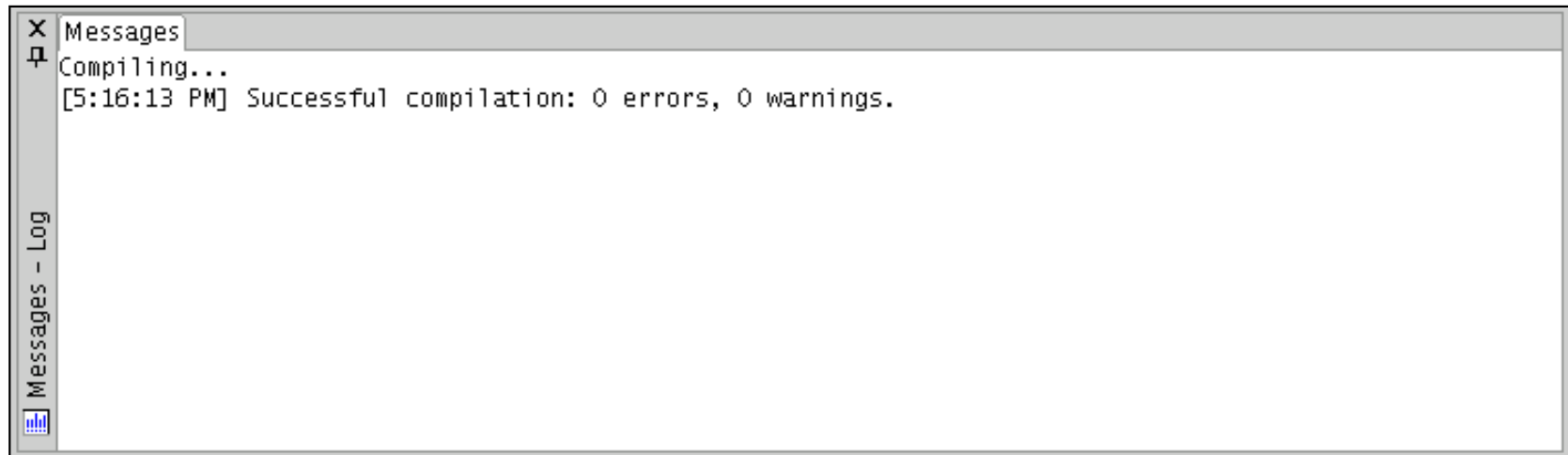
```
FUNCTION "TEST_JDEV" RETURN VARCHAR2  
AS  
BEGIN  
    RETURN('');  
END;
```

Skeleton of the function

Compiling

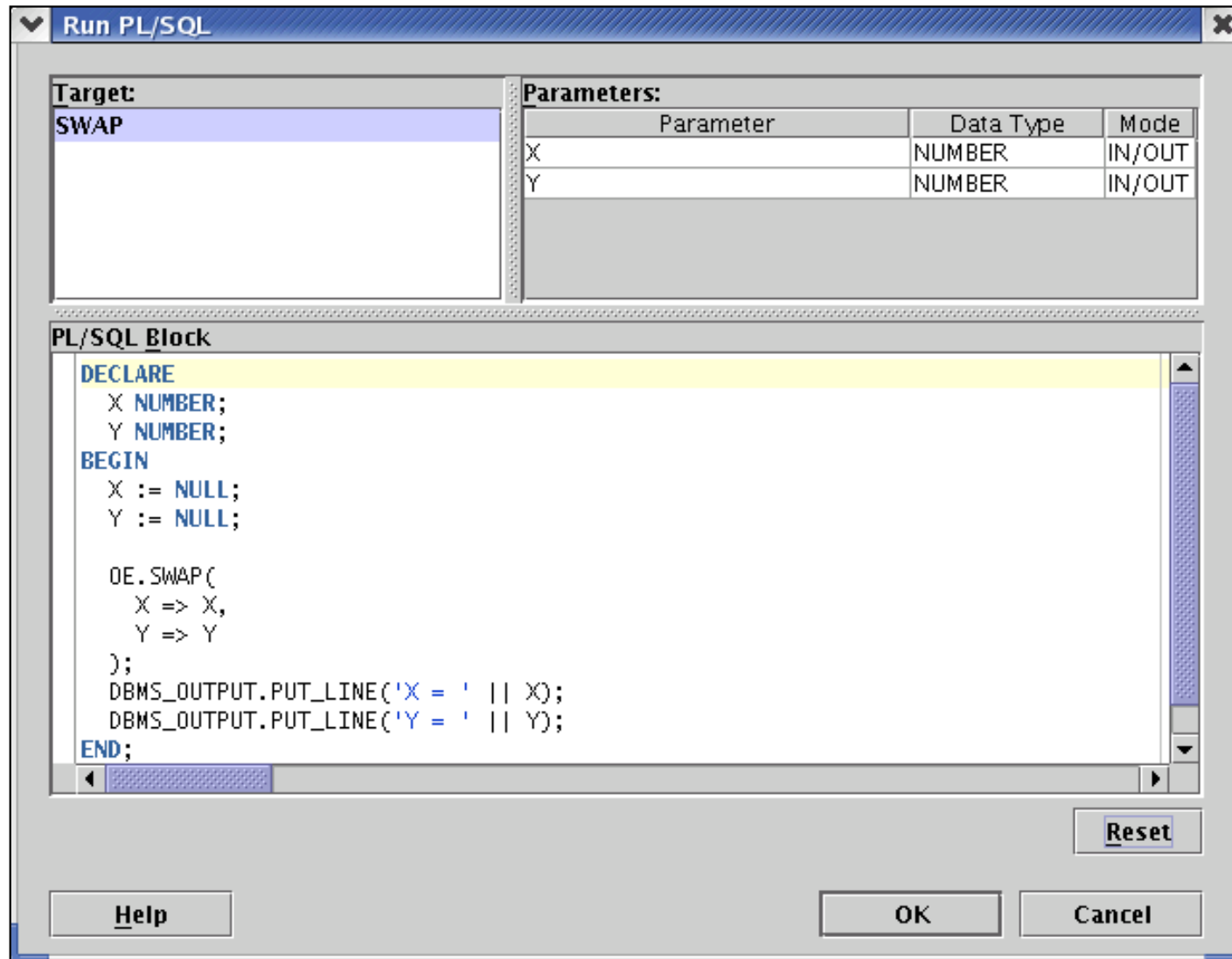


Compilation with errors

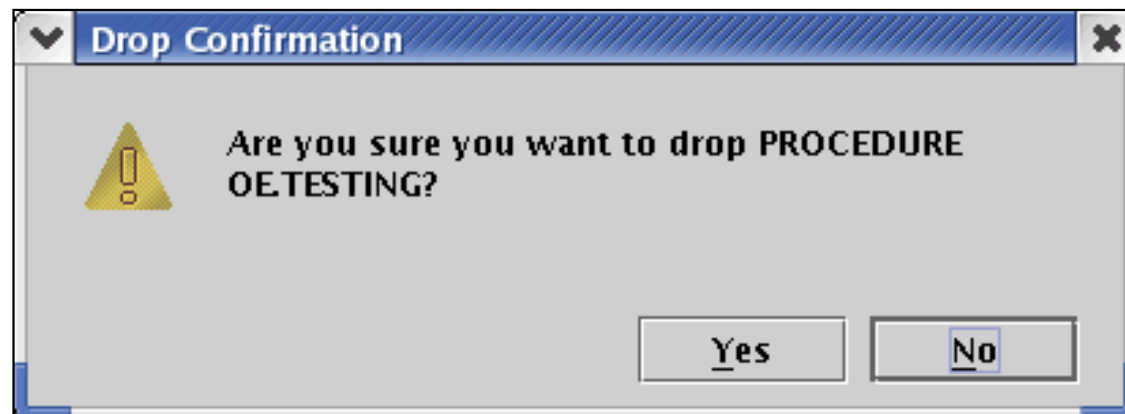


Compilation without errors

Running a Program Unit



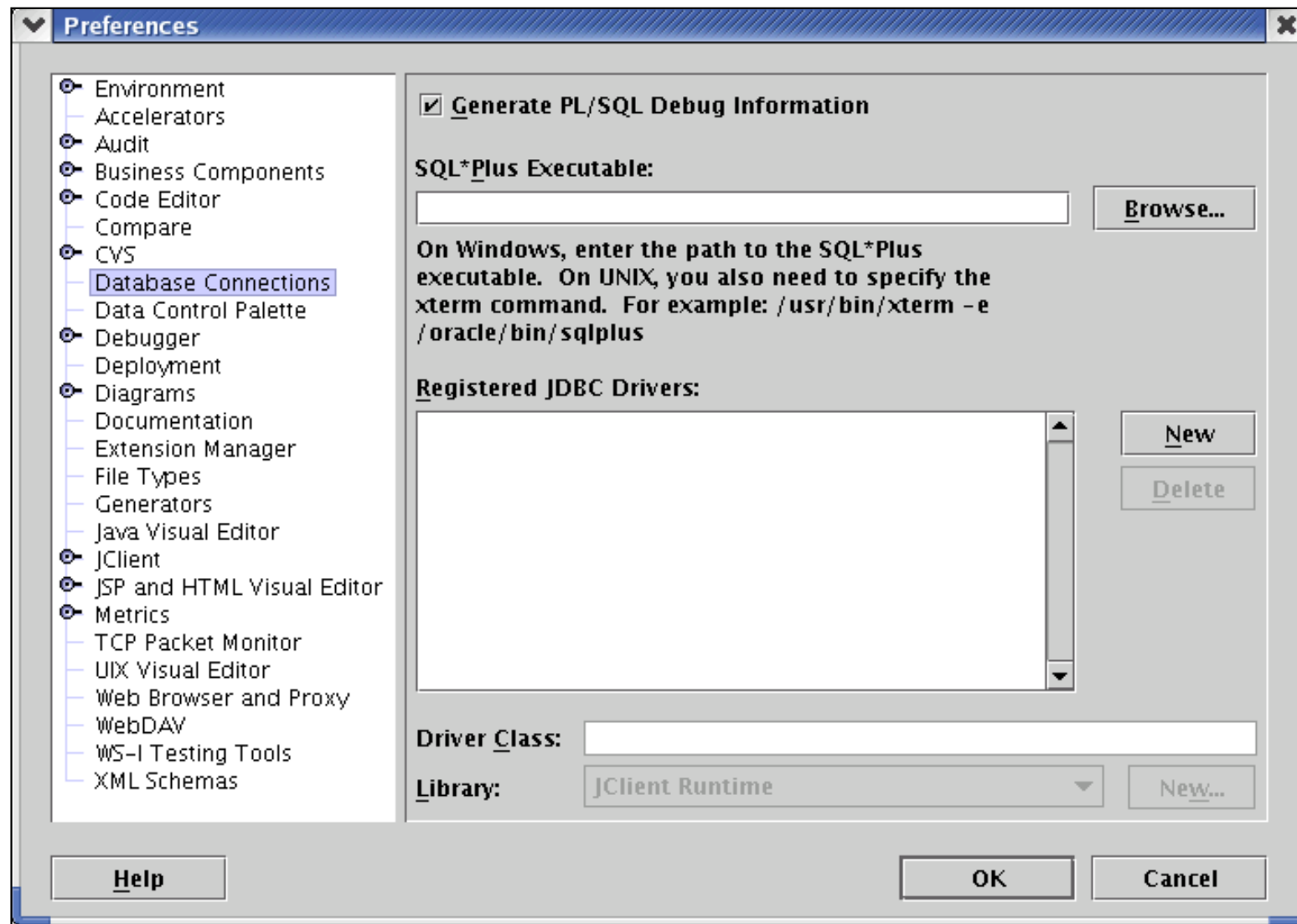
Dropping a Program Unit



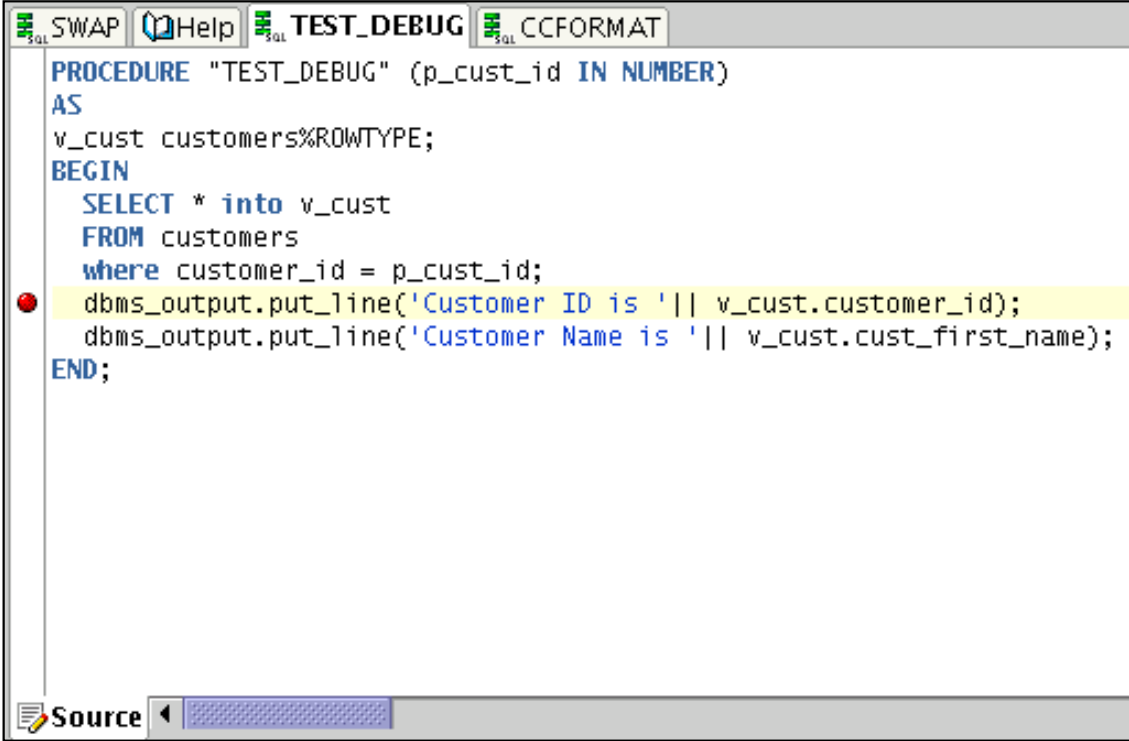
Debugging PL/SQL Programs

- JDeveloper support two types of debugging:
 - Local
 - Remote
- You need the following privileges to perform PL/SQL debugging:
 - `DEBUG ANY PROCEDURE`
 - `DEBUG CONNECT SESSION`

Debugging PL/SQL Programs



Setting Breakpoints



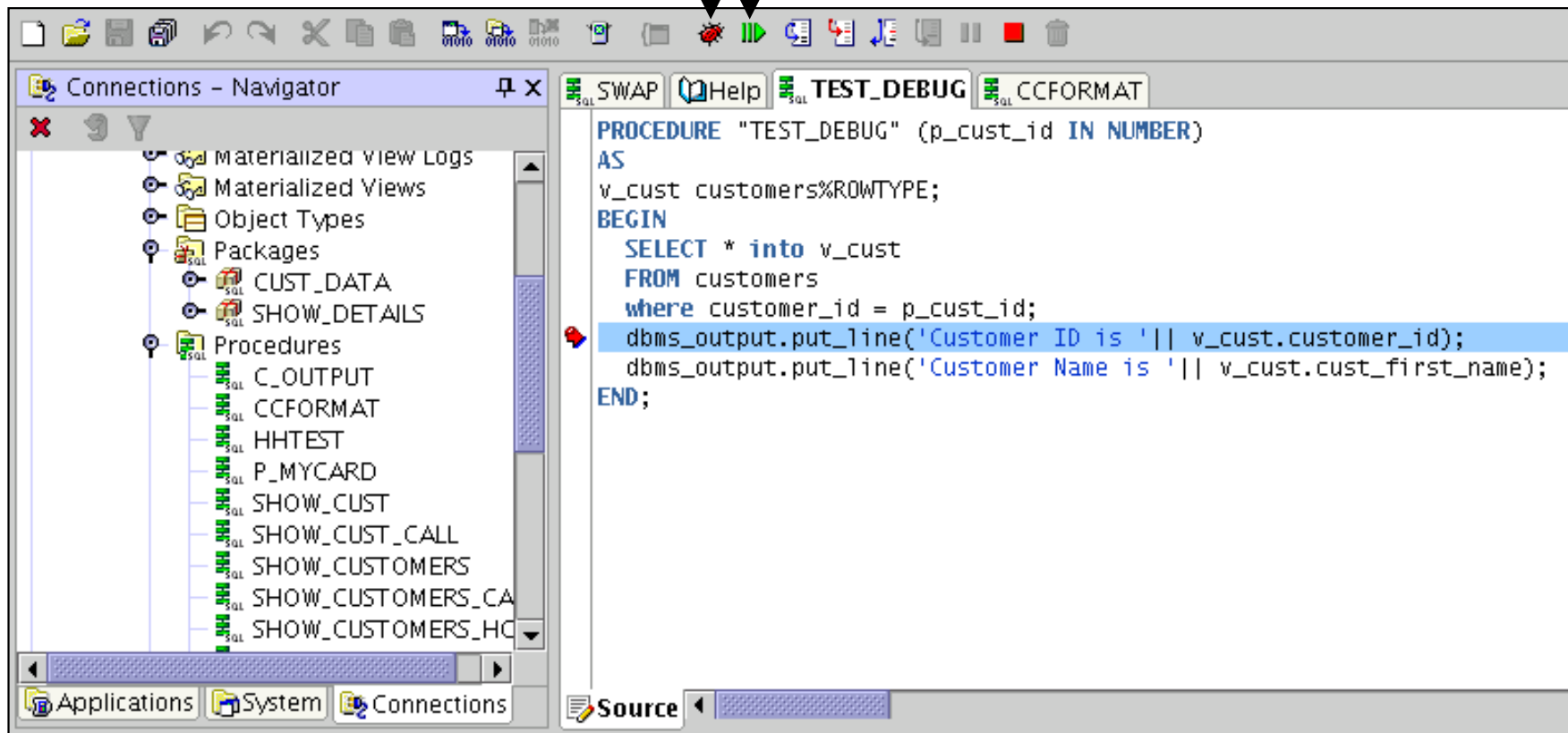
The screenshot shows a window titled "TEST_DEBUG" with a menu bar containing "SWAP", "Help", "TEST_DEBUG", and "CCFORMAT". The main area contains the following PL/SQL code:

```
PROCEDURE "TEST_DEBUG" (p_cust_id IN NUMBER)
AS
v_cust customers%ROWTYPE;
BEGIN
  SELECT * into v_cust
  FROM customers
  where customer_id = p_cust_id;
  dbms_output.put_line('Customer ID is ' || v_cust.customer_id);
  dbms_output.put_line('Customer Name is ' || v_cust.cust_first_name);
END;
```

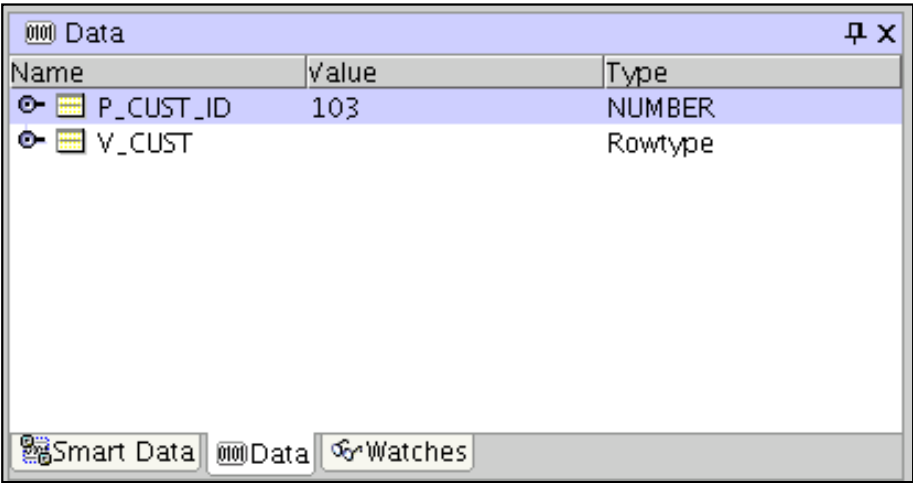
A red circular breakpoint icon is positioned to the left of the line `dbms_output.put_line('Customer ID is ' || v_cust.customer_id);`, which is highlighted in yellow. The bottom status bar shows a "Source" tab and a scroll bar.

Stepping Through Code

Debug Resume



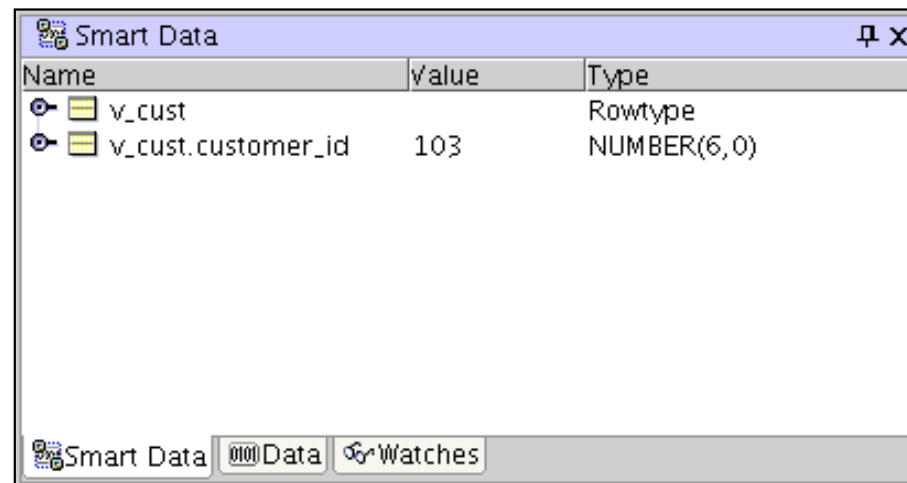
Examining and Modifying Variables



Name	Value	Type
P_CUST_ID	103	NUMBER
V_CUST		Rowtype

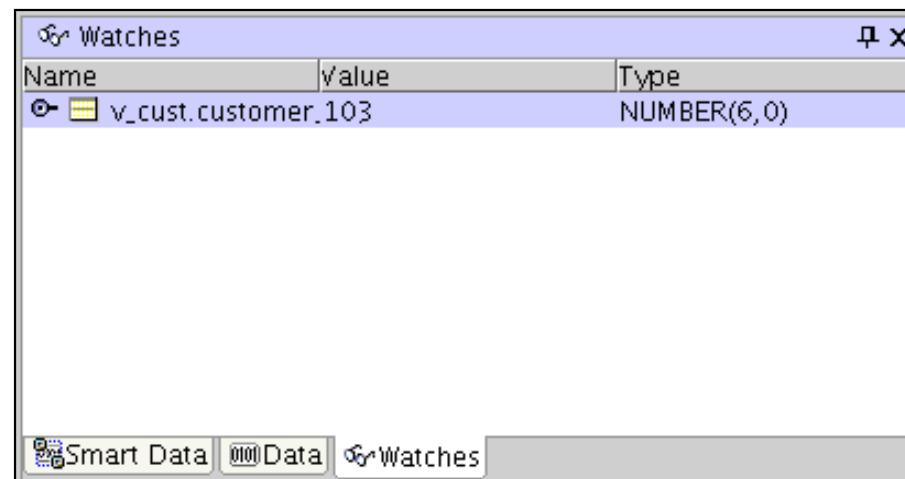
Data window

Examining and Modifying Variables



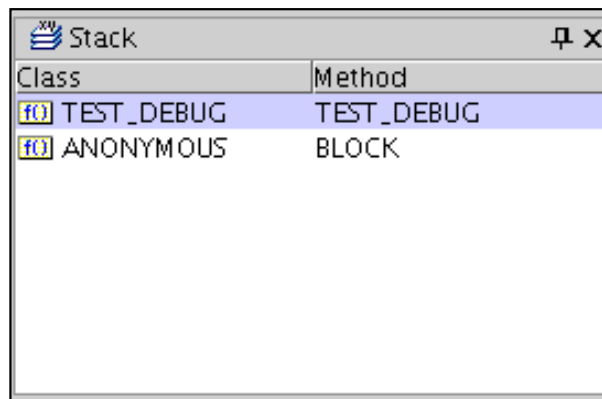
Smart Data window

Examining and Modifying Variables



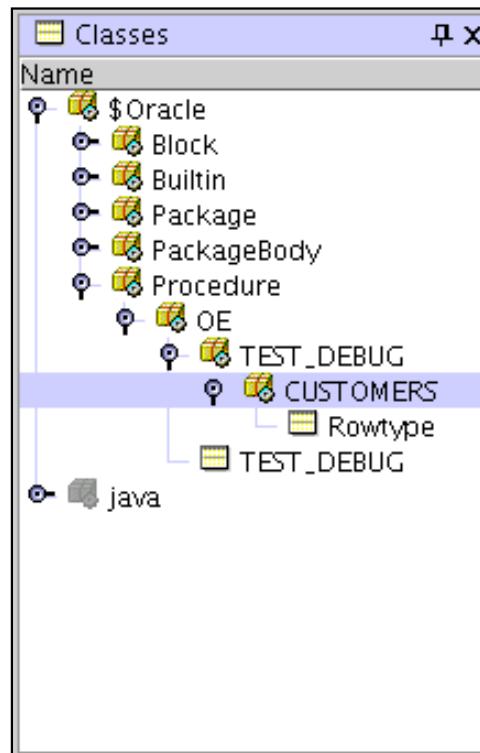
Watches window

Examining and Modifying Variables



Stack window

Examining and Modifying Variables



Classes window